



Guía Didáctica Completa — Proyecto Intermodular

Sergi Garcia Barea

CC BY-SA 4.0

Guía Didáctica Completa — Proyecto Intermodular

Guia Didactica Proyecto 1 Index

 *Lo primero: aprender a planificar antes de programar*

Proyecto 1 no es solo escribir código. Es aprender a **pensar** antes de construir: identificar un problema, definir requisitos, elegir tecnologías, comunicar ideas y documentar el proceso. Todo lo que aprendas aquí te servirá en Proyecto 2 y en tu vida profesional.

¿Qué es Proyecto 1?

Proyecto 1 es la primera fase del módulo de Proyecto. Se desarrolla durante el primer curso y consta de **7 unidades didácticas** que te guían desde la idea inicial hasta la preparación de una propuesta de aplicación web.

Las 7 unidades

UD	Qué harás	Enfoque
 UD 1.1: MVP	Definir qué es un producto mínimo viable	Conceptos clave
 UD 1.2: SCRUM Lite	Metodología ágil adaptada a estudiantes	Organización
 UD 1.3: Requisitos	Capturar y documentar qué necesita el sistema	Análisis
 UD 1.4: Tecnologías	Elegir stack tecnológico con criterio	Decisiones
 UD 1.5: Comunicación	Presentar y defender tu proyecto	Soft skills
 UD 1.6: Documentación	Crear documentación técnica moderna	Docs as Code
 UD 1.7: Propuesta	Ejemplos de proyectos y cómo empezar	Práctica

Evaluación

Cada UD se evalúa de forma independiente. Consulta la guía de evaluación para conocer los criterios y pesos de cada unidad.

Competencias Proyecto 1

Competencias generales

- **CG1:** Capacidad de análisis y síntesis para identificar problemas y proponer soluciones
- **CG2:** Capacidad de organización y planificación del trabajo
- **CG3:** Comunicación oral y escrita en la presentación de resultados
- **CG4:** Trabajo en equipo y colaboración con otros compañeros
- **CG5:** Aprendizaje autónomo y búsqueda de información técnica

Competencias específicas

- **CE1:** Definir un Producto Mínimo Viable (MVP) con alcance claro
- **CE2:** Aplicar metodología ágil (SCRUM Lite) en un proyecto individual
- **CE3:** Capturar y documentar requisitos e historias de usuario

- **CE4:** Elegir tecnologías con criterio arquitectónico
- **CE5:** Comunicar y defender un proyecto ante un tribunal
- **CE6:** Crear documentación técnica con enfoque Docs as Code
- **CE7:** Diseñar y proponer una aplicación web con alcance claro y persistencia de datos

Relación con las asignaturas

Asignatura	Competencias que aporta
Programación	Lógica, estructuras de datos, POO
Bases de Datos	Diseño y consulta de bases de datos
Lenguajes de Marcas	HTML, XML, documentación
Entornos de Desarrollo	Git, testing, depuración
Sistemas Informáticos	Instalación y configuración

Evaluación Proyecto 1

Criterios de evaluación

Criterio	Peso orientativo
Cumplimiento de objetivos	30%
Calidad técnica del trabajo	25%
Documentación y memoria	20%
Presentación y defensa	15%
Trabajo en equipo (si aplica)	10%

Sistema de calificación

Cada UD se califica de 0 a 10. La nota final es la media ponderada:

UD	Peso orientativo
UD 1.1: MVP	15%
UD 1.2: SCRUM Lite	10%
UD 1.3: Requisitos	15%
UD 1.4: Tecnologías	15%
UD 1.5: Comunicación	15%
UD 1.6: Documentación	10%
UD 1.7: Propuesta	20%

Nota: los pesos son orientativos; el profesor concretará los porcentajes al inicio del curso.

Criterios de calidad

Para obtener más de un 5 en cada UD:

- **UD 1.1:** MVP viable, alcance definido, justificación clara
- **UD 1.2:** Tablero funcional, sprints planificados, retrospectiva
- **UD 1.3:** Requisitos completos, historias con criterios de aceptación
- **UD 1.4:** Stack justificado, arquitectura por capas, decisiones documentadas
- **UD 1.5:** Presentación clara, demo funcional, defensa solvente
- **UD 1.6:** README completo, documentación técnica, código comentado
- **UD 1.7:** Proyecto funcional, CRUD completo, persistencia en BD

Observaciones importantes

- Las UD's se entregan en las fechas indicadas
- La **falta de entrega** de una UD supone no superar la evaluación continua
- Las faltas de ortografía en la documentación penalizan
- El código debe ser original (no copiado de internet sin atribución)

Guia Didactica Proyecto 1 Uds U1 1 Mvp

Un MVP no es un proyecto a medias. Es la versión más pequeña de tu idea que **resuelve un problema real**. Si no puedes explicar tu MVP en una frase, es demasiado grande. Esta unidad te enseña a encontrar esa versión mínima que tiene sentido.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

- 🎯 Definir qué es un MVP y por qué importa
- 🔍 Identificar un problema real que merezca la pena resolver
- 📋 Listar y priorizar funcionalidades con criterio
- ✂️ Cortar el alcance sin miedo para tener algo terminado

Mapa de la unidad

Punto	Tema	Idea clave
01	¿Qué es un MVP?	Definición, origen y mitos
02	Identificar el problema	Cómo encontrar problemas reales
03	Definir features	Listar, clasificar y priorizar
04	Cortar sin piedad	Reglas de alcance y cuándo parar
05	Ejemplos reales	Ideas que funcionaron y por qué
06	Errores comunes	Los 7 pecados del MVP
07	Herramientas	Qué usar para organizar tu MVP
08	Template	Plantilla rellenable para tu proyecto
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 08 \(template\)](#) y empieza a rellenar
- **Si no tienes idea:** empieza por el [punto 02 \(identificar problema\)](#)
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

 **SCRUM:** trabajar en equipo sin morir en el intento

Tener una buena idea (UD 1.1) no sirve de nada si no sabes **cómo organizar el trabajo**. SCRUM es un método que obliga a trabajar en ciclos cortos, terminar cosas de verdad y hablar entre el equipo. Esta unidad es “SCRUM Lite”: lo esencial sin la burocracia de las empresas grandes.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Entender qué es SCRUM y por qué funciona para proyectos reales
-  Organizar tu trabajo en sprints con objetivos claros
-  Usar un tablero Kanban para ver qué está hecho y qué falta
-  Escribir user stories que tengan sentido para el usuario
-  Estimar esfuerzo sin experiencia previa
-  Evitar los errores más comunes de SCRUM en el aula

Mapa de la unidad

Punto	Tema	Idea clave
01	¿Qué es ágil?	Waterfall vs Agile, el Manifiesto
02	¿Qué es SCRUM?	Roles, ceremonias, artefactos
03	El Sprint	Ciclos de 1-2 semanas con objetivo
04	Tablero Kanban	To Do → In Progress → Done
05	User Stories	“Como... quiero... para que...”
06	Estimación	T-shirt sizes, Planning Poker
07	Errores comunes	Los 6 pecados de SCRUM en el aula
08	Template	Planning, checklist y retrospectiva
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 08 \(template\)](#) y empieza a rellenar
- **Si no tienes idea:** empieza por el [punto 02 \(¿Qué es SCRUM?\)](#)
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

Guia Didactica Proyecto 1 Uds U1 3 Requisitos

 *Los requisitos: saber qué construir antes de construirlo*

Programar sin requisitos es como cocinar sin receta: quizá salga algo comestible, pero seguramente no es lo que querías. Esta unidad te enseña a **definir qué debe hacer tu app**, para quién y con qué restricciones, antes de escribir una sola línea de código.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Diferenciar requisitos funcionales y no funcionales
-  Escribir historias de usuario claras y testables
-  Definir criterios de aceptación en formato Dado/Cuando/Entonces
-  Priorizar requisitos con MoSCoW, Kano y valor vs esfuerzo
-  Evitar los errores más comunes en definición de requisitos
-  Generar un documento de requisitos completo para tu proyecto

Mapa de la unidad

Punto	Tema	Idea clave
01	¿Qué son los requisitos?	Funcionales vs no funcionales
02	Requisitos funcionales	Qué debe hacer la app
03	Requisitos no funcionales	Rendimiento, seguridad, usabilidad
04	Historias de usuario	“Como... quiero... para que...”
05	Criterios de aceptación	Dado/Cuando/Entonces
06	Priorización	MoSCoW, Kano, valor vs esfuerzo
07	Errores comunes	“El usuario puede hacer todo” y más
08	Template requisitos	Documento listo para usar
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 08 \(template\)](#) y rellena el documento
- **Si no tienes idea:** empieza por el [punto 01 \(¿Qué son los requisitos?\)](#)
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

UD 1.4: Elección de Tecnologías

 *Elegir bien ahorra semanas de sufrimiento*

Cada proyecto necesita tecnologías: un lenguaje, una base de datos, un framework. Pero elegir mal puede costarte semanas de refactorización o un producto que no funciona. Esta unidad te enseña a **evaluar opciones con criterio**, no por moda ni por lo que usa tu amigo.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

- 🗨️ Entender por qué la elección tecnológica importa y cuál es el coste de equivocarse
- 📊 Evaluar tecnologías con criterios objetivos (rendimiento, curva de aprendizaje, ecosistema, comunidad)
- 🔍 Comparar lenguajes y frameworks: Python, Java, JavaScript, PHP
- 💾 Elegir la base de datos correcta: SQL vs NoSQL, SQLite, PostgreSQL, MongoDB
- ⚖️ Decidir entre frontend y backend, MVC, monolito vs microservicios
- 📋 Comparar stacks completos: LAMP, MERN, Python+SQLite, Java+MySQL
- ✍️ Redactar la justificación técnica de tu proyecto
- 📄 Documentar tu stack con el template oficial

Mapa de la unidad

Punto	Tema	Idea clave
01	¿Por qué elegir?	El coste de elegir mal
02	Criterios de elección	Qué valorar antes de decidir
03	Lenguajes y frameworks	Python, Java, JavaScript, PHP
04	Bases de datos	SQL vs NoSQL, cuándo usar cada una
05	Frontend vs Backend	MVC, monolito, arquitectura
06	Comparativa stack	LAMP, MERN, Python+SQLite, Java+MySQL
07	Justificación	Cómo argumentar tus decisiones
08	Template stack	Documento listo para usar
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 08 \(template\)](#) y documenta tu stack
- **Si no sabes qué elegir:** empieza por el [punto 02 \(criterios\)](#) para aprender a evaluar
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

 *Comunicar es tan importante como programar*

Puedes tener el mejor proyecto del mundo, pero si no sabes explicarlo, el tribunal no lo valorará. Esta unidad te enseña a **presentar tu trabajo** de forma clara, profesional y convincente: pitch de 5 minutos, diapositivas efectivas, demo en vivo y defensa ante preguntas del tribunal.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Preparar un pitch de 5 minutos con estructura clara
-  Modular tu lenguaje verbal: tono, ritmo y confianza
-  Gestionar tu lenguaje no verbal: postura, gestos, contacto visual
-  Escribir un memoria/proyecto clara y bien estructurada
-  Crear diapositivas efectivas (menos texto, más impacto)
-  Hacer una demo en vivo sin problemas
-  Responder preguntas del tribunal con seguridad

Mapa de la unidad

Punto	Tema	Idea clave
01	¿Por qué comunicar?	La presentación es tu primera impresión
02	Pitch 5 minutos	Estructura: problema, solución, demo, tech, cierre
03	Lenguaje verbal	Tono, ritmo, muletillas y confianza
04	Lenguaje no verbal	Postura, contacto visual, gestos
05	Memoria del proyecto	Estructura y estilo del documento
06	Diapositivas	Menos texto, más visuales, herramientas
07	Demo práctica	Preparar, probar y plan B
08	Preguntas del tribunal	Preguntas frecuentes y cómo responder
09	Cierre	Mini-chequeo, vocabulario y resumen

- **Si tienes prisa:** ve directamente al [punto 02 \(pitch\)](#) y ensaya
- **Si no tienes idea:** empieza por el [punto 01 \(¿Por qué comunicar?\)](#)
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

UD 1.6: Documentación

 *Documentar no es opcional, es parte del proyecto*

Un proyecto sin documentación es como un libro sin índice: puede ser genial, pero nadie sabe cómo usarlo. Esta unidad te enseña a **escribir documentación clara, útil y profesional** que haga que tu proyecto sea entendible por otros — y por ti mismo dentro de 6 meses.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Entender la filosofía Docs as Code y por qué importa
-  Escribir un README claro y completo
-  Crear documentación técnica: arquitectura, setup, API
-  Redactar un manual de usuario para no técnicos
-  Usar Markdown con soltura
-  Elegir las herramientas de documentación adecuadas
-  Evitar los errores más comunes al documentar
-  Usar una plantilla para documentar tu proyecto

Mapa de la unidad

Punto	Tema	Idea clave
01	¿Por qué documentar?	La documentación como parte del desarrollo
02	README	La puerta de entrada a tu proyecto
03	Doc técnica	Arquitectura, setup y API
04	Manual de usuario	Escribir para no técnicos
05	Markdown	Sintaxis, formato y consejos
06	Herramientas doc	GitBook, Notion, draw.io y más
07	Errores de documentación	“Lo documento luego”, demasiado técnico, demasiado vago
08	Plantilla de doc	Template listo para usar
09	Cierre	Mini-chequeo, vocabulario y resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 08 \(plantilla\)](#) y adapta
- **Si no tienes idea:** empieza por el [punto 01 \(¿Por qué documentar?\)](#)
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

UD 1.7: Propuesta de Proyecto

 *Antes de programar, hay que decidir qué hacer*

Tienes las habilidades de comunicación y documentación. Ahora toca **definir qué vas a construir**. La propuesta de proyecto es el documento que recoge tu idea, la justifica, la define y la plantea como un plan de acción realista. Es el contrato entre tu equipo y el tribunal.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Entender qué es una propuesta de proyecto y para qué sirve

-  Leer y analizar un briefing de proyecto
-  Generar ideas originales mediante técnicas de brainstorming
-  Seleccionar la mejor idea según criterios claros
-  Definir el alcance de tu proyecto (qué entra y qué no)
-  Crear un cronograma realista con hitos
-  Redactar una propuesta completa y profesional
-  Evitar los errores más comunes en propuestas

Mapa de la unidad

Punto	Tema	Idea clave
01	¿Qué es una propuesta?	Estructura, purpose y formato
02	Briefing	Cómo leer y entender el encargo
03	Ideas de proyecto	Técnicas de brainstorming
04	Selección de idea	Criterios y viabilidad
05	Definición de alcance	Qué entra y qué no
06	Cronograma	Hitos, deadlines y planificación
07	Ejemplo de propuesta	Propuesta completa de ejemplo
08	Errores de propuesta	Demasiado ambicioso, demasiado vago
09	Cierre	Mini-chequeo, vocabulario y resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 07 \(ejemplo\)](#) y úsalo como referencia
- **Si no tienes idea:** empieza por el [punto 02 \(briefing\)](#) para entender qué te piden
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

Guia Didactica Proyecto 2 Index



Ya tienes experiencia de Proyecto 1: sabes lo que es planificar mal, lo que es no llegar a tiempo, lo que es tener un código que da vergüenza enseñar. Ahora sabes mejor lo que NO hacer. Y eso te da ventaja.

¿Qué es Proyecto 2?

Proyecto 2 es la continuación del módulo. Partes de lo aprendido en Proyecto 1 y lo conviertes en un **producto completo y profesional**: aplicación web Full-Stack, desplegada, monitorizada y con procesos de calidad.

Las 10 unidades

UD	Qué harás	Enfoque
 UD 2.1: Requisitos Avanzado	Refinamiento y criterios de aceptación	Análisis
 UD 2.2: SCRUM Avanzado	Métricas, burndown, retrospectivas	Metodología
 UD 2.3: Git Avanzado	GitFlow, hooks, CI/CD básico	Control versiones
 UD 2.4: Testing	Tipos de test, herramientas, TDD	Calidad
 UD 2.5: Patrones de Diseño	Patrones creacionales, estructurales, comportamiento	Arquitectura
 UD 2.6: Diagramas	C4, UML esencial, secuencia, infraestructura	Modelado
 UD 2.7: IA como Copiloto	LLMs, prompt engineering, testing con IA	Herramientas
 UD 2.8: Despliegue	Docker, CI/CD, zero-downtime	DevOps
 UD 2.9: Monitorización	Prometheus, Grafana, logs estructurados	Operaciones
 UD 2.10: Propuesta Final	Ejemplos de proyectos Full-Stack	Práctica

Evaluación

Cada UD se evalúa de forma independiente. Consulta la guía de evaluación para conocer los criterios y pesos de cada unidad.

Competencias Proyecto 2

Competencias generales

- **CG1:** Capacidad para evolucionar y mantener software existente
- **CG2:** Integración de tecnologías y servicios externos
- **CG3:** Automatización de procesos (testing, despliegue)
- **CG4:** Trabajo en equipo avanzado (roles, metodologías ágiles)
- **CG5:** Compromiso con la calidad y la seguridad

Competencias específicas

- **CE1:** Refinar requisitos con criterios de aceptación e INVEST
- **CE2:** Aplicar SCRUM avanzado con métricas y retrospectivas
- **CE3:** Usar Git con estrategias de branching (GitFlow, trunk-based)
- **CE4:** Asegurar la calidad con tests unitarios, de integración y manuales
- **CE5:** Aplicar patrones de diseño en arquitectura web
- **CE6:** Modelar sistemas con diagramas C4 y UML
- **CE7:** Integrar IA como herramienta de desarrollo
- **CE8:** Desplegar aplicaciones con Docker y CI/CD
- **CE9:** Monitorizar aplicaciones con Prometheus y Grafana
- **CE10:** Desarrollar un proyecto Full-Stack completo

Relación con las asignaturas de 2º

Asignatura	Competencias que aporta
Desarrollo Web en Entorno Cliente	Frontend avanzado, frameworks
Desarrollo Web en Entorno Servidor	APIs, seguridad, despliegue
Despliegue de Aplicaciones Web	CI/CD, cloud, contenedores
Diseño de Interfaces	UX, usabilidad, prototipado
Empresa	Modelos de negocio, emprendimiento

Entregables Proyecto 2

Entregables por UD

UD 2.1: Requisitos Avanzado

- Backlog priorizado con historias INVEST
- Criterios de aceptación para cada historia
- Estimación en puntos de historia

UD 2.2: SCRUM Avanzado

- Velocity del equipo (o proyección)
- Burndown chart del sprint
- Retrospectiva documentada

UD 2.3: Git Avanzado

- Estrategia de branching definida
- Hooks de Git configurados
- Pipeline CI básico funcionando

UD 2.4: Testing

- Tests unitarios de las funciones core
- Un test de integración del flujo principal
- Protocolo de pruebas manuales documentado
- Checklist de calidad completado antes de cada entrega

UD 2.5: Patrones de Diseño

- Documento de arquitectura con patrones aplicados
- Justificación de cada patrón elegido
- Código refactorizado según patrones

UD 2.6: Diagramas

- Diagrama de contexto (C4 nivel 1)
- Diagrama de componentes o contenedores
- Diagrama ER de la base de datos
- Al menos 1 diagrama de secuencia

UD 2.7: IA como Copiloto

- Documento con prompts utilizados
- Ejemplos de código generado y adaptado
- Reflexión sobre uso de IA

UD 2.8: Despliegue

- Dockerfile funcional
- docker-compose.yml (si aplica)
- CI/CD configurado (GitHub Actions)
- App desplegada con HTTPS

UD 2.9: Monitorización

- Logs estructurados en la aplicación
- Endpoint /metrics funcionando
- Dashboard básico en Grafana

UD 2.10: Propuesta Final

- Código fuente completo y organizado
- Aplicación desplegada y accesible
- Documentación técnica completa
- Memoria del proyecto
- Presentación y demo

Todos los entregables se suben al repositorio de GitHub. La memoria y documentación pueden estar en Markdown dentro del repositorio o en PDF.

✔ Evaluación Proyecto 2

Criterios de evaluación

Criterio	Peso orientativo
Funcionalidad y requisitos	25%
Calidad técnica y arquitectura	20%
Testing y calidad	15%
Despliegue y DevOps	15%
Documentación y memoria	10%
Monitorización	5%
Presentación y defensa	10%

Sistema de calificación

Cada UD se califica de 0 a 10. La nota final es la media ponderada:

UD	Peso orientativo
UD 2.1: Requisitos Avanzado	10%
UD 2.2: SCRUM Avanzado	10%
UD 2.3: Git Avanzado	10%
UD 2.4: Testing	10%
UD 2.5: Patrones de Diseño	10%
UD 2.6: Diagramas	10%
UD 2.7: IA como Copiloto	5%
UD 2.8: Despliegue	15%
UD 2.9: Monitorización	10%
UD 2.10: Propuesta Final	20%

Nota: los pesos son orientativos; el profesor concretará los porcentajes al inicio del curso.

Criterios de calidad

Para obtener más de un 5 en cada UD:

- **UD 2.1:** Historias con INVEST, criterios de aceptación, backlog priorizado
- **UD 2.2:** Velocity calculada, burndown chart, retrospectiva efectiva
- **UD 2.3:** Estrategia de branching clara, hooks configurados, CI funcionando
- **UD 2.4:** Tests unitarios del core, integración del flujo principal, pruebas manuales antes de entregar
- **UD 2.5:** Patrones aplicados correctamente, justificación de decisiones
- **UD 2.6:** Diagrama C4 mínimo, ER completo, al menos 1 secuencia
- **UD 2.7:** Uso responsable de IA, prompts efectivos, código entendido
- **UD 2.8:** Docker funcional, CI/CD configurado, app desplegada con HTTPS
- **UD 2.9:** Logs estructurados, métricas básicas, dashboard funcional
- **UD 2.10:** Proyecto Full-Stack completo, documentado, desplegado

Observaciones

- La **evolución** respecto a Proyecto 1 se valora positivamente
- El despliegue en producción (aunque sea gratuito) suma puntos
- Los tests automatizados son obligatorios en Proyecto 2

- La memoria debe incluir la comparativa con Proyecto 1

UD 2.1: Requisitos Avanzado

 *Requisitos al nivel siguiente: de vagos a verificables*

En Proyecto 1 aprendiste a escribir historias de usuario y criterios de aceptación. Ahora toca **eleva el nivel**: estimar con story points, refinar el backlog como un equipo profesional y establecer trazabilidad entre requisitos y código.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Evaluar historias de usuario con los criterios **INVEST**
-  Priorizar con **MoSCoW avanzado** y el modelo **Kano**
-  Estimar con **story points** y **Planning Poker**
-  Realizar **sesiones de refinamiento** de backlog
-  Escribir criterios de aceptación con **Given/When/Then** y casos extremos
-  Construir una **matriz de trazabilidad** de requisitos
-  Identificar y evitar **errores avanzados** de requisitos

Mapa de la unidad

Punto	Tema	Idea clave
01	Criterios INVEST	Las 6 cualidades de una buena historia
02	MoSCoW avanzado y Kano	Priorización con criterio, no por capricho
03	Story points y Planning Poker	Estimar esfuerzo relativo sin morir en el intento
04	Refinamiento de backlog	La sesión que salva sprints
05	Criterios de aceptación avanzado	Given/When/Then y casos extremos
06	Trazabilidad	De requisito a código y viceversa
07	Errores avanzados	Los 7 pecados capitales de los requisitos
08	Template avanzado	Documento listo para Proyecto 2
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 08 \(template\)](#) y adapta el documento a tu proyecto
- **Si no tienes idea:** empieza por el [punto 01 \(INVEST\)](#) para entender qué hace buena a una historia
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

UD 2.2: SCRUM Avanzado

 *SCRUM no es solo tablero Kanban. Es un sistema de mejora continua*

En Proyecto 1 aprendiste lo básico de SCRUM: sprints, Kanban, user stories. Ahora toca **eleva el nivel**: medir la velocidad real del equipo, detectar bloqueos antes de que te destruyan, hacer retrospectivas que sirvan de verdad y entender el papel del SCRUM Master como facilitador, no como jefe.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

- 🎲 Usar **Planning Poker** como técnica formal de estimación
- 📊 Medir y usar la **velocidad del equipo** para planificar
- 📉 Interpretar gráficos de **burndown y burnup**
- 🚧 Detectar y resolver **bloqueos y dependencias** antes de que te destruyan
- 🗣️ Liderar **retrospectivas avanzadas** que generen mejoras reales
- 📈 Definir **métricas y KPIs** para medir la salud del proyecto
- 🧑 Ejercer el rol de **SCRUM Master** como estudiante

Mapa de la unidad

Punto	Tema	Idea clave
01	Planning Poker	Estimación colectiva con cartas
02	Velocidad del equipo	Points por sprint para predecir
03	Burndown y burnup	Gráficos que muestran el progreso real
04	Bloqueos y dependencias	Qué hacer cuando algo se atasca
05	Retrospectivas avanzadas	Más allá de “¿qué salió bien?”
06	Métricas y KPIs	Qué medir y cómo usar los datos
07	SCRUM Master estudiante	Tu rol como facilitador del equipo
08	Template SCRUM avanzado	Planning, retrospectiva y métricas
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 08 \(template\)](#) y adapta los formatos a tu proyecto
- **Si no tienes idea:** empieza por el [punto 01 \(Planning Poker\)](#) para entender la estimación formal
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

UD 2.3: Git Avanzado

 *Git no es solo hacer commit. Es una estrategia de trabajo*

En Proyecto 1 aprendiste lo básico: commit, push, pull, branch. Ahora toca **dominar Git**: estrategias de branching como GitFlow y Trunk-Based, hooks que automatizan calidad, CI/CD con GitHub Actions y convenciones de commit que hacen tu historial legible.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Aplicar la estrategia **GitFlow** para gestionar features y releases
-  Entender **Trunk-Based Development** y cuándo usarlo
-  Resolver **conflictos de merge** sin morir en el intento
-  Configurar **Git hooks** (pre-commit, commit-msg) con código
-  Configurar **CI/CD** con GitHub Actions con código real
-  Usar **Conventional Commits** para un historial limpio
-  Hacer **code review** efectiva

Mapa de la unidad

Punto	Tema	Idea clave
01	GitFlow	Branches para features, releases y hotfixes
02	Trunk-Based	Desarrollo directo en main con branches cortos
03	Conflictos	Cómo resolverlos sin perder trabajo
04	Git Hooks	Automatizar calidad antes del commit
05	CI/CD	GitHub Actions para build, test y deploy
06	Convenciones de commit	Conventional Commits para historial limpio
07	Code Review	Cómo revisar código sin destruir relaciones
08	Template Git	Workflow listo para tu proyecto
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 08 \(template\)](#) y adapta el workflow a tu proyecto
- **Si no tienes idea:** empieza por el [punto 01 \(GitFlow\)](#) para entender la estrategia de branches
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09



UD 2.4: Testing

 *El seguro de tu proyecto: que no falle cuando más importa*

En Proyecto 1 tu app funcionaba “en tu ordenador”. En Proyecto 2 toca demostrar que funciona **en cualquier ordenador, cualquier día, delante del tribunal**. El testing es la herramienta que te da esa certeza: automatizada, verificable y documentada.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Explicar **por qué los tests** son esenciales para tu proyecto

-  Distinguir **unitario, integración, E2E y manual** y cuándo usar cada uno
-  **Priorizar** qué probar con las 3 reglas (uso, fragilidad, cambio)
-  Integrar tests en tu **flujo de trabajo** (TDD, pre-commit, CI)
-  Aplicar un **protocolo de pruebas manuales** antes de cada entrega
-  Elegir las **herramientas** adecuadas a tu stack (Vitest, JUnit, Playwright)
-  Compartir la **responsabilidad del testing** con tu equipo
-  Usar un **checklist de calidad** completo antes del tribunal

Mapa de la unidad

Punto	Tema	Idea clave
01	¿Por qué hacer tests?	Los tests son un seguro, no un trámite
02	Tipos de test	Unitario, integración, E2E y manual
03	Qué probar en tu proyecto	Prioriza: lo que más usa, rompe y cambia
04	Tests en tu flujo	TDD, pre-commit y CI sin morir en el intento
05	Pruebas manuales	La red de seguridad antes de cada entrega
06	Herramientas de testing	Vitest, JUnit, Playwright y Testing Library
07	Testing en equipo	Code review, acuerdos y pair testing
08	Checklist de calidad	Todo lo que verificar antes de entregar
09	Cierre	Retos, laboratorio y preguntas para pensar

Competencias que desarrollarás

Competencia	Cómo se trabaja
Pensamiento crítico	Decidir qué probar y qué no, priorizar esfuerzo
Resolución de problemas	Detectar y diagnosticar fallos con tests
Trabajo en equipo	Compartir la responsabilidad de la calidad
Comunicación técnica	Documentar comportamiento con tests legibles
Autonomía	Saber que tu código funciona antes de entregarlo

¿Por dónde empiezo?

- **Si tienes prisa:** ve al [punto 08 \(checklist\)](#) y aplica lo mínimo antes de la próxima entrega
- **Si no has escrito un test nunca:** empieza por el [punto 01](#) y el [punto 02](#)
- **Si quieres entender el testing de verdad:** sigue el orden del 01 al 09

UD 2.5: Patrones de Diseño

 *No reinventes la rueda: usa patrones que ya funcionan*

Los **patrones de diseño** son soluciones probadas a problemas que aparecen una y otra vez en el desarrollo de software. No son código copiado: son **plantillas de diseño** que te ayudan a crear código más limpio, flexible y mantenible. Conocerlos te hace programador mejor.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Entender qué son los patrones de diseño y por qué existen
-  Usar patrones **creacionales**: Singleton, Factory, Builder
-  Usar patrones **estructurales**: Adapter, Facade, Composite
-  Usar patrones **comportamiento**: Observer, Strategy, Command
-  Entender **MVC** y **Repository** como arquitecturas completas
-  Saber **cuándo usar** cada patrón (y cuándo no)

Mapa de la unidad

Punto	Tema	Idea clave
01	¿Qué son los patrones?	Soluciones reutilizables a problemas comunes
02	Patrones creacionales	Singleton, Factory, Builder
03	Patrones estructurales	Adapter, Facade, Composite
04	Patrones de comportamiento	Observer, Strategy, Command
05	MVC	Model-View-Controller explicado
06	Repository	Separar acceso a datos de la lógica
07	Cuándo usar cada patrón	Guía práctica de selección
08	Template patrones	Selección de patrón para tu proyecto
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 07 \(cuándo usar\)](#) para ver qué patrón necesitas
- **Si no tienes idea:** empieza por el [punto 01 \(¿Qué son?\)](#) para entender el concepto
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

UD 2.6: Diagramas

 *Un diagrama dice más que mil líneas de código*

Los diagramas no son burocracia: son la **forma más rápida de comunicar** cómo funciona tu sistema. Un diagrama de secuencia explica un flujo en 30 segundos. Un diagrama de componentes muestra la arquitectura en un vistazo. Son la documentación visual que tu proyecto necesita.

Objetivos de la unidad

-  Entender qué son los diagramas y por qué son importantes
-  Usar el **modelo C4** para documentar arquitectura en 4 niveles
-  Crear **diagramas de entidad-relación** para modelar datos
-  Crear **diagramas de secuencia** para flujos de interacción
-  Conocer los **símbolos UML** esenciales
-  Escribir diagramas con **Mermaid** (sintaxis en texto)
-  Usar **herramientas** como draw.io, Excalidraw y Lucidchart

Mapa de la unidad

Punto	Tema	Idea clave
01	¿Qué son los diagramas?	Comunicación visual del sistema
02	Modelo C4	4 niveles de arquitectura
03	Diagrama ER	Modelar base de datos
04	Diagrama de secuencia	Flujos de interacción
05	Símbolos UML	Referencia de notación
06	Mermaid	Diagramas en código de texto
07	Herramientas	draw.io, Excalidraw, Lucidchart
08	Template diagrams	Plantillas para tu proyecto
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 06 \(Mermaid\)](#) para empezar a crear diagramas en código
- **Si no tienes idea:** empieza por el [punto 01 \(¿Qué son?\)](#) para entender por qué importan
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

La IA no programa por ti, pero programa mejor contigo

La inteligencia artificial ha llegado al desarrollo de software para quedarse. Pero no es un atajo mágico: es una herramienta que, usada bien, te hace más rápido y mejor. Usada mal, te genera código que no entiendes, bugs escondidos y problemas de seguridad. Esta unidad te enseña a usar la IA como un buen copiloto: que ayude, no que tome el volante.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Entender qué hace bien y qué hace mal la IA en desarrollo
-  Escribir prompts efectivos que obtienen código útil
-  Revisar y testear código generado por IA
-  Evitar problemas de seguridad al usar IA
-  Usar la IA de forma ética y responsable

Mapa de la unidad

Punto	Tema	Idea clave
01	Qué hace bien y mal la IA	Fortalezas y limitaciones reales
02	Prompts efectivos	Cómo escribir para obtener resultados
03	GitHub Copilot	Uso práctico del copiloto
04	Testing con IA	Cómo testear código generado
05	Seguridad con IA	Riesgos y cómo mitigarlos
06	Ética de la IA	Uso responsable
07	Errores comunes	Los fallos más frecuentes
08	Template de uso	Plantilla para documentar tu uso de IA
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- Si tienes prisa: ve directamente al [punto 08 \(template\)](#) y documenta tu uso de IA

- **Si eres novato con IA:** empieza por el [punto 01 \(qué hace bien y mal\)](#)
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

UD 2.8: Despliegue

 *Tu proyecto no existe hasta que alguien puede usarlo en producción*

Un proyecto que solo funciona en localhost es un borrador. El despliegue es el paso que convierte tu código en un **producto real**: accesible desde cualquier navegador, con HTTPS, actualizaciones automáticas y sin downtime. Es lo que separa un proyecto de clase de algo profesional.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Entender qué es Docker y cómo empaquetar aplicaciones
-  Configurar pipelines CI/CD con GitHub Actions
-  Desplegar en plataformas reales (GitHub Pages, Netlify, Vercel, Railway)
-  Aplicar estrategias de zero-downtime
-  Seguir un checklist de despliegue completo

Mapa de la unidad

Punto	Tema	Idea clave
01	Qué es el despliegue	De localhost a producción
02	Docker	Empaquetar aplicaciones
03	CI/CD	Automatizar build y deploy
04	Plataformas	Dónde desplegar
05	Zero-downtime	Sin interrupciones
06	Checklist	Lista completa de pasos
07	Errores comunes	Qué falla y por qué
08	Template	Plantilla de despliegue
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 06 \(checklist\)](#) y sigue los pasos
- **Si no sabes qué es Docker:** empieza por el [punto 02 \(Docker\)](#)
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

UD 2.9: Monitorización

 *Lo que no mides, no puedes mejorar*

Desplegar tu app es solo el paso intermedio. Una vez en producción, necesitas **saber qué está pasando**: ¿cuántos usuarios tiene?, ¿hay errores?, ¿es rápida o lenta?, ¿se cae a veces?. La monitorización te da esas respuestas con datos, no con suposiciones.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Entender qué son los niveles de log y cómo usarlos
-  Definir las métricas clave de tu aplicación
-  Configurar Prometheus y Grafana como stack de monitorización

- 🚨 Crear alertas para detectar problemas antes de que los usuarios
- 📊 Construir dashboards que muestren el estado de tu app

Mapa de la unidad

Punto	Tema	Idea clave
01	Qué son los logs	Niveles de log y estructura
02	Métricas clave	Latencia, errores, throughput
03	Prometheus + Grafana	El stack estándar
04	Alertas	Detectar problemas en tiempo real
05	Stack mínimo	Qué necesitas para empezar
06	Configuración	Winston, prom-client y más
07	Dashboard	Construir dashboards útiles
08	Template	Plantilla de configuración
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 05 \(stack mínimo\)](#) y configura lo básico
- **Si no sabes qué son los logs:** empieza por el [punto 01 \(logs\)](#)
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

🚀 UD 2.10: Propuesta Final

📖 *Todo se une: el proyecto que demuestra lo que sabes hacer*

Esta es la unidad final. Aquí convergen todo lo aprendido: requisitos, SCRUM, Git, patrones, diagramas, IA, despliegue y monitorización. No es solo “hacer un proyecto”: es demostrar que puedes llevar una idea de la conceptualización a producción con metodología profesional.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Definir un proyecto Full-Stack completo con requisitos claros
-  Generar y seleccionar la mejor idea de proyecto
-  Elegir el stack tecnológico adecuado
-  Planificar el cronograma con hitos realistas
-  Preparar los entregables para la evaluación

Mapa de la unidad

Punto	Tema	Idea clave
01	Briefing final	Qué se espera del proyecto
02	Ideas finales	Generar opciones de proyecto
03	Selección	Elegir la mejor idea
04	Stack final	Elegir tecnologías
05	Features	Definir funcionalidades
06	Cronograma	Planificar el tiempo
07	Entregables	Qué entregar
08	Evaluación	Cómo se evalúa
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 07 \(entregables\)](#) y prepara lo que hay que entregar
- **Si no tienes idea de proyecto:** empieza por el [punto 02 \(ideas\)](#)
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

 *El día que entendieron que la metodología no era burocracia*

“¿Otra reunión?”, “¿Otra columna en el tablero?”, “¿Otra estimación?”. Al principio, todo lo que sonaba a “metodología” les parecía pérdida de tiempo. Querían programar, no hacer reuniones.

Hasta que llegó la semana antes de la entrega y descubrieron: - Una funcionalidad que daban por hecha no estaba empezada - Dos personas habían trabajado en lo mismo sin saberlo - Nadie sabía qué quedaba por hacer exactamente

La metodología no es burocracia. Es **no llegar en pánico al final**.

¿Qué metodología usar?

No hay una única respuesta. Depende de tu equipo y tu proyecto.

✅ Scrum (SCRUM Lite)

Sprints de 1-2 semanas. Reuniones cortas con propósito. Roles rotativos. Ideal para fechas de entrega fijas. Se trabaja a fondo en la UD 1.2 (SCRUM Lite) y la UD 2.2 (SCRUM Avanzado).

✅ Kanban

Flujo continuo sin sprints. Límites de trabajo en curso (WIP). Menos reuniones. Ideal para mantenimiento y mejoras. También se cubre en la UD 1.2.

SCRUM (adaptado)

Para equipos de 2-4 personas, con sprints de 1-2 semanas.

 Esta sección resume lo esencial. En profundidad, verás **SCRUM Lite** en la [UD 1.2](#) y **SCRUM Avanzado** (métricas, burndown, retrospectivas) en la [UD 2.2](#).

Qué hacer: - Sprint planning al empezar cada sprint (qué haremos, quién lo hará) - Daily (15 min, de pie o sentados, pero rápidos) - Sprint review al terminar (qué se ha conseguido) - Retrospectiva (qué mejorar para el próximo sprint)

Qué NO hacer: - Scrum Master oficial (rotad el rol) - Ceremonias de 2 horas - Estimar en puntos de historia (usad horas o días) - Sprint backlog inmutable (se puede renegociar)

Kanban

Para equipos que prefieren fluidez continua sin sprints fijos.

Qué hacer: - Tablero con columnas: Pendiente → En curso → Revisión → Hecho - Límite de tareas en “En curso” (máximo 2 por persona) - Flujo continuo sin fechas fijas

Ventaja: más flexible, menos reuniones. **Desventaja:** menos estructura, fácil procrastinar.

Git Flow

No es solo para el código. También organiza el trabajo:

Rama	Propósito
main	Código estable y desplegado
develop	Integración de funcionalidades
feature/*	Una funcionalidad nueva (una por persona)
hotfix/*	Corrección urgente

 En profundidad: [UD 2.3 — Git Avanzado](#).

El ciclo del proyecto

Sea cual sea la metodología, el proyecto sigue este ciclo:

**  Planificar →

→ → **  Ejecutar →

→ → **  Revisar →

→ → **  Aprender → →

Cada iteración (semana, sprint) deberías: 1. Elegir qué hacer 2. Hacerlo 3. Comprobar que funciona 4. Reflexionar sobre lo aprendido

Sin el paso 4, repites los mismos errores. Sin el paso 1, trabajas sin dirección.

Caso 1: El equipo que no planificaba

Un equipo de 3 personas decidió que “la metodología era pérdida de tiempo”. Se pusieron a programar directamente desde el día 1.

Resultado: - A 3 días de la entrega, descubrieron que dos personas habían implementado la misma funcionalidad - Una feature crítica no se había empezado - Tuvieron que dormir 4 horas al día para

llegar

Lección: 30 minutos de planificación al inicio de la semana habrían ahorrado 3 días de caos.

Ejemplo de sprint de 2 semanas

Semana 1

Día	Mañana	Tarde
Lunes	Sprint planning (30 min)	Desarrollo
Martes	Daily (15 min)	Desarrollo
Miércoles	Daily (15 min)	Desarrollo
Jueves	Daily (15 min)	Desarrollo
Viernes	Daily (15 min)	Revisión parcial

Semana 2

Día	Mañana	Tarde
Lunes	Daily (15 min)	Desarrollo
Martes	Daily (15 min)	Desarrollo
Miércoles	Daily (15 min)	Desarrollo
Jueves	Daily (15 min)	Últimas pruebas
Viernes	Sprint review + Retro (45 min)	🎉 Descanso

Total de reuniones: **~3 horas en 2 semanas** (menos del 5% del tiempo).

Errores comunes

- **Sobreplanificar:** perder 2 días planificando un sprint de 1 semana
- **Infraplanificar:** empezar a programar sin saber qué hace cada uno
- **No hacer retrospectiva:** repetir los mismos errores sprint tras sprint
- **Daily que se alarga:** las daily son de 15 min, no de 45

Herramientas recomendadas

Herramienta	Para qué
Trello / GitHub Projects	Tablero Kanban
Notion / Confluence	Documentación compartida
Discord / Slack	Comunicación diaria
Google Calendar	Reuniones y fechas límite
Git + GitHub / GitLab	Control de versiones

Herramientas

 *El día que se dieron cuenta de que no necesitaban 20 herramientas*

El equipo empezó con entusiasmo: un chat para hablar, otro para compartir archivos, un tablero para tareas, un documento para ideas, otro documento para la memoria, un repositorio, un gestor de contraseñas...

A las dos semanas, la información estaba dispersa en 8 sitios distintos. Nadie recordaba dónde estaba cada cosa. Perdían más tiempo buscando información que creándola.

Menos herramientas. Mejor organizadas.

Herramientas esenciales

No necesitas más de 4-5 herramientas bien usadas.

1. Repositorio de código (Git + GitHub/GitLab)

Tu repositorio no es solo para el código. También es para: - Documentación técnica (README, Wiki) - Issues (tareas, bugs, ideas) - Pull requests (revisión de código) - Projects (tablero Kanban integrado)

Tip: usa GitHub Projects como tablero. Así tienes código y tareas en el mismo sitio. Una herramienta menos que gestionar.

2. Comunicación (Discord, WhatsApp, Slack)

Elegid una. Solo una. No combinéis WhatsApp para “urgente” y Discord para “lo demás”. Todo en el mismo sitio.

Consejo: cread canales o grupos separados para: - General (anuncios, dudas generales) - Técnico (problemas de código) - Off-topic (memes, no interrumpir a los demás)

3. Documentación (Markdown en el repo)

La memoria del proyecto, las decisiones técnicas, los diagramas. Todo en un sitio accesible para todos.

Recomendación: usad Markdown en el propio repositorio. Así la documentación viaja con el código y tenéis control de versiones.

4. Gestión de tareas (GitHub Projects, Trello, Notion)

Un tablero con tres columnas basta:

** 📄 Por hacer →

→ → ** ⚙️ En proceso →

→ → ** ✅ Hecho → →

Si necesitas más, añade “Revisión” o “Bloqueado”. Pero no te pases. Cuantas más columnas, más tiempo pierdes moviendo tarjetas.

📄 Caso 2: El día que consolidaron sus herramientas

Un equipo usaba: - WhatsApp para hablar - Trello para tareas - Google Docs para documentación - GitHub para código - Figma para diseños - Drive para assets

El problema: cada vez que necesitaban algo, no sabían dónde buscar. La información estaba fragmentada.

La solución: - Movieron todo a GitHub (Issues + Projects + Wiki) - Markdown para documentación (dentro del repo) - Discord para comunicación (integrado con GitHub) - Figma solo para prototipos iniciales

Resultado: 4 herramientas → 3. La información, centralizada.

5. IDE (VS Code recomendado)

VS Code es el estándar por su ecosistema de extensiones:

Extensión	Para qué
ESLint / Prettier	Código limpio y formateado
GitLens	Historial y blame en línea
GitHub Pull Requests	Revisar PRs sin salir del IDE
Live Share	Programar en pareja en remoto
Docker	Gestionar contenedores
Thunder Client	Probar APIs (alternativa a Postman)

Herramientas complementarias

Herramienta	Para qué	Alternativas
Postman / Insomnia	Probar APIs	Hoppscotch
Draw.io / Figma	Diagramas y prototipos	Excalidraw, Mermaid
Docker	Contenedores	Podman
Vercel / Netlify	Despliegue frontend	Render, GitHub Pages
UptimeRobot	Monitorizar que la app responde	—

 Estas herramientas se trabajan en las UD's: - [UD 1.4 — Elección de Tecnologías](#) - [UD 2.6 — Diagramas](#) - [UD 2.8 — Despliegue](#) - [UD 2.9 — Monitorización](#)

Flujo de trabajo típico

1. Abres una Issue en GitHub → tarea pendiente
2. Mueves la Issue a "En proceso" → estás trabajando
3. Creas una rama feature/ desde develop → código
4. Abres un Pull Request → revisión del compañero
5. Mergeas a develop → integración
6. Cierras la Issue → 

Lo que NO necesitas

- Un IDE distinto cada uno (cada uno que use el que quiera)

- Una herramienta de gestión de proyectos solo porque está de moda
- Un servicio cloud que nadie sabe configurar
- Una base de datos externa cuando SQLite local es suficiente
- Un canal de comunicación distinto para cada tema

Recursos

 *El día que Stack Overflow era su mejor amigo sin Internet*

“No me funciona este código”, dijo una voz en el grupo. “Búscalo en Google”, respondió otra.

Buscaron. Encontraron una respuesta en Stack Overflow de 2015. La probaron. No funcionaba. Buscaron otra. Esa sí.

Internet es tu mejor recurso. Pero hay que saber buscar.

Cómo buscar ayuda técnica

Busca en el orden correcto

1. **Documentación oficial** (la fuente más fiable)
2. **Stack Overflow** (problemas comunes ya resueltos)
3. **GitHub Issues** (bugs conocidos, soluciones de la comunidad)
4. **Tutoriales / blogs** (guías paso a paso)
5. **Pregunta a un compañero o al profesor** (cuando lleves 30 min atascado)

Cómo preguntar bien

Cuando preguntes (en Stack Overflow, al profesor, a un compañero), sé específico:

 “No me funciona el login”  “Al enviar el formulario de login con email y contraseña válidos, recibo un error 500 en la consola del servidor. El código relevante es...”

Incluye siempre: - Qué esperabas que pasara - Qué pasa realmente - El código relevante (no todo el archivo) - Los mensajes de error completos

Un alumno llevaba 3 horas atascado con un error de conexión a base de datos. Había probado “cosas” sin saber bien qué hacía.

Su proceso: - Pegó el error en Google - Probó la primera respuesta (no funcionó) - Probó la segunda (tampoco) - Se rindió y preguntó al profesor

El profesor hizo: - Copió el error exacto en Google - Encontró un GitHub Issue con el mismo error - La solución era cambiar una línea de configuración

Lección: el error estaba documentado en GitHub Issues, pero él no sabía buscar allí. Aprender a buscar es tan importante como aprender a programar.

Referencias por tema

Frontend

- [MDN Web Docs](#) — la biblia del desarrollo web
- [CSS-Tricks](#) — guías visuales de CSS
- Documentación oficial de React / Vue / Svelte

Backend

- [Express.js](#) guía oficial
- [Node.js](#) documentación de la API
- [Spring Boot](#) guía oficial

Bases de datos

- Documentación oficial de PostgreSQL / MySQL
- [SQL Tutorial](#) (w3schools, para empezar)

Git

- [Pro Git](#) — libro gratuito y completo
- [Oh Shit, Git!?!](#) — cuando algo sale mal

Docker

- [Docker Curriculum](#) — tutorial interactivo
- Documentación oficial de Docker

IA como recurso de aprendizaje

El recurso más infravalorado: preguntar

No estás solo en el proyecto. Tu profesor, tus compañeros y la comunidad son recursos que muchos no usan por miedo o pereza. Pregunta:

- ¿Voy por buen camino?
- ¿Esta decisión técnica tiene sentido?
- ¿Cómo harías tú esto?
- ¿Qué priorizarías ahora?

Una consulta de 5 minutos a una persona con experiencia puede ahorrarte 2 días de trabajo. Preguntar bien (con el error concreto y el contexto) es una habilidad profesional en sí misma.

 Sobre cómo usar la IA como recurso de aprendizaje, verás más en la [UD 2.7 — IA como Copiloto](#).

Licencia

 *Compartir es aprender*

Todo el contenido de esta guía se publica con licencia abierta para que puedas usarlo, adaptarlo y compartirlo. El conocimiento no se guarda, se distribuye.

Esta guía didáctica y todos los materiales asociados (apuntes, plantillas, ejemplos) se publican bajo la licencia **Creative Commons Atribución-CompartirIgual 4.0 Internacional (CC BY-SA 4.0)**.

Eres libre de

- **Compartir** — copiar y redistribuir el material en cualquier medio o formato
- **Adaptar** — remezclar, transformar y crear a partir del material

Bajo las siguientes condiciones

- **Atribución** — debes reconocer la autoría de forma adecuada, proporcionar un enlace a la licencia e indicar si se han realizado cambios
- **CompartirIgual** — si remezclas, transformas o creas a partir del material, deberás difundir tus contribuciones bajo la **misma licencia** que el original

Autoría

Sergi Garcia Barea — CC BY-SA 4.0



Más información en: <https://creativecommons.org/licenses/by-sa/4.0/>

? Preguntas Frecuentes

 *Las dudas que todo el mundo tiene*

Da igual el curso: siempre surgen las mismas preguntas. Aquí tienes las respuestas para que no pierdas tiempo ni te lleves sorpresas.

Sobre el proyecto

► ? ¿Puedo cambiar de idea a mitad de proyecto?

► ? ¿Qué pasa si mi equipo se rompe?

► ? ¿Puedo hacer el proyecto individual?

► ? ¿Qué pasa si el profesor rechaza mi idea?

► ? ¿Puedo usar inteligencia artificial?

► ? ¿Qué peso tiene cada unidad didáctica?

► ? ¿Qué pasa si no entrego una UD a tiempo?

► ? ¿La presentación oral cuenta?

► ? ¿Se evalúa el trabajo en equipo?

Sobre el código

► ? ¿Puedo usar código de internet?

► ? ¿Cuánto código tengo que escribir?

► ? ¿Tengo que hacer tests?

► ? ¿Qué pasa si mi código no compila?

Sobre la documentación

► ? ¿La memoria se entrega al final?

► ? ¿Qué extensión debe tener?

► ? ¿Tengo que hacer una memoria en PDF?

► ? ¿Tienes otra duda?

 *El día que no se les ocurría nada*

“Vale, tenéis que hacer un proyecto. Ideas, ideas...” Silencio en la clase. Nadie sabía qué hacer.

No hace falta una idea revolucionaria. Hace falta una idea que **resuelva un problema real**, aunque sea pequeño.

Cómo elegir una buena idea

	Buena idea	Mala idea
Alcance	Resuelve un problema concreto	“Una red social para todo”
Usuario	Tiene un usuario definido	“Es para todo el mundo”
Tecnología	Usa lo que sabes	“Necesito aprender IA en una semana”
Motivación	Te interesa el tema	“Es lo único que se me ocurre”

Ideas para PI1

Gestión y organización

- **Gestor de tareas** para un equipo pequeño
- **Control de inventario** para un pequeño negocio
- **Sistema de reservas** para una peluquería, taller o pista deportiva
- **Gestor de gastos** personales o de un viaje
- **Organizador de eventos** (bodas, cumpleaños, conferencias)

Educación

- **Plataforma de apuntes** compartidos entre estudiantes
- **Gestor de trabajos** para un instituto (entrega, corrección, notas)
- **Sistema de tutorías** (reservar y gestionar sesiones)
- **Generador de horarios** de clase
- **App de Flashcards** para estudiar

Comunidad

- **Directorio de recursos** de un barrio (talleres, asociaciones)
- **Red de trueque** de objetos entre vecinos
- **Gestor de incidencias** para una comunidad de vecinos
- **App para compartir coche** entre compañeros de clase

Tiempo real / Datos

- **Tablero Kanban** para equipos pequeños
- **Gestor de hábitos** y seguimiento personal
- **Sistema de encuestas** con resultados en tiempo real
- **Gestor de recetas** con filtros por ingredientes

Cómo validar tu idea

Antes de empezar a programar, valida que tu idea tiene sentido:

1. **Háblala con 3 personas:** ¿les parece útil? ¿la usarían?
2. **Búscala en Google:** ¿ya existe algo así? ¿qué harías diferente?
3. **Pregunta a tu profesor o a la comunidad:** ¿es viable para el tiempo que tenemos?
4. **Recorta alcance:** ¿cuál es la versión más pequeña que sigue siendo útil?

 En la [UD 1.7 — Propuesta de Proyecto](#) aprenderás a transformar estas ideas en una propuesta formal (briefing, alcance y cronograma).

Ideas para PI2

En PI2 partís del proyecto de PI1. Las ideas se centran en **evolucionar**:

- Añadir autenticación con Google/GitHub
- Integrar pagos simulados (Stripe test)
- Añadir mapas (tiendas cercanas, rutas)
- Implementar chat en tiempo real (WebSockets)
- Dashboard con gráficos y estadísticas
- Exportar datos a PDF/CSV
- Notificaciones por email
- Versión móvil responsive avanzada

- Panel de administración
- Tests automatizados y CI/CD

Y si no tienes ni idea

Empieza por un problema que **tú mismo tengas**:

- “Necesito organizar mis apuntes”
- “Mi madre necesita gestionar las citas de su negocio”
- “En mi club deportivo no tienen web”
- “Odio hacer la lista de la compra y siempre se me olvida algo”

El mejor proyecto es el que te sirve a ti. Porque si a ti te sirve, lo harás con ganas. Y si lo haces con ganas, sale mejor.

Guia Didactica Proyecto 1 Uds U1 1 Mvp 01 Que Es Un Mvp

 **Estás en:**  **UD 1.1 • MVP** → 01 • ¿Qué es un MVP?

 *Antes de construir, hay que entender qué estás construyendo*

Antes de abrir un editor de código o diseñar una base de datos, necesitas entender **qué es exactamente lo que vas a hacer**. Y eso empieza por una palabra que suena complicada pero que es muy sencilla: MVP.

La idea en una frase

Un MVP es la versión más simple de tu producto que funciona, enseña algo y está terminado.

No es “hacer menos”. Es hacer **solo lo que importa** y dejarlo terminado.

El origen: por qué existe el MVP

El término **MVP** (Minimum Viable Product) viene del mundo de las *startups* y fue popularizado por **Eric Ries** en su libro *The Lean Startup* (2011).

La idea central es esta:

En vez de pasar 6 meses construyendo algo perfecto que nadie puede que quiere, construye en 2 semanas algo sencillo que puedas **probar con personas reales** y aprende de lo que te digan.

Ejemplo real: Airbnb

Airbnb no empezó como la plataforma que conoces hoy. Empezó como **una página web simple** donde dos fundadores alquilaban un colchón en su salón durante una conferencia. No había app, no había pagos online, no había sistema de reseñas.

¿Qué hicieron? - Pusieron fotos del colchón en una web - Cobraron por transferencia bancaria - Hablaron con los inquilinos para saber qué les parecía

¿Qué aprendieron? - La gente estaba dispuesta a pagar por alojamientos únicos - Las fotos eran importantes (mejoraron las fotos y las reservas subieron) - Necesitaban un sistema de pagos seguro

¿Eso es un MVP? Sí. Era lo mínimo que necesitaban para **validar su idea** con dinero real.

MVP vs Producto Final

Aspecto	MVP	Producto Final
Funcionalidades	Pocas, esenciales	Muchas, completas
Calidad	Funciona, pero puede ser “feo”	Pulido, profesional
Alcance	Resuelve 1 problema bien	Resuelve varios problemas
Tiempo	Semanas	Meses
Objetivo	Aprender si la idea funciona	Generar beneficios
Usuarios	Pocos, de prueba	Miles, reales

La confusión más común

Mucha gente piensa que un MVP es “un proyecto que falta por terminar”. **No es así.**

Un MVP **está terminado**. Simplemente tiene menos features de las que querrías en el futuro. Es como una casa con un dormitorio: está terminada, se puede vivir en ella, pero puedes añadir más habitaciones después.

Los 3 mitos del MVP

Mito 1: “Un MVP es algo chapucero”

Realidad: Un MVP debe **funcionar bien** en lo que hace. Puede tener pocas features, pero esas features deben funcionar correctamente. Si tu CRUD no guarda bien los datos, eso no es un MVP, es un bug.

Mito 2: “Un MVP no necesita documentación”

Realidad: Un MVP necesita **lo mínimo de documentación** para que alguien lo entienda. Un README con instrucciones de ejecución es suficiente, pero no puedes entregar código sin explicar qué es.

Mito 3: “Un MVP es solo para emprendedores”

Realidad: El concepto de MVP sirve en **cualquier proyecto**: una asignatura, un proyecto personal, una herramienta para tu club. No necesitas ser empresario para pensar en términos de MVP.

¿Por qué importa el MVP en Proyecto 1?

En Proyecto 1 tienes un **tiempo limitado** (unas 4-6 semanas). Si intentas hacer todo lo que se te ocurre, no terminarás nada.

El MVP te obliga a:

1. **Pensar antes de construir** — ¿Qué es lo imprescindible?
2. **Enfocarte en lo que importa** — ¿Qué resuelve el problema?
3. **Terminar** — Tener algo funcional que entregar
4. **Aprender** — Descubrir si tu idea funciona

Dato: El 70% de los proyectos que fracasan en FP lo hacen porque el alumno **no terminó**, no porque la idea fuera mala. El MVP evita eso.

El test del “¿y además...?”

Imagina que estás explicando tu proyecto a alguien y te dice:

- “¿Y además tiene login?”
 - Si tu respuesta es “no, pero tiene CRUD completo” → **bien**
 - Si tu respuesta es “eh... no sé” → **mal**
- “¿Y también tiene chat?”
 - Si tu respuesta es “no, eso es para después” → **bien**
 - Si tu respuesta es “pues lo intentaré añadir” → **mal**

El “y además” es el enemigo del MVP. Cada vez que alguien dice “y además...”, estás añadiendo complejidad que no necesitas.

Mini-chequeo

► ¿Puedes explicar qué es un MVP en una frase?

► ¿Un MVP está “a medias” o “terminado”?

► ¿Cuál es el objetivo principal de un MVP?

✓ Resumen en 3 frases

1. Un MVP es la versión más pequeña de tu producto que **funciona, enseña algo y está terminado**.

2. No es “un proyecto a medias”, es un proyecto con **alcance reducido intencionadamente**.

3. El MVP te obliga a **pensar antes de construir** y a terminar en vez de empezar 10 cosas.

 Volver al [índice de la unidad](#) · Siguiendo: [02 — Identificar el problema](#)

Guia Didactica Proyecto 1 Uds U1 1 Mvp 02 Identificar Problema

 **Estás en:**  **UD 1.1 · MVP** → [02 · Identificar el problema](#)

 *El problema correcto es la mitad de la solución*

No es lo mismo “quiero hacer una app” que “quiero resolver **este problema concreto** para **estas personas**”. El primer paso de tu proyecto no es código: es encontrar un problema que merezca la pena resolver.

La idea en una frase

Un buen proyecto empieza por un problema real que alguien tiene, no por una función que quieres programar.

Si no identificas bien el problema, construirás algo que nadie necesita.

Por qué no puedes empezar por “quiero hacer una app de...”

El error más común en Proyecto 1 es empezar por la solución en vez del problema:

Enfoque incorrecto	Enfoque correcto
“Quiero hacer una app de recetas”	“No sé qué cocinar con lo que tengo en la nevera”
“Quiero hacer un gestor de tareas”	“Siempre olvido qué tengo que hacer para el institute”
“Quiero hacer una red social”	“Mis compañeros pierden apuntes por WhatsApp”

¿Cuál es la diferencia? El enfoque incorrecto empieza por la **solución**. El correcto empieza por el **problema**. Si empiezas por la solución, puedes construir algo que no necesita nadie.

3 fuentes de problemas reales

Fuente 1: Tu día a día

Los mejores problemas son los que **tú mismo tienes**:

- ¿Qué gestionas manualmente que podría automatizarse?
- ¿Qué datos siempre pierdes u olvidas?
- ¿Qué haría tu familia o amigos más fácil?
- ¿Qué apps existentes hacen mal?

Ejemplo: “Odio hacer la lista de la compra y siempre se me olvida algo” → App de listas con recordatorios.

Fuente 2: Tu entorno

Pregúnta a personas de tu alrededor:

- **Familia:** “¿Qué te gustaría que una app hiciera por ti?”
- **Compañeros:** “¿Qué os gustaría tener para el institute?”
- **Amigos:** “¿Qué problemas tienes con las apps que usas?”

Ejemplo: “Mi madre tiene una peluquería y gestiona las citas en un papel” → App de reservas.

Fuente 3: Apps existentes

Busca apps que ya existen y pregúntate:

- ¿Qué hacen bien?
- ¿Qué hacen mal?
- ¿Qué les falta?
- ¿Para quién son demasiado complicadas?

Ejemplo: “Existen gestores de tareas, pero todos son demasiado complicados para algo simple” → Gestor de tareas minimalista.

El ejercicio de los 10 problemas

Objetivo: En 10 minutos, escribe 10 problemas reales de tu entorno.

Reglas: 1. **No filtrar** — escribe todo lo que se te ocurra, sin juzgar 2. **Problemas, no soluciones** — “no sé qué cocinar” no “app de recetas” 3. **Concretos** — “pierdo apuntes de DAW” no “pierdo cosas” 4. **De otros** — también vale problemas de familiares o amigos

1. No sé qué cocinar con lo que tengo en la nevera
2. Siempre olvido la fecha de exámenes
3. Mi madre pierde citas de la peluquería
4. Mis compañeros pierden apuntes por WhatsApp
5. No encuentro dónde guardar fotos organizadas
6. Mi club deportivo no tiene sistema de reservas
7. Odio hacer la lista de la compra
8. No sé cuánto dinero gasto al mes
9. Mi padre no encuentra herramientas en el garaje
10. No recuerdo qué libros he leído

Después de escribir 10, elige el que más te motive. Si no te motiva ninguno, vuelve a escribir 10 más.

El “test de la ducha”

Una buena forma de saber si un problema es real es el **test de la ducha**:

Si estás en la ducha y piensas en ese problema, ¿te sigue pareciendo importante?

- Si piensas “sí, esto es un rollo y debería tener solución” → **problema real**
- Si piensas “bueno, no es para tanto” → **problema inventado**

Validar el problema

Una vez tengas un problema, **valida que es real** antes de construir nada:

Método 1: Habla con 3 personas

Pregunta a 3 personas que tengan ese problema: - “¿Tienes este problema?” - “¿Cómo lo resuelves ahora?” - “¿Pagarías por una solución?” - “¿Qué tendría que tener una solución para que la usaras?”

Si las 3 personas te dicen “sí, tengo ese problema” → **problema validado**.

Método 2: Busca soluciones existentes

Si ya existen soluciones, pregunta: - ¿Por qué no las usan? - ¿Qué les falta? - ¿Son demasiado caras, complicadas o lentas?

Si existen soluciones pero tienen problemas → **oportunidad**.

Método 3: El “test del Mínimo Viable”

Pregunta: “¿Cuál es la versión más simple que usarías?”

Si la respuesta es algo que puedes construir en 2-4 semanas → **buen problema para MVP**.

Problema real

Mis compañeros pierden apuntes por WhatsApp y no los encuentran cuando los necesitan

Problema inventado

Estaría bien tener una app para organizar apuntes, por si acaso alguien la necesita

Problema demasiado grande

Crear una plataforma educativa completa para todos los institutos de España

Problema de MVP

Que mis compañeros de DAW tengan sus apuntes organizados por asignatura

Errores comunes al identificar problemas

Error	Ejemplo	Por qué es malo
Empezar por la solución	“Quiero hacer una app con React”	No sabes si es necesaria
Problema demasiado grande	“Resolver la educación en España”	No puedes resolverlo en 4 semanas
Problema demasiado pequeño	“Recordar el color de la pared”	No tiene valor real
Problema ficticio	“Una app para...” sin tener el problema	No lo usarías tú
Problema sin usuario	“Algo para todo el mundo”	Si es para todos, es para nadie

Mini-chequeo

► ¿Cuáles son las 3 fuentes de problemas reales?

► ¿Qué es el “test de la ducha”?

✓ Resumen en 3 frases

1. Empieza por el **problema**, no por la solución: “no sé qué cocinar” es mejor que “app de recetas”.
2. Los mejores problemas vienen de **tu día a día** y de las **personas de tu entorno**.
3. **Valida** que el problema es real antes de construir: habla con 3 personas y busca soluciones existentes.

 Volver al [índice de la unidad](#) · Anterior: [01 — ¿Qué es un MVP?](#) · Siguiente: [03 — Definir features](#)

Guia Didactica Proyecto 1 Uds U1 1 Mvp 03 Definir Features

 **Estás en:**  **UD 1.1 • MVP** → [03](#) · Definir features

 *De la idea suelta a la lista concreta de funcionalidades*

Ya tienes un problema claro. Ahora necesitas decidir **qué va a hacer tu aplicación** para resolverlo. Pero no todo lo que se te ocurra: solo lo imprescindible. Este punto te enseña a listar, clasificar y priorizar con criterio.

La idea en una frase

Las features son lo que tu app hace. Las features del MVP son solo las que necesitas para que resuelva el problema.

No pongas todo lo que se te ocurra. Pon solo lo que **necesitas**.

Paso 1: Lista todo (sin filtrar)

Abre un documento y escribe **todas** las funcionalidades que se te ocurran. Sin juzgar, sin ordenar, sin pensar si son posibles.

Ejemplo: App de recetas para “¿Qué cocino con lo que tengo?”

- Introducir los ingredientes que tengo
- Buscar recetas por ingrediente
- Buscar recetas por nombre
- Filtrar por tiempo de cocina
- Filtrar por dificultad
- Guardar recetas favoritas
- Crear listas de compra
- Compartir recetas
- Valorar recetas
- Subir fotos de recetas propias
- Modo oscuro
- Notificaciones de recetas nuevas
- Estadísticas de lo que más cocinas
- Integración con supermercados
- Modo offline
- Accesibilidad (voz)
- Multiidioma

Regla importante: en esta fase **no quites nada**. Escríbelo todo. Luego priorizarás.

Paso 2: Clasifica por prioridad

Una vez tengas tu lista, clasifica cada feature en una de estas categorías:

Categoría	Significado	Ejemplo
MVP	Sin esto, la app no tiene sentido	Introducir ingredientes, buscar recetas
Deseable	Mejora la experiencia si hay tiempo	Favoritos, filtrar por tiempo
Futuro	Ideas para después	Compartir, notificaciones, estadísticas

El test de la pregunta mágica

Para cada feature, hazte esta pregunta:

“Si esta feature no está, ¿la app sigue resolviendo el problema?”

- Si la respuesta es **NO** → es MVP
- Si la respuesta es **SÍ, pero peor** → es deseable
- Si la respuesta es **SÍ, igual** → es futuro

Ejemplo aplicado:

Feature	¿La app resuelve sin ella?	Categoría
Introducir ingredientes	NO — no puedes buscar sin ellos	MVP
Buscar recetas	NO — no tiene sentido sin búsqueda	MVP
Filtrar por dificultad	SÍ, pero peor — puedes buscar sin filtro	Deseable
Modo oscuro	SÍ, igual — es cosmético	Futuro
Notificaciones	SÍ, igual — no es esencial	Futuro

Paso 3: El test del “¿y además...?”

Imagina que alguien te pregunta por tu app. Cada vez que digas “y además...”, estás añadiendo una feature que **probablemente no necesitas**.

Tú: "Mi app busca recetas por ingredientes"
Amigo: "¿Y también busca por nombre?"
Tú: "Sí, y además filtra por tiempo"
Amigo: "¿Y tiene favoritos?"
Tú: "Eh... sí, y además notificaciones"

Cada “y además” es una feature que complica tu MVP. En este punto, para y pregúntate: “¿Es esto imprescindible o es un ‘deseable’?”

Paso 4: El “test del abuelo”

Explica tu app a alguien que no sea de informática (tu abuelo, tu madre, un amigo de otra carrera). Si no lo entiende en 30 segundos, tu MVP es demasiado complicado.

Ejemplo malo: “Mi app es una plataforma educativa multiidioma con gamificación y notificaciones push que integra un sistema de recomendaciones basado en IA para sugerir recetas personalizadas según el perfil nutricional del usuario.”

Ejemplo bueno: “Mi app es una lista de la compra que se convierte en recetas según lo que tengas en casa.”

Si no puedes explicarlo en una frase, tienes demasiadas features.

MoSCoW: el método de priorización

MoSCoW es un acrónimo que te ayuda a clasificar features:

Letra	Significado	Pregunta clave
M	Must have	¿Sin esto, la app no funciona?
S	Should have	¿Es importante pero no vital?
C	Could have	¿Mejora la experiencia pero no es necesario?
W	Won't have	¿Es para el futuro?

Ejemplo completo: App de recetas

Feature	MoSCoW	Justificación
Introducir ingredientes	M	Sin esto no hay app
Buscar recetas	M	Función principal
Ver receta completa	M	Sin esto no se puede cocinar
Filtrar por dificultad	S	Mejora la experiencia
Guardar favoritos	C	Útil pero no vital
Notificaciones	W	Futuro
Multiidioma	W	Futuro
Modo offline	W	Futuro

Regla del MVP: solo las **M** (Must have) van en la primera versión.

El error de las “features de emergencia”

Cuando estás priorizando, es fácil pensar: “esto es rápido de hacer, lo pongo”. **No caigas en esa trampa.**

Features “rápidas” que parecen inocentes	Por qué son peligrosas
“Modo oscuro”	Requiere diseño, testing en ambos modos
“Botón de compartir”	Requiere integración con redes sociales
“Notificaciones”	Requiere backend, permisos, diseño
“Filtros avanzados”	Requiere diseño de UI, testing

Regla: si una feature no resuelve directamente el problema central, **no va en el MVP** aunque sea “rápida”.

Cuenta las features: el límite del MVP

Un MVP de Proyecto 1 debería tener **entre 2 y 4 funcionalidades principales**.

¿Por qué tan pocas?

- **Tiempo limitado:** tienes 4-6 semanas para todo (planificación + desarrollo + documentación)
- **Calidad > cantidad:** mejor 2 features perfectas que 5 a medias
- **Aprendizaje:** con pocas features puedes feedback real
- **Terminado:** necesitas **entregar algo completo**

Número de features	¿Es un buen MVP?
1-2	Demasiado simple, pero viable
3-4	Óptimo — suficiente para demostrar la idea
5-6	Posible si son muy sencillas
7+	Probablemente demasiado para un proyecto FP

Mini-chequeo

► ¿Cómo clasificas las features por prioridad?

► ¿Qué es MoSCoW?

► ¿Cuántas features debería tener tu MVP?

✓ Resumen en 3 frases

1. Primero **lista todo** lo que se te ocurra sin filtrar, luego clasifica por prioridad con MoSCoW.
2. Usa el test de la pregunta mágica: “¿Sin esta feature, la app resuelve el problema?” Si la respuesta es sí, **no es MVP**.
3. Un buen MVP tiene **2-4 funcionalidades principales** que funcionen perfectamente.

Guia Didactica Proyecto 1 Uds U1 1 Mvp 04 Cortar Sin Piedad

 Estás en:  **UD 1.1 • MVP** → 04 · Cortar sin piedad

 *El arte de decir ‘no’ a las buenas ideas*

Tienes tu lista de features priorizadas. Ahora viene la parte difícil: **cortar**. No es destruir tu proyecto, es enfocarlo. Cada feature que quitas es tiempo que ahorras para hacer bien lo que queda.

La idea en una frase

Cortar features no es perder, es ganar tiempo y calidad.

Menos features = más tiempo para cada una = mejor resultado = proyecto terminado.

Por qué cortar es tan difícil

Cortar features se siente mal porque:

1. **Inversión emocional** — te ha costado pensarlo, quitarlo es como desperdiciar ese esfuerzo
2. **Miedo a decepcionar** — “¿y si el profesor piensa que es poco?”
3. **Síndrome del “y además...”** — cada feature parece rápida de añadir
4. **Optimismo irracional** — “ya veré cómo lo meto”

Un proyecto con 3 features perfectas gana siempre a un proyecto con 8 features rotas.

Las 5 reglas para cortar

Regla 1: El “test del ISBN”

Imagina que tu proyecto tiene un ISBN (un libro). ¿Podrías poner en la portada una descripción de 10 palabras?

-  “App que convierte tu nevera en recetas concretas”
-  “Plataforma educativa multiidioma con gamificación, notificaciones y IA”

Si la descripción es larga, tienes demasiadas features.

Regla 2: El “test del usuario imaginario”

Crea un usuario imaginario (ej: “María, 20 años, estudiante de DAW”). Ahora pregúntate:

“¿María usaría mi app todos los días para resolver SU problema?”

Si la respuesta es “no sé” o “a veces”, es que tu app no es suficientemente focused.

Regla 3: El “test del delete”

Pregunta: “Si quito esta feature, ¿el usuario se va a furioso?”

- Si la respuesta es “se va a molestar un poco” → **quítala**
- Si la respuesta es “no puede usar la app” → **déjala**

La mayoría de las features que “parecen importantes” en realidad causan una molestia menor si no están.

Regla 4: El “test del tiempo”

Calcula cuánto tiempo te tomaría cada feature:

Feature	Tiempo estimado	¿Vale la pena?
CRUD de recetas	3 días	Sí — esencial
Filtrar por dificultad	1 día	Tal vez
Modo oscuro	2 días	No — es cosmético
Notificaciones	3 días	No — es complejo

Regla: si una feature tarda más de 2 días y no es esencial, **quítala**.

Regla 5: El “test de la navaja de Ockham”

“Entre varias soluciones, la más simple es la correcta.”

Si puedes resolver un problema de 2 formas, elige la más simple. Siempre.

El error del “tengo tiempo”

El error más peligroso en un proyecto FP es pensar: “todavía me queda tiempo, puedo añadir esto”.

Realidad: el tiempo siempre se acaba más rápido de lo que crees.

Semana	Lo que piensas	Lo que pasa
1	“Tengo mucho tiempo”	Estás en la planificación
2	“Empiezo a programar”	Aparecen bugs que no esperabas
3	“Va bien”	Te enfrentas a problemas técnicos
4	“Estoy terminando”	Falta documentación y testing
5	“Ya casi está”	El tribunal pide más detalles
6	“Entrego lo que tengo”	Entregas a medias

Regla: planifica para 4 semanas, no para 6. La última semana es para imprevistos.

¿Cuándo parar de cortar?

No es “cortar todo”. Es cortar **hasta que quede lo imprescindible**.

Señales de que has cortado suficiente:

- Tu app se puede explicar en 1 frase
- Tienes 2-4 funcionalidades principales
- Cada feature se puede implementar en 1-3 días
- Te sientes cómodo con el alcance

Señales de que has cortado de más:

- Tu app hace solo 1 cosa (eso es un script, no un proyecto)
- No hay nada interesante que mostrar
- El proyecto no tiene sentido sin al menos 3 features

El “y además” final

Antes de decidir tu MVP final, haz una última revisión. Para cada feature que quede, pregúntate:

“Si esta feature no está, ¿el profesor va a pensar que el proyecto está incompleto?”

Si la respuesta es “no, va a pensar que está bien enfocado” → **béla**. Si la respuesta es “sí, va a pensar que falta algo” → **déjala**.

► ? ¿Y si mi profesor dice que es poco?

Checklist de corte final

Antes de cerrar tu alcance, verifica:

- ☐ ¿Puedo explicar mi proyecto en una frase?
- ☐ ¿Tiene entre 2 y 4 funcionalidades principales?
- ☐ ¿Cada feature resuelve directamente el problema central?
- ☐ ¿He quitado todo lo que es “bonito pero no necesario”?
- ☐ ¿Sé exactamente qué voy a construir?
- ☐ ¿Estoy cómodo con el alcance (no asustado ni sobrado)?

Si respondes sí a todo → **tu MVP está definido.**

Mini-chequeo

► ¿Por qué cortar features es bueno?

► ¿Qué es el “test del ISBN”?

► ¿Cuánto tiempo debes planificar para el desarrollo?

✓ Resumen en 3 frases

1. **Cortar no es perder**, es ganar tiempo y calidad para hacer bien lo que queda.
2. Usa los tests (ISBN, usuario, delete, tiempo, navaja) para decidir qué quitar.
3. Un MVP con **2-4 features perfectas** gana siempre a uno con 8 features rotas.



Volver al [índice de la unidad](#) · Anterior: [03 — Definir features](#) · Siguiente: [05 — Ejemplos reales](#)

 **Estás en:**  **UD 1.1 • MVP** → 05 • Ejemplos reales

 *Ver ejemplos de otros es la mejor forma de entender un MVP*

La teoría está bien, pero ver **qué hicieron otros** y **por qué funcionó** es mucho más útil. Aquí tienes 8 ejemplos de ideas que empezaron como MVPs y cómo se convirtieron en algo grande.

La idea en una frase

Los mejores proyectos empezaron como algo pequeño que alguien terminó, no como algo grande que nadie acabó.

Cada ejemplo sigue el mismo patrón: idea grande → MVP pequeño → éxito.

Ejemplo 1: Facebook — Directorio de compañeros

Aspecto	Detalle
Problema	En Harvard no había un directorio fácil de usar para conocer compañeros
Idea grande	Red social completa con grupos, eventos, fotos, mensajería
MVP	Directorio de alumnos con foto, nombre y estado de relación
¿Por qué funcionó?	Resolvía 1 problema concreto: “¿Quién es esta persona?”
Features del MVP	Ver perfil, buscar compañero, estado de relación
Features que NO estaban	Fotos, grupos, eventos, mensajería, anuncios

Lección: Empezar por algo simple y dejar que los usuarios pidan más.

Ejemplo 2: Twitter — Microblogging

Aspecto	Detalle
Problema	Los blogs son largos y nadie lee posts de 1000 palabras
Idea grande	Plataforma de contenido con hashtags,趋势, verificación
MVP	Mensajes de texto de 140 caracteres, solo texto
¿Por qué funcionó?	Era fácil de usar, rápido, adictivo
Features del MVP	Escribir tweet, seguir usuarios, timeline
Features que NO estaban	Hashtags, fotos, verified, trends, DMs

Lección: La simplicidad puede ser una fortaleza, no una debilidad.

Ejemplo 3: Instagram — Fotos con filtros

Aspecto	Detalle
Problema	Las fotos del móvil eran feas y compartir las complicado
Idea grande	Red social de fotos con stories, reels, shopping
MVP	App para añadir filtros a fotos y compartirlas
¿Por qué funcionó?	Resolvía 1 problema: “mis fotos del móvil son feas”
Features del MVP	Tomar foto, aplicar filtro, compartir
Features que NO estaban	Stories, reels, comentarios, likes, shopping

Lección: Un solo feature bien hecho puede ser más poderoso que 10 a medias.

Ejemplo 4: Uber — Coche con un botón

Aspecto	Detalle
Problema	En San Francisco, taxis son caros y difíciles de encontrar
Idea grande	Plataforma de movilidad con UberX, UberEats, Uber Freight
MVP	Botón para pedir un coche, pago con tarjeta
¿Por qué funcionó?	“Pulsa un botón, llega un coche” — más simple imposible
Features del MVP	Pedir coche, pagar, valorar conductor
Features que NO estaban	UberX, UberPool, UberEats, precio dinámico

Lección: La experiencia del usuario puede ser perfecta con una sola acción.

Ejemplo 5: Dropbox — Video explicativo

Aspecto	Detalle
Problema	Sincronizar archivos entre dispositivos era complicado
Idea grande	Servicio completo de almacenamiento en la nube
MVP	Un video de 3 minutos explicando cómo funcionaría
¿Por qué funcionó?	Validó la idea sin escribir una línea de código
Features del MVP	Solo el video y una landing page
Features que NO estaban	La app en sí — no existía todavía

Lección: A veces tu MVP no es una app, es una **demostración de que la idea funciona**.

Ejemplo 6: Zappos — Fotos de zapatos

Aspecto	Detalle
Problema	No se podían comprar zapatos online cómodamente
Idea grande	Tienda online de zapatos con envío gratis y devoluciones
MVP	Web con fotos de zapatos de tiendas locales, compra manual
¿Por qué funcionó?	El fundador iba a la tienda, compraba el zapato y lo enviaba
Features del MVP	Catálogo de fotos, compra por email
Features que NO estaban	Inventario automático, pagos online, envío gratis

Lección: Un MVP puede ser “trampa” — lo importante es validar que la gente compra.

Ejemplo 7: Airbnb — Colchón en el salón

Aspecto	Detalle
Problema	En una conferencia no había alojamiento barato
Idea grande	Plataforma global de alquiler vacacional
MVP	Página web con fotos de un colchón en un salón
¿Por qué funcionó?	Dos personas pagaron por dormir en un colchón
Features del MVP	Fotos del colchón, pago por transferencia
Features que NO estaban	Reservas online, pagos seguros, reseñas, seguro

Lección: Si alguien paga por algo tan simple, la idea tiene potencial.

Ejemplo 8: Landbot — Chatbot sin código

Aspecto	Detalle
Problema	Crear chatbots requiere programar
Idea grande	Plataforma completa de automatización con IA
MVP	Herramienta visual para crear chatbots arrastrando bloques
¿Por qué funcionó?	Resolvía 1 problema: “quiero un chatbot pero no sé programar”
Features del MVP	Arrastrar bloques, conectar mensajes, publicar
Features que NO estaban	IA, integraciones, analytics, plantillas

Lección: Hacer accesible algo que antes era complicado es un gran MVP.

Patrones comunes de éxito

Patrón	Ejemplos	Lección
1 problema, 1 solución	Facebook, Instagram, Uber	No intentes resolver todo, resuelve 1 cosa
Simplicidad extrema	Twitter, Uber	Menos features = más fácil de usar
Validar antes de construir	Dropbox, Zappos	A veces tu MVP es un vídeo o una foto
Empezar pequeño	Airbnb, Facebook	Empieza por 1 ciudad o 1 campus
Dejar que los usuarios pidan más	Todos	Construye lo mínimo, escucha, luego amplía

Mini-chequeo

► ¿Qué patrón siguió Dropbox con su MVP?

► ¿Qué feature NO tenía Instagram en su MVP?

► ¿Qué tienen en común todos estos MVPs?

✓ Resumen en 3 frases

1. Los mejores proyectos empezaron como algo **pequeño y terminado**, no como algo grande y a medias.
2. Cada ejemplo resolvía **1 problema concreto** con **pocas features** perfectamente hechas.
3. A veces tu MVP no es una app: puede ser un **vídeo, una foto o una landing page** que valide la idea.

 Volver al [índice de la unidad](#) · Anterior: [04 — Cortar sin piedad](#) · Siguiente: [06 — Errores comunes](#)

Guia Didactica Proyecto 1 Uds U1 1 Mvp 06 Errores Comunes

 Estás en:  **UD 1.1 • MVP** → [06](#) · Errores comunes

 *Aprender de los errores de otros es más barato que cometer los tuyos*

Cada año, cientos de alumnos de FP cometen los mismos errores al definir su MVP. Conocer estos errores **antes de empezar** te ahorra semanas de trabajo desperdiciado.

Los errores más comunes no son técnicos: son de alcance, de priorización y de no terminar.

El 70% de los proyectos que fracasan en FP lo hacen por estos 7 errores.

Los 7 pecados del MVP

Pecado 1: “Quiero hacer todo”

El error: Empezar con una lista de 15 funcionalidades porque “todas parecen importantes”.

Consecuencia: No terminas ninguna bien. Entregas un proyecto a medias con 10 features rotas.

Solución: Usa MoSCoW y cíñete a las “Must have”. Si tu lista tiene más de 4 features, corta.

Pecado 2: “Es rápido de hacer”

El error: Añadir features que “parecen rápidas” sin calcular el tiempo real.

Consecuencia: Esas “features rápidas” se comen tu tiempo y las esenciales quedan a medias.

Ejemplo real:

```
Alumno: "Modo oscuro es rápido, lo añado"  
Resultado: 3 días en hacerlo funcionar bien  
          + 2 días en que el modo claro se rompió  
          + 1 día en arreglar bugs de contraste  
          = 6 días que podría haber usado para testing
```

Solución: Si una feature no es esencial, **no la añadas** aunque sea “rápida”.

Pecado 3: “Ya veré cómo lo integro”

El error: Añadir features sin pensar en cómo se conectan entre sí.

Consecuencia: Tienes 5 features sueltas que no forman un producto coherente.

Ejemplo real:

Features aisladas:

- CRUD de usuarios ✓
- CRUD de tareas ✓
- Login ✓
- Dashboard ✓
- Notificaciones ✓

Resultado: 5 módulos que no se hablan entre sí

Solución: Antes de añadir, pregúntate: “¿Cómo se conecta esto con lo que ya tengo?”

Pecado 4: “No necesito documentar”

El error: Pensar que documentar es “perder tiempo” o “algo para el final”.

Consecuencia: El tribunal no entiende tu proyecto. Parece que no sabes qué has hecho.

Solución: Documenta **mientras** desarrollas, no después. Un README básico cada semana.

Pecado 5: “No necesito testear”

El error: “Funciona en mi ordenador, luego lo pruebo”.

Consecuencia: Bugs que aparecen en la demo del tribunal. Embarazo.

Solución: Prueba cada feature **inmediatamente** después de implementarla.

Pecado 6: “Ya tengo tiempo”

El error: Planificar para 6 semanas cuando realmente tienes 4 de desarrollo real.

Consecuencia: La última semana entras en pánico y entregas a medias.

Cronograma real de un proyecto FP: | Semana | Actividad | ¿Realmente es desarrollo? | |----|
----|-----| | 1 | Planificación, setup, dudas | 20% desarrollo | | 2 | Empiezas en serio |
70% desarrollo | | 3 | Bugs, problemas técnicos | 50% desarrollo | | 4 | Integración, más bugs | 40%
desarrollo | | 5 | Documentación, testing | 20% desarrollo | | 6 | Preparar presentación | 10%
desarrollo |

Solución: Planifica para 4 semanas de desarrollo. La semana 5 es para documentación, la 6 para presentación.

Pecado 7: “Mi idea es original”

El error: No buscar soluciones existentes porque “ya se ha inventado todo”.

Consecuencia: Inventas la rueda, pierdes tiempo, o descubres que alguien lo hizo mejor.

Solución: Busca apps similares. Aprende de ellas. Diferénciate en algo concreto.

Caso práctico: Los 7 pecados en acción

Laura y el gestor de tareas imposible

Laura quiso hacer un “gestor de tareas con IA” para su Proyecto 1. Empezó con esta lista:

1. CRUD de tareas ✓
2. Login con JWT ✓
3. IA que sugiere prioridades ✓
4. Calendario visual ✓
5. Estadísticas con gráficos ✓
6. Notificaciones por email ✓
7. Modo oscuro ✓
8. Exportar a PDF ✓
9. Chat en tiempo real ✓
10. App móvil ✓

Semana 1: Hizo el CRUD y el login. Iba bien.

Semana 2: Empezó la IA. Se atasó 5 días en un algoritmo de priorización.

Semana 3: El calendario no funcionaba bien con la IA. Empezó a estresarse.

Semana 4: Las notificaciones no iban. El chat era imposible. Laura estaba furiosa.

Semana 5: No tenía documentación, no había testeado nada. Entró en pánico.

Semana 6: Entregó un CRUD con login, una IA a medias, y nada más. Suspenso.

¿Qué salió mal? Los 7 pecados: quiso hacer todo, añadió features “rápidas”, no documentó, no testeo, no planificó bien.

¿Qué debería haber hecho? Un CRUD de tareas con priorización manual, documentación básica y testing. 3 features bien hechas.

La lista de “NO” definitiva

Antes de empezar tu proyecto, decide qué **NO** vas a hacer:

NO haré...	¿Por qué?
Notificaciones push	Requiere backend, permisos, diseño complejo
Multiidioma	Más traducciones que features
App móvil	Web responsive es suficiente
Chat en tiempo real	Requiere WebSockets, backend persistente
IA/Machine Learning	Requiere datos, entrenamiento, tiempo
Modo offline	Requiere Service Workers, sincronización
Pagos	Requiere integración con pasarela, SSL
Redes sociales	Requiere APIs, OAuth, moderación

Imprime esta póster y pégalo junto a tu pantalla. Cada vez que digas “y además...”, míralo.

Mini-chequeo

► ¿Cuáles son los 7 pecados del MVP?

► ¿Por qué “ya tengo tiempo” es un error?

► ¿Qué hacer si te atascas en una feature?

✓ Resumen en 3 frases

1. Los 7 pecados más comunes son: **querer todo, features rápidas, no integrar, no documentar, no testear, no planificar tiempo y no buscar soluciones existentes.**
2. El error más peligroso es “**ya tengo tiempo**” — siempre se acaba antes de lo que crees.

3. Decide **qué NO vas a hacer** antes de empezar. Eso te salvará más que decidir qué sí.

 Volver al [índice de la unidad](#) · Anterior: [05 — Ejemplos reales](#) · Siguiente: [07 — Herramientas](#)

Guia Didactica Proyecto 1 Uds U1 1 Mvp 07 Herramientas

 **Estás en:**  **UD 1.1 • MVP** → [07](#) · [Herramientas](#)

 *Herramientas simples para no perder el tiempo*

No necesitas una suite empresarial para organizar tu proyecto. Necesitas algo **simple, rápido y que todos entiendan**. Aquí tienes las mejores opciones para cada cosa.

La idea en una frase

La mejor herramienta es la que usas, no la que es “la mejor”.

No pierdas tiempo configurando herramientas. Usa la más simple y empieza.

¿Qué necesitas organizar?

Para tu Proyecto 1 necesitas 3 cosas:

Necesidad	Para qué	Ejemplo
Tablero de tareas	Ver qué has hecho, qué falta	Trello, GitHub Projects
Documentación	Escribir apuntes, README	Notion, Markdown, Google Docs
Comunicación	Hablar con tu compañero	WhatsApp, Discord, Telegram

Tablero de tareas: Trello vs GitHub Projects

Trello

Aspecto	Detalle
Qué es	Tablero kanban con tarjetas arrastrables
Ventajas	Muy visual, fácil, gratis, sin configurar
Inconvenientes	Sin integración con código, sin métricas
Ideal para	Equipo pequeño, proyecto simple

Cómo usarlo para tu MVP:

Columnas:			
Por hacer	En progreso	Hecho	
-----	-----	-----	
Ejemplo:			
Por hacer	En progreso	Hecho	
-----	-----	-----	
CRUD tareas	Login JWT	Setup proyecto	
Calendario		README	

GitHub Projects

Aspecto	Detalle
Qué es	Tablero integrado con tu repositorio GitHub
Ventajas	Gratis, integrado con Git, métricas automáticas
Inconvenientes	Más complejo que Trello, requiere GitHub
Ideal para	Equipo que ya usa GitHub, proyecto con Git

Cómo usarlo: Ve a tu repositorio → pestaña “Projects” → “New project” → Kanban board.

Trello

Soy más fácil. Arrastras tarjetas y listo. Sin configurar.

GitHub Projects

Estoy integrado con tu código. Cada commit puede mover una tarjeta.

Recomendación: Si no sabes cuál usar, **empieza con Trello**. Es más fácil y no necesitas configurar nada.

Documentación: Notion vs Markdown vs Google Docs

Notion

Aspecto	Detalle
Qué es	Wiki/ documentación todo-en-uno
Ventajas	Bonito, fácil, colaborativo, plantillas
Inconvenientes	Sin exportar a PDF fácil, requiere cuenta
Ideal para	Documentación del proyecto, apuntes

Markdown (en VS Code)

Aspecto	Detalle
Qué es	Texto simple con formato (<code>.md</code>)
Ventajas	Gratis, rápido, portable, versionable con Git
Inconvenientes	Sin preview en tiempo real (necesitas extensión)
Ideal para	README, documentación técnica, notas

Google Docs

Aspecto	Detalle
Qué es	Procesador de textos online
Ventajas	Colaborativo, familiar, exporta PDF
Inconvenientes	Sin versionado, sin formato técnico
Ideal para	Memoria del proyecto, documento para el tribunal

Recomendación: Usa **Markdown** para documentación técnica (README, apuntes) y **Google Docs** para la memoria del proyecto.

Comunicación del equipo

Herramienta	Ventaja principal	Cuándo usarla
WhatsApp	Ya la tienes, es rápida	Dudas rápidas, acuerdos
Discord	Canales organizados, voz	Reuniones, discusiones largas
Telegram	Canales, archivos grandes	Compartir documentos
GitHub Issues	Trazabilidad	Bugs, tareas concretas

Recomendación: **WhatsApp** para comunicación diaria. **GitHub Issues** para bugs y tareas formales.

Herramientas de diseño (opcional)

Si necesitas hacer mockups o diagramas:

Herramienta	Para qué	Precio
Figma	Mockups de UI, wireframes	Gratis (plan personal)
draw.io	Diagramas ER, secuencia, fluxogramas	Gratis
Excalidraw	Bocetos rápidos, pizarra digital	Gratis
Lucidchart	Diagramas profesionales	Gratis (limitado)

Recomendación: **draw.io** para diagramas (se integra con VS Code) y **Excalidraw** para bocetos rápidos.

La stack mínima recomendada

Para tu Proyecto 1, esta es la **stack mínima** de herramientas:

Categoría	Herramienta	Por qué
Tablero	Trello	Fácil, visual, gratis
Documentación	Markdown + VS Code	Rápido, versionable
Memoria	Google Docs	Fácil de exportar a PDF
Comunicación	WhatsApp	Ya la tienes
Código	GitHub	Versionado, backup, portfolio
Diagramas	draw.io	Gratis, se integra con VS Code

Total: 6 herramientas. No necesitas más.

► ? ¿Necesito pagar por herramientas?

Errores comunes con herramientas

Error	Consecuencia	Solución
Elegir herramientas complicadas	Más tiempo configurando que trabajando	Usa la más simple
Usar 20 herramientas	Caos, nada conectado	Máximo 6-7
No usar el tablero	Pierdes el control de qué falta	Revisa cada día
Documentar solo al final	No recuerdas qué hiciste	Documenta mientras
No usar Git	Pierdes código, no puedes volver atrás	Git desde el día 1

Mini-chequeo

► ¿Cuáles son las 3 cosas que necesitas organizar?

► ¿Trello o GitHub Projects?

► ¿Necesito pagar por herramientas?

✓ Resumen en 3 frases

1. Necesitas **6 herramientas**: tablero (Trello), docs (Markdown), memoria (Google Docs), comunicación (WhatsApp), código (GitHub), diagramas (draw.io).
2. **La mejor herramienta es la que usas**, no la más potente. No pierdas tiempo configurando.
3. No gastes dinero. Todo lo necesario es **gratis**.

 Volver al [índice de la unidad](#) · Anterior: [06 — Errores comunes](#) · Siguiente: [08 — Template MVP](#)

Guia Didactica Proyecto 1 Uds U1 1 Mvp 08 Template Mvp

 **Estás en:**  **UD 1.1 • MVP** → 08 · Template MVP

 *Copia, rellena y ya tienes tu MVP definido*

Si has leído los puntos anteriores, ya sabes qué es un MVP, cómo identificar un problema, cómo priorizar features y qué errores evitar. Ahora toca **ponerlo por escrito**. Esta

plantilla te guía paso a paso.

La idea en una frase

Un buen MVP se define en 1 página. Si necesitas más, es que es demasiado grande.

Rellena esta plantilla y tendrás tu MVP listo para empezar a desarrollar.

Plantilla: Definición de MVP

Copia este esquema en un documento (Markdown, Notion o Google Docs) y rellénalo:

1. Problema

¿Qué problema resuelve tu proyecto?

[Escribe aquí el problema en 1-2 frases. Ejemplo: “Mis compañeros de DAW pierden apuntes por WhatsApp y no los encuentran cuando los necesitan.”]

¿Quién tiene este problema?

[Define tu usuario objetivo. Ejemplo: “Estudiantes de DAW de 1º y 2º del IES X”].

¿Cómo lo resuelven ahora?

[Describe la solución actual. Ejemplo: “Búscan en WhatsApp, preguntan en grupo, o directamente no encuentran.”].

2. Solución (MVP)

En 1 frase, ¿qué hace tu app?

[Ejemplo: “Una web donde los alumnos de DAW guardan y buscan apuntes por asignatura.”]

¿Por qué esta solución y no otra?

[Justificación breve. Ejemplo: “Porque WhatsApp no tiene organización por asignatura y la búsqueda es mala.”].

3. Features (MoSCoW)

Feature	MoSCoW	Tiempo estimado	Justificación
[Feature 1]	M/S/C/W	[días]	[Por qué es M/S/C/W]
[Feature 2]	M/S/C/W	[días]	[Por qué es M/S/C/W]
[Feature 3]	M/S/C/W	[días]	[Por qué es M/S/C/W]
[Feature 4]	M/S/C/W	[días]	[Por qué es M/S/C/W]
[Feature 5]	M/S/C/W	[días]	[Por qué es M/S/C/W]

Features M (MVP): [Lista solo las M] **Features S (Deseable):** [Lista las S] **Features C (Podría tener):** [Lista las C] **Features W (Futuro):** [Lista las W]

4. Límites del MVP

¿Qué NO hace tu MVP?

[Lista explícitamente lo que NO vas a hacer. Ejemplo: “No tiene login, no tiene notificaciones, no tiene modo oscuro.”]

¿Qué pasaría si alguien pide una de estas features?

[Tu respuesta. Ejemplo: “Se puede añadir en el futuro, pero el MVP se queda así.”]

5. Usuario objetivo

¿Para quién es?

[Descripción del usuario. Ejemplo: “Alumnos de DAW que necesitan organizar apuntes por asignatura.”]

¿Qué necesita hacer?

[Acciones principales. Ejemplo: “Guardar un apunte, buscarlo, verlo, borrarlo.”]

¿Qué NO necesita hacer?

[Acciones que no van en el MVP. Ejemplo: “Compartir apuntes, valorar, chatear.”]

6. Criterios de éxito

Tu MVP es exitoso si:

- ☐ [Criterio 1. Ejemplo: “Un alumno puede guardar un apunte en menos de 30 segundos.”]
- ☐ [Criterio 2. Ejemplo: “Un alumno puede encontrar un apunte por asignatura.”]
- ☐ [Criterio 3. Ejemplo: “El profesor puede ver que el proyecto funciona y está documentado.”]

7. Cronograma estimado

Semana	Actividad
1	Planificación + setup
2-3	Desarrollo features MVP
4	Testing + documentación
5	Preparar presentación
6	Buffer (imprevistos)

8. Stack tecnológico

Capa	Tecnología	Por qué
Frontend	[Ej: Vue, React, HTML+JS]	[Justificación]
Backend	[Ej: Node, Python, Ninguno]	[Justificación]
BD	[Ej: SQLite, JSON, Ninguna]	[Justificación]
Despliegue	[Ej: GitHub Pages, Netlify]	[Justificación]

Ejemplo relleno: App de apuntes DAW

Para que veas cómo queda, aquí tienes un ejemplo:

1. Problema

¿Qué problema resuelve tu proyecto? > Mis compañeros de DAW pierden apuntes por WhatsApp y no los encuentran cuando los necesitan antes de un examen.

¿Quién tiene este problema? > Estudiantes de Desarrollo de Aplicaciones Web de 1º y 2º.

¿Cómo lo resuelven ahora? > Buscan en WhatsApp (llorando), preguntan en grupo, o directamente no encuentran y estudian sin apuntes.

2. Solución (MVP)

En 1 frase, ¿qué hace tu app? > Una web donde los alumnos guardan y buscan apuntes por asignatura.

¿Por qué esta solución y no otra? > Porque WhatsApp no tiene organización por asignatura y la búsqueda es terrible.

3. Features (MoSCoW)

Feature	MoSCoW	Tiempo	Justificación
Guardar apunte (título, asignatura, contenido)	M	2 días	Función principal
Buscar por asignatura	M	1 día	Esencial para encontrar
Ver apunte completo	M	1 día	Sin esto no se puede leer
Editar apunte	M	1 día	Para corregir errores
Eliminar apunte	S	0.5 días	Útil pero no vital
Filtrar por autor	C	1 día	Mejora la experiencia
Login de usuarios	W	3 días	Futuro
Notificaciones	W	2 días	Futuro

Features M: Guardar, buscar, ver, editar (6 días) **Features S:** Eliminar (0.5 días) **Features C:** Filtrar por autor (1 día) **Features W:** Login, notificaciones

4. Límites del MVP

¿Qué NO hace tu MVP? > No tiene login, no tiene notificaciones, no tiene chat, no tiene valoraciones, no tiene modo oscuro.

5. Usuario objetivo

¿Para quién es? > Alumnos de DAW que necesitan apuntes organizados por asignatura.

¿Qué necesita hacer? > Guardar un apunte, buscarlo por asignatura, verlo, editarlo, borrarlo.

6. Criterios de éxito

- ☐ Un alumno puede guardar un apunte ^{89 / 743} en menos de 30 segundos

- ☐ Un alumno puede encontrar un apunte por asignatura en menos de 10 segundos
- ☐ El proyecto tiene README con instrucciones de ejecución

Checklist final del MVP

Antes de empezar a desarrollar, verifica:

- ☐ El problema está claro y es real
- ☐ El usuario objetivo está definido
- ☐ Las features están priorizadas con MoSCoW
- ☐ Sé exactamente qué NO voy a hacer
- ☐ Los criterios de éxito están definidos
- ☐ El cronograma es realista (4 semanas de desarrollo)
- ☐ El stack tecnológico está decidido

Si todo está marcado → tu MVP está listo para empezar.

Mini-chequeo

► ¿Cuántas secciones tiene la plantilla?

► ¿Cuál es el paso más importante de la plantilla?

► ¿Qué hago si el cronograma no cuadra?

Resumen en 3 frases

1. Un buen MVP se define en **1 página**: problema, solución, features, límites, usuario, criterios, cronograma y stack.
2. El paso más importante es **saber qué NO vas a hacer** — eso define tu alcance.
3. Rellena la plantilla, verifica la checklist y **empieza a desarrollar**.

Guia Didactica Proyecto 1 Uds U1 1 Mvp 09 Cierre

 Estás en:  **UD 1.1 • MVP** → [09](#) · [Cierre](#)

 *Ya sabes qué es un MVP. Ahora toca usarlo.*

Has recorrido toda la unidad: desde qué es un MVP hasta cómo definirlo con la plantilla. Ahora toca **verificar que lo entiendes** y **prepararte para la siguiente unidad**.

La idea en una frase

Un MVP es la versión más pequeña de tu producto que funciona, enseña algo y está terminado.

Si recuerdas esto, has entendido la unidad.

Mini-chequeo final

Responde estas preguntas antes de seguir:

Preguntas de comprensión

► ¿Qué es un MVP y qué NO es?

► ¿Cuál es el primer paso para definir tu MVP?

► ¿Cómo priorizas features con MoSCoW?

► ¿Cuántas features debería tener tu MVP?

► ¿Qué es el “test del ISBN”?

Ejercicio práctico

En 10 minutos, define tu MVP usando la plantilla del punto 08:

1. Escribe el problema (1-2 frases)
2. Define el usuario objetivo
3. Lista 5-8 features y clasifícalas con MoSCoW
4. Decide qué NO vas a hacer
5. Estima el cronograma

Si consigues definir tu MVP en 1 página → has entendido la unidad.



Tabla resumen de la unidad

Concepto	Idea clave
MVP	Versión más pequeña que funciona y enseña
Problema	Empieza por el problema, no por la solución
Features	Lista todo, luego clasifica con MoSCoW
Cortar	Menos features = más calidad
Ejemplos	Los mejores empezaron pequeños
Errores	Los 7 pecados: querer todo, no documentar, etc.
Herramientas	Trello, Markdown, GitHub, draw.io (todo gratis)
Template	Define tu MVP en 1 página

Vocabulario rápido

Término	Definición
MVP	Minimum Viable Product — versión más pequeña que funciona
MoSCoW	Método de priorización: Must, Should, Could, Won't
Feature	Funcionalidad que tiene tu aplicación
Alcance	Qué incluye y qué no incluye tu proyecto
Validar	Comprobar que la idea funciona antes de construir
Sprint	Período de tiempo en el que se trabaja en algo concreto
Criterio de éxito	Condición que debe cumplirse para considerar exitoso el MVP

Conexión con las siguientes unidades

Unidad	Qué aprenderás	Cómo se conecta
1.2 SCRUM Lite	Cómo organizar tu trabajo en sprints	Usarás el MVP que definiste aquí
1.3 Requisitos	Cómo documentar qué hace tu app	Ampliarás las features del MVP
1.5 Comunicación	Cómo presentar tu proyecto	Presentarás tu MVP al tribunal
1.7 Propuesta	Cómo escribir la propuesta final	Tu MVP será la base de la propuesta

💡 Consejo para el siguiente punto

Si todavía no tienes claro qué proyecto hacer:

1. Repasa el [punto 02 \(identificar problema\)](#)
2. Haz el ejercicio de los 10 problemas
3. Elige el que más te motive
4. Aplica la plantilla del [punto 08](#)

No necesitas una idea perfecta. Necesitas una idea que puedas terminar.

✅ Resumen en 3 frases

1. Un MVP es la versión más pequeña de tu producto que **funciona, enseña y está terminado**.
2. Define tu MVP en 1 página: problema, features (MoSCoW), límites, cronograma.
3. **Empieza con poco, termina mucho** — mejor 3 features perfectas que 10 rotas.

 Volver al [índice de la unidad](#) · Anterior: [08 — Template MVP](#) · Siguiente: [UD 1.2 — SCRUM Lite](#)

Guía Didáctica Proyecto 1 Uds U1 2 Scrum Lite 01 Que Es Ágil

📖 Estás en: 📄 UD 1.2 • SCRUM Lite → 01 • ¿Qué es Ágil?

📖 *Antes de SCRUM, hay que entender por qué existe*

SCRUM no nació de la nada. Nació de un problema real: los proyectos de software **fallaban constantemente** cuando se intentaban planificar todo al principio. Esta unidad te explica qué cambió y por qué.

📖 La idea en una frase

Metodología ágil = trabajar en ciclos cortos, entregar algo útil cada vez, y adaptarte cuando las cosas cambian.

No es “no planificar”. Es planificar **menos, pero mejor**.

Waterfall: el método “tradicional”

El modelo **Waterfall** (cascada) es el método clásico de gestión de proyectos. Se llama así porque las fases caen como una cascada: una después de otra, sin volver atrás.

Las fases de Waterfall

- | | |
|------------------|---|
| 1. Requisitos | → Recoger TODO lo que necesita el cliente |
| 2. Diseño | → Planificar la arquitectura completa |
| 3. Desarrollo | → Programar todo de una vez |
| 4. Testing | → Probar cuando ya está "todo hecho" |
| 5. Despliegue | → Entregar al usuario |
| 6. Mantenimiento | → Arreglar lo que se rompa |

¿Por qué falla?

Problema	Ejemplo real
El cliente no sabe qué quiere al principio	Pide “un gestor de tareas” pero cuando lo ve quiere “un gestor de tareas con calendario y notificaciones”
Los requisitos cambian	En la semana 2 descubres que necesitas login, pero el diseño no lo contemplaba
El testing viene al final	Cuando pruebas, encuentras bugs que requieren reescribir código de hace 3 semanas
No hay feedback temprano	Entregas en la semana 8 algo que el cliente no quería desde la semana 3

La analogy del viaje

Imagina que vas de Madrid a París:

- **Waterfall:** Planificas cada paso, cada comida, cada museo. Si el día 3 llueve, tu plan se arruina.
- **Ágil:** Tienes un mapa general, pero decides cada día según el tiempo y tus ganas. Si llueve, visitas un museo en vez de pasear.

¿Qué es el enfoque ágil?

El enfoque **ágil** es un conjunto de metodologías que comparten una filosofía común:

1. **Entregar valor frecuentemente** (no esperar al final)
2. **Adaptarse al cambio** (los requisitos cambian, y eso está bien)
3. **Trabajar en equipo** (el conocimiento se comparte)
4. **Hablar con el usuario** (feedback continuo)

Los 4 valores del Manifiesto Ágil (2001)

17 expertos en software se reunieron en Utah y firmaron el **Manifiesto Ágil**. Sus valores:

Valoramos más...	Que sobre...
Individuos e interacciones	Procesos y herramientas
Software funcionando	Documentación extensiva
Colaboración con el cliente	Negociación de contratos
Respuesta al cambio	Seguir un plan

Nota: No dice que lo de la derecha no importe. Dice que lo de la **izquierda importa más**.

Los 12 principios (resumidos)

1. Nuestra prioridad es **satisfacer al cliente** con entregas tempranas
2. **Bienvenidos los cambios** de requisitos, incluso tarde en el desarrollo
3. Entregar software funcionando **en semanas**, no en meses
4. **Equipos auto-organizados** confían más que los jerárquicos
5. La **comunicación cara a cara** es la más efectiva
6. Software funcionando es la medida principal de progreso

Comparativa: Waterfall vs Ágil

Waterfall

Planificas todo al principio (2 meses), construyes todo de golpe (3 meses), entregas al final. Si el cliente cambia de opinión, penas.

Ágil

Planificas poco (1 semana), construyes en ciclos de 2 semanas, entregas algo cada ciclo. Si el cliente cambia, te adaptas.

Aspecto	Waterfall	Ágil
Planificación	Todo al principio	Continua, iterativa
Entrega	Una vez, al final	Cada 1-2 semanas
Cambios	Son un problema	Son oportunidades
Feedback	Al final	Continuo
Riesgo	Se detecta tarde	Se detecta pronto
Documentación	Muy extensa	La justa necesaria

¿Por qué importa para ti?

En Proyecto Intermodular tienes **poco tiempo** (unas 6 semanas). El enfoque ágil te sirve porque:

1. **No pierdes tiempo** planificando cosas que no vas a hacer
2. **Detectas problemas pronto** (en vez de descubrirlos la semana 5)
3. **Terminas algo real** (no un plan perfecto que nunca se hizo)
4. **Adaptas tu proyecto** si descubres que algo no funciona

Dato real: Según el CHAOS Report (Standish Group), los proyectos ágiles tienen un 42% de éxito frente al 14% de los proyectos Waterfall en pequeños equipos.

El “mino” ágil en tu proyecto

No necesitas ser una empresa para trabajar ágil. En tu proyecto:

Enfoque ágil	Cómo lo aplicas
Ciclos cortos	Sprints de 1 semana
Entrega frecuente	Cada semana tienes algo funcionando
Feedback	Pides opinión a tu profe o compañeros
Adaptación	Si algo no funciona, cambias el plan
Trabajo en equipo	Repartís tareas, habláis cada día

Aclaración: “Ágil no es hacer rápido”

- ⚠️ ¿Ágil significa ir de prisa y sin planificar?

Mini-chequeo

- ¿Cuál es la diferencia principal entre Waterfall y Ágil?

- ¿Qué dice el Manifiesto Ágil sobre la documentación?

- ¿Por qué el enfoque ágil es mejor para proyectos pequeños?

✓ Resumen en 3 frases

1. El enfoque ágil consiste en **trabajar en ciclos cortos**, entregar valor cada vez y adaptarse al cambio.
2. El Manifiesto Ágil prioriza **personas sobre procesos**, software sobre documentación y colaboración sobre contratos.
3. Para tu proyecto, ágil significa **sprints de 1 semana**, feedback frecuente y no aferrarte a un plan rígido.

Guia Didactica Proyecto 1 Uds U1 2 Scrum Lite 02 Que Es Scrum

 Estás en:  **UD 1.2 · SCRUM Lite** → [02 · ¿Qué es SCRUM?](#)

 *SCRUM es un framework, no una religión*

SCRUM es la metodología ágil más popular del mundo. Pero en el ámbito educativo se suele simplificar demasiado o se copia tal cual de las empresas. Esta unidad te presenta **SCRUM Lite**: lo esencial para tu proyecto, sin la burocracia innecesaria.

La idea en una frase

SCRUM es un marco de trabajo que organiza el desarrollo en ciclos cortos (sprints), con roles claros y reuniones rápidas.

No es una metodología completa. Es un **framework** que puedes adaptar.

Los 3 roles de SCRUM

En un equipo SCRUM hay 3 roles. En un proyecto de FP, estos roles se simplifican:

1. Product Owner (PO)

En la empresa: Representa al cliente. Decide ~~qué se~~ ^{qué se} hace y en qué orden.

En tu proyecto: Es el que **define qué features son prioritarias**. Normalmente es el profesor o el propio equipo que decide qué va primero.

Responsabilidad	Ejemplo
Definir el producto	“Nuestra app es un gestor de tareas para alumnos”
Priorizar features	“El login es Must, el modo oscuro es Won’t”
Aceptar el trabajo	“Sí, esta feature está bien hecha”

2. Scrum Master (SM)

En la empresa: Facilita el proceso. Quita impedimentos. Protege al equipo.

En tu proyecto: Es el que se **asegura de que SCRUM se cumple**. En equipos de FP, este rol se rota o lo asume el que más organización tenga.

Responsabilidad	Ejemplo
Facilitar reuniones	“La daily empieza a las 8:30, no a las 9”
Quitar impedimentos	“No podéis deployar porque no hay servidor → busco alternativa”
Proteger al equipo	“El profe pidió una feature extra → la metemos en el siguiente sprint”

3. Development Team (el equipo)

En la empresa: 3-9 personas que desarrollan el producto.

En tu proyecto: **Todos vosotros**. No hay subequipos. Cada uno aporta lo que puede.

Responsabilidad	Ejemplo
Auto-organizarse	“Yo hago frontend, tú backend, ella testing”
Estimar esfuerzo	“Esta feature me cuesta 2 días”
Entregar al final del sprint	“El viernes tenemos la feature terminada”

Las 5 ceremonias (reuniones)

SCRUM tiene 5 reuniones principales. En “SCRUM Lite” las simplificamos:

1. Sprint Planning (Inicio del sprint)

¿Qué es? Reunión donde el equipo decide **qué se va a hacer** en el sprint.

Duración: 30-60 minutos.

Preguntas clave: - ¿Qué podemos entregar este sprint? - ¿Quién hace qué? - ¿Qué puede salir mal?

En tu proyecto: El lunes por la mañana, 30 minutos. Decidís las tareas de la semana.

2. Daily Scrum (diaria)

¿Qué es? Reunión diaria de 15 minutos donde cada uno dice 3 cosas.

Duración: 15 minutos máximo.

Las 3 preguntas:

1. ¿Qué hice ayer?
2. ¿Qué haré hoy?
3. ¿Qué me bloquea?

En tu proyecto: Cada mañana, 15 minutos. Si sois 4 personas, 4 minutos cada uno.

3. Sprint Review (revisión)

¿Qué es? Reunión al final del sprint donde se **muestra lo que se ha hecho**.

Duración: 30-45 minutos.

En tu proyecto: El viernes por la tarde. Mostráis lo que funciona al profe o al equipo.

4. Sprint Retrospective (retrospectiva)

¿Qué es? Reunión donde el equipo reflexiona sobre **cómo ha ido el sprint**.

Duración: 30 minutos.

Preguntas clave: - ¿Qué salió bien? - ¿Qué salió mal? - ¿Qué podemos mejorar?

En tu proyecto: El viernes después de la review. 15 minutos hablando de qué mejorar.

5. Backlog Refining (refinamiento)

¿Qué es? Reunión para **preparar el siguiente sprint**: estimar features, aclarar dudas.

Duración: 30-60 minutos.

Los 3 artefactos

1. Product Backlog

¿Qué es? La lista de **todas las features** que quieres en tu producto, ordenadas por prioridad.

En tu proyecto: Un documento o tabla con todas tus features clasificadas (MoSCoW de la UD 1.1).

2. Sprint Backlog

¿Qué es? La lista de features que el equipo se **compromete a hacer** en este sprint.

En tu proyecto: Las 3-5 tareas que os proponéis completar esta semana.

3. Increment

¿Qué es? El **resultado tangible** del sprint: software funcionando que se puede demostrar.

En tu proyecto: Lo que podéis mostrar el viernes. Si no funciona, no hay increment.

Comparativa: SCRUM formal vs SCRUM Lite

SCRUM formal (empresas)

3 roles definidos, 5 ceremonias, velocity tracking, burndown charts, sprint de 2 semanas, product backlog refinado, definition of done estricta.

SCRUM Lite (tu proyecto)

Roles rotativos, daily de 15 min, planning de 30 min, retrospectiva rápida, sprint de 1 semana, tablero Trello, y sentido común.

Aclaración: “SCRUM no es para equipos pequeños”

► 🤔 ¿SCRUM funciona solo para equipos grandes?

El ciclo de SCRUM en tu proyecto

```
Sprint Planning (lunes 30 min)
↓
Daily Scrum (cada día 15 min)
↓
Desarrollo (el grueso del tiempo)
↓
Sprint Review (viernes 30 min)
↓
Sprint Retrospective (viernes 15 min)
↓
Siguiendo Sprint...
```

Mini-chequeo

► ¿Cuáles son los 3 roles de SCRUM?

► ¿Cuáles son las 5 ceremonias de SCRUM?

► ¿Qué es un Sprint?

► ¿Qué es el Product Backlog?

✅ Resumen en 3 frases

1. SCRUM organiza el trabajo en **sprints** (ciclos de 1-2 semanas) con roles claros y reuniones cortas.

2. Los 3 roles son **Product Owner** (prioriza), **Scrum Master** (facilita) y **Development Team** (ejecuta).
3. En tu proyecto, aplica SCRUM Lite: **planning de 30 min, daily de 15 min, review y retrospectiva al final del sprint.**

 Volver al [índice de la unidad](#) · Anterior: [01 — ¿Qué es Ágil?](#) · Siguiendo: [03 — El Sprint](#)

Guia Didactica Proyecto 1 Uds U1 2 Scrum Lite 03 Sprint

 Estás en:  **UD 1.2 · SCRUM Lite** → [03 · El Sprint](#)

 *El sprint es el latido de SCRUM*

Un sprint es un período de tiempo fijo en el que el equipo se compromete a entregar algo concreto. No es “trabajar deprisa”. Es **trabajar enfocado** durante un tiempo limitado, con un objetivo claro.

La idea en una frase

Un sprint es una semana (o dos) donde el equipo se compromete a entregar algo que funciona, sin cambiar de opinión a mitad.

Si cambias de objetivo a mitad del sprint, no estás haciendo SCRUM.

La anatomy de un sprint

Duración

Duración	Cuándo usarla
1 semana	Proyectos de FP, equipos pequeños, alta incertidumbre
2 semanas	Equipos más grandes, proyectos más estables
4 semanas	Nunca en un proyecto de 6 semanas (solo 1.5 sprints)

Recomendación para tu proyecto: Sprints de **1 semana**. Con 6 semanas de proyecto, tienes 6 sprints. Es suficiente para entregar algo bien hecho.

El ciclo de un sprint de 1 semana

Lunes (30 min) → Sprint Planning: ¿qué hacemos esta semana?
Martes-Jueves → Desarrollo: cada uno con su tarea
Viernes (45 min) → Review + Retrospectiva: ¿qué conseguimos? ¿qué mejoramos?

Sprint Planning: el lunes por la mañana

El planning es la reunión más importante del sprint. Aquí se decide **qué se va a hacer** y **quién lo hace**.

Paso 1: Revisar el Product Backlog (10 min)

Miráis la lista de features priorizadas y decidís cuáles entran en este sprint.

Regla clave: No podéis comprometeros con más de lo que podéis hacer. Si tenéis 3 personas y cada una puede hacer 1 feature por semana, el sprint tiene como máximo 3 features.

Paso 2: Estimar esfuerzo (10 min)

Para cada feature, decidís cuánto cuesta. No necesitáis Planning Poker formal: basta con preguntar “¿cuántos días crees que te lleva?” y multiplicar por 1.5 (porque siempre cuesta más de lo que crees).

Paso 3: Repartir trabajo (10 min)

Cada persona elige o recibe una tarea. Idealmente, cada uno trabaja en **una sola feature** por sprint.

Plantilla de Sprint Planning

Feature	Responsable	Estimación	Estado
Login con JWT	Ana	2 días	Pendiente
CRUD de tareas	Pedro	3 días	Pendiente
Filtro por prioridad	Luis	1 día	Pendiente

Daily Scrum: los 15 minutos diarios

La daily es la reunión más corta y más importante. Sirve para **alinear al equipo** y **detectar bloqueos**.

Las 3 preguntas (cada persona)

1. ¿Qué hice ayer?
2. ¿Qué haré hoy?
3. ¿Qué me bloquea?

Ejemplo de daily

Persona	Ayer	Hoy	Bloqueo
Ana	Empecé el login	Terminar el login	Ninguno
Pedro	Diseñé la BD de tareas	Implementar CRUD	No sé cómo hacer el endpoint de borrar
Luis	Estudié Trello	Empezar el filtro	Ninguno

Nota: Pedro tiene un bloqueo. El Scrum Master (o el equipo) debe ayudarle **después** de la daily, no durante.

Reglas de la daily

1. **15 minutos máximo.** Cronometrad.
2. **De pie** (o levantados si es por Discord/Teams)
3. **No se resuelven dudas técnicas** aquí. Se apuntan y se hablan después.
4. **Si no hay nada que decir:** “Todo va bien, sigo con lo mismo.” 30 segundos.

Desarrollo: el grueso del sprint

El 80% del tiempo del sprint es **desarrollo real**. Aquí van algunos consejos:

Una persona, una tarea

No asignéis 3 features a la misma persona. Si Ana tiene el login y el registro y el perfil, va a estar sobrecargada mientras Luis solo tiene el filtro.

Working Agreement

Definid desde el primer sprint cómo trabajáis:

Aspecto	Decisión
Horario de código	9:00 - 14:00
Comunicación	Discord canal #proyecto
Ramas	feature/nombre-feature
Commits	Mensajes descriptivos en inglés
Code review	Al menos 1 persona revisa antes de mergear

Definition of Done (DoD)

¿Cuándo consideraréis que una feature está “hecha”? Definid unos criterios:

- ☐ El código compila sin errores
- ☐ Funciona en el navegador del profesor
- ☐ Tiene documentación básica
- ☐ Otra persona lo ha probado
- ☐ Está en la rama principal

Sprint Review: el viernes por la tarde

La review es donde **mostráis lo que habéis hecho**. No es una presentación bonita: es una demo.

1. **Cada persona muestra su feature** (5 minutos por persona)
2. **El Product Owner (o profe) acepta o rechaza** la feature
3. **Se actualiza el tablero Kanban:** las features hechas van a “Done”

Qué NO hacer en la review

- No hacer una presentación de PowerPoint
- No explicar el código línea por línea
- No discutir por qué algo no funciona (eso es para la retrospectiva)

Sprint Retrospective: qué mejorar

La retrospectiva es donde el equipo reflexiona sobre **cómo ha funcionado** el sprint, no sobre qué features se hicieron.

Las 3 preguntas

1. **¿Qué salió bien?** (para seguir haciéndolo)
2. **¿Qué salió mal?** (para dejar de hacerlo)
3. **¿Qué podemos mejorar?** (para empezar a hacerlo)

Ejemplo de retrospectiva

Categoría	Observación	Acción
✅ Bien	La daily nos ayudó a detectar el bloqueo de Pedro rápido	Seguir con la daily
❌ Mal	Pedro no pudo terminar el CRUD porque la API no estaba lista	El backend se termina ANTES de empezar el frontend
🔄 Mejorar	No sabemos cuánto queda del proyecto	Añadir un “burndown” sencillo en el tablero

Retrospectiva 4Ls (alternativa)

Si la de 3 preguntas se queda corta, usa el formato 4Ls:

Letra	Pregunta
Liked	¿Qué nos gustó?
Learned	¿Qué aprendimos?
Lacked	¿Qué nos faltó?
Longed for	¿Qué echamos en falta?

Comparativa: sprint de 1 semana vs 2 semanas

Sprint de 1 semana

6 sprints en 6 semanas. Ciclos rápidos, feedback frecuente, pero menos tiempo por feature. Ideal para proyectos de FP.

Sprint de 2 semanas

3 sprints en 6 semanas. Más tiempo por feature, pero menos ciclos de feedback. Solo si las features son complejas.

Aclaración: “Un sprint no es una semana de trabajo normal”

► ⚠ ¿Un sprint es solo trabajar como siempre pero llamándolo sprint?

Los sprints de tu proyecto (ejemplo)

Semana	Sprint	Objetivo	Features
1	Sprint 1	Setup + MVP básico	Repo, estructura, primera feature
2	Sprint 2	Core funcional	CRUD principal, login básico
3	Sprint 3	Features secundarias	Filtros, validación, errores
4	Sprint 4	Pulir + testing	Bugs, UX, pruebas
5	Sprint 5	Documentación	README, manual, diagrams
6	Sprint 6	Preparar entrega	Presentación, deploy, último review

Mini-chequeo

► ¿Cuánto dura un sprint recomendado para un proyecto de FP?

► ¿Qué se hace en el Sprint Planning?

► ¿Qué son las 3 preguntas de la daily?

► ¿Cuál es la diferencia entre Review y Retrospectiva?

✓ Resumen en 3 frases

1. Un sprint es un período de **1-2 semanas** con un objetivo claro y una entrega al final.
2. Las ceremonias esenciales son: **planning** (lunes), **daily** (cada día) y **review + retrospectiva** (viernes).
3. En tu proyecto, usa **sprints de 1 semana** para tener feedback rápido y terminar a tiempo.

Guia Didactica Proyecto 1 Uds U1 2 Scrum Lite 04 Kanban

 **Estás en:**  **UD 1.2 • SCRUM Lite** → 04 • Tablero Kanban

 *Si no puedes verlo, no puedes gestionarlo*

Un tablero Kanban es la herramienta más simple y más poderosa para organizar el trabajo en equipo. Es un tablero con columnas donde mueves tarjetas desde “por hacer” hasta “hecho”. Si no tenéis un tablero, estáis trabajando a ciegas.

La idea en una frase

Un tablero Kanban visualiza TODO el trabajo del equipo en un solo lugar: lo que falta, lo que se está haciendo y lo que está terminado.

Si alguien pregunta “¿dónde estamos?”, la respuesta está en el tablero.

La estructura básica

Un tablero Kanban tiene al menos 3 columnas:

To Do	In Progress	Done
-----	-----	-----
Login	CRUD tareas	Estructura repo
Filtros		Portada app
Validación		
Documentación		

Las 3 columnas esenciales

Columna	Significado	Regla
To Do	Tareas pendientes	No empieces una nueva si hay algo en In Progress
In Progress	Tareas en las que alguien está trabajando ahora mismo	Máximo 1-2 tareas por persona
Done	Tareas terminadas y verificadas	Solo se mueve aquí cuando OTRA persona lo confirma

Columnas opcionales

Columna	Cuándo usarla
Blocked	Cuando algo está atascado (dependencia, bug, duda)
In Review	Cuando el código está listo pero falta que alguien lo revise
Testing	Cuando la feature está implementada pero falta probarla

La columna **In Review** es la más importante de las opcionales. Si solo quien programa revisa su propio código, los bugs se escapan. La **revisión cruzada** (otra persona revisa tu código antes de moverlo a Done) es la práctica que más errores evita en equipos pequeños.

Las reglas de oro del tablero

Regla 1: WIP Limits (Work In Progress)

WIP = cuántas cosas puedes tener en “In Progress” a la vez.

Equipo	WIP recomendado
2 personas	2-3 tareas máximo
3 personas	3-4 tareas máximo
4 personas	4-5 tareas máximo

¿Por qué? Porque si Ana tiene 5 tareas a medias, ninguna está terminada. Es mejor terminar 1 y empezar otra.

Regla 2: Nada entra en Done sin confirmación

Una tarea no está “hecha” porque el programador diga que sí. Está hecha cuando: - Funciona correctamente - Al menos otra persona lo ha probado - No tiene errores conocidos

Regla 3: El tablero se actualiza cada día

Si el tablero no se actualiza, no sirve de nada. Actualizadlo **en la daily** o **al final del día**.

Regla 4: Las tareas deben ser pequeñas

Si una tarea tiene más de 2 días de trabajo, divídela en tareas más pequeñas. “Hacer el login” es demasiado grande. “Implementar el formulario de login” y “Conectar el login con el backend” son mejores.

Cómo usar Trello

Trello es la herramienta gratuita más popular para tableros Kanban.

Paso 1: Crear un tablero

1. Ve a trello.com y crea una cuenta gratuita
2. Crea un tablero nuevo: “Proyecto Intermodular - Sprint 1”
3. Crea las columnas: To Do, In Progress, Done

Paso 2: Crear tarjetas

Cada tarjeta es una tarea. En la tarjeta pon: - **Título claro:** “Implementar formulario de login” - **Descripción:** Qué hay que hacer exactamente - **Responsable:** Asigna la tarjeta a alguien - **Etiqueta de color:** Por tipo (frontend=azul, backend=verde, docs=amarillo) - **Fecha límite:** Cuando tiene que estar

Paso 3: Mover tarjetas

Cuando empiezas una tarea, la mueves a “In Progress”. Cuando la terminas, a “Done”.

Ejemplo de tablero Trello

To Do	In Progress	Done
-----	-----	-----
Formulario login	CRUD de tareas	Estructura repo
Filtros de búsqueda		Configuración DB
Manual de usuario		Portada app
Exportar a PDF		README

Cómo usar GitHub Projects

Si ya usáis GitHub para el código, **GitHub Projects** es una alternativa integrada.

Paso 1: Crear un proyecto

1. Ve a tu repositorio en GitHub
2. Pestaña “Projects” → “New project”
3. Selecciona “Board” (tablero)
4. Crea las columnas: To Do, In Progress, Done

Paso 2: Vincular Issues

Cada Issue de GitHub puede ser una tarjeta del tablero. Cuando creas una Issue, puedes añadirla al tablero.

Paso 3: Usar etiquetas

Crea etiquetas como en Trello: - `feature` (verde) - `bug` (rojo) - `docs` (amarillo) - `urgent` (naranja)

Comparativa: Trello vs GitHub Projects

Trello

Fácil de usar, visual, funciona para cualquier tipo de proyecto. Sin integración directa con código. Gratis hasta 10 tableros.

Integrado con Issues, Pull Requests y código. Ideal si ya usáis GitHub. Un poco menos visual que Trello.

Aspecto	Trello	GitHub Projects
Facilidad	Muy fácil	Fácil
Integración código	No	Sí (Issues, PRs)
Precio	Gratis (limitado)	Gratis
Mobile	App muy buena	Web responsive
Automatización	Power-Ups	Workflows
Recomendado para	Cualquier proyecto	Si ya usáis GitHub

El tablero como herramienta de comunicación

El tablero no es solo para vosotros. También sirve para:

- **El profesor:** Puede ver el estado del proyecto en 30 segundos
- **El tribunal:** Puede ver qué se hizo y qué no
- **Vosotros mismos:** En 6 semanas no recordaréis qué hicisteis la semana 2

Tablero = transparencia

Si el tablero muestra que “Login” lleva 3 semanas en “In Progress”, hay un problema. El tablero **hace visible lo que de otro modo sería invisible**.



Métricas del tablero: Lead time y Throughput

El tablero no solo organiza: también **mide**. Dos métricas sencillas:

- **Lead time:** días desde que se crea una tarea hasta que se completa. Si el promedio es 3 días y una tarea lleva 7, hay un bloqueo
- **Throughput:** tareas completadas por semana. Si baja de una semana a otra, algo está pasando

No necesitas herramientas complejas: basta con mirar el tablero y contar cuántas tarjetas pasan a “Done” cada semana.

Aclaración: “El tablero es solo para gestión”

- ▶ 🤔 ¿El tablero es solo una lista de tareas?

Mini-chequeo

- ▶ ¿Cuáles son las 3 columnas básicas de un tablero Kanban?

- ▶ ¿Qué es un WIP limit?

- ▶ ¿Cuándo se mueve una tarjeta a “Done”?

- ▶ ¿Qué son el lead time y el throughput?

- ▶ ¿Qué herramienta es mejor: Trello o GitHub Projects?

✅ Resumen en 3 frases

1. Un tablero Kanban tiene 3 columnas: **To Do**, **In Progress**, **Done** y visualiza todo el trabajo del equipo.
2. Usa **WIP limits** (máximo 1-2 tareas por persona) para evitar empezar muchas cosas sin terminar.
3. El tablero es una **herramienta de comunicación** (actualízalo cada día) y sus métricas (**lead time** y **throughput**) detectan bloqueos antes de que se conviertan en crisis.

Guia Didactica Proyecto 1 Uds U1 2 Scrum Lite 05 User Stories

 **Estás en:**  **UD 1.2 · SCRUM Lite** → [05 · User Stories](#)

 *Las user stories convierten requisitos en lenguaje humano*

“Necesito un CRUD” no es un requisito. Es un和技术 jerga que no dice nada sobre **para qué** sirve ni **quién** lo necesita. Las user stories obligan a pensar en el **usuario** y en el **valor** que aporta cada feature.

La idea en una frase

Una user story describe una funcionalidad desde el punto de vista del usuario: quién la necesita, qué quiere hacer y por qué.

Si no puedes explicar para qué sirve una feature en una frase, no la entiendes lo suficiente.

El formato clásico

```
Como [tipo de usuario]
quiero [hacer algo]
para [obtener un beneficio]
```

Ejemplo: gestor de tareas

User Story	Valor
Como alumno , quiero añadir tareas con fecha límite para no olvidarme de entregar trabajos	Organización personal
Como profesor , quiero ver las tareas de mis alumnos para saber quién va retrasado	Supervisión
Como alumno , quiero marcar tareas como hechas para ver mi progreso	Motivación

¿Por qué este formato?

El formato “Como... quiero... para que...” obliga a pensar en 3 cosas:

1. **¿Quién lo necesita?** (no todos los usuarios son iguales)
2. **¿Qué quiere hacer?** (la funcionalidad concreta)
3. **¿Para qué?** (el valor real)

Las 3 capas de una user story

INVEST

Una buena user story debe ser:

Letra	Significado	Ejemplo
Independiente	No depende de otras stories para completarse	“Añadir tarea” no depende de “Editar tarea”
Negotiable	Se puede discutir cómo implementarla	“Login” puede ser con email o con usuario
Valuable	Aporta valor al usuario	“Exportar a PDF” sí, “cambiar color de fondo” probablemente no
Estimable	Puedes estimar cuánto cuesta	Si no puedes estimarla, es demasiado grande
Small	Pequeña enough para terminarla en un sprint	Si dura más de 2 días, divídela
Testable	Puedes verificar que funciona	“Funciona el login” se puede probar

Criterios de aceptación

Las user stories necesitan **criterios de aceptación**: condiciones que deben cumplirse para considerar la story “terminada”.

Formato

```
Criterio de aceptación:  
- Dado que [contexto]  
- Cuando [acción]  
- Entonces [resultado]
```

Ejemplo: “Añadir tarea”

```
Criterios de aceptación:  
  
DADO que estoy en la pantalla de tareas  
CUANDO escribo el nombre de la tarea y pulso "Añadir"  
ENTONCES la tarea aparece en la lista  
  
DADO que añado una tarea sin nombre  
CUANDO pulso "Añadir"  
ENTONCES aparece un error: "El nombre es obligatorio"  
  
DADO que añado una tarea con fecha límite  
CUANDO la fecha es anterior a hoy  
ENTONCES aparece un warning: "La fecha es en el pasado"
```

Por qué importan los criterios de aceptación

Sin criterios de aceptación, cada persona interpreta la story de forma diferente:

Sin criterios	Con criterios
“Hacer el login” → Ana hace login con usuario, Pedro quiere login con email	“Login con email y contraseña” → todos saben exactamente qué hay que hacer
“Funciona” → ¿para quién? ¿en qué navegador?	“Funciona en Chrome, Firefox y Safari” → claro y testeable

User Story	Requisito técnico
“Como alumno, quiero añadir tareas para organizarme”	“Implementar endpoint POST /tareas con validación”
“Como profesor, quiero ver tareas de alumnos”	“Implementar endpoint GET /tareas?alumno=xxx”
“Como alumno, quiero marcar tareas hechas”	“Implementar endpoint PATCH /tareas/:id/completar”

Regla: Primero las user stories, luego los requisitos técnicos. Las stories son el “qué” y el “por qué”. Los requisitos técnicos son el “cómo”.

Comparativa: user story bien escrita vs mal escrita

Mal escrita

Como usuario, quiero un CRUD de tareas para gestionarlas. (¿Quién es el usuario? ¿Qué es ‘gestionar’? ¿Para qué sirve?)

Bien escrita

Como alumno, quiero añadir tareas con nombre y fecha límite para no olvidarme de entregar trabajos. (Claro, concreto, con valor)

Ejemplos de user stories para tu proyecto

Gestor de tareas

#	User Story	Prioridad
USo1	Como alumno, quiero crear tareas con nombre y fecha para organizarme	Must
USo2	Como alumno, quiero marcar tareas como hechas para ver mi progreso	Must
USo3	Como alumno, quiero filtrar tareas por fecha para ver las urgentes	Should
USo4	Como alumno, quiero editar tareas para corregir errores	Should
USo5	Como alumno, quiero eliminar tareas que ya no necesito	Could
USo6	Como alumno, quiero exportar mis tareas a PDF para imprimirlas	Could
USo7	Como alumno, quiero recibir notificaciones antes de la fecha límite	Won't

Portal de notas

#	User Story	Prioridad
USo1	Como alumno, quiero ver mis notas de cada asignatura	Must
USo2	Como profesor, quiero introducir notas de mis alumnos	Must
USo3	Como alumno, quiero ver la media de mis notas	Should
USo4	Como profesor, quiero importar notas desde un CSV	Should
USo5	Como alumno, quiero ver gráficos de mi evolución	Could
USo6	Como padre, quiero ver las notas de mi hijo	Won't

Aclaración: “Las user stories son para empresas”

► 🤔 ¿Las user stories solo sirven en empresas grandes?

Mini-chequeo

► ¿Cuál es el formato de una user story?

► ¿Qué son los criterios de aceptación?

► ¿Cuál es la diferencia entre una user story y un requisito técnico?

► ¿Cuántas user stories debería tener tu proyecto?

✓ Resumen en 3 frases

1. Las user stories describen funcionalidades desde el punto de vista del usuario: **“Como... quiero... para que...”**.
2. Los **criterios de aceptación** definen cuándo una story está terminada (formato Dado/Cuando/Entonces).
3. Primero las stories (el “qué”), luego los requisitos técnicos (el “cómo”). No al revés.

 Volver al [índice de la unidad](#) · Anterior: [04 — Tablero Kanban](#) · Siguiente: [06 — Estimación](#)

Guia Didactica Proyecto 1 Uds U1 2 Scrum Lite 06 Estimacion

 Estás en:  **UD 1.2 · SCRUM Lite** → 06 · Estimación

 *Estimar no es adivinar, es pensar antes de empezar*

“¿Cuánto te cuesta esa feature?” es la pregunta más importante de un sprint. Si no puedes estimar, no puedes planificar. Si no puedes planificar, vas a trabajar a ciegas y seguramente no termines a tiempo.

La idea en una frase

Estimar es responder “cuánto esfuerzo cuesta” antes de empezar, para poder planificar y evitar sorpresas.

No necesitas ser exacto. Necesitas ser **realista**.

¿Por qué estimar?

Sin estimación	Con estimación
“¿Qué hacemos esta semana?” → “No sé, ya veremos”	“Esta semana hacemos login (2 días), CRUD (2 días) y testing (1 día)”
“Llevamos 3 semanas y no sabemos cuánto queda”	“Llevamos 6 de 10 story points, vamos por buen camino”
“El profe pregunta cuándo estará listo →”No sé”	“El viernes que viene tenemos el MVP funcionando”

Método 1: T-shirt sizes

El método más simple. En vez de estimar en horas o días, usas tallas de camiseta:

Talla	Significado	Equivalencia aproximada
XS	Trivial	< 1 hora
S	Pequeño	1-2 horas
M	Mediano	4-8 horas (1 día)
L	Grande	2-3 días
XL	Muy grande	> 3 días (¡dividir!)

Ejemplo

Feature	Talla	Responsable
Añadir tarea	S	Ana
Login básico	M	Pedro
Filtros	S	Luis
Exportar PDF	L	Ana
Modo oscuro	M	Pedro

Regla: Si algo es **XL**, divídelo en 2-3 features más pequeñas. “Hacer el login completo” puede ser “Formulario de login” (M) + “Conectar con backend” (M) + “Recordar sesión” (S).

Método 2: Story Points

Los **story points** son una medida relativa de esfuerzo. No son horas ni días: son una comparación entre features.

La escala Fibonacci

1, 2, 3, 5, 8, 13, 21...

¿Por qué Fibonacci? Porque a medida que una feature es más grande, la incertidumbre crece. No tiene sentido distinguir entre “13” y “14”: ambas son “bastante grande”.

1. El equipo elige una feature como referencia (ej: “añadir tarea” = 2 points)
2. Para cada feature nueva, se pregunta: “¿Es más grande, igual o más pequeña que la de referencia?”
3. Se asigna el número más cercano

Feature	Referencia	Story Points
Añadir tarea	Referencia	2
Editar tarea	Un poco más que añadir	3
Login	Más que editar	5
CRUD completo	Mucho más	8
Exportar PDF	Bastante complejo	5

Ventaja de los story points

No necesitas saber **cuánto tiempo** tarda cada feature. Solo necesitas saber **qué es más grande** que qué. Eso es más fácil de estimar, especialmente sin experiencia.

Método 3: Planning Poker

El **Planning Poker** es una técnica para que el equipo estime **de forma consensuada**.

Cómo funciona

1. Cada persona tiene tarjetas con números (1, 2, 3, 5, 8, 13)
2. Se presenta una feature
3. Cada persona piensa cuánto cuesta
4. Al mismo tiempo, todos muestran su tarjeta
5. Si hay分歧 (ej: uno pone 3 y otro pone 8), se discute por qué
6. Se repite hasta llegar a un consenso

Ejemplo de Planning Poker

Feature	Ana	Pedro	Luis	Consenso
Login	5	5	8	5 (Luis ajusta)
CRUD tareas	3	3	3	3 (unanimidad)
Filtros	2	3	2	2 (Pedro ajusta)

¿Por qué funciona?

- Obliga a que **todos piensen** antes de estimar
- Detecta **malentendidos** (“¿Por qué crees que es 8? Yo creo que es 5”)
- Crea **consenso** sin imponer la opinión de uno

Comparativa: los 3 métodos

T-shirt sizes

XS, S, M, L, XL. El más rápido y simple. Ideal para empezar. No da un número exacto pero es suficiente para planificar.

Story Points

1, 2, 3, 5, 8, 13. Más preciso que T-shirts pero requiere una referencia. Ideal cuando ya tienes un sprint hecho.

Planning Poker

El más preciso pero el más lento. Obliga a que todos participen. Ideal para features complejas donde hay分歧.

La regla del “por 1.5”

Estimación inicial	Realidad (×1.5)
2 horas	3 horas
1 día	1.5 días
3 días	4.5 días

¿Por qué? Porque siempre hay cosas que no preves: - Bugs que no esperabas - Integración con código existente - Dudas que requieren investigación - Interrupciones

Aclaración: “Estimar es perder el tiempo”

► ⚠️ ¿Para qué estimar si nunca se cumple?

Ejercicio práctico

Toma las 5 features de tu proyecto y estímalas con los 3 métodos:

Feature	T-shirt	Story Points	Horas (×1.5)
Feature 1			
Feature 2			
Feature 3			
Feature 4			
Feature 5			

Si consigues estimar las 5 en 5 minutos → has entendido la unidad.

► ¿Cuáles son los 3 métodos de estimación?

► ¿Por qué se usa la secuencia de Fibonacci para story points?

► ¿Qué hacer si una feature es XL o > 13 points?

► ¿Cuál es la regla del “por 1.5”?

✓ Resumen en 3 frases

1. Estimar es **pensar en el esfuerzo** antes de empezar, no dar un número exacto.
2. Usa **T-shirt sizes** para empezar rápido y **Story Points** cuando tengas más experiencia.
3. Si una feature es XL o > 13 points, **divídela** en partes más pequeñas.

 [Volver al índice de la unidad](#) · Anterior: [05 — User Stories](#) · Siguiente: [07 — Errores comunes](#)

Guia Didactica Proyecto 1 Uds U1 2 Scrum Lite 07 Errores Scrum

 **Estás en:**  **UD 1.2 · SCRUM Lite** → [07 · Errores comunes](#)

 *SCRUM no funciona si no lo haces bien*

SCRUM es un framework poderoso, pero es fácil de hacer mal. Cada año, cientos de equipos de FP cometen los mismos errores. Conocerlos **antes de empezar** te ahorra

La idea en una frase

Los errores más comunes de SCRUM no son técnicos: son de actitud, de no seguir el proceso y de confundir SCRUM con “reuniones”.

El 80% de los fracasos de SCRUM en el aula se reducen a estos 6 errores.

Los 6 pecados de SCRUM en el aula

Pecado 1: “SCRUM es demasiado para nosotros”

El error: “Somos 3 personas, no necesitamos todo eso. Mejor programamos directamente.”

Consecuencia: Trabajáis sin plan, sin roles, sin reuniones. Cuando surge un problema (que siempre surge), no tenéis ni idea de por qué ni cómo 解決arlo.

Solución: No necesitas SCRUM completo. Usa **SCRUM Lite**: planning de 30 min, daily de 15 min, review y retrospectiva al final del sprint. Son 1 hora por semana de inversión que te ahorra días de confusión.

Pecado 2: “Las reuniones son una pérdida de tiempo”

El error: “¿Para qué nos reunimos si podemos programar? Las reuniones no producen código.”

Consecuencia: Cada uno trabaja por su cuenta. No sabéis qué hace el otro. Las features no se integran. El viernes descubrís que tenéis 3 versiones diferentes del mismo módulo.

Solución: Las reuniones son **cortas y concretas**. La daily son 15 minutos. El planning son 30 minutos. Si una reunión se alarga, es porque no tenéis un moderador o no seguís un formato.

Pecado 3: “No necesitamos tablero”

El error: “Me lo apunto en mi cuaderno y ya está. ¿Para qué un tablero?”

Consecuencia: Nadie sabe qué está haciendo el otro. El profe no puede ver el estado del proyecto. Cuando alguien se atasca, nadie se entera hasta que es demasiado tarde.

Solución: Un tablero Trello o GitHub Projects cuesta 5 minutos en crearlo y 2 minutos en actualizarlo cada día. Es la herramienta de comunicación más barata que existe.

Pecado 4: “La retrospectiva es para quejarse”

El error: “En la retrospectiva nos ponemos a hablar de lo mal que todo va y no se soluciona nada.”

Consecuencia: La retrospectiva se convierte en una sesión de quejas. Nadie propone mejoras concretas. El equipo se desmoraliza.

Solución: La retrospectiva tiene 3 preguntas claras: ¿qué salió bien? ¿qué salió mal? ¿qué mejoramos? Cada respuesta debe tener **una acción concreta**. Si no hay acción, no hay retrospectiva.

Pecado 5: “El Sprint Backlog es opcional”

El error: “Sabemos lo que hay que hacer, no hace falta escribirlo.”

Consecuencia: No podéis estimar, no podéis ver el progreso, no podéis saber si vais bien o mal. El sprint se convierte en “lo que salga”.

Solución: El Sprint Backlog es la **lista de tareas del sprint**. Escríbelo en el tablero. Actualízalo cada día. Si no lo haces, estás trabajando sin brújula.

Pecado 6: “No hacemos retrospectiva porque vamos bien”

El error: “Si todo va bien, ¿para qué nos reunimos a hablar?”

Consecuencia: Las cosas que van bien dejan de ir bien porque no se repiten. Los problemas pequeños se convierten en grandes porque no se detectan a tiempo.

Solución: La retrospectiva **siempre se hace**, tanto si vas bien como si vas mal. Si vas bien, descubres **por qué** vas bien para seguir haciéndolo. Si vas mal, descubres **qué cambiar**.

Caso práctico: los 6 errores en acción

El equipo que no hizo SCRUM

Carlos, María y Pablo eran un equipo de 3 personas. Empezaron el proyecto con muchas ganas pero decidieron que SCRUM era “demasiado formal” para ellos.

Semana 1: Cada uno empezó a programar lo que le parecía. No se reunieron. No había tablero.

Semana 2: Carlos hizo un login. María hizo un CRUD. Pablo hizo una base de datos. Pero el login no conectaba con la BD de Pablo, y el CRUD de María usaba una estructura diferente.

Semana 3: Se dieron cuenta de que tenían 3 módulos que no se hablaban entre sí. Empezaron a arreglarlo. No había documentación de lo que habían hecho.

Semana 4: El profe preguntó “¿cómo vamos?” y nadie pudo responder. No había tablero, no había plan, no había nada que mostrar.

Semana 5: Entraron en pánico. Empezaron a hacer horas extra. Las integraciones eran un caos.

Semana 6: Entregaron un proyecto a medias. Login funcionando, CRUD a medias, BD sin conectar. Suspenso.

¿Qué salió mal? Los 6 errores: no quisieron SCRUM, no se reunieron, no usaron tablero, no planificaron, no tuvieron retrospectiva y no documentaron.

¿Qué debería haber hecho? SCRUM Lite: planning de 30 min el lunes, daily de 15 min, tablero en Trello, retrospectiva de 15 min el viernes. 1 hora por semana que les habría ahorrado 5 semanas de confusión.

La tabla de los errores

Error	Consecuencia	Solución
“SCRUM es mucho”	Sin estructura, sin control	SCRUM Lite: 1 h/semana
“Reuniones pierden tiempo”	Trabajo aislado, sin integración	Daily 15 min, planning 30 min
“No necesito tablero”	Nadie sabe qué hace el otro	Trello o GitHub Projects
“Retrospectiva es quejarse”	Sin mejoras concretas	Acción por cada problema
“Sprint Backlog opcional”	Sin plan, sin visibilidad	Tablero actualizado cada día
“No hacemos retro si va bien”	Los buenos hábitos desaparecen	Siempre retro, bien o mal
132 / 743		

Aclaración: “SCRUM no es para todo”

► 🤔 ¿Debemos hacer SCRUM siempre?

Mini-chequeo

► ¿Cuáles son los 6 errores más comunes de SCRUM en el aula?

► ¿Cuánto tiempo se invierte en reuniones SCRUM por semana?

► ¿Por qué la retrospectiva se hace siempre, incluso si va bien?

✅ Resumen en 3 frases

1. Los 6 errores más comunes son: **rechazar SCRUM, odiar las reuniones, no usar tablero, hacer retrospectivas sin acciones, no planificar el sprint y saltarse la retro cuando va bien.**
2. SCRUM Lite invierte **1 hora por semana** y te ahorra días de confusión.
3. La retrospectiva **siempre se hace**: si vas bien, para seguir haciéndolo; si vas mal, para cambiar.

 Volver al [índice de la unidad](#) · Anterior: [06 — Estimación](#) · Siguiente: [08 — Template Sprint](#)

📖 **Estás en:** 📋 **UD 1.2 • SCRUM Lite** → o8 • Template Sprint

📖 *Las plantillas te ahorran tiempo y te obligan a pensar*

No reinventes la rueda cada sprint. Usa estas plantillas como punto de partida y adáptalas a tu equipo. Lo importante no es la plantilla: es **llenarla**.

📌 La idea en una frase

Una plantilla bien diseñada te obliga a pensar en lo que importa antes de empezar a programar.

Si una plantilla te parece inútil, no la estás usando bien.

Plantilla 1: Sprint Planning

Copia esta tabla cada lunes por la mañana y rellénala en 30 minutos.

Datos del sprint

Campo	Valor
Sprint #	
Fecha inicio	
Fecha fin	
Objetivo del sprint	

#	User Story	Responsable	Estimación	Estado
1				Pendiente
2				Pendiente
3				Pendiente
4				Pendiente
5				Pendiente

Riesgos identificados

Riesgo	Probabilidad	Impacto	Mitigación
	Alta/Media/Baja	Alto/Medio/Bajo	

Criterio de éxito del sprint

¿Cómo sabemos que este sprint ha sido exitoso?

[Escribe aquí el criterio de éxito]

Plantilla 2: Daily Scrum Checklist

Imprime o copia esta lista para cada día de la semana.

Lunes

- ☐ Sprint Planning completado
- ☐ Tareas asignadas en el tablero
- ☐ Daily Scrum realizada (15 min)
- ☐ Tablero actualizado

Martes

- ☐ Daily Scrum realizada (15 min)
- ☐ Tablero actualizado

- ☐ ¿Alguien tiene bloqueos?

Miércoles

- ☐ Daily Scrum realizada (15 min)
- ☐ Tablero actualizado
- ☐ ¿Vamos en camino para el objetivo del sprint?

Jueves

- ☐ Daily Scrum realizada (15 min)
- ☐ Tablero actualizado
- ☐ Preparar demo para la review

Viernes

- ☐ Daily Scrum realizada (15 min)
- ☐ Sprint Review (30 min)
- ☐ Sprint Retrospective (15 min)
- ☐ Tablero limpio para el siguiente sprint

Plantilla 3: Retrospectiva 3Ls

Copia esta tabla cada viernes y rellénala con tu equipo.

¿Qué salió bien? (Liked)

Acción concreta	Para qué sirve	¿Seguimos haciéndolo?
		Sí / No

¿Qué salió mal? (Learned)

Problema	Causa raíz	Acción correctiva

¿Qué podemos mejorar? (Longed for)

Mejora propuesta	Responsable	Cuándo implementarla
		Sprint siguiente / Ahora

Acciones del sprint siguiente

#	Acción	Responsable	Estado
1			
2			
3			

Plantilla 4: Retrospectiva 4Ls (alternativa)

Si la de 3Ls se queda corta, usa esta versión extendida.

Letra	Pregunta	Respuesta
Liked	¿Qué nos gustó del sprint?	
Learned	¿Qué aprendimos?	
Lacked	¿Qué nos faltó?	
Longed for	¿Qué echamos en falta?	

Plantilla 5: Burndown Chart simplificado

Un **burndown chart** muestra cuánto trabajo queda por hacer día a día. Es la forma más visual de saber si vais por buen camino.

Tabla de progreso

Día	Tareas totales	Tareas hechas	Tareas pendientes
Lunes		0	
Martes			
Miércoles			
Jueves			
Viernes			

Cómo interpretarlo

- **Línea recta hacia abajo:** vais bien
- **Línea plana:** no habéis avanzado (problema)
- **Línea que sube:** habéis añadido trabajo (mal signo)

Plantilla 6: Definition of Done (DoD)

Define qué significa “hecho” para tu equipo. Copia esta lista y personalízala.

Para features

- ☐ El código compila sin errores
- ☐ Funciona en el navegador del profesor
- ☐ Tiene documentación básica
- ☐ Al menos otra persona lo ha probado
- ☐ Está en la rama principal
- ☐ No tiene errores conocidos

Para el sprint

- ☐ Todas las features del sprint están en Done
- ☐ El tablero está actualizado
- ☐ Se hizo la retrospectiva
- ☐ Las acciones correctivas están apuntadas
- ☐ El profe ha visto la demo

Aclaración: “Las plantillas son burocracia”

► ⚠ ¿Para qué tantas plantillas? No es más fácil programar directamente?

Mini-chequeo

► ¿Cuáles son las 6 plantillas de SCRUM Lite?

► ¿Cuánto tiempo se tarda en rellenar el Sprint Planning?

► ¿Qué es un burndown chart?

✓ Resumen en 3 frases

1. Las plantillas te obligan a **pensar antes de programar**: planning (30 min), daily (15 min), retrospectiva (15 min).
2. La **Definition of Done** define qué significa “hecho” para evitar malentendidos.
3. Un **burndown chart** simplificado te dice si vais por buen camino con solo mirar la tabla.

 Volver al [índice de la unidad](#) · Anterior: [07 — Errores comunes](#) · Siguiente: [09 — Cierre](#)

 Ya sabes qué es SCRUM. Ahora toca usarlo.

Has recorrido toda la unidad: desde qué es el enfoque ágil hasta cómo planificar sprints con plantillas. Ahora toca **verificar que lo entiendes** y **prepararte para la siguiente unidad**.

La idea en una frase

SCRUM Lite es trabajar en ciclos cortos con objetivos claros, reuniones rápidas y un tablero visible.

Si recuerdas esto, has entendido la unidad.

Mini-chequeo final

Responde estas preguntas antes de seguir:

Preguntas de comprensión

▶ ¿Cuál es la diferencia entre Waterfall y Agile?

▶ ¿Cuáles son los 3 roles de SCRUM?

▶ ¿Cuánto dura un sprint recomendado para tu proyecto?

▶ ¿Qué es un tablero Kanban y por qué es importante?

▶ ¿Cuál es el formato de una user story?

► ¿Qué hacer si una feature es XL en estimación?

► ¿Cuánto tiempo se invierte en reuniones SCRUM por semana?

Ejercicio práctico

En 15 minutos, prepara tu primer sprint:

1. Crea un tablero en Trello o GitHub Projects con las columnas To Do, In Progress, Done
2. Añade 3-5 user stories de tu proyecto como tarjetas en “To Do”
3. Estima cada story con T-shirt sizes (S, M, L)
4. Asigna responsables
5. Define el objetivo del sprint

Si consigues tener un tablero listo con 3-5 tarjetas → has entendido la unidad.



Tabla resumen de la unidad

Concepto	Idea clave
Ágil	Trabajar en ciclos cortos, entregar valor, adaptarse al cambio
SCRUM	Framework con roles, ceremonias y artefactos
Sprint	Ciclo de 1-2 semanas con objetivo y entrega
Kanban	Tablero con To Do → In Progress → Done
User Stories	“Como... quiero... para que...” con criterios de aceptación
Estimación	T-shirt sizes, Story Points, Planning Poker
Errores	Los 6 pecados: rechazar SCRUM, odiar reuniones, no usar tablero...
Templates	Planning, daily, retrospectiva, burndown, DoD

Término	Definición
Ágil	Enfoque de desarrollo en ciclos cortos con feedback continuo
SCRUM	Framework ágil con roles, ceremonias y artefactos definidos
Sprint	Período de tiempo (1-2 semanas) con objetivo y entrega
Product Owner	Rol que define qué se hace y prioriza el trabajo
Scrum Master	Rol que facilita el proceso y quita impedimentos
Daily Scrum	Reunión diaria de 15 minutos (¿qué hice? ¿qué haré? ¿qué bloquea?)
Sprint Planning	Reunión para decidir qué se hace en el sprint
Sprint Review	Reunión para mostrar lo que se ha hecho
Retrospectiva	Reunión para mejorar cómo se trabaja
Tablero Kanban	Herramienta visual con columnas To Do, In Progress, Done
WIP	Work In Progress — máximo de tareas simultáneas
User Story	Descripción de una funcionalidad desde el punto de vista del usuario
Story Points	Medida relativa de esfuerzo (escala Fibonacci: 1, 2, 3, 5, 8, 13)
Planning Poker	Técnica de estimación consensuada en equipo
T-shirt Sizes	Método de estimación con tallas: XS, S, M, L, XL
Burndown Chart	Gráfico que muestra el trabajo restante día a día
Definition of Done	Criterios que deben cumplirse para considerar una tarea terminada
Backlog	Lista de todas las features pendientes, ordenadas por prioridad
Increment	Resultado tangible del sprint: software funcionando

Unidad	Qué aprenderás	Cómo se conecta
1.3 Requisitos	Cómo documentar qué hace tu app	Ampliarás las user stories del sprint
1.4 Diseño	Cómo diseñar la interfaz	Diseñarás lo que planificaste en el sprint
1.5 Comunicación	Cómo presentar tu proyecto	Mostrarás el resultado de tus sprints
1.7 Propuesta	Cómo escribir la propuesta final	El historial de sprints documenta tu progreso

Consejo para el siguiente punto

Si todavía no tienes claro cómo organizar tu proyecto:

1. Crea un tablero en Trello ahora mismo (5 minutos)
2. Añade las 5 features principales de tu proyecto
3. Estímalas con T-shirt sizes
4. Elige las 3 que entran en el primer sprint
5. Empieza el lunes con el planning

No necesitas SCRUM perfecto. Necesitas empezar.

Resumen en 3 frases

1. SCRUM Lite es trabajar en **sprints de 1 semana** con planning, daily y retrospectiva: **1 hora por semana** de inversión.
2. Un **tablero Kanban** visualiza todo el trabajo y hace visible el progreso y los bloqueos.
3. Las **user stories** y la **estimación** te obligan a pensar en el usuario y en el esfuerzo antes de programar.

Guia Didactica Proyecto 1 Uds U1 3 Requisitos 01 Que Son Requisitos

 **Estás en:**  **UD 1.3 · Requisitos** → 01 · ¿Qué son los requisitos?

 *Antes de programar, hay que saber qué construir*

El 40% de los problemas en proyectos de software se deben a requisitos mal definidos o incompletos. Los requisitos son el contrato entre **lo que el usuario necesita** y **lo que tú vas a construir**. Sin ellos, estás adivinando.

La idea en una frase

Un requisito es una declaración de lo que la app debe hacer (o cómo debe comportarse) para satisfacer una necesidad del usuario.

No es un deseo vago ni una idea suelta. Es una afirmación concreta, verificable y documentada.

¿Qué es un requisito?

En términos sencillos:

Requisito = Necesidad del usuario + Cómo se satisface + Cómo se verifica

Sin requisito	Con requisito
“Quiero una app de tareas”	“El alumno debe poder crear tareas con nombre, fecha límite y prioridad, y verlas ordenadas por fecha”
“Que sea rápida”	“La app debe cargar la lista de tareas en menos de 2 segundos en un móvil de gama baja”
“Que sea bonita”	“La interfaz debe usar colores del instituto y ser legible en pantalla de 5 pulgadas”

Los dos tipos de requisitos

Requisitos funcionales (RF)

¿Qué hace la app? describen funcionalidades concretas.

- “El usuario puede registrarse con email y contraseña”
- “El alumno puede crear, editar y eliminar tareas”
- “El profesor puede ver las tareas de todos sus alumnos”

Requisitos no funcionales (RNF)

¿Cómo se comporta la app? describen restricciones y cualidades.

- “La app debe cargar en menos de 2 segundos”
- “Los datos deben persistir localmente sin conexión”
- “La interfaz debe ser accesible para personas con discapacidad visual”

Comparativa: funcional vs no funcional

Requisito funcional

El alumno puede crear tareas con nombre y fecha. Describe UNA COSA que la app HACE.

La app carga en menos de 2 segundos. Describe CÓMO se hace, no qué se hace.

Característica	Requisito funcional	Requisito no funcional
Pregunta	¿Qué hace?	¿Cómo se comporta?
Ejemplo	“Crear tarea”	“Carga en 2s”
Verificable	Sí (funciona o no)	Sí (se mide con métrica)
Priorización	MoSCoW	MoSCoW

¿Por qué importan los requisitos?

Sin requisitos...

- Cada miembro del equipo imagina features diferentes
- Programas cosas que nadie pidió
- Descubres errores en la última semana
- El profesor no sabe qué esperar de ti

Con requisitos...

- Todos saben exactamente qué hay que construir
- Estimas esfuerzo real (porque sabes qué implica)
- Detectas conflictos al principio (no al final)
- Puedes priorizar y evitar features innecesarias

La pirámide de los requisitos

Visión	← Tu app en una frase
Requisitos no func.	← Qué debe hacer (RF + RNF)
Requisitos funcionales	← Funcionalidades concretas
User Stories	← "Como... quiero... para que..."
Criterios aceptación	← Dado/Cuando/Entonces

La visión es el faro. Los requisitos son el mapa. Las user stories son los pasos.

Aclaración: “Requisitos son solo para empresas grandes”

► 🤔 ¿Necesito documentar requisitos para un proyecto de 6 semanas?

Mini-chequeo

► ¿Cuál es la diferencia entre requisito funcional y no funcional?

► ¿Por qué son importantes los requisitos?

► ¿Necesito un documento de 50 páginas?

✅ Resumen en 3 frases

1. Un requisito es una declaración concreta de **qué debe hacer la app** o **cómo debe comportarse** para satisfacer una necesidad del usuario.

2. Los **requisitos funcionales** describen funcionalidades (“el usuario puede crear tareas”). Los **no funcionales** describen restricciones (“carga en 2 segundos”).
3. Documentar requisitos ligero (no burocrático) te ahorra semanas de trabajo desperdiciado y malentendidos en el equipo.

 Volver al [índice de la unidad](#) · Siguiendo: [02 — Requisitos funcionales](#)

Guia Didactica Proyecto 1 Uds U1 3 Requisitos 02

Requisitos Funcionales

 **Estás en:**  **UD 1.3 · Requisitos** → [02 · Requisitos funcionales](#)

 *Los requisitos funcionales son el ‘qué’ de tu app*

“Mi app tiene login, un CRUD y exporta a PDF.” Eso no son requisitos: es una lista de palabras sueltas. Los requisitos funcionales bien escritos dicen **exactamente qué debe hacer cada funcionalidad**, para quién y con qué condiciones.

La idea en una frase

Un requisito funcional describe una acción concreta que la app debe realizar: quién la ejecuta, qué hace y cuál es el resultado.

Si no puedes testear si se cumple, no es un requisito funcional.

Es una declaración que describe un **comportamiento observable** de la app. El usuario puede verlo, hacerlo o comprobarlo.

Estructura básica

[Actor] + [Acción] + [Objeto] + [Condición] + [Resultado]

Ejemplo desglosado

Actor	Acción	Objeto	Condición	Resultado
Alumno	Crea	Una tarea	Con nombre y fecha	La tarea aparece en su lista
Profesor	Ve	Las tareas de sus alumnos	Filtrando por alumno	Muestra solo las tareas de ese alumno
Alumno	Elimina	Una tarea	Que sea suya	La tarea desaparece de la lista

Cómo identificarlos

Método 1: Preguntar “¿Qué puede hacer el usuario?”

App de tareas escolares:

- ¿Qué puede hacer un ALUMNO?
 - Crear tareas, editarlas, eliminarlas, marcar como hechas, ver su lista
- ¿Qué puede hacer un PROFESOR?
 - Ver tareas de alumnos, asignar tareas, ver estadísticas
- ¿Qué puede hacer un PADRE?
 - Ver notas del hijo (si hay funcionalidad)

Método 2: Escuchar a los usuarios

Preguntas clave: - “¿Qué haces ahora con papel/cuaderno que podrías hacer en la app?” - “¿Qué información necesitas ver?” - “¿Qué errores quieres evitar?”

Método 3: Revisar requisitos previos

Si el profesor o el equipo ya tiene algo documentado, revísalo y tradúcelo a requisitos concretos.

Plantilla de requisito funcional

RF-[Número]: [Nombre descriptivo]

Descripción: [Qué hace la app]

Actor: [Quién lo ejecuta]

Precondiciones: [Qué debe ocurrir antes]

Flujo principal:

1. [Paso 1]
2. [Paso 2]
3. [Paso 3]

Flujo alternativo: [Qué pasa si algo cambia]

Postcondiciones: [Qué queda después]

Ejemplo completo: “Crear tarea”

RF-001: Crear tarea

Descripción: El alumno crea una nueva tarea escolar

Actor: Alumno

Precondiciones: Estoy registrado y en la pantalla principal

Flujo principal:

1. Pulso el botón "Nueva tarea"
2. Escribo el nombre de la tarea
3. Selecciono la fecha límite
4. Pulso "Guardar"
5. La tarea aparece en mi lista ordenada por fecha

Flujo alternativo:

- Si el nombre está vacío → muestra error "Nombre obligatorio"
- Si la fecha es pasada → muestra warning "La fecha ya pasó"

Postcondiciones: La tarea existe en la lista del alumno

Ejemplos de requisitos funcionales

App de tareas escolares

#	Requisito	Actor	Prioridad
RF-001	El alumno crea tareas con nombre, fecha y prioridad	Alumno	Must
RF-002	El alumno edita tareas existentes	Alumno	Must
RF-003	El alumno elimina tareas que ya no necesita	Alumno	Should
RF-004	El alumno marca tareas como completadas	Alumno	Must
RF-005	El alumno filtra tareas por fecha, prioridad o estado	Alumno	Should
RF-006	El profesor ve todas las tareas de sus alumnos	Profesor	Must
RF-007	La app exporta la lista de tareas a PDF	Alumno	Could
RF-008	La app muestra estadísticas de tareas completadas	Alumno	Could

Portal de notas

#	Requisito	Actor	Prioridad
RF-001	El profesor introduce notas de sus alumnos	Profesor	Must
RF-002	El alumno ve sus notas por asignatura	Alumno	Must
RF-003	El alumno ve la media de sus notas	Alumno	Should
RF-004	El profesor importa notas desde un CSV	Profesor	Could
RF-005	El alumno recibe alertas cuando saca un suspenso	Alumno	Could

Mal escrito

La app debe ser funcional. (¿Qué significa ‘funcional’? ¿Funcional para qué?)

RF-001: El alumno crea tareas con nombre y fecha límite. La tarea aparece en su lista ordenada por fecha.

Mal escrito	Bien escrito
“Tiene login”	“El usuario se registra con email y contraseña válidos. Recibe confirmación por email.”
“Se pueden ver estadísticas”	“El alumno ve un gráfico de barras con tareas completadas vs pendientes por semana”
“Funciona en móvil”	“La interfaz se adapta a pantallas de 320px a 428px de ancho”

Consejos para buenos requisitos funcionales

1. **Usa verbos de acción:** crear, editar, eliminar, ver, filtrar, exportar
2. **Sé específico:** no “el usuario puede gestionar tareas”, sino “el alumno crea, edita y elimina tareas”
3. **Incluye el actor:** ¿quién lo hace? No todos los usuarios hacen lo mismo
4. **Define el resultado:** ¿qué pasa después de la acción?
5. **Numéralos:** RF-001, RF-002... para poder referirte a ellos fácilmente

Aclaración: “¿Y si no sé qué requisitos poner?”

Mini-chequeo

► ¿Cuál es la estructura de un requisito funcional?

► ¿Cómo identifico los requisitos funcionales de mi app?

► ¿Cuántos requisitos funcionales debería tener mi proyecto?

✓ Resumen en 3 frases

1. Los requisitos funcionales describen **qué hace la app**: quién ejecuta la acción, qué hace y cuál es el resultado.
2. Usa la plantilla **Actor + Acción + Objeto + Condición + Resultado** para escribir requisitos claros y testables.
3. Si no puedes verificar que se cumple un requisito, **es demasiado vago**. Conviértelo en algo concreto.

 Volver al [índice de la unidad](#) · Anterior: [01 — ¿Qué son los requisitos?](#) · Siguiente: [03 — Requisitos no funcionales](#)

Guía Didáctica Proyecto 1 Uds U1 3 Requisitos 03 Requisitos No Funcionales

 Estás en:  **UD 1.3 · Requisitos** → 03 · Requisitos no funcionales

 *Los RNF son las reglas del juego que no se ven*

Un usuario nunca te pedirá “que cargue en 2 segundos”. Pero cuando tu app tarda 10 segundos, nadie la usa. Los requisitos no funcionales son las **restricciones invisibles** que definen si tu app es buena o insoportable.

La idea en una frase

Los requisitos no funcionales no dicen qué hace la app, sino CÓMO se comporta: qué tan rápido, qué tan segura, qué tan fácil de usar.

Son las “reglas del juego” que aplican a toda la app, no a una funcionalidad concreta.

¿Qué son los requisitos no funcionales?

Son restricciones que afectan a **toda la aplicación** o a componentes transversales. No describen una funcionalidad, sino una **cualidad del sistema**.

Requisito funcional: "El usuario crea una tarea"
Requisito no funcional: "La tarea se guarda en menos de 1 segundo"

Las 5 categorías principales de RNF

1. Rendimiento (Performance)

¿Qué tan rápido funciona la app?

Métrica	Ejemplo de RNF
Tiempo de carga	“La lista de tareas carga en menos de 2 segundos”
Tiempo de respuesta	“Al pulsar ‘Guardar’, la confirmación aparece en menos de 1 segundo”
Capacidad	“Soporta 100 usuarios simultáneos sin degradación”
Tamaño	“La app ocupa menos de 5MB de almacenamiento”

2. Usabilidad

¿Qué tan fácil es usar la app?

Métrica	Ejemplo de RNF
Aprendizaje	“Un nuevo usuario crea su primera tarea en menos de 2 minutos”
Accesibilidad	“Cumple WCAG 2.1 nivel AA (contraste, alt text, navegación por teclado)”
Consistencia	“Los botones de acción están siempre en la misma posición”
Adaptabilidad	“La interfaz se adapta a pantallas de 320px a 1920px”

3. Seguridad

¿Cómo protege los datos?

Métrica	Ejemplo de RNF
Autenticación	“Login con email y contraseña hasheada con bcrypt”
Autorización	“Solo el profesor puede ver las tareas de todos los alumnos”
Datos	“Las contraseñas nunca se almacenan en texto plano”
Sesiones	“La sesión expira tras 30 minutos de inactividad”

4. Fiabilidad

¿Qué tan estable es la app?

Métrica	Ejemplo de RNF
Disponibilidad	“Funciona offline (PWA) con datos cacheados”
Errores	“Si falla la conexión, muestra un mensaje claro y reintentar”
Datos	“Los datos no se pierden al cerrar el navegador (localStorage)”
Recuperación	“Si hay error, la app se recupera sin perder datos introducidos”

5. Compatibilidad

¿Dónde funciona la app?

Métrica	Ejemplo de RNF
Navegadores	“Funciona en Chrome 90+, Firefox 88+, Safari 14+, Edge 90+”
Dispositivos	“Funciona en móvil (iOS 14+, Android 10+), tablet y escritorio”
Resoluciones	“Diseño responsive desde 320px hasta 1920px”
Tecnologías	“Framework Vue 3, desplegado en GitHub Pages”

Plantilla de requisito no funcional

RNF-[Número]: [Nombre descriptivo] Categoría: [Rendimiento/Usabilidad/Seguridad/Fiabilidad/Compatibilidad] Descripción: [Qué restricción se aplica] Métrica: [Cómo se mide] Valor objetivo: [Cuánto debe valer] Verificación: [Cómo se comprueba]

Ejemplo completo

RNF-001: Tiempo de carga Categoría: Rendimiento Descripción: La lista de tareas debe cargar rápidamente Métrica: Tiempo desde que se pulsa "Mis tareas" hasta que se muestra la lista Valor objetivo: Menos de 2 segundos en un móvil de gama baja Verificación: Medir con DevTools > Performance en un Nexus 5X

Ejemplos de RNF para tu proyecto

#	RNF	Categoría	Métrica
RNF-001	La app carga en menos de 2 segundos	Rendimiento	Tiempo de carga
RNF-002	La interfaz es responsive (320px–1920px)	Usabilidad	Resolución
RNF-003	Los datos persisten sin conexión (PWA)	Fiabilidad	Disponibilidad
RNF-004	Cumple WCAG 2.1 nivel AA	Usabilidad	Accesibilidad
RNF-005	Funciona en Chrome, Firefox, Safari y Edge	Compatibilidad	Navegadores
RNF-006	Las contraseñas se hashean con bcrypt	Seguridad	Hashing

Comparativa: RNF bien escrito vs mal escrito

La app debe ser rápida. (¿Qué es ‘rápida’? ¿200ms? ¿5s?)

RNF-001: La lista de tareas carga en menos de 2 segundos en un móvil de gama baja (Nexus 5X). Se mide con DevTools.

Mal escrito	Bien escrito
“Que sea seguro”	“Las contraseñas se almacenan hasheadas con bcrypt (coste 12). Nunca en texto plano.”
“Que funcione en móvil”	“La interfaz se adapta a pantallas de 320px (iPhone SE) a 428px (iPhone 14 Plus)”
“Que sea fácil”	“Un nuevo usuario crea su primera tarea en menos de 2 minutos sin instrucciones”
“Que no se caiga”	“Si la conexión falla, la app muestra ‘Sin conexión’ y reintenta cada 5 segundos”

RNF y PWA

Si tu app es una PWA (Progressive Web App), estos son los RNF más importantes:

RNF PWA	Descripción
Offline	La app funciona sin conexión usando Service Worker
Cache	Los assets estáticos se cachean para carga rápida
Instalable	El usuario puede “instalarla” en su pantalla de inicio
Responsive	Se adapta a cualquier tamaño de pantalla
HTTPS	Obligatorio para Service Workers

Aclaración: “Los RNF son para empresas grandes”

► 🤔 ¿Necesito definir RNF para un proyecto escolar?

Mini-chequeo

► ¿Cuáles son las 5 categorías de RNF?

► ¿Cómo sé si un RNF es bueno o malo?

► ¿Cuántos RNF debería definir?

✅ Resumen en 3 frases

1. Los RNF no describen qué hace la app, sino **cómo se comporta**: rendimiento, usabilidad, seguridad, fiabilidad y compatibilidad.
2. Un buen RNF es **medible y verificable**: no “que sea rápido”, sino “que cargue en menos de 2 segundos”.
3. Para tu proyecto, define **3-5 RNF concretos** (responsive, offline, accesibilidad) que te toman 15 minutos.

 Volver al [índice de la unidad](#) · Anterior: [02 — Requisitos funcionales](#) · Siguiente: [04 — Historias de usuario](#)

📖 *Las historias de usuario convierten requisitos en lenguaje humano*

“Implementar endpoint POST /tarefas” no le dice nada a tu profe ni a tu compañero. Pero “Como alumno, quiero crear tareas para organizarme” lo entiende todo el mundo. Las historias de usuario son el puente entre **lo que el usuario necesita** y **lo que el equipo construye**.

📌 La idea en una frase

Una historia de usuario describe una funcionalidad desde el punto de vista de quien la usa: quién la necesita, qué quiere hacer y por qué le importa.

Si no puedes explicar el valor en una frase, la historia no está clara.

El formato clásico

```
Como [tipo de usuario]
quiero [hacer algo concreto]
para [obtener un beneficio]
```

Las 3 preguntas que obliga a responder

Pregunta	Parte de la historia	Por qué importa
¿Quién?	“Como [tipo de usuario]”	No todos los usuarios son iguales
¿Qué?	“Quiero [acción concreta]”	Define la funcionalidad exacta
¿Para qué?	“Para [beneficio]”	Justifica por qué merece el esfuerzo

Ejemplos de historias de usuario

App de tareas escolares

#	Historia de usuario	Valor
HU-001	Como alumno , quiero añadir tareas con nombre y fecha límite para no olvidarme de entregar trabajos	Organización personal
HU-002	Como alumno , quiero marcar tareas como hechas para ver mi progreso	Motivación
HU-003	Como alumno , quiero filtrar tareas por fecha para ver las urgentes primero	Eficiencia
HU-004	Como profesor , quiero ver las tareas de mis alumnos para saber quién va retrasado	Supervisión
HU-005	Como alumno , quiero editar tareas para corregir errores	Flexibilidad
HU-006	Como alumno , quiero eliminar tareas para limpiar mi lista	Organización
HU-007	Como alumno , quiero exportar mis tareas a PDF para imprimirlas	Accesibilidad offline

Portal de notas

#	Historia de usuario	Valor
HU-001	Como alumno , quiero ver mis notas por asignatura para saber mi nivel	Transparencia
HU-002	Como profesor , quiero introducir notas de mis alumnos para evaluar su progreso	Evaluación
HU-003	Como alumno , quiero ver la media de mis notas para saber cómo voy	Motivación
HU-004	Como profesor , quiero importar notas desde un CSV para ahorrar tiempo	Eficiencia

Una buena historia de usuario debe cumplir el acrónimo **INVEST**:

Letra	Significado	Significado	Ejemplo malo	Ejemplo bueno
I	Independiente	No depende de otras historias	“Crear usuario” antes de “Login”	Cada historia se puede implementar sola
N	Negociable	Se puede discutir el “cómo”	“Usar Vue 3 con Composition API”	“El usuario se registra” (el cómo se decide después)
V	Valuable	Aporta valor al usuario	“Añadir un botón decorativo”	“El alumno crea tareas para organizarse”
E	Estimable	Puedes estimar cuánto cuesta	“Hacer la app perfecta”	“El alumno filtra tareas por fecha” (2-3 días)
S	Small	Pequeña para un sprint	“Implementar todo el sistema de usuarios”	“El alumno crea una tarea con nombre y fecha” (1 día)
T	Testable	Puedes verificar que funciona	“Que sea fácil de usar”	“El alumno crea una tarea y aparece en su lista”

Historia de usuario vs requisito funcional

Aspecto	Historia de usuario	Requisito funcional
Formato	“Como... quiero... para que...”	“RF-001: El alumno crea tareas...”
Público	Equipo + usuarios	Analistas + desarrolladores
Detalle	Resumen + criterios de aceptación	Flujo detallado con pasos
Flexibilidad	Negociable	Fijo
Ejemplo	“Como alumno, quiero crear tareas para organizarme”	“RF-001: El alumno ingresa nombre, selecciona fecha y pulsa Guardar. La tarea aparece en la lista.”

Regla: Primero las historias (para entender el qué y el por qué), luego los requisitos funcionales (para definir el cómo exacto).

Comparativa: historia bien escrita vs mal escrita

Como usuario, quiero un CRUD de tareas para gestionarlas. (¿Quién es ‘el usuario’? ¿Qué es ‘gestionar’? ¿Qué valor tiene un CRUD?)

Como alumno, quiero crear tareas con nombre y fecha límite para no olvidarme de entregar trabajos. (Claro, concreto, con valor)

Mal escrita	Bien escrita
“Como admin, quiero gestionar usuarios”	“Como profesor, quiero ver las tareas de mis alumnos para saber quién va retrasado”
“Que tenga notificaciones”	“Como alumno, quiero recibir un aviso 24h antes de la fecha límite para no olvidarme”
“Funcionalidad de búsqueda”	“Como alumno, quiero buscar tareas por nombre para encontrar rápidamente una tarea concreta”

Aclaración: “Las historias de usuario son solo para empresas”

► 🤔 ¿Necesito escribir historias de usuario para mi proyecto de FP?

► ¿Cuál es el formato de una historia de usuario?

► ¿Qué significan las siglas INVEST?

► ¿Cuántas historias de usuario debería tener mi proyecto?

► ¿Cuál es la diferencia entre historia de usuario y requisito funcional?

✓ Resumen en 3 frases

1. Las historias de usuario usan el formato “**Como... quiero... para que...**” para describir funcionalidades desde el punto de vista del usuario.
2. Una buena historia cumple **INVEST**: Independiente, Negociable, Valuable, Estimable, Small y Testable.
3. Primero las historias (el qué y el por qué), luego los requisitos funcionales (el cómo exacto). No al revés.

 Volver al [índice de la unidad](#) · Anterior: [03 — Requisitos no funcionales](#) · Siguiente: [05 — Criterios de aceptación](#)

Guia Didactica Proyecto 1 Uds U1 3 Requisitos 05 Criterios Aceptacion

 Estás en:  **UD 1.3 · Requisitos** → [05 · Criterios de aceptación](#)

“Está hecho” es subjetivo. Para uno, “hecho” significa que compila. Para otro, significa que pasa tests. Los criterios de aceptación eliminan la ambigüedad: son **condiciones objetivas** que deben cumplirse para considerar una historia completa.

La idea en una frase

Un criterio de aceptación es una condición verificable que, cuando se cumple, garantiza que la historia de usuario aporta el valor prometido.

Si no puedes testear si se cumple, no es un criterio de aceptación.

¿Qué son los criterios de aceptación?

Son las **condiciones concretas** que deben darse para que una historia de usuario esté “terminada”. Responden a la pregunta: **¿cómo sé que esto funciona como debe?**

Sin criterios vs con criterios

Sin criterios	Con criterios
“Hacer el login” → cada uno interpreta algo diferente	“Login con email y contraseña. Si son correctos, redirige al dashboard. Si no, muestra error.”
“Funciona” → ¿para quién? ¿en qué navegador?	“Funciona en Chrome, Firefox y Safari. Se ve bien en móvil y escritorio.”
“Está terminado” → ¿según quién?	“Cumple los 3 criterios listados abajo.”

El formato Dado/Cuando/Entonces

Este formato viene de **BDD (Behavior-Driven Development)** y es el estándar para escribir criterios de aceptación:

```
DADO que [contexto o precondition]
CUANDO [acción del usuario]
ENTONCES [resultado esperado]
```

Variantes útiles

```
DADO que [contexto]
CUANDO [acción]
ENTONCES [resultado]

Y [otro resultado]

PERO SI [condición alternativa]
ENTONCES [otro resultado]
```

Ejemplos completos

Historia: “Como alumno, quiero crear tareas para organizarme”

Criterios de aceptación:

```
DADO que estoy en la pantalla de tareas
CUANDO pulso el botón "Nueva tarea"
ENTONCES se abre un formulario con campos nombre y fecha
```

```
DADO que el formulario está abierto
CUANDO escribo un nombre y selecciono una fecha futura
Y pulso "Guardar"
ENTONCES la tarea aparece en mi lista ordenada por fecha
```

```
DADO que el formulario está abierto
CUANDO pulso "Guardar" sin escribir nombre
ENTONCES aparece un error: "El nombre es obligatorio"
```

```
DADO que el formulario está abierto
CUANDO selecciono una fecha pasada
ENTONCES aparece un aviso: "La fecha ya pasó"
```

```
DADO que la tarea se ha creado correctamente
CUANDO vuelvo a abrir la app
ENTONCES la tarea sigue ahí (persiste)
```

Historia: “Como alumno, quiero marcar tareas como hechas”

Criterios de aceptación:

DADO que tengo una tarea pendiente en mi lista
CUANDO pulso el checkbox de la tarea
ENTONCES la tarea se marca como completada y se tacha visualmente

DADO que una tarea está marcada como hecha
CUANDO pulso el checkbox de nuevo
ENTONCES la tarea vuelve a estar pendiente

DADO que tengo 5 tareas y 3 están hechas
CUANDO miro el contador de progreso
ENTONCES muestra "3/5 completadas (60%) "

Los 5 tipos de criterios de aceptación

1. Happy path (camino feliz)

El escenario donde todo va bien:

DADO que estoy en la pantalla de login
CUANDO introduzco email y contraseña correctos
ENTONCES accedo al dashboard

2. Validación de errores

Qué pasa cuando algo va mal:

DADO que introduzco un email que no existe
CUANDO pulso "Iniciar sesión"
ENTONCES aparece: "Email o contraseña incorrectos"

3. Límites y restricciones

Los límites del sistema:

DADO que escribo un nombre de tarea
CUANDO el nombre tiene más de 100 caracteres
ENTONCES el campo no acepta más caracteres

4. Persistencia

Los datos sobreviven:

DADO que he creado 3 tareas
CUANDO cierro el navegador y lo vuelvo a abrir
ENTONCES las 3 tareas siguen en mi lista

5. Comportamiento responsive

La app se adapta:

DADO que abro la app en un móvil de 320px de ancho
CUANDO miro la lista de tareas
ENTONCES las tareas se muestran en una columna, legibles

Comparativa: criterio vago vs concreto

DADO que el usuario hace login, CUANDO mete sus datos, ENTONCES funciona. (¿Qué es ‘funciona’? ¿Qué datos?)

DADO que introduzco email ‘ana@test.com’ y contraseña ‘1234’, CUANDO pulso ‘Entrar’, ENTONCES accedo al dashboard en menos de 2 segundos.

Vago	Concreto
“Funciona el login”	“Con email y contraseña correctos, accedo al dashboard”
“Se ve bien en móvil”	“En pantallas de 320px, el texto es legible sin hacer zoom”
“La app es rápida”	“La lista carga en menos de 2 segundos en un Nexus 5X”
“Guarda los datos”	“Tras crear una tarea y cerrar la app, la tarea sigue al reabrirla”

Cuántos criterios por historia

Tamaño de la historia	Criterios recomendados
Simple (1 día)	2-3 criterios
Media (2-3 días)	3-5 criterios
Compleja (4-5 días)	5-7 criterios

Regla práctica: Si tienes más de 7 criterios, probablemente la historia es demasiado grande. Divídela.

Aclaración: “¿Y si no sé qué criterios poner?”

► 🤔 ¿Cómo sé si mis criterios de aceptación son suficientes?

Mini-chequeo

► ¿Cuál es el formato de un criterio de aceptación?

► ¿Cuántos criterios por historia?

► ¿Qué tipos de criterios debo incluir?

✅ Resumen en 3 frases

1. Los criterios de aceptación son **condiciones verificables** que definen cuándo una historia está terminada.

2. El formato estándar es **DADO** que [contexto], **CUANDO** [acción], **ENTONCES** [resultado].
3. Incluye siempre el **happy path**, los **errores** y la **persistencia** para que no quede ambigüedad.

 Volver al [índice de la unidad](#) · Anterior: [04 — Historias de usuario](#) · Siguiente: [06 — Priorización](#)

Guia Didactica Proyecto 1 Uds U1 3 Requisitos 06 Priorizacion

 **Estás en:**  **UD 1.3 · Requisitos** → [06 · Priorización](#)

 *No puedes hacer todo: hay que decidir qué importa más*

Tienes 6 semanas y 15 features en la lista. ¿Cuáles haces primero? Si no priorizas, acabarás haciendo features bonitas pero inútiles mientras olvidas las fundamentales. Priorizar es **decidir a qué le dices sí** y a qué le dices “no, ahora no”.

La idea en una frase

Priorizar es ordenar los requisitos por su importancia real, no por lo que suena bonito.

No todo es “Must”. Si todo es urgente, nada es urgente.

Método 1: MoSCoW

El método más usado y el más fácil de aplicar. Divide los requisitos en 4 categorías:

Las 4 categorías

Categoría	Significado	Pregunta clave	Ejemplo
Must	Obligatorio	“¿Sin esto, la app no funciona?”	Crear y ver tareas
Should	Importante	“¿Se puede sin esto, pero se nota?”	Filtrar tareas por fecha
Could	Deseable	“Estaría bien, pero no es vital”	Exportar a PDF
Won't	No ahora	“No entra en este proyecto”	Notificaciones push

Regla del 60/20/20

Una distribución saludable para tu proyecto:

Must:	60% de los requisitos (lo que ENTREGAS seguro)
Should:	20% de los requisitos (lo que intentas incluir)
Could:	20% de los requisitos (si sobra tiempo)
Won't:	0% (no los listes, son features futuras)

Ejemplo: app de tareas

#	Requisito	MoSCoW
HU-001	Crear tareas con nombre y fecha	Must
HU-002	Marcar tareas como hechas	Must
HU-003	Ver lista de tareas	Must
HU-004	Editar tareas	Should
HU-005	Filtrar por fecha	Should
HU-006	Eliminar tareas	Could
HU-007	Exportar a PDF	Could
HU-008	Notificaciones push	Won't

--

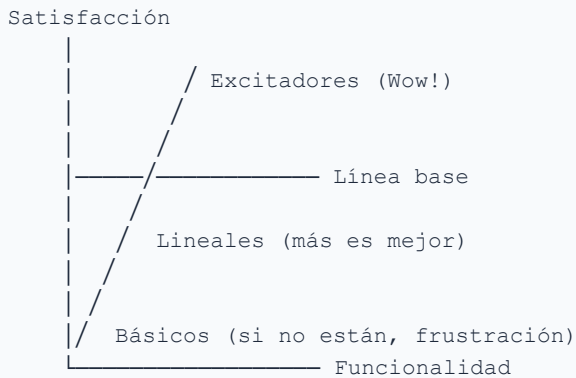
Método 2: Modelo Kano

El modelo Kano clasifica los requisitos según **cómo afectan a la satisfacción del usuario**. No todos los requisitos aportan la misma satisfacción.

Las 3 categorías de Kano

Tipo	Descripción	Ejemplo	Satisfacción si está
Básicos	Si no están, el usuario se va. Si están, no genera satisfacción	Login, crear tareas	Esperado (0 satisfacción extra)
Lineales	Cuanto mejor, más satisfecho. Relación lineal	Velocidad, filtros	Satisfecho (más es mejor)
Excitadores	Si no están, no pasa nada. Si están, generan satisfacción inesperada	Exportar PDF, estadísticas	De sorpresa (wow!)

La curva de Kano



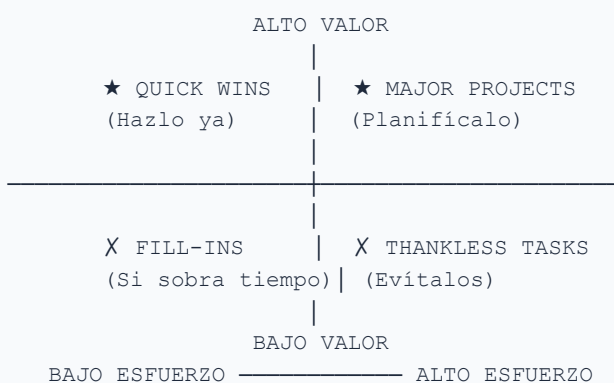
¿Cómo usar Kano en tu proyecto?

1. **Básicos:** Defínelos primero. Son los Must de MoSCoW.
2. **Lineales:** Priorízalos por impacto. Son los Should.
3. **Excitadores:** Añádelos si sobra tiempo. Son los Could.

Método 3: Matriz valor vs esfuerzo

Este método es **visual**: dibujas una matriz y sitúas cada requisito según su valor para el usuario y el esfuerzo para construirlo.

La matriz



Ejemplo: app de tareas

Requisito	Valor	Esfuerzo	Cuadrante	Acción
Crear tareas	Alto	Bajo	★ Quick Win	Hazlo primero
Marcar hechas	Alto	Bajo	★ Quick Win	Hazlo primero
Filtrar por fecha	Alto	Medio	★ Major Project	Planificalo
Exportar PDF	Medio	Alto	★ Major Project	Ver si vale la pena
Eliminar tareas	Bajo	Bajo	Fill-in	Si sobra tiempo
Notificaciones	Bajo	Alto	Thankless	No lo hagas

Caso práctico: priorizando un proyecto real

El equipo que priorizó mal

Laura, Diego y Ana tenían 4 semanas para entregar su app de notas.

Sin priorizar empezaron por: 1. Una pantalla de login bonita con animaciones 2. Un sistema de temas oscuro/claro 3. Un gráfico de evolución de notas 4. Estadísticas comparativas entre alumnos

Resultado: Llevaban 2 semanas y no podían ni introducir notas. El login funcionaba perfecto y el tema oscuro era precioso, pero la app no servía para nada.

Si hubieran priorizado con MoSCoW: - **Must:** Introducir notas, ver notas por alumno - **Should:** Media de notas, buscar alumno - **Could:** Gráficos, temas, animaciones

Habrían terminado la app funcional en 2 semanas y habrían añadido bonito en las 2 restantes.

Comparativa: los 3 métodos

Fácil y rápido. Clasifica en Must/Should/Could/Won't. Perfecto para tu primer proyecto.

Más sofisticado. Distingue básicos, lineales y excitadores. Útil para saber qué genera satisfacción real.

Visual y práctico. Si algo es alto valor y bajo esfuerzo, hazlo ya. Si es bajo valor y alto esfuerzo, ni lo mires.

Método	Ventaja	Cuándo usarlo
MoSCoW	Fácil de aplicar, todos lo entienden	Siempre (método base)
Kano	Entiende qué genera satisfacción real	Cuando quieres sorprender al usuario
Valor vs esfuerzo	Visual, eliminafeatures inútiles	Cuando tienes demasiadas opciones

Recomendación: Usa **MoSCoW como base** y complementa con la **matriz valor vs esfuerzo** para las features dudosas.

Aclaración: “¿Y si todo parece Must?”

► 🤔 Si todas las features parecen importantes, ¿cómo priorizo?

► ¿Qué significa cada categoría de MoSCoW?

► ¿Cuáles son las 3 categorías del modelo Kano?

► ¿Cómo uso la matriz valor vs esfuerzo?

✓ Resumen en 3 frases

1. **MoSCoW** clasifica en Must/Should/Could/Won't. Si todo es Must, nada es Must: sé honesto.
2. El **modelo Kano** distingue básicos (obligatorios), lineales (más es mejor) y excitadores (sorpresa positiva).
3. La **matriz valor vs esfuerzo** te dice qué hacer ya (Quick Win) y qué evitar (Thankless Task).

 Volver al [índice de la unidad](#) · Anterior: [05 — Criterios de aceptación](#) · Siguiente: [07 — Errores comunes](#)

Guia Didactica Proyecto 1 Uds U1 3 Requisitos 07 Errores Requisitos

 **Estás en:**  **UD 1.3 · Requisitos** → [07 · Errores comunes](#)

 *Los errores de requisitos se pagan caros*

Cada error de requisitos que pasas desapercibido se multiplica por 10 cuando lo implementas. Si lo detectas al programar, ^{176/743}te cuesta 1 hora. Si lo detectas al entregar, te

La idea en una frase

El error más caro no es un bug de código: es construir algo que nadie pidió o que no funciona como esperaba.

Conocer estos errores antes de empezar te ahorra semanas.

Los 7 pecados de los requisitos

Pecado 1: “El usuario puede hacer todo”

El error: Definir requisitos tan amplios que no dicen nada concreto.

Ejemplo de error	Versión corregida
“El usuario puede gestionar sus datos”	“El alumno crea, edita y elimina tareas con nombre y fecha”
“La app tiene funcionalidades de usuario”	“El alumno se registra con email y contraseña, y crea su primera tarea en menos de 2 minutos”
“Se pueden ver estadísticas”	“El alumno ve un gráfico de barras con tareas completadas por semana”

Solución: Usa verbos de acción concretos: crear, editar, eliminar, ver, filtrar, exportar. Nunca “gestionar” ni “administrar”.

Pecado 2: Requisitos no funcionales olvidados

El error: Definir solo funcionalidades y olvidar restricciones.

```
Todo el mundo define: "El alumno crea tareas"
Nadie define: "La app carga en menos de 2 segundos"
               "La interfaz funciona en móvil"
               "Los datos persisten sin conexión"
```

Solución: Después de listar los requisitos funcionales, pregunta: ¿Qué pasa si va lento? ¿Qué pasa si no hay internet? ¿Qué pasa si uso un móvil pequeño? Cada respuesta es un RNF.

Pecado 3: Historias de usuario vagas

El error: Historias sin actor concreto, sin valor claro o sin criterios de aceptación.

Historia vaga	Historia concreta
“Como usuario, quiero usar la app”	“Como alumno, quiero crear tareas con fecha para organizarme”
“Que tenga login”	“Como alumno, quiero registrarme con email y contraseña para acceder a mis tareas”
“Que se vea bien”	“Como alumno, quiero ver mis tareas en el móvil para comprobarlas en el bus”

Solución: Pregúntate: ¿Quién es el usuario concreto? ¿Qué quiere hacer exactamente? ¿Por qué le importa? Si no puedes responder, la historia no está lista.

Pecado 4: Confundir “cómo” con “qué”

El error: Escribir requisitos que definen la implementación en vez del comportamiento.

Cómo (mal)	Qué (bien)
“Usar Vue 3 con Composition API”	“La app carga la lista de tareas en menos de 2 segundos”
“Almacenar en IndexedDB”	“Los datos persisten sin conexión a internet”
“Usar Tailwind CSS”	“La interfaz es responsive y funciona en móvil y escritorio”

Solución: Los requisitos definen **qué** necesita el usuario y **cómo se comporta** la app. La tecnología es decisión del equipo, no del requisito.

Pecado 5: “Lo sabré cuando lo vea”

El error: No documentar requisitos y confiar en que el equipo “se entiende”.

Consecuencia: Cada persona imagina features diferentes. El alumno piensa en un gestor de tareas. El profe piensa en un portal de notas. El compañero piensa en algo con juegos. Cuando se juntan los trozos, no encajan nada.

Solución: Escribe los requisitos **antes de programar**. Una tarde de documentación te ahorra semanas de reescribir código.

Pecado 6: Todo es Must

El error: Clasificar todas las features como obligatorias.

Consecuencia: No puedes priorizar. Intentas hacer todo a la vez. No terminas nada. El sprint se llena de tareas a medias.

Solución: Usa MoSCoW y sé honesto. Si tu app pudiera hacer solo UNA cosa, ¿cuál sería? Esa es tu Must. Las demás son Should, Could o Won’t.

Pecado 7: Requisitos contradictorios

El error: Tener requisitos que se contradicen entre sí.

Requisito 1	Requisito 2 (contradictorio)
“La app funciona offline”	“La app se sincroniza en tiempo real con el servidor”
“Carga en menos de 1 segundo”	“Muestra 1000 tareas con imágenes en alta resolución”
“Accesible para discapacidad visual”	“Usa colores muy claros y texto pequeño”

Solución: Revisa todos los requisitos juntos al final. Si dos se contradicen, decide cuál es más importante y ajusta el otro.

Caso práctico: los 7 errores en acción

El proyecto que tenía todo mal

Elena, Marcos y Sofía hicieron su proyecto de DAW con estos errores:

Semana 1: Escribieron requisitos como “La app debe ser completa y funcional”. No definieron quién es el usuario ni qué hace exactamente.

Semana 2: Cada uno empezó a programar algo diferente. Elena hizo un login con OAuth. Marcos hizo un CRUD de tareas. Sofía hizo gráficos bonitos. No había coordinación.

Semana 3: Descubrieron que el login de Elena no conectaba con el CRUD de Marcos. Los gráficos de Sofía usaban datos que nadie había introducido.

Semana 4: El profe preguntó “¿qué hace vuestra app?” y no supieron explicarlo. “Pues... tiene login... y tareas... y gráficos...”

Semana 5: Intentaron arreglarlo, pero no tenían documentación de qué habían construido ni por qué.

Semana 6: Entregaron un proyecto que compila pero que nadie entiende. Las features no se integran.

¿Qué salió mal? Los 7 errores: requisitos vagos, sin RNF, sin historias claras, confundiendo qué con cómo, sin documentar, todo era Must y había contradicciones.

¿Qué debería haber hecho? 2 horas definiendo 8-10 requisitos funcionales, 4 RNF, 12 historias de usuario con criterios de aceptación. Les habría ahorrado 5 semanas de confusión.

La tabla de los errores

Error	Consecuencia	Solución
“El usuario puede hacer todo”	Requisitos vagos, sin valor	Verbos concretos: crear, editar, ver
Sin RNF	App lenta, que no funciona en móvil	Definir 3-5 RNF medibles
Historias vagas	Sin claridad sobre qué construir	Actor + Acción + Beneficio
“Lo sabré cuando lo vea”	Cada uno imagina algo diferente	Documentar antes de programar
Todo es Must	No priorizas, no terminas	MoSCoW honesto
Requisitos contradictorios	Imposible de implementar	Revisar todos juntos al final
Confundir qué con cómo	Ata tecnológica innecesaria	Definir comportamiento, no tecnología

Aclaración: “No necesito documentar para un proyecto pequeño”

► 🤔 Si mi proyecto es pequeño, ¿realmente necesito documentar requisitos?

Mini-chequeo

► ¿Cuáles son los 7 errores más comunes?

► ¿Cuál es el error más caro?

► ¿Cómo evito requisitos contradictorios?

✓ Resumen en 3 frases

1. Los 7 errores más comunes son: **requisitos vagos, olvidar RNF, historias sin claridad, no documentar, todo es Must, contradicciones y confundir qué con cómo.**
2. El error más caro es **no documentar**: cada persona imagina algo diferente y al final nada encaja.
3. **1.5 horas de documentación** te ahorran semanas de construir cosas que nadie pidió.

 Volver al [índice de la unidad](#) · Anterior: [06 — Priorización](#) · Siguiente: [08 — Template requisitos](#)

Guia Didactica Proyecto 1 Uds U1 3 Requisitos 08 Template Requisitos

 **Estás en:**  **UD 1.3 · Requisitos** → [08 · Template Requisitos](#)

 *No empieces desde cero: rellena esta plantilla*

Este template contiene todo lo que necesitas para documentar los requisitos de tu proyecto. Cópialo, rellénalo y tendrás un documento completo en 1-2 horas. No es burocracia: es el mapa que guiará todo tu desarrollo.

La idea en una frase

Un template bien diseñado te obliga a pensar en lo que importa antes de empezar a programar.

Template completo de requisitos

Copia este documento en tu repositorio como `REQUISITOS.md` y rellénalo.

1. Visión del proyecto

Nombre de la app: [Nombre]
Descripción breve (1 frase): [Qué hace y para quién]
Usuario principal: [Quién la usa]
Problema que resuelve: [Qué necesidad satisface]

Ejemplo:

Nombre: TaskSchool
Descripción: Gestor de tareas escolares para alumnos de FP
Usuario: Alumnos y profesores de FP
Problema: Los alumnos olvidan fechas de entrega y no organizan bien su trabajo

2. Requisitos funcionales

#	Requisito	Actor	Descripción	Prioridad
RF-001	[Nombre descriptivo]	[Quién]	[Qué hace la app]	Must/Should/Could
RF-002				
RF-003				
RF-004				
RF-005				
RF-006				
RF-007				
RF-008				

Rellena con 5-10 requisitos. Usa verbos de acción concretos.

3. Requisitos no funcionales

#	RNF	Categoría	Métrica	Valor objetivo
RNF-001	[Nombre]	Rendimiento/Usabilidad/Seguridad/Fiabilidad/Compatibilidad	[Cómo se mide]	[Cuánto debe valer]
RNF-002				
RNF-003				
RNF-004				
RNF-005				

Rellena con 3-6 RNF. Cada uno debe ser medible y verificable.

4. Historias de usuario

HU-001: Como [actor], quiero [acción], para [beneficio]

Criterios de aceptación:

- DADO que [contexto]
CUANDO [acción]
ENTONCES [resultado]
- DADO que [contexto alternativo]
CUANDO [acción]
ENTONCES [resultado]

Prioridad: Must/Should/Could

Estimación: S/M/L

Repite para cada historia (10-15 historias recomendadas).

5. Tabla resumen de historias

#	Historia de usuario	Actor	Prioridad	Estimación
HU-001	Como [actor], quiero [acción], para [beneficio]		Must/Should/Could	S/M/L
HU-002				
HU-003				
HU-004				
HU-005				
HU-006				
HU-007				
HU-008				
HU-009				
HU-010				

6. Priorización MoSCoW

Categoría	Requisitos
Must (60%)	RF-, RF-
Should (20%)	RF-, RF-
Could (20%)	RF-, RF-
Won't	[No listar, son features futuras]

7. Supuestos y dependencias

Supuestos:

- [Qué damos por hecho que será cierto]
- [Ejemplo: Los alumnos tienen smartphone con navegador actualizado]
- [Ejemplo: El profesor tiene acceso a un ordenador]

Dependencias:

- [Qué necesitamos de fuera para que funcione]
- [Ejemplo: GitHub Pages para desplegar]
- [Ejemplo: LocalStorage del navegador para persistir datos]

8. Fuera de alcance

Estas funcionalidades NO se incluyen en esta versión:

- [Feature 1 que no entra]
- [Feature 2 que no entra]
- [Feature 3 que no entra]

Importante: Definir qué NO haces es tan importante como definir qué SÍ haces.

Ejemplo relleno: app de tareas

1. Visión

Nombre: TaskSchool

Descripción: Gestor de tareas escolares para alumnos de FP

Usuario: Alumnos y profesores

Problema: Los alumnos olvidan fechas y no organizan su trabajo

2. Requisitos funcionales

#	Requisito	Actor	Descripción	Prioridad
RF-001	Crear tarea	Alumno	Crea tarea con nombre y fecha límite	Must
RF-002	Ver tareas	Alumno	Ve sus tareas ordenadas por fecha	Must
RF-003	Marcar hecha	Alumno	Marca tarea como completada	Must
RF-004	Editar tarea	Alumno	Modifica nombre o fecha de una tarea	Should
RF-005	Filtrar tareas	Alumno	Filtra por fecha, prioridad o estado	Should
RF-006	Ver tareas alumnos	Profesor	Ve tareas de todos sus alumnos	Must
RF-007	Eliminar tarea	Alumno	Elimina tareas que no necesita	Could
RF-008	Exportar PDF	Alumno	Exporta su lista de tareas a PDF	Could

3. RNF

#	RNF	Categoría	Métrica	Valor
RNF-001	Carga rápida	Rendimiento	Tiempo de carga lista	< 2 segundos
RNF-002	Responsive	Usabilidad	Rango de resolución	320px–1920px
RNF-003	Offline	Fiabilidad	Funciona sin conexión	Sí (PWA)
RNF-004	Compatibilidad	Compatibilidad	Navegadores	Chrome, Firefox, Safari, Edge

Aclaración: “¿Esto es obligatorio?”

Mini-chequeo

► ¿Cuáles son las secciones mínimas del template?

► ¿Dónde guardo el documento de requisitos?

► ¿Puedo modificar el template?

✓ Resumen en 3 frases

1. El template tiene **8 secciones**: visión, RF, RNF, historias, tabla, MoSCoW, supuestos y fuera de alcance.
2. Lo **mínimo** para tu proyecto es rellenar: visión, RF, historias y MoSCoW. Te toma **1 hora**.
3. Guarda el documento como `REQUISITOS.md` en la raíz de tu repositorio para que todo el equipo lo tenga visible.

 Volver al [índice de la unidad](#) · Anterior: [07 — Errores comunes](#) · Siguiente: [09 — Cierre](#)

Guia Didactica Proyecto 1 Uds U1 3 Requisitos 09 Cierre

 Estás en:  **UD 1.3 · Requisitos** → 09 · Cierre

 *Ya sabes qué construir. Ahora toca documentarlo.*

Has recorrido toda la unidad: desde qué son los requisitos hasta cómo priorizarlos y escribirlos correctamente. Ahora toca **verificar que lo entiendes** y **aplicarlo a tu proyecto**.

La idea en una frase

Los requisitos son el mapa de tu proyecto: sin ellos, estás programando a ciegas.

Si recuerdas esto, has entendido la unidad.

Mini-chequeo final

Responde estas preguntas antes de seguir:

Preguntas de comprensión

► ¿Cuál es la diferencia entre requisitos funcionales y no funcionales?

► ¿Cuál es el formato de una historia de usuario?

► ¿Qué son los criterios de aceptación?

► ¿Qué significa MoSCoW?

► ¿Cuáles son las 5 categorías de RNF?

► ¿Qué es el modelo Kano?

Ejercicio práctico

En 60 minutos, prepara tu documento de requisitos:

1. **Visión** (5 min): Escribe el nombre, descripción y usuario de tu app
2. **RF** (15 min): Lista 5-10 requisitos funcionales con actor y prioridad
3. **RNF** (10 min): Define 3-5 requisitos no funcionales medibles
4. **Historias** (20 min): Escribe 10-15 historias de usuario con criterios de aceptación
5. **MoSCoW** (10 min): Clasifica todos los requisitos en Must/Should/Could

Si consigues tener un `REQUISITOS.md` completo → has entendido la unidad.



Tabla resumen de la unidad

Concepto	Idea clave
Requisitos	Declaraciones de qué debe hacer la app o cómo debe comportarse
Requisitos funcionales	Qué hace la app: actor + acción + resultado
Requisitos no funcionales	Cómo se comporta: rendimiento, usabilidad, seguridad
Historias de usuario	“Como... quiero... para que...” con valor concreto
Criterios de aceptación	DADO/CUANDO/ENTONCES: cuándo una historia está terminada
INVEST	Independiente, Negociable, Valuable, Estimable, Small, Testable
MoSCoW	Must/Should/Could/Won't para priorizar
Kano	Básicos, lineales, excitadores para entender satisfacción
Errores	Vagos, sin RNF, todo Must, sin documentar, contradicciones

Vocabulario rápido

Término	Definición
Requisito funcional	Descripción de una funcionalidad concreta que la app debe realizar
Requisito no funcional	Restricción o cualidad que afecta a toda la app (rendimiento, seguridad...)
Historia de usuario	Descripción de una funcionalidad desde el punto de vista del usuario
Criterio de aceptación	Condición verificable que define cuándo una historia está terminada
INVEST	Acrónimo para evaluar la calidad de una historia de usuario
MoSCoW	Método de priorización: Must, Should, Could, Won't
Kano	Modelo de satisfacción: básicos, lineales, excitadores
Dado/Cuando/Entonces	Formato estándar para criterios de aceptación (BDD)
PWA	Progressive Web App: aplicación web que funciona offline
BDD	Behavior-Driven Development: desarrollo guiado por comportamiento
Valor	Beneficio que una funcionalidad aporta al usuario
Esfuerzo	Trabajo necesario para implementar una funcionalidad
Quick Win	Feature de alto valor y bajo esfuerzo: hazla primero
Thankless Task	Feature de bajo valor y alto esfuerzo: evítala

Conexión con las siguientes unidades

Unidad	Qué aprenderás	Cómo se conecta
1.4 Diseño	Cómo diseñar la interfaz	Diseñarás basándote en los requisitos que acabas de definir
1.5 Comunicación	Cómo presentar tu proyecto	Mostrarás el documento de requisitos como parte de la documentación
1.7 Propuesta	Cómo escribir la propuesta final	Los requisitos documentados son la base de la propuesta técnica
PI2 Sprints	Cómo ejecutar el proyecto	Cada sprint se basa en las historias de usuario priorizadas

Consejo para el siguiente punto

Si todavía no tienes claro qué poner en tus requisitos:

1. Abre un archivo `REQUISITOS.md` en tu repositorio
2. Copia el template del [punto 08](#)
3. Rellena la visión (5 min)
4. Lista 5 features principales (10 min)
5. Escribe 5 historias de usuario (15 min)

No necesitas un documento perfecto. Necesitas empezar.

Resumen en 3 frases

1. Los requisitos son el **mapa de tu proyecto**: definen qué construir, para quién y con qué restricciones antes de escribir código.
2. Usa **historias de usuario** (Como... quiero... para que...) con **criterios de aceptación** (Dado/Cuando/Entonces) para que no quede ambigüedad.
3. **Prioriza con MoSCoW** y completa el template en **1 hora** para tener tu proyecto documentado y listo para diseñar.

01 — ¿Por qué elegir tecnologías?

 Estás en:  **UD 1.4 · Elección de Tecnologías** → 01 · ¿Por qué elegir?

 *Elegir mal cuesta más que no empezar*

“Yo uso lo que sé” es la excusa más común para elegir tecnologías. Pero lo que sabes puede no ser lo que tu proyecto necesita. Elegir la tecnología equivocada no es solo un error técnico: es tiempo perdido, código que hay que reescribir y frustración. Esta unidad te enseña a elegir con criterio.

La idea en una frase

La elección tecnológica define qué tan fácil será construir, mantener y escalar tu proyecto. Elegir bien ahorra semanas. Elegir mal las duplica.

No es cuestión de moda ni de opiniones: es de **consecuencias reales**.

¿Por qué importa tanto?

El iceberg de la tecnología

Código	← Lo que ves (20%)
Aprendizaje	← Curva de aprendizaje (30%)
Ecosistema	← Librerías, herramientas, plugins (30%)
Mantenimiento	← Lo que nadie ve hasta que falla (20%)

Cuando eliges una tecnología, no eliges solo un lenguaje. Eliges: - Una **curva de aprendizaje** (¿cuánto tardas en ser productivo?) - Un **ecosistema** (¿hay librerías para lo que necesitas?) - Una **comunidad** (¿hay alguien que resuelva tus dudas?) - Un **futuro** (¿seguirá existiendo en 2 años?)

El coste de elegir mal

Ejemplo real: una app de tareas escolares

Decisión	Consecuencia	Coste
Elige PHP puro sin framework	No sabe organizar el código, todo en un archivo de 500 líneas	2 semanas reescribiendo
Elige MongoDB para datos relacionales	Duplica datos, no puede hacer JOINS, pierde integridad	1 semana migrando a SQL
Elige Angular sin saber TypeScript	3 días solo configurando el proyecto, sin escribir lógica	3 días perdidos
Elige React + Node (lo que usa YouTube)	La app es estática, no necesita backend, sobra complejidad	Días de setup innecesario

Lo que no te dicen en YouTube

Lo que ves

Un tutorial de 30 minutos muestra una app funcionando. Parece fácil.

La realidad

Ese tutorial usa una tecnología que no necesitas, para un problema que no tienes, sin mantenimiento futuro.

Las 3 trampas más comunes

1. “Uso lo que sé”

Es la excusa más frecuente. Pero si solo sabes JavaScript y tu proyecto necesita un backend pesado con autenticación compleja, tal vez Python o Java sean mejores opciones. **Saber una tecnología no significa que sea la correcta para todo.**

2. “Uso lo que es popular”

React tiene millones de descargas. Eso no significa que sea la mejor opción para tu proyecto escolar. La popularidad mide adoption, no adecuación.

3. “Lo visto en un tutorial”

Un tutorial de YouTube te muestra el camino feliz. No te muestra: configuración, errores, mantenimiento, escalabilidad. **El tutorial es el 10% de lo que necesitas.**

Aclaración: “¿Tengo que ser experto para elegir?”

► 🤖 ¿Necesito conocer todas las tecnologías para elegir bien?

Mini-chequeo

► ¿Por qué importa la elección tecnológica?

► ¿Cuáles son las 3 trampas más comunes?

✓ Resumen en 3 frases

1. La elección tecnológica define cuánto tardarás en construir y qué tan fácil será mantener tu proyecto.
2. Las 3 trampas son: usar solo lo que sabes, seguir la popularidad sin criterio, y copiar tutoriales sin contexto.
3. No necesitas ser experto: necesitas **conocer opciones, evaluar con criterio y entender las consecuencias**.

 Volver al [índice de la unidad](#) · Siguiendo: [02 — Criterios de elección](#)

02 — Criterios de elección

 Estás en:  UD 1.4 · Elección de Tecnologías → 02 · Criterios de elección

 *No elijas por instinto: elige por criterio*

“Me gusta Python” no es un criterio de elección. “Python tiene mejor ecosistema de datos y mi proyecto necesita procesar CSVs” sí lo es. Para elegir tecnologías necesitas **medir, no adivinar**. Esta unidad te da los 5 criterios que todo desarrollador debería evaluar.

La idea en una frase

Elegir una tecnología sin criterios es como comprar un coche mirando solo el color: funciona, pero probablemente no es lo que necesitas.

Cada proyecto tiene necesidades diferentes. Los criterios te ayudan a **alinear la tecnología con el problema**.

Los 5 criterios de elección

1. Rendimiento

¿Qué tan rápida y eficiente es la tecnología?

Pregunta clave	Qué mide
¿Cuántos usuarios concurrentes soporta?	Capacidad de escalabilidad
¿Cuánto tiempo tarda en procesar datos?	Velocidad de ejecución
¿Cuánta memoria consume?	Eficiencia de recursos

Para tu proyecto escolar: el rendimiento rara vez es el criterio decisivo. Tus 30 compañeros no van a colapsar un servidor. Pero sí importa si procesas datos pesados (imágenes, archivos grandes).

2. Curva de aprendizaje

¿Cuánto tardas en ser productivo con esta tecnología?

Tecnología	Curva de aprendizaje	Detalle
Python	 Baja	Sintaxis clara, ideal para principiantes
JavaScript	 Media	Sintaxis flexible pero muchas formas de hacer lo mismo
Java	 Alta	Verboso, boilerplate, tipos estáticos
PHP	 Media	Fácil de empezar, fácil de hacer mal

Regla práctica: si no has usado la tecnología antes, añade 1-2 semanas a tu estimación para aprenderla.

3. Ecosistema

¿Hay librerías, frameworks y herramientas para lo que necesitas?

Necesidad	Python	JavaScript	Java	PHP
Web framework	Django, Flask	Express, Next.js	Spring Boot	Laravel
Bases de datos	SQLite, PostgreSQL	SQLite, MongoDB	SQLite, MySQL	SQLite, MySQL
Autenticación	django-allauth	Passport.js	Spring Security	Laravel Breeze
Despliegue fácil	 Sí	 Sí	 Requiere JVM	 Sí

4. Comunidad

¿Hay alguien que resuelva tus dudas cuando te trabes?

Indicador	Python	JavaScript	Java	PHP
Preguntas en Stack Overflow	+2M	+2.5M	+1.8M	+1.2M
Documentación oficial	★ ★ ★	★ ★ ★	★ ★ ★	★ ★
Tutoriales en español	Muchos	Muchos	Algunos	Algunos
Actualización reciente	✓ Activa	✓ Activa	✓ Activa	✓ Activa

5. Mercado laboral

¿Te sirve para encontrar trabajo?

Tecnología	Demanda laboral (España)	Salario medio
JavaScript	🔥 Muy alta	30-45K €
Python	🔥 Alta	32-48K €
Java	● Alta	35-50K €
PHP	● Media	28-40K €

Nota: para un proyecto escolar, el mercado laboral es secundario. Pero si estás aprendiendo para工作, vale la pena considerarlo.

Cómo ponderar los criterios

No todos los criterios pesan igual. Depende de tu situación:

Situación	Criterio prioritario	Criterio secundario
Proyecto escolar (6 sem)	Curva de aprendizaje	Ecosistema
Proyecto personal	Ecosistema	Comunidad
Preparación laboral	Mercado laboral	Ecosistema
App con muchos usuarios	Rendimiento	Escalabilidad

Comparativa: elegir por gusto vs por criterio

Por gusto

Eligo React porque me gusta y lo he visto en YouTube. Si luego no funciona, ya veré qué hago.

Por criterio

Mi proyecto es una app estática sin backend. Necesito HTML/CSS/JS puro o un framework estático ligero. React sobra.

Aclaración: “¿Y si elijo mal?”

► 🤔 ¿Qué pasa si elijo una tecnología y luego me arrepiento?

Mini-chequeo

► ¿Cuáles son los 5 criterios de elección?

► ¿Qué criterio priorizo en un proyecto escolar de 6 semanas?

► ¿Puedo cambiar de tecnología a mitad de proyecto?

✅ Resumen en 3 frases

1. Los 5 criterios son: **rendimiento, curva de aprendizaje, ecosistema, comunidad y mercado laboral.**

2. En un proyecto escolar prioriza **curva de aprendizaje y ecosistema**: necesitas ser productivo rápido y tener soporte.
3. Evaluar con criterio antes de empezar te ahorra semanas de refactorización si la elección no era la correcta.

 [Volver al índice de la unidad](#) · Anterior: [01 — ¿Por qué elegir?](#) · Siguiente: [03 — Lenguajes y frameworks](#)

03 — Lenguajes y frameworks

 **Estás en:**  **UD 1.4 · Elección de Tecnologías** → [03 · Lenguajes y frameworks](#)

 *El lenguaje no es lo importante: el problema sí lo es*

“Python es el mejor lenguaje” es una frase sin sentido. Python es excelente para datos, pero no ideal para una app de chat en tiempo real. **No existe el mejor lenguaje**: existe el más adecuado para tu problema. Esta unidad te ayuda a elegir entre las 4 opciones principales.

La idea en una frase

Cada lenguaje tiene un “sweet spot”: un tipo de problema que resuelve mejor que otros. Tu trabajo es identificar qué sweet spot coincide con tu proyecto.

Los 4 candidatos principales

Aspecto	Detalle
Sintaxis	Limpia, legible, casi como pseudocódigo
Curva	 La más baja de los 4
Frameworks web	Django (completo), Flask (ligero)
Ideal para	APIs, datos, automatización, prototipos rápidos
No ideal para	Apps en tiempo real, rendimiento extremo

Ejemplo de sintaxis:

```
# Crear una tarea
def crear_tarea(nombre, fecha):
    return {"nombre": nombre, "fecha": fecha, "hecha": False}
```

Java

Aspecto	Detalle
Sintaxis	Verbosa, explícita, tipada estáticamente
Curva	 La más alta de los 4
Frameworks web	Spring Boot (completo), Jakarta EE
Ideal para	Empresas grandes, Android, sistemas robustos
No ideal para	Proyectos pequeños, prototipos rápidos

Ejemplo de sintaxis:

```
// Crear una tarea
public class Tarea {
    private String nombre;
    private LocalDate fecha;
    private boolean hecha;

    public Tarea(String nombre, LocalDate fecha) {
        this.nombre = nombre;
        this.fecha = fecha;
        this.hecha = false;
    }
}
```

Aspecto	Detalle
Sintaxis	Flexible, asincrónica, muchos patrones posibles
Curva	● Media (flexibilidad = confusión)
Frameworks web	Express (ligero), Next.js (completo), NestJS
Ideal para	APIs REST, apps en tiempo real, full-stack con un solo lenguaje
No ideal para	Procesamiento pesado de datos, apps enterprise

Ejemplo de sintaxis:

```
// Crear una tarea
function crearTarea(nombre, fecha) {
  return { nombre, fecha, hecha: false };
}
```

PHP

Aspecto	Detalle
Sintaxis	Sencilla para empezar, inconsistente
Curva	● Media (fácil de empezar, fácil de hacer mal)
Frameworks web	Laravel (moderno), Symfony (enterprise)
Ideal para	WordPress, webs dinámicas, hosting compartido
No ideal para	APIs modernas, apps en tiempo real

Ejemplo de sintaxis:

```
// Crear una tarea
function crearTarea(string $nombre, string $fecha): array {
  return ["nombre" => $nombre, "fecha" => $fecha, "hecha" => false];
}
```


Tabla comparativa completa

Criterio	Python	Java	JavaScript	PHP
Curva de aprendizaje	● Baja	● Alta	● Media	● Media
Rendimiento	● Medio	● Alto	● Medio	● Medio
Ecosistema web	● Excelente	● Excelente	● Excelente	● Bueno
Ideal para principiantes	✓ Sí	✗ No	⚠ Con guía	⚠ Con guía
Despliegue fácil	✓ Sí	⚠ JVM	✓ Sí	✓ Sí
Mercado laboral (España)	🔥 Alto	● Alto	🔥 Muy alto	● Medio
Proyecto escolar	✓ Recomendado	⚠ Posible	✓ Recomendado	⚠ Posible

¿Cuándo usar cada uno?

Usa Python si:

- Tu proyecto implica **procesamiento de datos** (CSV, JSON, APIs externas)
- Quieres un backend **rápido de montar** (Django en 1 día)
- Eres principiante y quieres **minimizar errores de sintaxis**
- Necesitas **scripts de automatización**

Usa JavaScript si:

- Quieres hacer **frontend y backend con el mismo lenguaje**
- Tu app necesita **interactividad en tiempo real**
- Conoces HTML/CSS y quieres **ampliar hacia el backend**
- Quieres aprender una tecnología **con alta demanda laboral**

Usa Java si:

- Tu proyecto es **enterprise** o necesita **altas prestaciones**

- El profesor pide **tipado fuerte y arquitectura formal**
- Quieres aprender algo que **se usa mucho en empresas grandes**
- No te importa la verbosidad y quieres **explícito sobre conciso**

Usa PHP si:

- Necesitas **desplegar en hosting compartido** barato
- Tu proyecto es un **CMS o web dinámica** con base de datos
- Ya tienes **experiencia con PHP y Laravel**
- El hosting solo soporta **PHP y MySQL**

Comparativa: Python vs JavaScript para un proyecto escolar

Python + Django

Backend robusto, ORM incluido, admin panel automático. Pero necesitas un servidor Python (no hosting compartido barato).

JavaScript + Express

Mismo lenguaje que el frontend, npm masivo, despliegue fácil en Vercel/Railway. Pero más configuración manual.

Aclaración: “¿Y si el profesor me obliga a usar un lenguaje?”

► 🤔 Mi profesor dice que use Java o PHP pero yo quiero Python. ¿Qué hago?

Mini-chequeo

► ¿Cuál es el lenguaje más recomendado para principiantes?

► ¿Qué ventaja tiene JavaScript sobre los demás?

► ¿Cuándo NO debo usar Java?

✓ Resumen en 3 frases

1. **Python** es ideal para principiantes y procesamiento de datos; **JavaScript** para full-stack con un solo lenguaje; **Java** para proyectos enterprise; **PHP** para hosting compartido.
2. No existe el “mejor lenguaje”: existe el **más adecuado para tu problema**, tu experiencia y tu contexto.
3. Para un proyecto escolar de 6 semanas, **Python o JavaScript** suelen ser las mejores opciones por su curva de aprendizaje y ecosistema.

 Volver al [índice de la unidad](#) · Anterior: [02 — Criterios de elección](#) · Siguiente: [04 — Bases de datos](#)

04 — Bases de datos

 Estás en:  **UD 1.4 · Elección de Tecnologías** → 04 · Bases de datos

 *Los datos son el corazón de tu app: elige bien cómo los guardas*

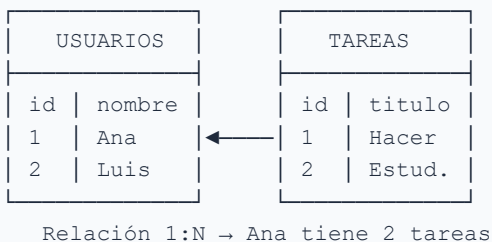
Tu app puede ser bonita y rápida, pero si pierde los datos del usuario, no sirve para nada. La base de datos es donde **viven tus datos**: cómo se almacenan, se consultan y se protegen. Elegir la base de datos correcta es tan importante como elegir el lenguaje.

La idea en una frase

SQL organiza datos como hojas de cálculo relacionadas. NoSQL los guarda como documentos flexibles. La pregunta no es cuál es mejor, sino cuál encaja con tu datos.

SQL vs NoSQL: las dos familias

SQL (Base de datos relacional)



- Datos en **tablas** con filas y columnas
- Relaciones entre tablas (foreign keys)
- Lenguaje de consulta: **SQL** (Structured Query Language)
- Garantiza **integridad referencial**

NoSQL (Base de datos No relacional)

```
{
  "_id": "ana123",
  "nombre": "Ana",
  "tareas": [
    { "titulo": "Hacer deberes", "fecha": "2026-09-10" },
    { "titulo": "Estudiar examen", "fecha": "2026-09-15" }
  ]
}
```

- Datos en **documentos** (JSON, BSON)

- Sin relaciones fijas (cada documento es independiente)
- Sin lenguaje de consulta estándar
- **Flexible** pero sin garantía de integridad

Las 3 opciones principales para tu proyecto

SQLite 🍂

Aspecto	Detalle
Tipo	SQL ligero, sin servidor
Instalación	Zero: viene con Python, Node, PHP
Archivo	Un solo archivo <code>.db</code> en tu carpeta
Ideal para	Proyectos pequeños, apps locales, prototipos
Limitación	No soporta concurrencia alta (1 usuario a la vez)

Cuándo usarla: tu proyecto escolar. Es la opción más simple. No necesitas instalar nada, no necesitas servidor, y funciona perfecto para 1-100 usuarios locales.

PostgreSQL 🐘

Aspecto	Detalle
Tipo	SQL completo, profesional
Instalación	Requiere servidor (o servicio cloud como Supabase)
Ideal para	Apps con datos complejos, relaciones múltiples, producción
Ventaja	El más potente y escalable de los SQL
Limitación	Más complejo de configurar que SQLite

Cuándo usarla: si tu proyecto tiene múltiples usuarios simultáneos, relaciones complejas, o necesitas features avanzados (triggers, vistas, funciones almacenadas).

Aspecto	Detalle
Tipo	NoSQL (documentos)
Instalación	Requiere servidor (o MongoDB Atlas free tier)
Ideal para	Datos flexibles, estructura que cambia, prototipos rápidos
Ventaja	Sin esquema fijo, JSON nativo
Limitación	Sin integridad referencial, datos duplicados

Cuándo usarla: si la estructura de tus datos cambia frecuentemente, o si vienes de JavaScript y piensas en JSON naturalmente.

Comparativa completa

Criterio	SQLite	PostgreSQL	MongoDB
Instalación	✔ Zero	⚠ Servidor	⚠ Servidor
Complejidad	● Muy baja	● Media	● Media
Rendimiento	● Bueno (1 user)	● Excelente	● Bueno
Escalabilidad	● Baja	● Alta	● Alta
Relaciones	✔ JOINS	✔ JOINS	⚠ Denormalizar
Integridad	✔ Foreign keys	✔ Foreign keys	✗ No garantizada
Ideal para proyecto escolar	✔ Sí	⚠ Si necesitas más	⚠ Si datos son flexibles

¿Cómo decidir?

```
¿Tu proyecto es escolar (1-30 usuarios)?
├─ Sí → ¿Necesitas relaciones complejas?
│   ├── No → SQLite ✅
│   └── Sí → ¿Servidor disponible?
│       ├── Sí → PostgreSQL
│       └── No → SQLite (suficiente)
└─ No (producción) → PostgreSQL o MongoDB según datos
```

Ejemplo práctico: app de tareas escolares

Modelo de datos	Mejor opción
Tareas con usuario, fecha, prioridad	SQLite (relaciones simples)
Tareas + archivos adjuntos + comentarios	PostgreSQL (relaciones complejas)
Tareas con campos variables (cada usuario define sus campos)	MongoDB (flexibilidad)

Comparativa: SQL vs NoSQL en la práctica

SQL (PostgreSQL)

Si un alumno tiene 5 tareas, cada tarea referencia al alumno por ID. Si borras al alumno, las tareas se borran automáticamente (integridad).

NoSQL (MongoDB)

Si un alumno tiene 5 tareas, cada tarea guarda el nombre del alumno completo. Si cambias el nombre, tienes que actualizar 5 documentos (duplicación).

Aclaración: “¿SQLite sirve para un proyecto real?”

► 🤖 ¿SQLite es demasiado simple para mi proyecto de FP?

Mini-chequeo

► ¿Cuál es la diferencia entre SQL y NoSQL?

► ¿Cuándo debo usar SQLite?

► ¿Necesito MongoDB para mi proyecto?

✓ Resumen en 3 frases

1. **SQL** (SQLite, PostgreSQL) organiza datos en tablas con relaciones; **NoSQL** (MongoDB) guarda documentos flexibles sin esquema fijo.
2. Para un proyecto escolar, **SQLite** es la mejor opción: cero configuración, integrado en todos los lenguajes, suficiente para 1-30 usuarios.
3. Elige **PostgreSQL** si necesitas relaciones complejas o múltiples usuarios simultáneos; **MongoDB** solo si tus datos son muy flexibles.

 Volver al [índice de la unidad](#) · Anterior: [03 — Lenguajes y frameworks](#) · Siguiente: [05 — Frontend vs Backend](#)

05 — Frontend vs Backend

 Estás en:  UD 1.4 · Elección de Tecnologías → 05 · Frontend vs Backend

 *Frontend es lo que se ve. Backend es lo que funciona.*

Tu app tiene dos partes: la interfaz que ve el usuario y la lógica que procesa datos. Muchos estudiantes se obsesionan con el frontend (colores, animaciones) y olvidan el backend (donde vive la lógica). O al revés. **Necesitas ambos.** Esta unidad te ayuda a decidir cuánto de cada uno.

La idea en una frase

Frontend = lo que el usuario ve y toca. Backend = lo que procesa, almacena y devuelve. Tu proyecto necesita ambos, pero la proporción depende de tu app.

Frontend: la cara de tu app

¿Qué es?

Todo lo que el usuario **ve e interactúa** en el navegador:

Componente	Ejemplo
HTML	Estructura de la página (botones, formularios, textos)
CSS	Diseño (colores, fuentes, espaciado, responsive)
JavaScript	Interactividad (validar formularios, actualizar sin recargar)
Framework	Vue, React, Angular (organizan el código frontend)

¿Cuándo necesitas más frontend?

- Tu app es **visual** (portafolio, juego, presentación)
- La lógica es **simple** (calcular, mostrar, filtrar)
- No necesitas **guardar datos** en servidor
- Todo vive en el **navegador del usuario**

Ejemplo: una calculadora de notas, un temporizador de exámenes, un portafolio personal.

Backend: el cerebro de tu app

¿Qué es?

Todo lo que **no se ve** pero hace que la app funcione:

Componente	Ejemplo
API	Endpoints que reciben y devuelven datos
Lógica	Reglas de negocio (calcular nota media, validar permisos)
Base de datos	Donde se guardan los datos persistentes
Autenticación	Login, registro, sesiones

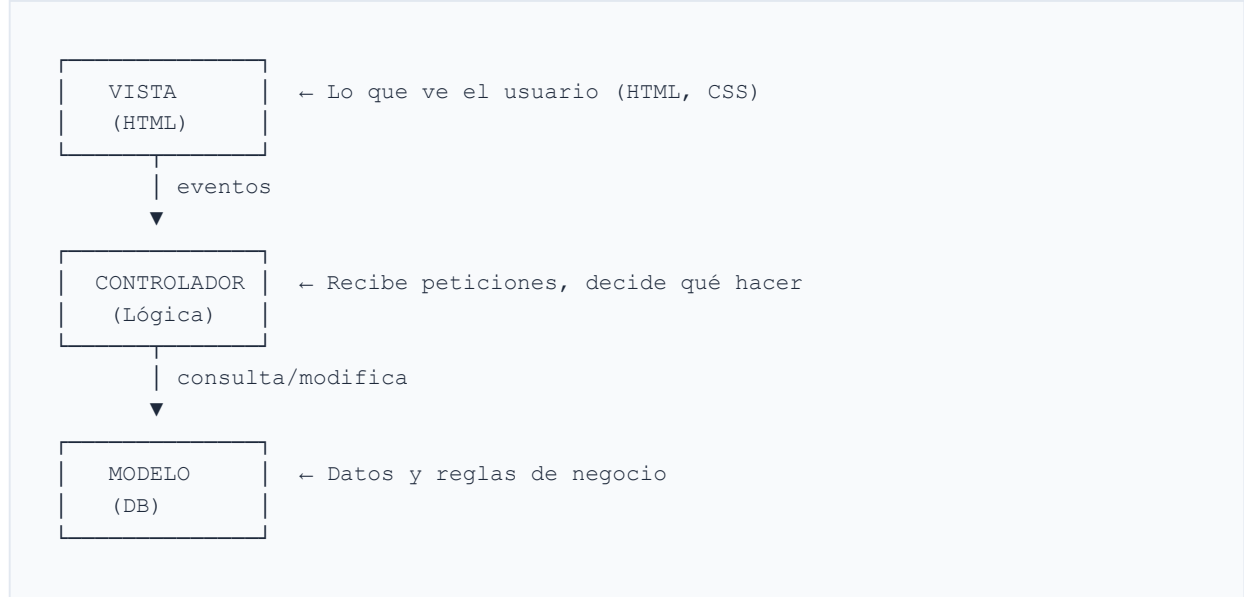
¿Cuándo necesitas más backend?

- Tu app tiene **usuarios con datos persistentes**
- Necesitas **autenticación** (login, roles)
- Procesas **datos en servidor** (envío de emails, cálculos pesados)
- Múltiples usuarios **comparten datos**

Ejemplo: un gestor de tareas con login, un portal de notas, una app de chat.

MVC: el patrón más común

MVC = Modelo-Vista-Controlador



Capa	Responsabilidad	Ejemplo
Vista	Mostrar datos al usuario	<code>tareas.html</code>
Controlador	Recibir peticiones, llamar al modelo	<code>tareas_controller.py</code>
Modelo	Acceder a la base de datos	<code>tarea.py</code>

¿Por qué importa? Porque separar responsabilidades hace el código **más fácil de mantener**. Si cambias el diseño, solo tocas la Vista. Si cambias la lógica, solo tocas el Controlador.

Monolito vs Microservicios

Monolito

Todo en una sola app: frontend, backend, base de datos. Fácil de desarrollar, difícil de escalar. Ideal para empezar.

Microservicios

Cada funcionalidad es un servicio independiente. Complejo de montar, fácil de escalar. Ideal para apps grandes.

Aspecto	Monolito	Microservicios
Complejidad	🟢 Baja	🔴 Alta
Despliegue	✅ Un solo deploy	⚠️ Múltiples deploys
Escalabilidad	⚠️ Todo o nada	✅ Por servicio
Ideal para	Proyectos pequeños, prototipos	Apps grandes, equipos grandes
Proyecto escolar	✅ Sí	❌ No

Regla: si tu proyecto tiene menos de 10.000 líneas de código, **monolito**. No te compliques.

¿Qué necesitan realmente los estudiantes?

El stack mínimo viable



Lo que NO necesitas (probablemente)

- ❌ **Microservicios** → monolito es suficiente
- ❌ **Kubernetes** → un solo servidor basta
- ❌ **GraphQL** → REST es más simple
- ❌ **Redis, RabbitMQ** → SQLite + HTTP bastan
- ❌ **TypeScript** → JavaScript puro funciona

Aclaracion: “¿Puro frontend o también backend?”

▶ 🤔 ¿Puedo hacer mi proyecto solo con frontend (sin servidor)?

Mini-chequeo

▶ ¿Cuál es la diferencia entre frontend y backend?

▶ ¿Qué es MVC?

▶ ¿Monolito o microservicios para mi proyecto?

✅ Resumen en 3 frases

1. **Frontend** es la interfaz visible; **backend** es la lógica que procesa datos. Tu proyecto necesita ambos, pero la proporción depende de la app.
2. Usa **MVC** para separar responsabilidades y hazlo **monolito**: todo en una sola app es suficiente para un proyecto escolar.
3. No necesitas microservicios, GraphQL, Redis ni Kubernetes. El **stack mínimo** (HTML/CSS/JS + framework backend + SQLite) es más que suficiente.

 Volver al [índice de la unidad](#) · Anterior: [04 — Bases de datos](#) · Siguiente: [06 — Comparativa stack](#)

📖 Estás en: 🛠 UD 1.4 · Elección de Tecnologías → 06 · Comparativa stack

📁 No elijas piezas sueltas: elige un stack completo

Un **stack tecnológico** es el conjunto de tecnologías que usas juntas para construir una app: lenguaje, framework, base de datos y despliegue. Elegir piezas sueltas es como comprar motor de un coche y ruedas de otro: pueden no encajar. Esta unidad compara los stacks más comunes.

📁 La idea en una frase

Un stack es un equipo de tecnologías que trabajan juntas. El mejor stack es el que tu equipo conoce y que resuelve tu problema sin sobrediseñar.

Los 4 stacks más comunes

1. LAMP 🏠

Capa	Tecnología
Lenguaje	PHP
Apache	Servidor web
MySQL	Base de datos
PHP	Lenguaje backend

Pros: - ✅ Hosting compartido barato (casi todos soportan LAMP) - ✅ WordPress, Joomla, Drupal funcionan con LAMP - ✅ Documentación masiva en español - ✅ Fácil de desplegar en 1&1, Hostinger, etc.

Contras: - ❌ PHP es verboso y propenso a errores - ❌ MySQL no es tan potente como PostgreSQL - ❌ Apache es más lento que Nginx - ❌ Menos moderno que otras opciones

Ideal para: webs dinámicas simples, CMS, hosting compartido barato.

2. MERN

Capa	Tecnología
MongoDB	Base de datos NoSQL
Express	Framework web (Node.js)
React	Framework frontend
Node.js	Runtime JavaScript backend

Pros: - ✅ **Un solo lenguaje** (JavaScript) en todo el stack - ✅ React es el framework frontend más popular - ✅ npm: el mayor ecosistema de paquetes - ✅ MongoDB es flexible y fácil de empezar

Contras: - ❌ MongoDB sin relaciones puede causar datos duplicados - ❌ Express es minimalista: hay que configurar mucho manualmente - ❌ React tiene curva de aprendizaje alta (JSX, hooks, estado) - ❌ Muchas dependencias npm: riesgo de “dependency hell”

Ideal para: apps full-stack JavaScript, APIs REST, prototipos rápidos.

3. Python + SQLite

Capa	Tecnología
Lenguaje	Python
Framework	Django o Flask
Base de datos	SQLite
Despliegue	GitHub Pages (frontend) + Railway/Render (backend)

Pros: - ✅ **Curva de aprendizaje más baja** de todos los stacks - ✅ Django incluye admin panel, ORM, auth, todo incluido - ✅ SQLite: ^{218 / 743} cero configuración, cero servidor - ✅ Ideal para

principiantes -  Python es el lenguaje más versátil (web, datos, IA)

Contras: -  Flask es muy minimalista (poca estructura) -  Django puede ser overkill para proyectos muy simples -  Python es más lento que Java o Node en algunos casos -  Despliegue backend requiere servicio externo (Railway, Render)

Ideal para: proyectos escolares, principiantes, apps con datos, prototipos.

4. Java + MySQL

Capa	Tecnología
Lenguaje	Java
Framework	Spring Boot
Base de datos	MySQL
Despliegue	Servidor con JVM

Pros: -  **Robustez y escalabilidad** comprobadas -  Spring Boot es el framework enterprise por defecto -  Tipado fuerte: menos errores en runtime -  Muy usado en empresas grandes (banca, telecomunicaciones)

Contras: -  **Verboso:** más código para hacer lo mismo -  Curva de aprendizaje alta (boilerplate, anotaciones, dependencias) -  Requiere JVM: más pesado que Python o Node -  Configuración inicial compleja (Maven, Spring Initializr) -  Overkill para un proyecto escolar de 6 semanas

Ideal para: proyectos enterprise, equipos grandes, alta escalabilidad.

Tabla comparativa de stacks

Criterio	LAMP	MERN	Python+SQLite	Java+MySQL
Curva aprendizaje	● Media	● Media	● Baja	● Alta
Complejidad	● Media	● Media	● Baja	● Alta
Ideal principiantes	⚠ Posible	⚠ Con guía	✅ Sí	❌ No
Despliegue fácil	✅ Sí	✅ Sí	⚠ Backend	⚠ JVM
Hosting barato	✅ Sí	⚠ No	⚠ Parcial	❌ No
Demanda laboral	● Media	🔥 Alta	🔥 Alta	● Alta
Proyecto escolar	⚠ Posible	⚠ Posible	✅ Recomendado	❌ No recomendado

¿Qué stack elijo?

Flujo de decisión

```
¿Eres principiante?  
├ Sí → Python + SQLite (Django o Flask)  
└ No → ¿Quieres un solo lenguaje en todo?  
    ├── Sí → MERN (JavaScript en frontend y backend)  
    └ No → ¿Necesitas hosting barato?  
        ├── Sí → LAMP (PHP + MySQL)  
        └ No → Java + MySQL (si buscas robustez enterprise)
```

Recomendación para proyecto escolar

Recomendación

Python + SQLite + Django/Flask. Curva baja, todo incluido, cero configuración de base de datos, admin panel automático.

JavaScript + SQLite + Express. Si ya conoces HTML/CSS y quieres un solo lenguaje. Más configuración manual pero misma base de datos.

Aclaracion: “¿Puedo mezclar tecnologías?”

► 🤔 ¿Puedo usar Vue en frontend y Python en backend?

Mini-chequeo

► ¿Qué stack es mejor para principiantes?

► ¿Qué stack tiene más demanda laboral?

► ¿Puedo usar Vue con Python?

✅ Resumen en 3 frases

1. Un **stack** es el conjunto de tecnologías que trabajan juntas: lenguaje, framework, base de datos y despliegue.
2. Para un proyecto escolar, **Python + SQLite** es la opción más práctica: curva baja, cero configuración, todo incluido.
3. Puedes **mezclar frontend y backend** libremente (Vue con Python, React con Java) mientras se comuniquen por API REST.

07 — Justificación técnica

 Estás en:  UD 1.4 · Elección de Tecnologías → 07 · Justificación técnica

 *No basta con elegir: hay que explicar por qué*

“Usé Python porque me gusta” no es una justificación. “Usé Python porque su curva de aprendizaje baja me permite ser productivo en 6 semanas, y Django incluye ORM y autenticación que necesito” sí lo es. La **justificación técnica** es la habilidad de argumentar decisiones con criterios objetivos, no con gustos personales.

La idea en una frase

Una justificación técnica conecta cada decisión tecnológica con un criterio concreto y una consecuencia medible.

No se trata de defender tu elección como si fuera un juicio. Se trata de **demostrar que pensaste antes de elegir**.

Las 4 partes de una justificación

1. Decisión

¿Qué elegiste?

Ejemplo: “Elegí Python como lenguaje backend y SQLite como base de datos.”

2. Criterio

¿Por qué con ese criterio?

Ejemplo: “Prioricé la curva de aprendizaje y la facilidad de despliegue porque el proyecto dura 6 semanas.”

3. Alternativa descartada

¿Qué otra opción consideraste y por qué no?

Ejemplo: “Consideré JavaScript + MongoDB pero descarté MongoDB porque mis datos son relacionales (usuarios con tareas).”

4. Consecuencia

¿Qué ganas o pierdes con esta elección?

Ejemplo: “Gano rapidez de desarrollo pero pierdo escalabilidad si el proyecto crece mucho.”

Ejemplo completo: justificación de un proyecto

Proyecto: TaskSchool (gestor de tareas escolares)

LENGUAJE: Python

- └─ Decisión: Python como lenguaje backend
- └─ Criterio: Curva de aprendizaje baja
- └─ Alternativa descartada: Java (verboso, curva alta, overkill para 6 semanas)
- └─ Consecuencia: Soy productivo en 1 semana, pero Python es más lento que Java

FRAMEWORK: Django

- └─ Decisión: Django como framework web
- └─ Criterio: "Baterías incluidas" (ORM, auth, admin panel)
- └─ Alternativa descartada: Flask (muy minimalista, hay que configurar todo manualmente)
- └─ Consecuencia: Aprendo más rápido pero el framework es más pesado

BASE DE DATOS: SQLite

- └─ Decisión: SQLite como base de datos
- └─ Criterio: Cero configuración, integrado en Python
- └─ Alternativa descartada: PostgreSQL (más potente pero requiere servidor)
- └─ Consecuencia: No necesito servidor de BD, pero no soporta concurrencia alta

DESPLIEGUE: GitHub Pages (frontend) + Railway (backend)

- └─ Decisión: GitHub Pages para archivos estáticos, Railway para API
- └─ Criterio: Gratis y fácil de configurar
- └─ Alternativa descartada: Heroku (dejó de ser gratuito en 2022)
- └─ Consecuencia: Despliegue automático con cada commit

Errores comunes en justificaciones

Error	Ejemplo malo	Ejemplo bueno
Gusto personal	“Usé React porque me gusta”	“Usé React porque su ecosistema de componentes me permite reutilizar UI”
Moda	“Todo el mundo usa MERN”	“MERN permite un solo lenguaje en todo el stack, reduciendo la curva de aprendizaje”
Sin alternativa	“Usé Python”	“Usé Python en vez de Java porque la verbosidad de Java haría el proyecto más lento”
Sin consecuencia	“SQLite es fácil”	“SQLite es fácil pero no soporta múltiples usuarios simultáneos”
Genérico	“Es el mejor framework”	“Django incluye ORM, auth y admin, ahorrando 2 semanas de desarrollo manual”

Comparativa: justificación buena vs mala

Mala justificación

Elegí JavaScript porque es el lenguaje más popular del mundo y lo usan todas las empresas grandes.

Buena justificación

Elegí JavaScript (Node.js + Express) porque ya conozco HTML/CSS, lo que reduce mi curva de aprendizaje. El ecosistema npm me da acceso a librerías para autenticación (Passport.js) y base de datos (SQLite driver) que necesito.

Estructura para tu documento

Copia esta estructura para la sección de justificación en tu documentación:

```
## Justificación Técnica
```

```
### Lenguaje: [Nombre]
```

- **Criterio**: [Qué priorizaste]
- **Alternativa descartada**: [Qué otra opción consideraste]
- **Por qué se descartó**: [Razón objetiva]
- **Consecuencia**: [Qué ganas/pierdes]

```
### Framework: [Nombre]
```

- **Criterio**: ...
- **Alternativa descartada**: ...
- **Por qué se descartó**: ...
- **Consecuencia**: ...

```
### Base de datos: [Nombre]
```

- **Criterio**: ...
- **Alternativa descartada**: ...
- **Por qué se descartó**: ...
- **Consecuencia**: ...

```
### Despliegue: [Plataforma]
```

- **Criterio**: ...
- **Alternativa descartada**: ...
- **Por qué se descartó**: ...
- **Consecuencia**: ...

Aclaracion: “¿Tengo que justificar todo?”

- 🤔 ¿Necesito justificar cada tecnología que uso?

Mini-chequeo

- ¿Cuáles son las 4 partes de una justificación?

- ¿Cuál es el error más común?

- ¿Qué tecnologías debo justificar?

✓ Resumen en 3 frases

1. Una justificación técnica conecta cada decisión con un **criterio concreto**, una **alternativa descartada** y una **consecuencia medible**.
2. Los errores más comunes son justificar con **gusto personal**, **moda** o **sin mencionar alternativas**.
3. Justifica las **decisiones principales** (lenguaje, framework, base de datos, despliegue), no las herramientas estándar (HTML, CSS, Git).

 Volver al [índice de la unidad](#) · Anterior: [o6 — Comparativa stack](#) · Siguiente: [o8 — Template stack](#)

o8 — Template Stack Tecnológico

 Estás en:  **UD 1.4 · Elección de Tecnologías** → [o8 · Template Stack](#)

 *Documenta tu stack: tu futuro yo te lo agradecerá*

Cuando empieces a programar, olvidarás por qué elegiste cada tecnología. Cuando el profesor pregunte “¿por qué Python y no Java?”, necesitarás una respuesta. Este template te ayuda a **documentar todo de forma clara** para que nunca tengas que adivinar.

La idea en una frase

Un template bien rellenado es la diferencia entre “lo elegí porque sí” y “lo elegí porque cumple estos 3 criterios”.

Template completo

Copia este documento en tu repositorio como `STACK.md` y rellénalo.

1. Resumen del stack

```
Proyecto: [Nombre de tu app]
Stack completo: [Lenguaje] + [Framework] + [Base de datos] + [Despliegue]

Ejemplo: Python + Flask + SQLite + GitHub Pages + Railway
```

2. Lenguaje backend

```
LENGUAJE: [Nombre]
Versión: [Ej: Python 3.11]

¿Por qué este lenguaje?
- Criterio principal: [Ej: Curva de aprendizaje baja]
- Alternativa descartada: [Ej: Java]
- Por qué se descartó: [Ej: Verboso, curva alta, overkill para 6 semanas]
- Consecuencia: [Ej: Soy productivo en 1 semana]

Fuentes consultadas:
- [Enlace a documentación oficial]
- [Enlace a comparativa o artículo]
```

3. Framework

FRAMEWORK: [Nombre]
Versión: [Ej: Django 4.2]

¿Por qué este framework?

- Criterio principal: [Ej: "Baterías incluidas" – ORM, auth, admin]
- Alternativa descartada: [Ej: Flask]
- Por qué se descartó: [Ej: Flask es minimalista, hay que configurar todo manualmente]
- Consecuencia: [Ej: Ahorro 2 semanas de configuración]

Features que aporta:

- [] ORM (mapeo objeto-relacional)
- [] Autenticación de usuarios
- [] Panel de administración
- [] Validación de formularios
- [] Gestión de sesiones

4. Base de datos

BASE DE DATOS: [Nombre]
Versión: [Ej: SQLite 3.x]

¿Por qué esta base de datos?

- Criterio principal: [Ej: Cero configuración, integrada en Python]
- Alternativa descartada: [Ej: PostgreSQL]
- Por qué se descartó: [Ej: Requiere servidor, overkill para 1-30 usuarios]
- Consecuencia: [Ej: No necesito instalar ni configurar nada]

Modelo de datos:

- [Tablas/colecciones principales]
- [Relaciones entre ellas]

5. Frontend

FRONTEND: [Tecnologías]

Opción A – Puro (sin framework):

- HTML5 + CSS3 + JavaScript vanilla
- [Librería externa si aplica: Bootstrap, Tailwind]

Opción B – Con framework:

- [Vue / React / Angular]
- Versión: [Ej: Vue 3]
- ¿Por qué? [Criterio]

¿Por qué esta elección?

- Criterio principal: [Ej: Simplicidad, no necesito framework]
- Alternativa descartada: [Ej: React]
- Por qué se descartó: [Ej: Overkill para una app con 3 pantallas]
- Consecuencia: [Ej: Menos componentes reutilizables, pero más ligero]

6. Despliegue

DESPLIEGUE:

Frontend (archivos estáticos):

- Plataforma: [GitHub Pages / Netlify / Vercel]

- URL: [https://tusuuario.github.io/tuproyecto/]

- Coste: Gratis

Backend (API / servidor):

- Plataforma: [Railway / Render / Vercel]

- URL: [https://tuproyecto.up.railway.app/]

- Coste: Gratis (tier free)

¿Por qué esta plataforma?

- Criterio: [Ej: Gratis, despliegue automático con Git]

- Alternativa descartada: [Ej: Heroku]

- Por qué se descartó: [Ej: dejó de ser gratuito en 2022]

7. Herramientas complementarias

HERRAMIENTA	USO	COSTE
Git	Control versiones	Gratis
GitHub	Repositorio	Gratis
VS Code	Editor de código	Gratis
Postman	Testing de APIs	Gratis
DB Browser	Gestionar SQLite	Gratis

8. Tabla resumen

Capa	Tecnología	Versión	Justificación (1 frase)
Backend			
Framework			
Base de datos			
Frontend			
Despliegue FE			
Despliegue BE			

Ejemplo relleno: TaskSchool

1. Resumen

Proyecto: TaskSchool
Stack: Python 3.11 + Flask + SQLite + HTML/CSS/JS + GitHub Pages + Railway

2. Lenguaje

LENGUAJE: Python 3.11
Criterio: Curva de aprendizaje baja
Alternativa: Java (descartado por verbosidad y curva alta)
Consecuencia: Productivo en 1 semana

3. Framework

FRAMEWORK: Flask 3.0
Criterio: Ligero, sin sobrecarga
Alternativa: Django (descartado por ser overkill para 3 endpoints)
Consecuencia: Configuro auth y DB manualmente, pero el proyecto es más ligero

4. Base de datos

BASE DE DATOS: SQLite 3.x
Criterio: Cero configuración
Alternativa: PostgreSQL (descartado por requerir servidor)
Consecuencia: Un archivo .db, sin instalar nada

Aclaracion: “¿Esto es obligatorio?”

► 🤔 ¿Tengo que rellenar todo el template?

Mini-chequeo

► ¿Dónde guardo el documento de stack?

► ¿Cuánto tarda en rellenar el template?

► ¿Qué hago si no sé justificar alguna elección?

✅ Resumen en 3 frases

1. El template tiene **8 secciones**: resumen, lenguaje, framework, base de datos, frontend, despliegue, herramientas y tabla resumen.
2. Lo **mínimo** es rellenar: resumen, lenguaje, framework, base de datos y tabla. Te toma **30 minutos**.
3. Guarda el documento como `STACK.md` en la raíz de tu repositorio, junto a `REQUISITOS.md`.

09 — Cierre

 Estás en:  UD 1.4 · Elección de Tecnologías → 09 · Cierre

 Ya sabes elegir. Ahora documenta y justifica.

Has recorrido toda la unidad: desde por qué importa elegir tecnologías hasta cómo documentar tu stack. Ahora toca **verificar que lo entiendes** y **aplicarlo a tu proyecto**. No basta con leer: hay que decidir.

La idea en una frase

Elegir tecnologías no es cuestión de gusto: es de criterios, consecuencias y justificación.

Si recuerdas esto, has entendido la unidad.

Mini-chequeo final

Responde estas preguntas antes de seguir:

Preguntas de comprensión

► ¿Cuáles son los 5 criterios de elección tecnológica?

► ¿Cuál es la diferencia entre SQL y NoSQL?

► ¿Qué es un stack tecnológico?

► ¿Qué es MVC y por qué importa?

► ¿Cuáles son las 4 partes de una justificación técnica?

► ¿Qué stack es mejor para un proyecto escolar de 6 semanas?

► ¿Qué tecnologías debo documentar en el template?

Ejercicio práctico

En 45 minutos, completa tu stack:

1. **Elige stack** (10 min): Usa el flujo de decisión del punto 06
2. **Rellena template** (20 min): Copia el template del punto 08 y rellena las 5 secciones mínimas
3. **Justifica** (15 min): Para cada tecnología, escribe criterio + alternativa + consecuencia

Si consigues tener un `STACK.md` completo con justificación → has entendido la unidad.



Tabla resumen de la unidad

Concepto	Idea clave
Criterios de elección	Rendimiento, curva, ecosistema, comunidad, mercado laboral
Python	Curva baja, ideal para principiantes, versátil
JavaScript	Full-stack con un solo lenguaje, alta demanda laboral
Java	Robustez enterprise, curva alta, overkill para proyectos pequeños
PHP	Hosting compartido barato, ideal para CMS
SQLite	Cero configuración, ideal para proyectos escolares
PostgreSQL	SQL completo, para proyectos con relaciones complejas
MongoDB	NoSQL flexible, para datos que cambian frecuentemente
MVC	Modelo-Vista-Controlador: separa responsabilidades
Monolito	Todo en una app: suficiente para proyectos pequeños
Justificación	Decisión + criterio + alternativa + consecuencia

Vocabulario rápido

Término	Definición
Stack tecnológico	Conjunto de tecnologías que trabajan juntas (lenguaje, framework, BD, despliegue)
LAMP	Linux + Apache + MySQL + PHP (stack clásico web)
MERN	MongoDB + Express + React + Node.js (stack JavaScript full-stack)
MVC	Modelo-Vista-Controlador: patrón de arquitectura para separar responsabilidades
Monolito	Arquitectura donde todo está en una sola aplicación
Microservicios	Arquitectura donde cada funcionalidad es un servicio independiente
ORM	Object-Relational Mapping: mapea objetos PHP/Python/Java a tablas SQL
SQL	Structured Query Language: lenguaje para consultar bases de datos relacionales
NoSQL	Bases de datos no relacionales (documentos, grafos, key-value)
SQLite	Base de datos SQL ligera, sin servidor, en un solo archivo
PostgreSQL	Base de datos SQL completa y profesional
MongoDB	Base de datos NoSQL por documentos (JSON)
Curva de aprendizaje	Tiempo que necesitas para ser productivo con una tecnología
Ecosistema	Librerías, frameworks y herramientas disponibles para una tecnología
Justificación técnica	Documento que argumenta decisiones tecnológicas con criterios objetivos
Despliegue	Proceso de publicar tu app en un servidor para que sea accesible

 **Conexión con las siguientes unidades**

Unidad	Qué aprenderás	Cómo se conecta
1.5 Comunicación	Cómo presentar tu proyecto	La justificación técnica es parte de la presentación
1.6 Herramientas	Qué herramientas usarás	Configurarás el entorno según el stack elegido
1.7 Propuesta	Cómo escribir la propuesta final	El stack documentado es la base de la propuesta técnica
PI2 Sprints	Cómo ejecutar el proyecto	Cada sprint usa las tecnologías que acabas de elegir

Consejo para el siguiente punto

Si todavía no tienes claro qué stack usar:

1. Abre un archivo `STACK.md` en tu repositorio
2. Copia el template del [punto 08](#)
3. Responde la pregunta: **¿necesitas backend?**
 - No → HTML/CSS/JS puro
 - Sí → **Python + Flask + SQLite** (recomendado por defecto)
4. Rellena las 5 secciones mínimas (15 min)
5. Justifica cada elección (15 min)

No necesitas un stack perfecto. Necesitas empezar.

Resumen en 3 frases

1. Elegir tecnologías se basa en **5 criterios objetivos**: rendimiento, curva de aprendizaje, ecosistema, comunidad y mercado laboral.
2. Para un proyecto escolar de 6 semanas, **Python + SQLite + Flask/Django** es la opción más práctica: curva baja, cero configuración, productividad rápida.
3. **Documenta y justifica** tu stack en `STACK.md`: decisión + criterio + alternativa + consecuencia. Te toma 30 minutos y te ahorra semanas de dudas.

01 — ¿Por qué comunicar?

 **Estás en:**  **UD 1.5 · Comunicación** → 01 · ¿Por qué comunicar?

 *Un proyecto excelente mal presentado es un proyecto mediocre*

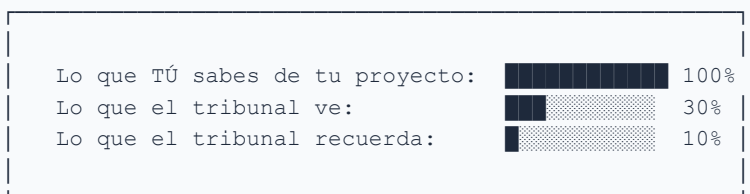
El tribunal no tiene tiempo de explorar tu código ni de leer tu README entero. Tiene **5 minutos** para entender qué has hecho, por qué importa y si lo has hecho bien. Tu presentación es la ventana por la que ven tu trabajo. Si está cerrada, no ven nada.

La idea en una frase

La calidad de tu presentación determina cuánto valora el tribunal la calidad de tu proyecto.

No es justo, pero es así. Un proyecto mediocre bien presentado suele sacar mejor nota que uno excelente mal explicado.

La paradoja del proyecto



Tu trabajo es **reducir esa brecha**. Cuanto más eficaz comuniques, más se acercará lo que el tribunal ve a lo que tú sabes.

¿Por qué la comunicación importa tanto?

1. Primera impresión

El tribunal forma una opinión en los **primeros 30 segundos**. Si empiezas con “Ehhh... pues esto es... una app de... bueno...”, ya estás en desventaja.

2. Evaluación implícita

Además de evaluar tu proyecto, el tribunal evalúa **tu capacidad profesional**: - ¿Sabes explicar lo que has hecho? - ¿Sabes vender tu trabajo? - ¿Trabajarías bien en un equipo real?

3. El 40% de la nota

En muchos centros, la presentación oral supone entre el 30% y el 50% de la nota final del proyecto. **Es la parte con mayor varianza**: puede subir o bajar tu nota 2-3 puntos fácilmente.

Comparativa: proyecto vs presentación

Alumno A

Hizo una app increíble con 15 funcionalidades, pero habló 3 minutos murmurando, no hizo demo y no supo responder preguntas. Nota: 6.

Alumno B

Hizo una app sencilla con 5 funcionalidades, pero explicó el problema, hizo una demo fluida y respondió con seguridad. Nota: 8.

¿Qué evalúa el tribunal?

Criterio	Qué miran	Peso típico
Claridad	¿Se entiende qué hace la app y por qué?	25%
Demo	¿Funciona? ¿Es fluida?	20%
Documentación	¿Está todo documentado?	20%
Tecnología	¿Usas las tecnologías correctamente?	15%
Presentación oral	¿Hablas con claridad y seguridad?	10%
Respuesta a preguntas	¿Entiendes lo que has hecho?	10%

Aclaración: “No soy bueno hablando”

► 😊 ¿Y si me pongo nervioso y me quedo en blanco?

Los 5 errores de comunicación más comunes

1. **Leer las diapositivas** — El tribunal puede leer solo. Tu valor está en explicar.
2. **Hablar demasiado rápido** — Los nervios aceleran. Consciente: respira y habla despacio.
3. **No hacer demo** — Sin demo, es solo una promesa. El tribunal quiere ver.
4. **No preparar preguntas** — Si no sabes responder a “¿por qué usaste X?”, parece que no entiendes tu propio proyecto.
5. **Empezar con disculpas** — “Es que no nos dio tiempo...”, “La culpa es de...”. Nunca. Empieza con fuerza.

Mini-chequeo

► ¿Cuánto dura la presentación típica de un proyecto intermodular?

► ¿Qué evalúa el tribunal además del código?

► ¿Por qué un proyecto sencillo bien presentado puede sacar mejor nota?

✓ Resumen en 3 frases

1. La presentación al tribunal es tu **primera impresión**: determina cuánto valora tu proyecto.
2. El tribunal evalúa **claridad, demo, documentación y seguridad** — no solo código.
3. Preparar la comunicación (ensayo, diapositivas, demo, preguntas) puede **subir o bajar tu nota 2-3 puntos**.

 Volver al [índice de la unidad](#) · Siguiendo: [02 — Pitch 5 minutos](#)

02 — Pitch 5 minutos

 Estás en:  UD 1.5 · Comunicación → 02 · Pitch 5 minutos

 5 minutos para convencer

Un pitch no es una explicación técnica exhaustiva. Es una **historia convincente** que deja al tribunal con ganas de ver más. En 5 minutos debes explicar qué problema resuelves, cómo lo resuelves y por qué tu solución es buena.

La idea en una frase

Un pitch responde a tres preguntas: ¿Qué problema resuelves? ¿Cómo lo resuelves? ¿Por qué funciona?

Si el tribunal no se queda con estas tres cosas, no se queda con nada.

La estructura del pitch

1. PROBLEMA	(1 min)	●
"¿Alguna vez os ha pasado que...?"		
2. SOLUCIÓN	(1 min)	●
"Nuestra app resuelve esto con..."		
3. DEMO RÁPIDA	(1.5 min)	💻
Muestra la app en funcionamiento		
4. TECNOLOGÍA	(0.5 min)	⚙️
"Hemos usado X porque..."		
5. CIERRE	(1 min)	🎯
"En resumen..." + llamada a la acción		

Detalle de cada fase

1. El Problema (1 minuto)

Objetivo: Que el tribunal sienta que el problema importa.

- Empieza con una **anécdota o pregunta retórica**
- Usa **datos concretos** si los tienes
- Conecta con la experiencia del tribunal

Ejemplo: > "¿Alguna vez habéis entrado a una web y no habéis encontrado la información de contacto en 5 minutos? Según el estudio X, el 67% de los usuarios abandonan una web si no encuentran la info de contacto en menos de 30 segundos."

Evita: Empezar con “Nuestra app es una plataforma que...” → eso no engancha.

2. La Solución (1 minuto)

Objetivo: Explicar qué hace tu app y por qué es buena.

- **Una frase** que resuma la app
- **2-3 funcionalidades principales** (no todas)
- **Diferenciador:** ¿qué la hace diferente de otras?

Ejemplo: > “Hemos creado ContactoFácil, una app que genera una página de contacto personalizada en 30 segundos. Sin registrar, sin configurar. El usuario introduce su nombre, email y redes, y tiene su enlace listo para compartir.”

3. Demo Rápida (1.5 minutos)

Objetivo: Mostrar que funciona de verdad.

- **No expliques qué vas a hacer**, solo hazlo
- Muestra el **flujo principal** (no todas las features)
- Habla mientras usas la app: “Aquí veis que al pulsar...”
- **Ten la demo preparada** antes de subir al escenario

Consejo: Graba una captura de pantalla como plan B por si falla internet.

4. Tecnología (30 segundos)

Objetivo: Demostrar que entiendes lo que has usado.

No es una lista de tecnologías. Es una **justificación breve**:

“Hemos usado Vue 3 porque necesitábamos reactividad en tiempo real. Tailwind nos permitió diseñar responsive en tiempo récord. IndexedDB para persistir datos offline.”

Evita: “Hemos usado HTML, CSS, JavaScript, Vue, Tailwind, Vite, GitHub Pages...” → eso es un listado, no una explicación.

5. Cierre (1 minuto)

Objetivo: Dejar una impresión final fuerte.

Estructura: 1. **Resumen en una frase:** “ContactoFácil resuelve el problema de la información de contacto dispersa.” 2. **Dato de impacto:** “En 2 semanas de prueba, 30 profesores crearon su página.” 3. **Llamada a la acción:** “Podéis probarla ahora mismo en esta URL.”

Nunca termines con: “Y eso es todo” o “No sé qué más decir.”

Comparativa: buen pitch vs mal pitch

Mal pitch

Nuestra app es una plataforma web desarrollada con Vue.js y Tailwind CSS que permite a los usuarios gestionar tareas de forma eficiente utilizando tecnologías modernas del desarrollo web.

Buen pitch

¿Alguna vez habéis perdido una tarea importante porque se os olvidó? TaskFlow os avisa por notificaciones, os organiza por prioridades y funciona offline. Mirad: creáis una tarea en 3 segundos, y aquí la tenéis en vuestra lista.

Plantilla de pitch (rellenable)


```
## PROBLEMA (1 min)
- ¿Qué problema resuelves?
- ¿A quién le afecta?
- Dato o anécdota que lo haga real

## SOLUCIÓN (1 min)
- Nombre de la app + una frase
- 2-3 funcionalidades principales
- ¿Qué la hace diferente?

## DEMO (1.5 min)
- Flujo principal: abrir → usar → resultado
- Hablar mientras se usa

## TECNOLOGÍA (30 seg)
- 2-3 tecnologías con JUSTIFICACIÓN
- No listados

## CIERRE (1 min)
- Resumen en 1 frase
- Dato de impacto
- Llamada a la acción
```

Aclaración: “¿Puedo leer?”

►  ¿Está bien leer las notas durante la presentación?

Mini-chequeo

► ¿Cuánto dura cada parte del pitch?

► ¿Por qué empezar por el problema y no por la solución?

► ¿Qué hago si se me olvidan los puntos del pitch?

► ¿Debo mostrar todas las funcionalidades en la demo?

✓ Resumen en 3 frases

1. Un pitch de 5 minutos sigue la estructura: **problema** → **solución** → **demo** → **tecnología** → **cierre**.
2. Empieza con un **problema que enganche**, no con “nuestra app es una plataforma...”.
3. **Ensayar 3-5 veces** con cronómetro es la diferencia entre un pitch confuso y uno profesional.

 Volver al [índice de la unidad](#) · Anterior: [01 — ¿Por qué comunicar?](#) · Siguiente: [03 — Lenguaje verbal](#)

03 — Lenguaje verbal

 Estás en:  UD 1.5 · **Comunicación** → 03 · Lenguaje verbal

 *Lo que dices importa, pero cómo lo dices importa más*

Puedes tener el mejor contenido del mundo, pero si lo dices monótono, rápido y lleno de muletillas, el tribunal no lo escuchará. El lenguaje verbal es tu herramienta principal para **mantener la atención** y **transmitir seguridad**.

La idea en una frase

Hablar con claridad, ritmo y confianza es más importante que el contenido perfecto.

Un contenido mediocre bien dicho siempre gana a un contenido excelente mal dicho.

Los 4 pilares del lenguaje verbal

1. Tono de voz

El tono comunica **actitud y profesionalidad**.

Tono	Efecto en el tribunal
Monótono	Aburrido, desinteresado
Demasiado alto	Agresivo, nervioso
Demasiado bajo	Inseguro, poco profesional
Modulado	Seguro, profesional, interesante

Consejo: Varía el tono según lo que estés diciendo. Cuando expliques el problema, tono serio. Cuando muestres la demo, tono entusiasta.

2. Ritmo (velocidad)

Velocidad ideal para presentaciones:

 140-160 palabras/min

Velocidad normal de conversación:

 180-200 palabras/min

Lo que pasa cuando tienes nervios:

 220+ palabras/min

Problema: Los nervios aceleran. Hablas más rápido de lo que crees.

Solución: - **Respira** antes de empezar - Haz **pausas de 1-2 segundos** entre ideas clave - Si crees que hablas despacio, estás hablando al ritmo correcto

3. Muletillas

Las muletillas son palabras de relleno que restan credibilidad.

Muletilla	Alternativa
“Ehhh...”	Pausa silenciosa
“O sea...”	Eliminar directamente
“Básicamente...”	Eliminar directamente
“Es que...”	“Esto se debe a que...”
“Como que...”	Eliminar directamente
“Vale, entonces...”	“Ahora veremos...”
“Sabeis...”	Pregunta concreta o afirmación

Truco para eliminar muletillas: Grábate hablando 2 minutos. Cuenta cuántas muletillas usas. Luego ensaya el mismo discurso intentando reducirlas a la mitad.

4. Confianza

La confianza se transmite por:

- **Afirmaciones:** “Hemos usado Vue porque...” (no “Creo que usamos Vue”)
- **Datos:** “Funciona en el 95% de los casos” (no “más o menos funciona”)
- **Seguridad:** “Esta es nuestra propuesta” (no “Bueno, esto es lo que hemos hecho”)
- **Gestión del error:** Si te equivocas, di “Corrijo: lo que quiero decir es...” y sigue

El poder de las pausas

"Nuestra app resuelve [PAUSA] el problema de [PAUSA] la organización de tareas [PAUSA] para alumnos de ciclos formativos."

vs.

"Nuestra app resuelve el problema de la organización de tareas para alumnos de ciclos formativos."

Las pausas: - Dan **tiempo al tribunal** para procesar - Marcan **ideas clave** - Transmiten **seguridad** (quien tiene prisa parece nervioso)

Comparativa: verbal vs paraverbal

Solo contenido

Hemos usado Vue 3, Tailwind CSS y IndexedDB. La app permite crear tareas, asignar prioridades y recibir notificaciones.

Contenido + lenguaje verbal

Hemos elegido Vue 3 [pausa] porque necesitábamos reactividad en tiempo real. [Pausa] Mirad: al crear esta tarea, la lista se actualiza al instante. [Entusiasmo] Funciona incluso sin internet.

Ejercicio práctico: el reto de las 30 segundos

1. **Toma la 功能 principal de tu app**
2. **Explícala en 30 segundos** sin usar muletillas
3. **Grábate** con el móvil
4. **Escucha:** ¿Suena profesional o como una conversación informal?
5. **Repite** hasta que suene natural

Si lo consigues en 30 segundos, podrás hacer un pitch de 5 minutos sin problemas.

Aclaración: “Hablo demasiado rápido”

► ⚡ ¿Cómo puedo controlar la velocidad cuando tengo nervios?

► ¿Cuál es la velocidad ideal para una presentación?

► ¿Qué hacer con las muletillas?

► ¿Está bien mostrar nervios?

► ¿Cómo transmito confianza si no la tengo?

✓ Resumen en 3 frases

1. El lenguaje verbal se compone de **tono, ritmo, muletillas y confianza** — todos entrenables.
2. **Habla a 140-160 palabras/min** (más lento de lo que crees) y usa **pausas de 1-2 segundos** entre ideas clave.
3. Grábate, identifica tus muletillas y **reemplaza cada muletilla por una pausa** — suena mucho más profesional.

 Volver al [índice de la unidad](#) · Anterior: [02 — Pitch 5 minutos](#) · Siguiente: [04 — Lenguaje no verbal](#)

04 — Lenguaje no verbal

 Estás en:  UD 1.5 · Comunicación → 04 · Lenguaje no verbal

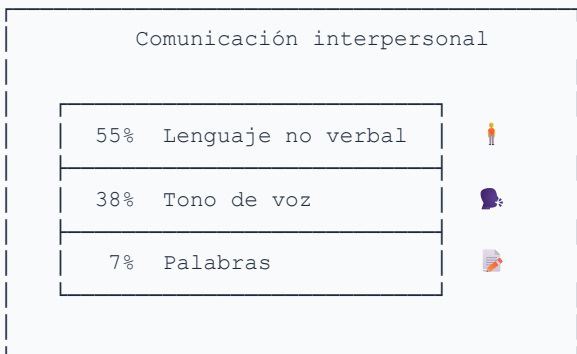
Según Albert Mehrabian, el 55% de la comunicación es no verbal. Esto significa que **más de la mitad de lo que el tribunal percibe** de ti no viene de tus palabras, sino de tu postura, gestos y expresiones. Un cuerpo inseguro puede arruinar un pitch perfecto.

La idea en una frase

Tu cuerpo comunica confianza o inseguridad antes de que digas una sola palabra.

El tribunal forma una opinión no verbal en los primeros **7 segundos**.

La regla de Mehrabian



Nota: Esta regla se aplica sobre todo cuando hay **incongruencia** entre lo que dices y lo que muestra tu cuerpo. Si dices “estoy seguro” pero temblaste, el tribunal cree a tu cuerpo, no a tus palabras.

Los 5 elementos del lenguaje no verbal

1. Postura

Postura	Comunica
Erguido, hombros atrás	Confianza, profesionalismo
Encorvado	Inseguridad, falta de preparación
Manos en los bolsillos	Nerviosismo, desinterés
Apoyado en la mesa	Informalidad, pode excesivo

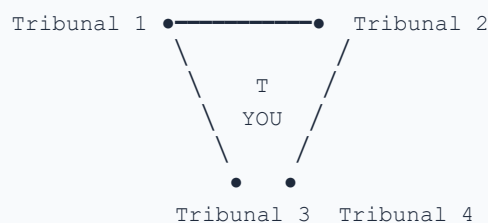
Postura ideal: Pies a la anchura de hombros, peso distribuido igualmente, hombros relajados. Piensa en un presentador de telediario.

2. Contacto visual

El contacto visual genera **confianza y conexión**.

- **Regla del 50/70:** Mantén contacto visual el 50-70% del tiempo mientras hablas
- **Distribúyelo:** No mires solo al profesor. Incluye a todos los miembros del tribunal
- **No escanees:** Mira a una persona 3-5 segundos, luego a otra
- **En la demo:** Mira a la pantalla cuando lo necesites, pero vuelve al tribunal

Distribución del contacto visual:



Mira a cada persona 3-5 segundos, rota entre ellos

3. Gestos de manos

Los manos son herramientas, no adornos.

Gestos efectivos	Gestos a evitar
Abrir las manos (transparencia)	Cruzar los brazos (defensa)
Señalar para enfatizar	Apuntar con un dedo (agresivo)
Contar con los dedos (1, 2, 3)	Juguetear con objetos (nervios)
Gesto de “imagina” (manos abiertas)	Bolsillos (falta de compromiso)

Regla: Tus manos deben estar por encima de la cintura y por debajo de los hombros cuando gesticulas.

4. Expresión facial

- **Sonríe** al empezar y al terminar — genera empatía
- **Ceño fruncido** cuando hables del problema — transmite seriedad
- **Ojos abiertos** cuando muestres la demo — transmite entusiasmo
- **Evita:** Cara de susto, mirada al techo, boca entreabierta

5. Movimiento

- **No te quedes clavado** en un sitio — muévete con propósito
- **Da un paso adelante** cuando quieras enfatizar algo
- **Da un paso atrás** cuando termines una idea
- **No camines de un lado a otro** — transmite nerviosismo

Comparativa: postura vs palabras

Incongruente

Estamos muy seguros de nuestro proyecto [brazos cruzados, mirada al suelo, voz baja] - El tribunal no te cree.

Congruente

Estamos muy seguros de nuestro proyecto [erguido, contacto visual, tono firme] - El tribunal te cree.

Errores no verbales más comunes

1. **Brazos cruzados** — “Estoy a la defensiva”
2. **Mirar al suelo** — “No estoy seguro de lo que digo”
3. **Jugar con el bolígrafo** — “Estoy nervioso”
4. **Ponerse las manos en la cara** — “Quiero que termine esto”
5. **No moverse** — “Estoy paralizado”
6. **Caminar sin parar** — “No puedo estar quieto del nerviosismo”

Aclaracion: “No sé qué hacer con las manos”

► 🙌 ¿Dónde pongo las manos cuando no estoy gesticulando?

Mini-chequeo

► ¿Cuánto contacto visual debo mantener?

► ¿Qué hago con las manos?

► ¿Debo sonreír durante toda la presentación?

► ¿Qué hago si noto que el tribunal está aburrido?

✓ Resumen en 3 frases

1. El lenguaje no verbal (postura, contacto visual, gestos) representa el **55% de la comunicación** según Mehrabian.
2. Mantén **postura erguida**, contacto visual **50-70%** del tiempo y **gestos abiertos** por encima de la cintura.
3. Tu cuerpo debe ser **congruente** con tus palabras: si dices “seguro”, tu postura debe transmitir seguridad.

 Volver al [índice de la unidad](#) · Anterior: [03 — Lenguaje verbal](#) · Siguiente: [05 — Memoria del proyecto](#)

05 — Memoria del proyecto

 Estás en:  UD 1.5 · Comunicación → 05 · Memoria del proyecto

 *La memoria es tu proyecto por escrito*

La memoria del proyecto es el documento que **queda después de la presentación**. Es lo que el tribunal revisa con calma, lo que demuestra que no solo sabes programar, sino también **organizar, documentar y comunicar** tu trabajo de forma profesional.

La idea en una frase

Una buena memoria convierte tu proyecto de “app funcional” a “trabajo profesional documentado”.

Estructura de la memoria

	MEMORIA DEL PROYECTO
1.	Portada
	└ Nombre del proyecto
	└ Autores
	└ Centro y ciclo
	└ Fecha
2.	Índice
	└ Con numeración de página
3.	Introducción (1-2 páginas)
	└ Contexto del problema
	└ Objetivos del proyecto
	└ Estructura del documento
4.	Análisis y requisitos (3-5 páginas)
	└ Análisis de la necesidad
	└ Requisitos funcionales
	└ Requisitos no funcionales
	└ Historias de usuario
5.	Diseño (3-5 páginas)
	└ Arquitectura de la aplicación
	└ Diagrama de componentes
	└ Diseño de la interfaz
	└ Modelo de datos
6.	Desarrollo (5-8 páginas)
	└ Tecnologías utilizadas
	└ Estructura del proyecto
	└ Funcionalidades implementadas
	└ Código destacado (no todo el código)
7.	Pruebas y resultados (2-3 páginas)
	└ Pruebas realizadas
	└ Resultados
	└ Mejoras pendientes
8.	Conclusiones (1-2 páginas)
	└ Logros alcanzados
	└ Dificultades encontradas
	└ Posibles mejoras
9.	Anexos (opcionales)
	└ Manual de usuario
	└ Enlaces al repositorio
	└ Capturas de pantalla

Reglas de estilo

Mal	Bien
“Se procedió a la implementación de la funcionalidad de autenticación”	“Implementamos el login con email y contraseña”
“Dicha solución fue desarrollada utilizando...”	“Usamos Vue 3 para...”
“Es menester destacar que...”	“Destacamos que...”

Regla: Si una frase tiene más de 25 palabras, divídela en dos.

Tono profesional pero accesible

- **No uses jerga técnica innecesaria:** Si puedes decir “almacenar datos”, no digas “persistir información en el mecanismo de storage del navegador”.
- **No uses coloquialismos:** Nada de “la app está chula” o “queda genial”.
- **Sé directo:** “La app permite crear tareas” > “La aplicación tiene la capacidad de llevar a cabo la creación de tareas”.

Longitud adecuada

```
Tamaño típico de la memoria:

Portada ..... 1 página
Introducción ..... 1-2 páginas
Análisis ..... 3-5 páginas
Diseño ..... 3-5 páginas
Desarrollo ..... 5-8 páginas
Pruebas ..... 2-3 páginas
Conclusiones ..... 1-2 páginas
Anexos ..... 2-5 páginas
-----
TOTAL:                18-31 páginas
```

No escribas por escribir. Cada párrafo debe aportar información útil.

Lo que el tribunal busca en la memoria

Memoria débil

Párrafos enormes sin estructura, sin imágenes, sin código, sin análisis. Solo describe qué se hizo sin explicar por qué ni cómo.

Memoria fuerte

Secciones claras, imágenes explicativas, fragmentos de código relevantes, análisis de alternativas, tablas comparativas. Cada sección responde a una pregunta concreta.

Errores comunes en la memoria

1. **Copiar el README** — La memoria no es un README ampliado. Tiene análisis, diseño y conclusiones.
2. **Pegar todo el código** — Solo incluye fragmentos relevantes. El código completo va al repositorio.
3. **Sin imágenes** — Diagramas, capturas, flujos. Una imagen vale más que un párrafo.
4. **Escribir en futuro** — “Se implementará...” → “Se implementó...” (escrito en pasado).
5. **Sin referencias** — Si usas una librería, cita su documentación.

Aclaracion: “¿Cuánto debo escribir?”

► 📄 ¿La memoria debe ser larga o corta?

Mini-chequeo

► ¿Cuál es la estructura básica de la memoria?

► ¿Debo incluir todo el código en la memoria?

► ¿Qué longitud es adecuada?

► ¿En qué tiempo verbal debo escribir?

✓ Resumen en 3 frases

1. La memoria sigue una estructura clara: **introducción, análisis, diseño, desarrollo, pruebas, conclusiones**.
2. Usa **tono profesional pero accesible**, evita párrafos largos y pega fragmentos de código relevantes.
3. **20-25 páginas** bien estructuradas es el rango ideal — calidad sobre cantidad.

 Volver al [índice de la unidad](#) · Anterior: [04 — Lenguaje no verbal](#) · Siguiente: [06 — Diapositivas](#)

06 — Diapositivas

 **Estás en:**  **UD 1.5 • Comunicación** → 06 • Diapositivas

 *Las diapositivas son un apoyo, no un guion*

Las diapositivas existen para **complementar** tu explicación, no para sustituirla. Si el tribunal puede entender tu proyecto solo leyendo tus diapositivas, no te necesitan a ti. Las mejores diapositivas son las que obligan a escuchar al presentador.

Menos texto, más visuales. Una idea por diapositiva. Si puedes decirla en una frase, no pongas un párrafo.

La regla de oro: el tribunal no puede leer y escucharte al mismo tiempo.

La regla 10-20-30

Guy Kawasaki popularizó esta regla para presentaciones:

Regla	Significado
10 diapositivas	Como máximo (para 20 min)
20 minutos	Duración máxima de la charla
30 puntos	Tamaño mínimo de fuente

Para un proyecto intermodular (10-15 min), aplica: **8-10 diapositivas, 14-16pt mínimo.**

Estructura de diapositivas recomendada

1. Portada Nombre, autores, fecha
2. El problema Qué problema resuelves (con dato)
3. La solución Qué hace tu app (una frase)
4. Demo Captura o enlace (no texto)
5. Tecnología 2-3 techs con justificación
6. Arquitectura Diagrama simple
7. Funcionalidades Lista visual (iconos)
8. Resultados Métricas o feedback
9. Conclusiones Logros + mejoras
10. ¿Preguntas? Cierre

Menos es más

Diapositiva con mucho texto	Diapositiva con poco texto
Párrafo de 100 palabras explicando el problema	“El 67% de los alumnos pierden tareas por desorganización” + icono
Lista de 8 tecnologías usadas	3 iconos con nombres y una frase cada una
Tabla comparativa con 6 columnas	2 opciones comparadas visualmente

Colores y contraste

Fondo oscuro + texto claro → Moderno, profesional

Fondo claro + texto oscuro → Limpio, accesible

- ✗ Fondo de color + texto de color → Difícil de leer
- ✗ Fondo gradiente con texto superpuesto → ilegible
- ✗ Imagen de fondo sin contraste → ilegible

Paleta segura: Un color principal + blanco + negro/gris. No uses más de 3 colores.

Imágenes y visuales

- **Usa capturas reales** de tu app, no mockups genéricos
- **Diagramas simples:** Usa draw.io, Mermaid o hasta PowerPoint
- **Iconos:** Lucide, Phosphor, o los de tu framework
- **Evita:** GIFs animados, imágenes stock, clipart

Comparativa: diapositiva buena vs mala

Diapositiva mala

Párrafo de 150 palabras sobre por qué se eligió Vue. Lista de 8 dependencias con versiones.

Tabla comparativa de 5 frameworks.

Diapositiva buena

Título: “¿Por qué Vue?” · 3 bullets: “Reactividad”, “Curva de aprendizaje suave”, “Ecosistema” · Captura de pantalla de la app · Logo de Vue.

Herramientas recomendadas

Herramienta	Ventaja	Coste
Google Slides	Fácil, colaborativo, gratis	Gratis
Canva	Plantillas bonitas, exportable	Gratis/Pro
Reveal.js	HTML-based, personalizable	Gratis
Marp	Markdown → diapositivas, VS Code	Gratis
Figma	Diseño custom, muy visual	Gratis
Pitch	Moderno, colaborativo	Gratis/Pro

Consejo: No pierdas 3 días eligiendo herramienta. Usa la que ya conozcas.

Aclaracion: “¿Puedo usar animaciones?”

► ✨ ¿Las animaciones ayudan o distraen?

Mini-chequeo

► ¿Cuántas diapositivas debo tener?

► ¿Cuánto texto por diapositiva?

► ¿Debo incluir capturas de la app?

► ¿Qué pasa si no sé diseñar?

✓ Resumen en 3 frases

1. Las diapositivas son un **apoyo visual**, no un guion: **máximo 6 líneas** por diapositiva, **1 idea por diapositiva**.
2. Usa **capturas reales** de tu app, diagramas simples y **máximo 3 colores** — evita bloques de texto.
3. Herramientas como **Google Slides, Canva o Marp** son más que suficientes — no pierdas tiempo en la herramienta, inviértelo en el contenido.

 Volver al [índice de la unidad](#) · Anterior: [05 — Memoria del proyecto](#) · Siguiente: [07 — Demo práctica](#)

07 — Demo práctica

 Estás en:  **UD 1.5 · Comunicación** → [07 · Demo práctica](#)

 *Si no hay demo, no hay prueba*

El tribunal quiere **ver tu app funcionando**. No confían en una presentación con solo texto y capturas. Una demo en vivo demuestra que tu proyecto funciona de verdad, pero también es el momento más delicado: si falla, la impresión se arruina.

La idea en una frase

Una demo preparada con plan B es infinitamente mejor que una demo improvisada con mucho riesgo.

La clave no es que la demo funcione siempre, sino que tengas un plan para cada escenario posible.

Las 3 reglas de oro de la demo

1. PREPARA
Antes de la demo, todo debe estar listo
2. PRUEBA
El día anterior, haz la demo 3 veces
3. PLAN B
Siempre ten una alternativa preparada

Fase 1: Preparación

Checklist de la demo

- ☐ **App desplegada** en producción (GitHub Pages, Vercel, etc.)
- ☐ **Datos de ejemplo** precargados (no empieces con pantalla vacía)
- ☐ **Cuenta de usuario** creada si hay login
- ☐ **Navegador actualizado** (Chrome, sin extensiones raras)
- ☐ **Ventanas cerradas**: email, WhatsApp, notificaciones
- ☐ **Zoom al 125%** para que se vea en el proyector

- ☐ **Resolución 1920×1080** (Full HD)
- ☐ **Internet probado** (o app offline si es posible)
- ☐ **Captura de pantalla** de la pantalla principal como plan B

Datos de ejemplo

El tribunal no quiere ver tu app vacía. Prepara:

- **3-5 elementos** de ejemplo (tareas, productos, mensajes, lo que sea)
- **Algunos con errores** a propósito (para mostrar validación)
- **Al menos uno “completado”** (para mostrar el flujo completo)

Fase 2: La ejecución

El flujo de la demo

1. INTRODUCIR (10 seg)
"Ahora os voy a mostrar la app en funcionamiento"
2. CONTEXTO (20 seg)
"Estamos en la pantalla principal. Aquí veis..."
3. ACCIÓN (30-60 seg)
"Voy a crear una tarea..."
[Mientras haces, explica lo que haces]
4. RESULTADO (10 seg)
"Como veis, la tarea aparece en la lista"
5. CIERRE (10 seg)
"En resumen: crear una tarea toma 3 segundos"

Total: 1-2 minutos. No más.

Habla mientras usas

- ✗ Silencio while (typing)
- ✓ "Aquí escribo el nombre de la tarea..."
"Ahora selecciono la fecha límite..."
"Pulso guardar y... mirad: aparece aquí"

El tribunal necesita saber qué está pasando. Si solo ves tu pantalla, no entienden lo que haces.

Escenarios y soluciones

Escenario	Solución
No funciona la app	Captura de pantalla + explicar el flujo verbalmente
No hay internet	App offline o video grabado previamente
Error en vivo	“Esto es un bug conocido que estamos arreando. Mientras tanto, os muestro...”
El proyector no muestra	Cambiar a modo espejo, o usar el portátil directamente
La app va lenta	Tener una versión local o una demo grabada

El video grabado como seguro de vida

Siempre graba una captura de pantalla de tu demo funcionando antes del día de la presentación. Si todo falla, tienes el video.

Herramientas: - **OBS Studio** (gratis, multiplataforma) - **Loom** (gratis, fácil de compartir) - **Captura de Windows** (Win + G)

Comparativa: demo bien vs mal preparada

Demo mal preparada

Abre la app, aparece vacía. Dice ‘no sé por qué no carga’. Cierra el navegador, lo abre de nuevo. Error 404. Se pone nervioso y termina la demo en 30 segundos sin mostrar nada.

Demo bien preparada

App ya abierta con datos de ejemplo. Flujo principal en 90 segundos. Explica cada paso. Si algo falla, cambia a captura de pantalla y sigue. Seguro y profesional.

Aclaracion: “¿Y si se rompe todo?”

- ▶ 🔥 ¿Qué pasa si la demo falla completamente?

Mini-chequeo

- ▶ ¿Cuánto debe durar la demo?

- ▶ ¿Qué hago si la app no está desplegada?

- ▶ ¿Debo mostrar errores a propósito?

- ▶ ¿Qué hacer si el internet falla?

✅ Resumen en 3 frases

1. La demo debe durar **1-2 minutos**, mostrar el **flujo principal** y tener **datos de ejemplo** precargados.
2. **Siempre ten un plan B**: captura de pantalla, video grabado, o explicación verbal — nunca dependas solo de que funcione en vivo.
3. **Habla mientras haces** la demo: el tribunal necesita escuchar qué estás haciendo, no solo ver tu pantalla.

 Volver al [índice de la unidad](#) · Anterior: [o6 — Diapositivas](#) · Siguiente: [o8 — Preguntas del tribunal](#)

📖 Estás en: 🗣️ UD 1.5 · Comunicación → o8 · Preguntas del tribunal

📖 *Las preguntas no son un interrogatorio, son una oportunidad*

Después de la presentación, el tribunal hace preguntas. Esto no es un examen oral ni un juicio. Es una **oportunidad para demostrar que entiendes tu proyecto** a fondo. Las preguntas buscan verificar que lo hiciste tú y que dominas las decisiones técnicas.

📖 La idea en una frase

El tribunal pregunta para entender tu nivel de comprensión, no para atraparte.

Si sabes qué hiciste y por qué lo hiciste, no tienes nada que temer.

Las 5 categorías de preguntas

- | | |
|----------------|-----------------------|
| 1. TÉCNICAS | "¿Por qué usaste X?" |
| 2. FUNCIONALES | "¿Qué hace la app?" |
| 3. PROCESO | "¿Cómo trabajasteis?" |
| 4. DECISIONES | "¿Por qué no Y?" |
| 5. FUTURO | "¿Qué mejorarías?" |

1. Preguntas técnicas

Son las más frecuentes. El tribunal quiere saber **por qué tomaste decisiones técnicas concretas**.

Pregunta	Buena respuesta	Mala respuesta
“¿Por qué Vue y no React?”	“Porque ya lo conocíamos y la curva de aprendizaje era menor. Para este proyecto, la reactividad de Vue era suficiente.”	“No sé, era lo que el profesor dijo.”
“¿Cómo persistes los datos?”	“Usamos localStorage porque los datos son del usuario local y no necesitamos sincronización en servidor.”	“Con JS, guardando en el navegador.”
“¿Por qué Tailwind?”	“Porque nos permitió hacer responsive rápidamente sin escribir CSS custom. El tiempo que ahorramos lo invertimos en funcionalidad.”	“Porque está de moda.”

Clave: Siempre responde con “**porque...**”. La justificación es lo que evalúan.

2. Preguntas funcionales

El tribunal quiere entender **qué hace tu app y cómo funciona**.

Ejemplos: - “¿Qué puede hacer un usuario que no puede hacer un invitado?” - “¿Cómo funciona el flujo de login?” - “¿Qué pasa si el usuario no tiene internet?”

Consejo: Si tienes la app abierta, **muéstralo**. No expliques verbalmente lo que puedes mostrar en 5 segundos.

3. Preguntas sobre el proceso

El tribunal evalúa **cómo trabajasteis como equipo**.

Pregunta	Buena respuesta
“¿Cómo os repartisteis el trabajo?”	“Cada uno se asignó un módulo según sus habilidades. Yo hice el backend, María el frontend, Carlos las pruebas. Nos reuníamos 15 min cada día.”
“¿Qué dificultasteis?”	“Al principio no teníamos requisitos claros. Tuvimos que redefinir historias de usuario en la semana 2.”
“¿Usasteis Git?”	“Sí, hicimos branching: main para producción, develop para desarrollo. Hacíamos PRs y revisiones antes de mergear.”

Nunca digas: “Trabajamos todos juntos en todo” → eso suena a que no hubo organización.

4. Preguntas sobre decisiones

Son las más interesantes. El tribunal quiere saber **por qué no hiciste algo diferente**.

Pregunta	Buena respuesta
“¿Por qué no usaste una base de datos?”	“Porque el proyecto es local-first y los datos son del usuario. IndexedDB era suficiente y evitábamos la complejidad de un backend.”
“¿Por qué no tests unitarios?”	“Sí los hicimos, pero nos centramos en E2E porque para una app de estas características era más valioso probar el flujo completo.”
“¿Por qué no PWA completa?”	“El tiempo nos obligó a priorizar. La funcionalidad offline era Must, pero las notificaciones push eran Could.”

Si no sabes: Di “No lo consideramos, pero veo que habría sido buena idea. ¿Podría ampliar esa pregunta?”

5. Preguntas sobre mejoras

Son las más fáciles de responder. El tribunal sabe que tu proyecto no es perfecto. **Ser honesto sobre las mejoras demuestra madurez**.

Plantilla: > “Hay tres cosas que mejoraría: [1] ..., [2] ..., [3] La primera es la más prioritaria porque...”

El marco STAR para responder

Paso	Significado	Ejemplo
Situación	Contexto de la pregunta	“Cuando implementamos el login...”
Tarea	Qué teníais que hacer	“Necesitábamos autenticar usuarios sin backend”
Acción	Qué hicisteis	“Usamos JWT con localStorage”
Resultado	Resultado obtenido	“Funciona offline y los datos se persisten localmente”

Comparativa: respuesta defensiva vs profesional

Defensiva

No, sí que lo hicimos bien. Es que el tiempo no nos dio para más. No es justo que nos pregunte eso.

Profesional

Eso es un punto que mejoraremos. En la versión actual usamos X porque..., pero en el futuro implementaríamos Y porque... Tiene más sentido?

Aclaracion: “¿Qué hago si no sé la respuesta?”

► 🙋 ¿Y si me preguntan algo que no sé?

1. “¿Por qué esta tecnología y no otra?”
2. “¿Cómo persistes los datos?”
3. “¿Qué pasaría si se usa con muchos usuarios?”
4. “¿Cómo os organizasteis en el equipo?”
5. “¿Qué fue lo más difícil?”
6. “¿Qué mejorarías?”
7. “¿Qué aprendisteis?”
8. “¿Cómo probasteis la app?”
9. “¿Por qué esta interfaz y no otra?”
10. “¿Cómo despliegas la app?”

Prepara respuestas para las 10. Te toma 30 minutos y te da una seguridad enorme.

Mini-chequeo

► ¿Qué hago si no sé la respuesta a una pregunta?

► ¿Cuántas preguntas suele hacer el tribunal?

► ¿Debo contestar siempre con “porque”?

► ¿Qué pasa si me contradice el tribunal?

✓ Resumen en 3 frases

1. El tribunal pregunta en 5 categorías: **técnicas, funcionales, proceso, decisiones y mejoras** — prepara respuestas para las 10 más frecuentes.
2. Usa el marco **STAR** (Situación, Tarea, Acción, Resultado) para responder con estructura y claridad.
3. **Nunca mientas** si no sabes algo: admite, contextualiza y pide ampliación — eso demuestra profesionalidad.

09 — Cierre

 Estás en:  UD 1.5 · Comunicación → 09 · Cierre

 *Ya sabes comunicar. Ahora toca ensayar.*

Has recorrido toda la unidad: desde por qué comunicar es importante, hasta cómo responder preguntas del tribunal. Ahora toca **verificar que lo entiendes** y **aplicarlo a tu proyecto**. El conocimiento sin práctica no sirve de nada.

La idea en una frase

La comunicación es una habilidad que se entrena: ensaya, graba, revisa y repite.

Si recuerdas esto, has entendido la unidad.

Mini-chequeo final

Responde estas preguntas antes de seguir:

Preguntas de comprensión

► ¿Cuáles son las 5 fases del pitch de 5 minutos?

► ¿Qué porcentaje de la comunicación es no verbal?

► **¿Cuál es la regla de las diapositivas?**

► **¿Qué es el plan B de una demo?**

► **¿Qué hacer si no sabes responder a una pregunta del tribunal?**

► **¿Cuáles son los 5 errores de comunicación más comunes?**

Ejercicio práctico

En 90 minutos, prepara tu presentación completa:

1. **Escribe el pitch** (15 min): Usa la plantilla del punto 02
2. **Crea las diapositivas** (30 min): 8-10 diapositivas, menos texto más visual
3. **Prepara la demo** (15 min): Datos de ejemplo, flujo principal, plan B
4. **Responde 10 preguntas** (15 min): Escribe respuestas a las preguntas frecuentes
5. **Ensayar 1 vez** (15 min): Con cronómetro, grabándote con el móvil

Si consigues hacer todo esto → estás listo para presentar.



Tabla resumen de la unidad

Concepto	Idea clave
Pitch	Problema → Solución → Demo → Tecnología → Cierre (5 min)
Lenguaje verbal	Tono modulado, ritmo 140-160 palabras/min, sin muletillas
Lenguaje no verbal	Postura erguida, contacto visual 50-70%, gestos abiertos
Memoria	20-25 páginas, estructura clara, tono profesional
Diapositivas	Máximo 6 líneas, 1 idea/diapo, capturas reales
Demo	1-2 minutos, flujo principal, plan B siempre
Preguntas	Responder con STAR, nunca mentir, preparar las 10 más frecuentes



Vocabulario rápido

Término	Definición
Pitch	Presentación breve y persuasiva de un proyecto o idea
Mehrabian	Regla 55-38-7: comunicación no verbal, tono y palabras
Lenguaje no verbal	Comunicación a través de postura, gestos y expresiones
Lenguaje verbal	Comunicación a través de palabras habladas (tono, ritmo)
Muletillas	Palabras de relleno que restan credibilidad (ehhh, o sea)
STAR	Marco de respuesta: Situación, Tarea, Acción, Resultado
Plan B	Alternativa preparada para cuando falla la demo en vivo
Demo	Demostración en vivo de la app funcionando
Memoria	Documento escrito que describe el proyecto completo
Diapositivas	Presentación visual de apoyo a la explicación oral
Tribunal	Grupo de profesores que evalúa el proyecto
Contacto visual	Mirar al tribunal mientras hablas para generar confianza
Pausa	Silencio deliberado entre ideas para dar énfasis
Tono modulado	Variación del tono de voz para mantener interés

Conexión con las siguientes unidades

Unidad	Qué aprenderás	Cómo se conecta
1.6 Propuesta	Cómo escribir la propuesta técnica	La memoria que escribiste es la base de la propuesta
1.7 Validación	Cómo presentar el proyecto final	Aplicarás todo lo aprendido en esta unidad
PI2 Sprints	Cómo ejecutar el proyecto	Comunicarás avances en la daily y en la revisión

Consejo para el siguiente punto

Si todavía no tienes claro cómo preparar tu presentación:

1. **Escribe tu pitch** en 5 frases (una por fase)
2. **Ensáyalo** en voz alta 3 veces
3. **Grábate** y escucha
4. **Identifica** dónde te trabas
5. **Mejora** y repite

No necesitas una presentación perfecta. Necesitas practicar.

Resumen en 3 frases

1. La comunicación se compone de **5 habilidades**: pitch, lenguaje verbal, lenguaje no verbal, diapositivas y manejo de preguntas.
2. **Ensayar 3-5 veces** con cronómetro y grabación es la diferencia entre una presentación confusa y una profesional.
3. **Prepara siempre un plan B** para la demo y respuestas para las 10 preguntas más frecuentes — la seguridad viene de la preparación.

 Volver al [índice de la unidad](#) · Anterior: [08 — Preguntas del tribunal](#) · Siguiente: [UD 1.6 — Propuesta](#)

01 — ¿Por qué documentar?

 Estás en:  **UD 1.6 · Documentación** → 01 · ¿Por qué documentar?

 *¿Documentar o programar? La respuesta es: ambos*

“No tengo tiempo para documentar, estoy programando.” Es la excusa más común. Pero piénsalo: si mañana otro compañero tiene que entender tu código, ¿qué prefiere ver? Un README claro o 500 líneas sin comentario alguno?

La idea en una frase

Documentar no es perder tiempo: es ahorrar tiempo a quien viene después — incluido tú mismo.

La filosofía Docs as Code

La idea es simple: **tratar la documentación como tratamos el código.**

```
Código fuente → Repositorio → Versionado → Revisión → Despliegue
      ↓           ↓           ↓           ↓           ↓
Documentación → Repositorio → Versionado → Revisión → Publicación
```

¿Qué significa esto en la práctica?

1. **Escribir en Markdown** (no en Word)
2. **Versionar con Git** (mismo repositorio que el código)
3. **Revisar con Pull Requests** (iguales que el código)
4. **Publicar automáticamente** (con herramientas estáticas)

¿Por qué importa?

1. Para ti mismo

En 3 meses no recordarás por qué decidiste usar Vue en vez de React, o por qué esa función se llama `filtrarDatosRaros`. La documentación es tu memoria externa.

2. Para tu equipo

Si trabajas en equipo, la documentación evita: - Preguntas repetitivas (“¿cómo instalo el proyecto?”)
- Conocimiento perdido (“¿quién hizo esto y por qué?”) - Duplicación de trabajo (“¿ya existía esto?”)

3. Para el tribunal

El tribunal **evalúa la documentación** como parte del proyecto. Un README claro, una memoria bien estructurada y un manual de usuario suman puntos.

4. Para la empresa real

En el mundo laboral, la documentación es lo que separa un proyecto profesional de uno amateur. Las empresas lo saben y lo valoran.

Comparativa: con y sin documentación

Sin documentación

Nuevo joins el equipo. No sabe qué hacer. Pregunta 15 veces. Tarda 2 semanas en ser productivo. Se frustra.

Con documentación

Nuevo joins el equipo. Lee el README, sigue la guía de setup, entiende la arquitectura. Tarda 2 días en ser productivo.

Aclaración: “No sé escribir bien”

► 🙌 ¿Y si no se me da bien escribir?

Los 4 mitos sobre la documentación

Mito	Realidad
“Documentar es perder tiempo”	Ahorra más tiempo del que invierte
“El código se explica solo”	Solo para quien lo escribió
“Nadie lee la documentación”	El tribunal la evalúa, y los compañeros la necesitan
“Es aburrido”	Puede serlo, pero es más aburrido no tenerla

Mini-chequeo

► ¿Qué es Docs as Code?

► ¿Por qué documentar es importante en un proyecto intermodular?

► ¿Qué formato se recomienda para documentación?

✓ Resumen en 3 frases

1. Documentar no es opcional: es una **habilidad profesional** que el tribunal evalúa.
2. La filosofía **Docs as Code** trata la documentación igual que el código: versionada, revisada, publicada.
3. La documentación **ahorra tiempo** a tu equipo, al tribunal y a tu yo futuro.



Volver al [índice de la unidad](#) · Siguiente: [02 — README](#)

 Estás en:  UD 1.6 · Documentación → 02 · README

 *El README es lo primero que ven — y lo último que escriben*

El archivo `README.md` es la **puerta de entrada** a tu proyecto. Es lo primero que ve el tribunal, lo primero que ve un compañero y lo primero que vería un reclutador. Un README malo da mala impresión aunque el código sea excelente.

La idea en una frase

Un buen README responde en 30 segundos: qué es, cómo se instala y cómo se usa.

Estructura de un README completo

```
# Nombre del Proyecto
```

```
Breve descripción de una línea.
```

```
## 🚀 Características
```

- Funcionalidad 1
- Funcionalidad 2
- Funcionalidad 3

```
## 🛠️ Tecnologías
```

- Vue 3
- Tailwind CSS
- Vite

```
## 📦 Instalación
```

1. Clonar el repositorio
2. Instalar dependencias
3. Ejecutar en desarrollo

```
## 🎮 Uso
```

```
Cómo usar la aplicación (capturas, pasos).
```

```
## 📁 Estructura del proyecto
```

```
Árbol de carpetas.
```

```
## 👥 Equipo
```

```
Nombre y rol de cada miembro.
```

```
## 📄 Licencia
```

```
Tipo de licencia.
```

Secciones detalladas

1. Nombre y descripción

```
# GestorTareas
```

```
Aplicación web para gestionar tareas de clase  
con sistema de prioridades y recordatorios.
```

Regla: Si en 2 líneas no se entiende qué hace, la descripción es mala.

2. Características

Lista con viñetas de las funcionalidades principales:

```
## Características
- CRUD de tareas con prioridades (alta, media, baja)
- Filtro por estado y fecha
- Vista de calendario
- Exportación a PDF
- Modo oscuro
```

3. Tecnologías

```
## Tecnologías
- **Frontend**: Vue 3, Composition API
- **Estilos**: Tailwind CSS
- **Build**: Vite
- **Despliegue**: GitHub Pages
```

4. Instalación

```
## Instalación

1. Clonar el repositorio
  ```bash
 git clone https://github.com/usuario/proyecto.git
```

#### 2. Instalar dependencias

```
npm install
```

#### 3. Ejecutar en desarrollo

```
npm run dev
```

...

### 5. Uso

```
Uso

1. Abrir la aplicación en el navegador
2. Añadir una tarea con el botón "+"
3. Asignar prioridad y fecha límite
4. Marcar como completada cuando termines
```

Incluye capturas de pantalla si es posible.

### 6. Estructura del proyecto

```
Estructura del proyecto
```

```
src/
├ components/ # Componentes reutilizables
├ views/ # Vistas principales
├ stores/ # Estado global
├ utils/ # Funciones auxiliares
└ assets/ # Imágenes y estilos
```

## Comparativa: README bueno vs malo

### README malo

Nombre del proyecto. Hecho con Vue. Para instalar npm install. El profesor lo entenderá.

### README bueno

Nombre + descripción clara. 5 características. 3 tecnologías. 4 pasos de instalación con código. 3 capturas. Estructura de carpetas. Equipo. Licencia.

## Aclaración: “¿Y si mi proyecto es sencillo?”

► 🤔 ¿Necesito un README largo si mi app es sencilla?

## Los 5 errores del README

1. **Sin descripción** — Solo pone el nombre. ¿Qué hace?
2. **Sin instrucciones de instalación** — “npm install” no es suficiente
3. **Sin capturas** — El tribunal quiere ver la app, no imaginarla
4. **Sin equipo** — ¿Quién hizo qué?
5. **Sin licencia** — ¿Es open source? ¿Se puede copiar?



## Mini-chequeo

► ¿Cuáles son las secciones mínimas de un README?

► ¿Qué formatos puede tener un README?

► ¿Cuánto debe medir un README?

### ✓ Resumen en 3 frases

1. El README es la **puerta de entrada** a tu proyecto: el tribunal lo lee primero.
2. Debe responder en 30 segundos: **qué es, cómo se instala y cómo se usa**.
3. Incluye siempre: descripción, tecnologías, instalación, uso, estructura, equipo y licencia.

 Volver al [índice de la unidad](#) · Anterior: [01 — ¿Por qué documentar?](#) · Siguiente: [03 — Doc técnica](#)

## 03 — Documentación técnica

 Estás en:  **UD 1.6 · Documentación** → 03 · Doc técnica

 *Lo técnico se entiende mejor con un mapa*

Tu proyecto tiene componentes, rutas, estados, servicios, dependencias... Sin documentación técnica, todo eso vive solo en la cabeza del que lo programó. La doc técnica

es el **plano de tu proyecto**: cómo está construido y cómo funciona por dentro.

## La idea en una frase

**La documentación técnica explica CÓMO está hecho tu proyecto, no QUÉ hace.**

## Tipos de documentación técnica

### 1. Arquitectura del proyecto

Describe cómo se organizan las partes:

Interfaz (UI) Componentes Vue + Tailwind
Estado (Store) Pinia / localStorage
Lógica (Logic) Composables / Funciones puras
Datos (Data) JSON / IndexedDB / API

### 2. Guía de setup

Cómo levantar el proyecto desde cero:

```
Requisitos previos
- Node.js 18+
- npm o pnpm

Instalación
1. Clonar: `git clone <url>`
2. Instalar: `npm install`
3. Variables de entorno: copiar `.env.example` a `.env`
4. Ejecutar: `npm run dev`
```

### 3. Documentación de API (si aplica)

Si tu app consume una API externa o expone endpoints:

```
Endpoints

GET /api/tareas
Devuelve todas las tareas del usuario.

Parámetros:
- `page` (número, opcional): Página a mostrar
- `limit` (número, opcional): Resultados por página (máx. 50)

Respuesta:
{
 "data": [...],
 "total": 42,
 "page": 1
}
```

## 4. Decisiones de diseño

Por qué tomaste ciertas decisiones:

```
Decisiones de diseño

¿Por qué Vue y no React?
- Equipo conocía mejor Vue
- Comunidad en español más activa
- Mejor documentación oficial en castellano

¿Por qué Tailwind y no CSS puro?
- Desarrollo más rápido
- Consistencia visual automática
- Soporte oscuro incluido
```

## Comparativa: doc técnica vs doc de usuario

### Doc técnica

Para el desarrollador: arquitectura, dependencias, estructura de código, APIs, decisiones de diseño.

### Doc de usuario

Para el usuario final: qué hace la app, cómo usarla, pasos con capturas, soluciones a problemas comunes.

## Aclaración: “Mi proyecto es sencillo, ¿necesito doc técnica?”

► 🤔 ¿Y si mi app solo tiene 3 pantallas?

## Plantilla de doc técnica

```
Documentación Técnica — [Nombre del Proyecto]

Arquitectura
[Diagrama o descripción de capas]

Tecnologías
| Tecnología | Versión | Propósito |
|-----|-----|-----|
| Vue | 3.x | Framework UI |
| Tailwind | 3.x | Estilos |
| Vite | 5.x | Build tool |

Estructura del proyecto
[Árbol de carpetas con descripción]

Instalación
[Pasos detallados]

Variables de entorno
| Variable | Descripción | Ejemplo |
|-----|-----|-----|
| VITE_API_URL | URL del backend | http://localhost:3000 |

Decisiones de diseño
[Por qué cada tecnología y patrón]

Dependencias principales
| Paquete | Uso |
|-----|-----|
| axios | Peticiones HTTP |
| pinia | Estado global |
```

## Mini-chequeo

► ¿Cuáles son los 4 tipos de documentación técnica?

► ¿Cuál es la diferencia entre doc técnica y doc de usuario?

► ¿Qué es una decisión de diseño?

## ✓ Resumen en 3 frases

1. La documentación técnica explica **cómo está construido** tu proyecto: arquitectura, setup, APIs y decisiones.
2. Incluso proyectos sencillos necesitan doc técnica: **una página con la estructura y las razones** es suficiente.
3. Usa plantillas y escribe para un **desarrollador que no conoce tu proyecto** — no para ti mismo.

 Volver al [índice de la unidad](#) · Anterior: [02 — README](#) · Siguiente: [04 — Manual de usuario](#)

## 04 — Manual de usuario

 **Estás en:**  **UD 1.6 · Documentación** → [04 · Manual de usuario](#)

 *El usuario no quiere saber cómo está hecho, quiere saber cómo se usa*

Tu app puede usar Vue, Pinia y 15 librerías más. Al usuario le da igual. El usuario quiere saber: **¿qué hago para conseguir lo que quiero?**. El manual de usuario es la documentación que responde esa pregunta.

Escribe como si le explicaran a alguien que nunca ha visto una computadora.

## Estructura del manual

```
Manual de usuario — [Nombre]

Introducción
Qué es la app y para qué sirve.

Primeros pasos
Cómo empezar: registrarse, configurar, etc.

Funcionalidades principales
Una por una, con capturas y pasos.

Preguntas frecuentes
Problemas comunes y soluciones.

Contacto
Cómo pedir ayuda.
```

## Reglas de escritura para no técnicos

### 1. Evita la jerga técnica

 Mal	 Bien
“Haz clic en el botón de submit”	“Pulsa el botón Enviar”
“El endpoint retorna un 404”	“No se ha encontrado la página”
“La cookie ha expirado”	“Tu sesión ha caducado, vuelve a iniciar sesión”
“Configura las variables de entorno”	“Abre el archivo .env y escribe la dirección del servidor”

### 2. Un paso por línea

❌ Mal:  
Abre la app, ve a ajustes y cambia el tema.

✅ Bien:  
1. Abre la aplicación  
2. Pulsa el icono de ajustes (engranaje)  
3. Selecciona "Tema oscuro"  
4. Pulsa "Guardar"

### 3. Capturas de pantalla

Una imagen vale más que mil palabras. Incluye: - Captura de cada pantalla importante - Flechas o recuadros señalando los botones - Texto alternativo descriptivo

### 4. Lenguaje amigable

❌ "Error: el campo email es obligatorio"  
✅ "¿Olvidaste poner tu correo? Escríbelo para continuar"

## Comparativa: manual para técnicos vs no técnicos

#### Para técnicos

Configura el archivo .env con VITE\_API\_URL=http://localhost:3000. Ejecuta npm run build. Despliega en /dist.

#### Para no técnicos

1. Abre el archivo de configuración. 2. Escribe la dirección del servidor. 3. Pulsa el botón Guardar. 4. La app está lista para usar.

## Aclaracion: “¿Y si mi usuario es técnico?”

## Mini-chequeo

► ¿Cuáles son las secciones de un manual de usuario?

► ¿Qué es “jerga técnica”?

► ¿Por qué son importantes las capturas de pantalla?

### ✓ Resumen en 3 frases

1. El manual de usuario escribe para **personas no técnicas**: evita jerga, usa pasos claros e incluye capturas.
2. **Un paso por línea** y lenguaje amigable hacen que el usuario no se pierda.
3. Incluye siempre: introducción, primeros pasos, funcionalidades, FAQ y contacto.

 Volver al [índice de la unidad](#) · Anterior: [03 — Doc técnica](#) · Siguiente: [05 — Markdown](#)

## 05 — Markdown

 **Estás en:**  **UD 1.6 · Documentación** → 05 · Markdown

 *Markdown es el lenguaje de la documentación moderna*

Si vas a escribir documentación, necesitas Markdown. Es el formato estándar para READMEs, wikis, documentación técnica y notas. Es simple, legible y se convierte a



## La idea en una frase

**Markdown es texto plano con reglas: lo escribes en cualquier editor y lo lees en cualquier plataforma.**

## Sintaxis básica

### Encabezados

```
H1 - Título principal
H2 - Sección
H3 - Subsección
H4 - Sub-subsección
```

**Regla:** No saltes de H1 a H3. Usa en orden.

### Texto con formato

```
Negrita → Negrita
Cursiva → Cursiva
~~Tachado~~ → Tachado
`código inline` → código inline
```

### Listas

```
- Elemento 1
- Elemento 2
 - Sub-elemento

1. Paso 1
2. Paso 2
3. Paso 3
```

## Enlaces e imágenes

[Texto del enlace] (https://ejemplo.com)  
![Texto alternativo] (imagen.png)

## Bloques de código

```
```javascript
const x = 10;
console.log(x);
```
```

## Citas

```
> Esto es una cita
> Puede ocupar varias líneas
```

## Tablas

|           |           |           |
|-----------|-----------|-----------|
| Columna 1 | Columna 2 | Columna 3 |
| -----     | -----     | -----     |
| Dato 1    | Dato 2    | Dato 3    |
| Dato 4    | Dato 5    | Dato 6    |

## Separadores

---

## Comparativa: Markdown vs Word

### Word / Google Docs

Formato binario, pesado, depende de un programa, difícil de versionar, mal para código.

### Markdown

Texto plano, ligero, funciona en cualquier editor, se versiona con Git, perfecto para código y documentación técnica.

## Aclaracion: “¿Markdown es difícil?”

► 🤔 ¿Necesito aprender mucho para usar Markdown?

## Consejos avanzados

### 1. Tabla de contenido

```
Contenido
- [Introducción] (#introducción)
- [Instalación] (#instalación)
- [Uso] (#uso)
```

### 2. Checkboxes

```
- [x] Tarea completada
- [] Tarea pendiente
```

### 3. Emoji con código

```
:rocket: → 🚀
:warning: → ⚠️
:white_check_mark: → ✅
```

### 4. HTML inline

```
<details>
<summary>Clic para expandir</summary>

Contenido colapsable aquí.

</details>
```

## Mini-chequeo

► ¿Qué símbolo crea un encabezado H2?

► ¿Cómo se escribe negrita en Markdown?

► ¿Cómo se inserta un enlace?

► ¿Qué es un bloque de código en Markdown?

## ✓ Resumen en 3 frases

1. Markdown es **texto plano con reglas**: se escribe en cualquier editor y se publica en múltiples formatos.
2. Dominar `#`, `**`, `-`, `>` y `[texto](url)` cubre el **80% de lo que necesitas**.
3. Usa Markdown para READMEs, documentación técnica, wikis y notas: es el **estándar de la industria**.

 [Volver al índice de la unidad](#) · Anterior: [04 — Manual de usuario](#) · Siguiente: [06 — Herramientas doc](#)

📖 Estás en: 📁 UD 1.6 · Documentación → 06 · Herramientas doc

📁 La herramienta correcta para cada tipo de doc

No toda la documentación se escribe igual. Un README va en Markdown, un diagrama va en draw.io, una wiki colaborativa va en Notion. Elegir la herramienta correcta te ahorra tiempo y mejora el resultado.

📁 La idea en una frase

No existe una herramienta perfecta: usa la adecuada para cada tipo de documentación.

Mapa de herramientas

Herramienta	Tipo	Ideal para	Coste
Markdown	Texto plano	READMEs, doc técnica, notas	Gratis
GitHub Wiki	Wiki versionada	Wiki del proyecto	Gratis
Notion	Wiki colaborativa	Equipo, notas, databases	Freemium
GitBook	Doc técnica	Documentación de API, guías	Freemium
draw.io	Diagramas	Arquitectura, flujos, UML	Gratis
Excalidraw	Bocetos	Wireframes, diagramas rápidos	Gratis
VitePress	S estática	Doc técnica generada desde MD	Gratis
Docusaurus	S estática	Doc de Facebook, open source	Gratis

# Detalle de cada herramienta

## Markdown ( `.md` )

El estándar. Funciona en GitHub, VS Code, Obsidian y casi cualquier editor.

**Ventajas:** Simple, universal, versionable con Git. **Inconvenientes:** Sin formato visual, sin colaboración en tiempo real.

## GitHub Wiki

Wiki integrada en cada repositorio de GitHub.

**Ventajas:** Integrada con el repo, Markdown, gratis. **Inconvenientes:** Fuera del repo (no versionada con el código), menos conocida.

## Notion

Wiki colaborativa con bases de datos, tablas y galerías.

**Ventajas:** Muy visual, colaborativa, múltiples vistas. **Inconvenientes:** No versionable con Git, depende de internet, coste en equipo.

## GitBook

Plataforma de documentación técnica con Markdown y UI bonita.

**Ventajas:** Resultado profesional, búsqueda, versionado. **Inconvenientes:** Coste en planes avanzados, menos flexible.

## draw.io (diagrams.net)

Editor de diagramas online y de escritorio.

**Ventajas:** Gratis, muchas plantillas, exporta PNG/SVG/PDF. **Inconvenientes:** Curva de aprendizaje, diagramas pueden quedar pesados.

## Excalidraw

Bocetos y diagramas estilo “dibujado a mano”.

**Ventajas:** Rápido, intuitivo, estilo informal. **Inconvenientes:** No es profesional, limitado para diagramas complejos.

## Notion

Visual, colaborativo, bases de datos, múltiples vistas. Pero no se versiona con Git y depende de internet.

## GitHub Wiki

Integrada con el repo, Markdown, gratis, versionable. Pero menos visual y menos conocida.

## Aclaracion: “¿Cuál uso para mi proyecto?”

► 🤔 ¿Qué herramienta elijo?

## Mini-chequeo

► ¿Qué herramienta es la mejor para diagramas de arquitectura?

► ¿Qué ventaja tiene Markdown sobre Word?

► ¿Cuándo usar Notion vs GitHub Wiki?

## ✅ Resumen en 3 frases

1. Cada tipo de documentación necesita una herramienta: **Markdown para texto, draw.io para diagramas, Notion para wikis.**

2. Para un proyecto intermodular, la combinación básica es **Markdown + draw.io + GitHub.**

 [Volver al índice de la unidad](#) · Anterior: [05 — Markdown](#) · Siguiente: [07 — Errores de documentación](#)

## 07 — Errores de documentación

 **Estás en:**  **UD 1.6 · Documentación** → [07 · Errores de documentación](#)

 *Los errores de documentación son predecibles — y evitables*

Todos cometemos los mismos errores. La buena noticia es que, conociéndolos, puedes evitarlos. Esta sección enumera los errores más frecuentes que cometen los proyectos intermodulares y cómo solucionarlos.

### La idea en una frase

**Los errores de documentación no son de ortografía: son de claridad, utilidad y oportunidad.**

## Los 7 errores más comunes

### 1. “Lo documento luego”

El error #1. La documentación se deja para el final, cuando ya no hay tiempo ni ganas. Resultado: un README vacío y una memoria <sup>抄</sup>ada la noche antes.



**Solución:** Documenta **mientras desarrollas**, no después. Cada funcionalidad nueva = su documentación.

## 2. Demasiado técnico

Llenar el manual de usuario con “ejecuta npm run build” y “configura el webpack”. El usuario no técnico no entiende nada.

**Solución:** Piensa en tu audiencia. Si es para usuarios, sin jerga. Si es para desarrolladores, con detalle técnico.

## 3. Demasiado vago

“La app hace cosas”. “Funciona con bases de datos”. “Se puede usar”.

**Solución:** Sé específico. ¿Qué hace? ¿Cómo se usa? ¿Qué problemas resuelve? Detalles, no generalidades.

## 4. Obsoleto

La documentación no se actualiza cuando el código cambia. El README dice “npm run dev” pero ahora es “npm run serve”.

**Solución:** Actualiza la documentación **en el mismo commit** que modificas el código. Docs as Code.

## 5. Sin estructura

Un párrafo largo sin encabezados, sin listas, sin separación. Nadie lee eso.

**Solución:** Usa encabezados, listas, tablas y separadores. Organiza por secciones lógicas.

## 6. Sin capturas

Descripciones textuales de la interfaz cuando una imagen lo explicaría mejor.

**Solución:** Incluye capturas de pantalla de cada pantalla importante. Señala los botones relevantes.

## 7. Solo para el tribunal

Documentar “para que quede bien” en vez de “para que sea útil”. El tribunal nota la diferencia.

**Solución:** Escribe documentación que tú mismo usarías si tuvieras que volver al proyecto en 3 meses.

# Comparativa: error vs corrección

## Error

Lo documento luego. Ya veré eso cuando termine todo (Spoiler: nunca se documenta bien).

## Corrección

Documento cada funcionalidad cuando la termino. 10 minutos por feature = documentación siempre al día.

## Aclaracion: “¿Y si ya cometí todos estos errores?”

► 🤔 ¿Todo está perdido si ya hice mi proyecto sin documentar?

## Mini-chequeo

► ¿Cuál es el error #1 de documentación?

► ¿Qué significa “documentación obsoleta”?

► ¿Por qué es malo documentar “solo para el tribunal”?

## ✓ Resumen en 3 frases

1. Los errores más comunes son: **“lo documento luego”, demasiado técnico, demasiado vago y obsoleto.**

2. La solución es documentar **mientras desarrollas**, no al final, y actualizar en el mismo commit.
3. Escribe documentación que **tú mismo usarías** si tuvieras que volver al proyecto.

 Volver al [índice de la unidad](#) · Anterior: [o6 — Herramientas doc](#) · Siguiente: [o8 — Plantilla de doc](#)

## o8 — Plantilla de documentación

 Estás en:  **UD 1.6 · Documentación** → [o8](#) · [Plantilla de doc](#)

 *No empieces desde cero: usa esta plantilla*

La mejor forma de documentar es **partir de una estructura ya hecha**. Esta plantilla incluye todo lo que el tribunal espera ver. Cópiala, adapta los campos y ya tienes tu documentación lista.

### La idea en una frase

**Una plantilla bien diseñada te garantiza que no olvidas nada importante.**

## Plantilla completa

**README.md**

```
[Nombre del Proyecto]

[Descripción de 1-2 líneas: qué hace la app y para quién es]

Características

- [Funcionalidad 1]
- [Funcionalidad 2]
- [Funcionalidad 3]
- [Funcionalidad 4]

Tecnologías utilizadas

| Tecnología | Propósito |
|-----|-----|
| Vue 3 | Framework frontend |
| Tailwind CSS | Estilos |
| Vite | Build tool |
| GitHub Pages | Despliegue |

Instalación

Requisitos previos
- Node.js 18+ (https://nodejs.org)
- npm o pnpm

Pasos

1. Clonar el repositorio
  ```bash
  git clone https://github.com/usuario/proyecto.git
  cd proyecto
```

2. Instalar dependencias

```
npm install
```

3. Iniciar en modo desarrollo

```
npm run dev
```

4. Abrir en el navegador

```
http://localhost:5173
```

Uso

[Descripción breve de cómo usar la aplicación]

[Incluir 1-2 capturas de pantalla]

```
src/
├── components/      # Componentes reutilizables
├── views/           # Vistas/páginas
├── stores/          # Estado global (Pinia)
├── utils/           # Funciones auxiliares
├── assets/          # Imágenes y estilos
└── App.vue          # Componente raíz
```

Equipo

Nombre	Rol
[Nombre 1]	Frontend + Diseño
[Nombre 2]	Backend + BD
[Nombre 3]	Documentación + Testing

Licencia

Este proyecto está bajo la licencia [MIT / CC BY-SA 4.0] — mira el archivo [LICENSE](#) para detalles.

```
---

## Plantilla de documentación técnica

```markdown
Documentación Técnica — [Nombre]

Arquitectura general

[Diagrama de capas o descripción]

Modelo de datos

| Entidad | Campos | Tipo |
|-----|-----|-----|
| Tarea | id, titulo, prioridad, fecha | JSON |

Funciones principales

[Nombre de la función]
- **Propósito**: [Qué hace]
- **Parámetros**: [Qué recibe]
- **Retorna**: [Qué devuelve]
- **Ejemplo**: [Código de ejemplo]

Decisiones de diseño

| Decisión | Alternativa descartada | Razón |
|-----|-----|-----|
| Vue 3 | React | Equipo conocía mejor Vue |
| Tailwind | CSS puro | Desarrollo más rápido |
```

# Plantilla de manual de usuario

```
Manual de Usuario — [Nombre]

Introducción

[Nombre] es una aplicación que [qué hace en 1 frase].
Está dirigida a [usuarios: profesores, alumnos, etc.]

Primeros pasos

1. Abre la aplicación en el navegador
2. [Paso 2]
3. [Paso 3]

Funcionalidades

[Funcionalidad 1]
[Descripción]
1. [Paso 1 con captura]
2. [Paso 2 con captura]

[Funcionalidad 2]
[Descripción]
1. [Paso 1]
2. [Paso 2]

Preguntas frecuentes

¿No se abre la app?
Comprueba que Node.js está instalado: `node -v`

¿No se guardan los datos?
[Solución]

Contacto

Si tienes problemas, contacta con [email o GitHub].
```

## Comparativa: plantilla vs empezar de cero

### Sin plantilla

Abres un archivo en blanco. No sabes qué poner. Pasas 2 horas pensando en la estructura. Olvidas secciones importantes.

### Con plantilla

Copias la plantilla. Rellenas los campos. En 30 minutos tienes una documentación completa y estructurada.

## Aclaracion: “¿Puedo modificar la plantilla?”

►  ¿La plantilla es fija o puedo adaptarla?

## Mini-chequeo

► ¿Cuáles son las 3 plantillas que ofrece esta sección?

► ¿Qué información debe llevar el apartado “Equipo”?

► ¿Se puede modificar la plantilla?

## Resumen en 3 frases

1. La plantilla incluye **todo lo que el tribunal espera**: README, doc técnica y manual de usuario.
2. **Copia, adapta y rellena**: en 30 minutos tienes una documentación completa.
3. La plantilla es un **punto de partida**, no una camisa de fuerza: modifícala según tu proyecto.

## 09 — Cierre

 Estás en:  UD 1.6 · Documentación → 09 · Cierre

 *Ya sabes documentar. Ahora toca aplicarlo.*

Has recorrido toda la unidad: desde por qué documentar es importante, hasta cómo usar una plantilla. Ahora toca **verificar que lo entiendes** y **aplicarlo a tu proyecto**. El conocimiento sin práctica no sirve de nada.

### La idea en una frase

**La documentación es una habilidad profesional: escríbela mientras desarrollas, no al final.**

Si recuerdas esto, has entendido la unidad.

### Mini-chequeo final

Responde estas preguntas antes de seguir:

#### Preguntas de comprensión

► ¿Qué es Docs as Code?

► ¿Cuáles son las secciones mínimas de un README?



► ¿Cuál es la diferencia entre doc técnica y manual de usuario?

► ¿Qué es el error #1 de documentación?

► ¿Qué herramientas se recomiendan para documentar?

► ¿Cómo se escribe Markdown?

---

## Ejercicio práctico

**En 60 minutos, documenta tu proyecto completo:**

1. **Crea el README** (15 min): Usa la plantilla del punto 08
2. **Escribe la doc técnica** (15 min): Arquitectura, setup, decisiones
3. **Crea el manual de usuario** (15 min): 3 funcionalidades con pasos y capturas
4. **Revisa errores** (15 min): ¿Tienes los 7 errores? Corrígelos

**Si consigues documentar todo esto → tu proyecto está listo para el tribunal.**

---



## Tabla resumen de la unidad



## Conexión con las siguientes unidades

Unidad	Qué aprenderás	Cómo se conecta
<b>1.7 Propuesta</b>	Cómo escribir la propuesta técnica	La documentación es la base de la propuesta
<b>PI2 Sprints</b>	Cómo ejecutar el proyecto	Documentarás cada sprint y cada funcionalidad
<b>Entrega final</b>	Cómo presentar el proyecto	Tu documentación será evaluada por el tribunal

## Consejo para el siguiente punto

Si todavía no tienes claro cómo documentar tu proyecto:

1. **Empieza por el README** — es lo que el tribunal verá primero
2. **Usa la plantilla** — no reinventes la rueda
3. **Documenta una funcionalidad** — practica antes de documentar todo
4. **Pide feedback** — que un compañero lea tu doc y te diga si la entiende

**No necesitas documentación perfecta. Necesitas documentación útil.**

## Resumen en 3 frases

1. La documentación se compone de **3 tipos**: README (entrada), doc técnica (cómo está hecho) y manual de usuario (cómo se usa).
2. **Docs as Code** es la filosofía: escribe en Markdown, versiona con Git, publica automáticamente.
3. **Documenta mientras desarrollas**, no al final — y usa la plantilla para no olvidar nada.

## 01 — ¿Qué es una propuesta?

 Estás en:  **UD 1.7 · Propuesta de Proyecto** → 01 · ¿Qué es una propuesta?

 *La propuesta es tu plan de acción por escrito*

Antes de escribir una línea de código, necesitas **definir qué vas a hacer, por qué lo vas a hacer y cómo lo vas a hacer**. La propuesta de proyecto es ese documento. Es el contrato entre tu equipo y el tribunal: lo que prometes entregar, en qué plazo y con qué alcance.

### La idea en una frase

**Una propuesta bien escrita evita malentendidos, define expectativas y da seguridad al tribunal.**

## ¿Qué es una propuesta de proyecto?

Es un documento formal que responde a **5 preguntas fundamentales**:

Pregunta	Sección de la propuesta
¿Qué vamos a hacer?	Descripción del proyecto
¿Por qué lo hacemos?	Justificación y objetivos
¿Para quién lo hacemos?	Usuario objetivo
¿Cómo lo haremos?	Tecnologías y metodología
¿Cuándo lo entregaremos?	Cronograma y hitos

---

## Estructura de una propuesta

1. Portada
  - Nombre del proyecto
  - Equipo
  - Centro y curso
  - Fecha
2. Introducción y justificación
  - Problema que resuelve
  - Por qué es relevante
  - Beneficios esperados
3. Objetivos
  - Objetivo general
  - Objetivos específicos
4. Usuario objetivo
  - ¿Quién lo usará?
  - ¿Qué necesita?
5. Funcionalidades
  - Lista priorizada
  - Descripción de cada una
6. Tecnologías y metodología
  - Stack tecnológico
  - Metodología de trabajo
7. Cronograma
  - Fases y hitos
  - Fechas clave
8. Equipo
  - Roles y distribución

---

## ## 02 - Briefing

> 🗺️ **\*\*Estás en:\*\*** 📄 **\*\*UD 1.7 · Propuesta de Proyecto\*\*** → 02 · Briefing

> 🗺️ El briefing es tu mapa: léelo antes de dibujar la ruta\_

>

> Antes de generar ideas, necesitas **\*\*entender qué te piden\*\***. El briefing

> (o encargo) es el documento del profesor que define las reglas del juego:

> qué debe incluir el proyecto, qué tecnologías son obligatorias, cómo se

> evalúa y cuándo se entrega.

## ## 🗨️ La idea en una frase

> **\*\*Un briefing bien leído evita hacer un proyecto que no cumple los requisitos.\*\***

---

## ## Qué buscar en el briefing

### ### 1. Requisitos funcionales

¿Qué **\*\*debe hacer\*\*** la aplicación?

Ejemplo: - CRUD de tareas - Sistema de usuarios con login - Filtros por categoría y prioridad - Exportación a PDF

### 2. Requisitos no funcionales

¿Cómo debe **funcionar** la aplicación?

Ejemplo: - Responsive (móvil y escritorio) - Tiempo de carga < 3 segundos - Compatible con Chrome, Firefox, Safari - Accesibilidad básica (WCAG 2.1 AA)

### 3. Restricciones técnicas

¿Qué **tecnologías** son obligatorias?

Ejemplo: - Frontend: Vue 3 o React - Estilos: Tailwind CSS o CSS Modules - Build: Vite o Webpack - Despliegue: GitHub Pages

### 4. Criterios de evaluación

¿Qué **valora el tribunal**?

Ejemplo: - Funcionalidad: 30% - Código: 25% - Diseño: 15% - Documentación: 15% - Presentación: 15%

### 5. Fechas y entregables

¿Cuándo **se entrega** qué?

Ejemplo: - Propuesta: semana 3 - Prototipo: semana 6 - Versión alpha: semana 10 - Entrega final: semana 14

---

## Comparativa: briefing bien vs mal leído

> **Mal leído**

>

> El equipo empieza a programar sin leer el briefing. A mitad de proyecto descubren que

> **Bien leído**

>

> El equipo lee el briefing, extrae requisitos, tecnologías y fechas. Lo pegan en un doc

---

## Aclaracion: "¿Y si el briefing es ambiguo?"

<details>

<summary>🤔 ¿Qué hago si no entiendo algo del briefing?</summary>

Pregunta al profesor. Es mejor preguntar **al principio** que descubrir un error al final. Usa un canal claro (email, foro, clase) y sé específico: "En el punto 3 dice 'sistema de autenticación'. ¿Implica login con contraseña o solo roles sin login?"

No asumas. Pregunta.

</details>

---

## Mini-chequeo

<details>

<summary>¿Cuáles son las 5 cosas que debes buscar en un briefing?</summary>

**Respuesta:** Requisitos funcionales, requisitos no funcionales, restricciones técnicas

**Para qué sirve:** Para no olvidar ningún requisito al diseñar tu propuesta.

</details>

<details>

<summary>¿Qué es un requisito funcional?</summary>

**Respuesta:** Algo que la aplicación **debe hacer**: CRUD, filtros, exportación, etc.

**Para qué sirve:** Para definir el alcance de tu proyecto.

</details>

<details>

<summary>¿Qué es un requisito no funcional?</summary>

**Respuesta:** Cómo debe funcionar: rendimiento, accesibilidad, compatibilidad, responsi

**Para qué sirve:** Para que la app no solo funcione, sino que funcione bien.

</details>

<details>

<summary>¿Qué hacer si el briefing es ambiguo?</summary>

**Respuesta:** Preguntar al profesor. No asumas: pregunta al principio, no al final.

**Para qué sirve:** Para evitar malentendidos costosos.

</details>

---



##  Resumen en 3 frases

1. El briefing define las **reglas del juego**: requisitos, tecnologías, evaluación y feedback.
2. Léelo completo antes de generar ideas: **extrae requisitos funcionales y no funcionales**.
3. Si algo es ambiguo, **pregunta al profesor** – no asumas.

---

>  Volver al [índice de la unidad](../ul-7-propuesta) · Anterior: [01 – ¿Qué es una propuesta?]

---

## 03 – Ideas de proyecto

>  **Estás en:**  **UD 1.7 · Propuesta de Proyecto** → 03 · Ideas de proyecto

>  La mejor idea no siempre es la primera que se te ocurre\_

>

- > "¿Qué hacemos para el proyecto?" Es la pregunta que genera más nervios.
- > No tiene por qué ser así. Existen técnicas para **generar ideas de forma estructurada**, no solo esperar a que te caiga del cielo. La clave es
- > generar muchas ideas primero, y filtrar después.

##  La idea en una frase

> **Genera muchas ideas sin juzgar, y solo después evalúa cuál es la mejor.**

---

## Técnicas de brainstorming

### 1. Lluvia de ideas clásica

1. Escribe todas las ideas que se te ocurran (sin ~~过滤~~ar)
2. No critiques ninguna en esta fase
3. Busca cantidad, no calidad
4. Después, agrupa y evalúa

**Ejemplo:** En 10 minutos, un equipo de 3 puede generar 30-50 ideas.

### 2. SCAMPER

Modifica algo existente:

Letra	Acción	Pregunta
-----	-----	-----
<b>S</b>	Sustituir	¿Qué puedo cambiar?
<b>C</b>	Combinar	¿Qué puedo unir?
<b>A</b>	Adaptar	¿A qué se parece?
<b>M</b>	Modificar	¿Qué puedo ampliar o reducir?
<b>P</b>	Poner en otro uso	¿Para qué más sirve?
<b>E</b>	Eliminar	¿Qué puedo quitar?
<b>R</b>	Revertir	¿Y si lo hago al revés?

**Ejemplo:** "App de tareas" → Sustituir tareas por retos → App de gamificación.

### 3. Mapa mental

└─ Calendario  
└─ Tareas

Quiz

Aprendizaje —|— Flashcards —|— Línea temporal

Encuesta

Comunicación —|— Foro —|— Murmura

### ### 4. Analizar apps existentes

Busca apps que te gusten y pregunta: ¿qué le falta? ¿Cómo lo haría diferente? ¿Qué problema no resuelve?

### ### 5. Empatizar con usuarios

Pregunta a compañeros, profes, padres: ¿qué problema tienen que una app podría resolver? Los mejores proyectos nacen de problemas reales.

---

## ## Comparativa: idear solo vs en equipo

> **Solo**

>

> Tienes 5 ideas, todas parecidas. No tienes feedback. Te quedas con la primera porque r

> **En equipo**

>

> 3 personas generan 30 ideas. Se complementan: uno piensa en UX, otro en técnica, otro

---

## ## Aclaracion: "¿Y si no se me ocurre nada?"

<details>

<summary>😬 ¿Qué hago si no tengo ni una sola idea?</summary>

Esto es normal. Prueba estas estrategias:

1. **Busca problemas reales**: ¿Qué te frustra del día a día?
2. **Mira apps existentes**: ¿Qué le falta a las que usas?
3. **Cambia de contexto**: Pasea, duerme, ablación. Las ideas vienen cuando no las busca
4. **Usa SCAMPER**: Modifica una app que ya conoces.
5. **Pide a otros**: "¿Qué app te gustaría que existiera?"

La primera idea nunca es la mejor. Genera 20 y elige 1.

</details>

---

## ## Mini-chequeo

<details>

<summary>¿Cuáles son las 5 técnicas de brainstorming?</summary>

**Respuesta**: Lluvia de ideas clásica, SCAMPER, mapa mental, analizar apps existentes y

**Para qué sirve**: Para tener múltiples formas de generar ideas cuando no se te ocurre  
</details>

<details>

<summary>¿Qué significa SCAMPER?</summary>

**Respuesta**: Sustituir, Combinar, Adaptar, Modificar, Poner en otro uso, Eliminar, Rev

**Para qué sirve**: Para no empezar desde cero: parte de lo que ya existe.

</details>

<details>

<summary>¿Por qué es mejor generar ideas en equipo?</summary>

**Respuesta**: Porque 3 personas generan más ideas y más diversas. Cada uno aporta una y

```

Para qué sirve: Para no quedarte con la primera idea que se te ocurre.
</details>

✅ Resumen en 3 frases

1. Usa técnicas estructuradas (SCAMPER, mapa mental, lluvia de ideas) para generar ideas.
2. Genera muchas, juzga pocas: la primera idea nunca es la mejor.
3. Empatiza con usuarios reales: los mejores proyectos nacen de problemas reales, no de ideas abstractas.

> 📖 Volver al [índice de la unidad](../ul-7-propuesta) · Anterior: [02 – Briefing](../ul-2-briefing)

04 – Selección de idea

> 📖 Estás en: 📄 UD 1.7 · Propuesta de Proyecto → 04 · Selección de idea

> 🎬 Tienes 20 ideas. ¿Cuál es la correcta?_
>
> Generar ideas es fácil. Elegir la correcta es lo difícil. No basta con
> que sea original o divertida: tiene que ser viabile, evaluada y
> alineada con el briefing. Esta sección te da criterios objetivos para
> tomar esa decisión.

🗂 La idea en una frase

> La mejor idea no es la más ambiciosa: es la que mejor equilibra originalidad, viabilidad y alineación con el briefing.

Criterios de selección

Evalúa cada idea del 1 al 5 en estos criterios:

| Criterio | Pregunta | Peso |
|-----|-----|-----|
| Viabilidad técnica | ¿Podemos hacerlo con lo que sabemos? | 25% |
| Alineación con el briefing | ¿Cumple los requisitos obligatorios? | 25% |
| Originalidad | ¿Es diferente a lo que ya existe? | 15% |
| Utilidad | ¿Resuelve un problema real? | 15% |
| Esfuerzo | ¿Es realista en el tiempo disponible? | 20% |

Fórmula de puntuación

```

**Puntuación = (Viabilidad × 0.25) + (Briefing × 0.25) + (Originalidad × 0.15) + (Utilidad × 0.15) + (Esfuerzo × 0.20)**

---

## ## Ejemplo de evaluación

Idea	Viab.	Briefing	Orig.	Util.	Esf.	**Total**
App de tareas escolares	5	5	2	3	5	**4.05**
Generador de horarios	4	4	3	4	4	**3.85**
Simulador de laboratorio	2	4	5	3	2	**3.15**
Portfolio gamificado	5	4	4	4	4	**4.25**

**\*\*Ganadora\*\***: Portfolio gamificado – equilibra viabilidad, originalidad y utilidad.

---

## ## Comparativa: idea ambiciosa vs realista

> **\*\*Idea ambiciosa\*\***

>

> Simulador de laboratorio de química en 3D con física real. Suena increíble. Pero neces

> **\*\*Idea realista\*\***

>

> Portfolio gamificado con logros y niveles. Usa Vue + Tailwind, lo que ya sabes. Origin

---

## ## Aclaracion: "¿Y si mi idea ya existe?"

<details>

<summary>🤔 ¿Puedo hacer algo que ya existe?</summary>

Sí, pero **\*\*con un enfoque diferente\*\***. No se trata de copiar, sino de mejorar o adaptar:

- ¿Qué le falta a la app existente?
- ¿Para quién no está pensada?
- ¿Qué función podría añadir?
- ¿Cómo lo harías más simple?

"Ya existe" no es un impedimento. "Ya existe y lo hago igual" sí lo es.

</details>

---

## ## Mini-chequeo

<details>

<summary>¿Cuáles son los 5 criterios de selección?</summary>

**\*\*Respuesta\*\***: Viabilidad técnica, alineación con el briefing, originalidad, utilidad y

**\*\*Para qué sirve\*\***: Para evaluar ideas de forma objetiva, no por capricho.

</details>

<details>

<summary>¿Qué peso tiene la viabilidad técnica?</summary>

**\*\*Respuesta\*\***: 25%. Es uno de los criterios con mayor peso porque si no puedes hacerlo,

**\*\*Para qué sirve\*\***: Para no elegir ideas que luego no puedes construir.

</details>

<details>

```

<summary>¿Qué hacer si mi idea ya existe?</summary>

Respuesta: Adaptarla: ¿qué le falta? ¿Para quién no está pensada? ¿Cómo la harías d...

Para qué sirve: Para no descartar buenas ideas solo porque no son 100% originales.
</details>

✅ Resumen en 3 frases

1. Evalúa cada idea con **5 criterios ponderados**: viabilidad, briefing, originalidad,
2. **La idea ganadora equilibra todo**: no es la más ambiciosa, sino la más realista y c
3. Si ya existe algo similar, **adapta y mejora**: no copies, innova.

> 📖 Volver al [índice de la unidad](../ul-7-propuesta) · Anterior: [03 – Ideas de proy

05 – Definición de alcance

> 📖 **Estás en:** 📄 **UD 1.7 · Propuesta de Proyecto** → 05 · Definición de alcance

> 🎬 Decidir qué NO hacer es tan importante como decidir qué hacer_
>
> El error más común en proyectos es querer hacer **demasiado**. El alcance
> define los límites de tu proyecto: qué funcionalidades incluye, cuáles
> quedan para después y cuáles no entran en absoluto. Sin alcance claro,
> el proyecto crece hasta explotar.

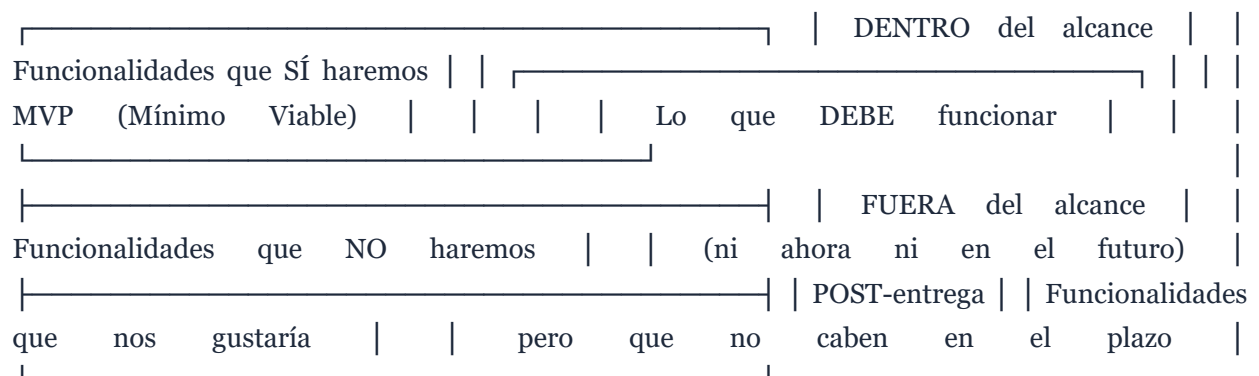
🗒 La idea en una frase

> **Un proyecto pequeño bien terminado vale más que uno grande a medias.**

¿Qué es el alcance?

El alcance es la **frontera** de tu proyecto. Divide todo en tres zonas:

```



```

Cómo definir el alcance

1. Lista de funcionalidades

Escribe todo lo que la app podría hacer. Después, clasifícalo:

| Prioridad | Funcionalidad | Estado |
|-----|-----|-----|
|  P0 | CRUD de tareas | Obligatorio |
|  P0 | Login de usuarios | Obligatorio |
|  P1 | Filtros y búsqueda | Importante |
|  P1 | Exportación a PDF | Importante |
|  P2 | Modo oscuro | Deseable |
|  P2 | Notificaciones push | Deseable |
|  P3 | Chat en tiempo real | Futuro |

2. Regla del 80/20

El 80% del valor viene del 20% de las funcionalidades. Identifica ese 20% y enfócate ahí primero.

3. Definición de "hecho"

Para cada funcionalidad, define cuándo está "hecha":

```markdown
### CRUD de tareas
- [x] Crear tarea con título y descripción
- [x] Editar tarea existente
- [x] Eliminar tarea con confirmación
- [x] Marcar como completada
- [ ] Drag and drop para reordenar (POST-entrega)

```

Comparativa: proyecto sin alcance vs con alcance

Sin alcance

Empezamos con 10 funcionalidades. A mitad de proyecto llevamos 2 hechas y 8 a medias. El código es un caos. Entregamos algo que no funciona del todo.

Con alcance

Empezamos con 5 funcionalidades P0. Las terminamos todas. Añadimos 2 P1. Entregamos algo sólido, funcional y bien hecho.

Aclaracion: “¿Y si el tribunal pide más?”

▶ 🤔 ¿Qué pasa si el profesor espera más funcionalidades?

Mini-chequeo

▶ ¿Cuáles son las 3 zonas del alcance?

▶ ¿Qué es un MVP?

▶ ¿Qué es la regla del 80/20?

✓ Resumen en 3 frases

1. El alcance define **qué entra, qué no y qué queda para después** en tu proyecto.
2. **Un proyecto pequeño bien terminado** vale más que uno grande a medias.
3. Usa prioridades (Po, P1, P2) y la **regla del 80/20** para enfocarte en lo esencial.

 Volver al [índice de la unidad](#) · Anterior: [04 — Selección de idea](#) · Siguiente: [06 — Cronograma](#)

06 — Cronograma

📅 *Un sin fecha es un sindeadline: nunca se termina*

“Lo tenemos para la semana que viene” es la frase más peligrosa de un proyecto. Sin fechas claras, las tareas se expanden hasta llenar todo el tiempo disponible. El cronograma es tu mecanismo de control: divide el proyecto en fases, asigna fechas y cumple.

📌 La idea en una frase

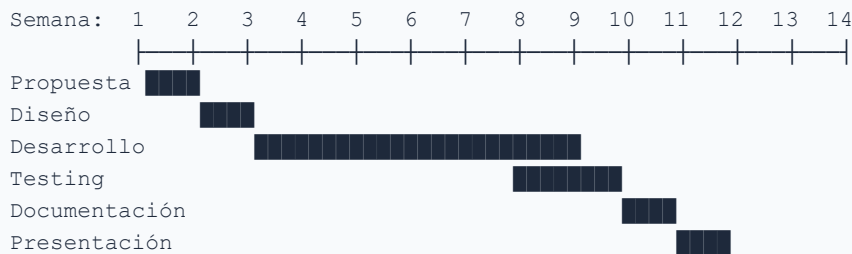
Un cronograma realista se basa en hitos, no en la esperanza de que “saliendo bien”.

Estructura de un cronograma

1. Fases del proyecto

Fase	Duración típica	Entregable
Propuesta	1-2 semanas	Documento de propuesta
Diseño	1-2 semanas	Prototipo, wireframes
Desarrollo	6-8 semanas	App funcional
Testing	1-2 semanas	Bugs corregidos
Documentación	1 semana	README, memoria, manual
Presentación	1 semana	Pitch, diapositivas, demo

2. Diagrama de Gantt simplificado



3. Hitos (milestones)

Los hitos son **puntos de control** donde verificas que todo va bien:

Hito	Semana	Qué debe estar listo
Hito 1	2	Propuesta aprobada
Hito 3	4	Prototipo visual validado
Hito 5	8	MVP funcional
Hito 7	12	Versión completa
Hito 8	14	Entrega final

Cómo estimar tiempos

1. Estimación por partes

Descompón cada funcionalidad en tareas pequeñas:

```

CRUD de tareas:
- Diseño de la vista: 2h
- Componente de formulario: 3h
- Lógica de crear/editar: 4h
- Lógica de eliminar: 1h
- Estilos y responsive: 2h
- Testing: 2h
Total: 14h → 2 días
  
```

2. Multiplica por 1.5

Los tiempos siempre se pasan. Multiplica tu estimación por 1.5 para obtener un tiempo realista.

3. Deja margen

Reserva un **20% del tiempo** para imprevistos: bugs, dudas, cambios de última hora.

Comparativa: cronograma vs “ya veremos”

Sin cronograma

Empezamos con ganas. A mitad de proyecto llevamos un tercio. Las prisas finales generan código malo. Entregamos algo incompleto.

Con cronograma

Cada semana sabemos qué hacer. Si nos retrasamos, lo vemos en el hito siguiente. Ajustamos a tiempo. Entregamos completo.

Aclaracion: “¿Y si no cumplimos un hito?”

► 🤔 ¿Qué pasa si llegamos al hito y no estamos donde queríamos?

Mini-chequeo

► ¿Cuáles son las 6 fases de un proyecto?

► ¿Qué es un hito?

► ¿Por qué multiplicar por 1.5?

✓ Resumen en 3 frases

1. Un cronograma realista se basa en **fases, hitos y deadlines**, no en la esperanza.
2. **Estima por partes, multiplica por 1.5 y deja margen** para imprevistos.
3. Si fallas un hito, **detecta, ajusta y comunica** — no esperes al final.

 Volver al [índice de la unidad](#) · Anterior: [05 — Definición de alcance](#) · Siguiente: [07 — Ejemplo de propuesta](#)

07 — Ejemplo de propuesta

 **Estás en:**  **UD 1.7 · Propuesta de Proyecto** → 07 · Ejemplo de propuesta

 *Un ejemplo vale más que mil explicaciones*

Esta sección muestra una **propuesta completa de ejemplo**. Úsala como referencia para estructurar la tuya. No la copies literalmente: adapta los contenidos a tu proyecto real.

La idea en una frase

La propuesta es un documento profesional: clara, completa y concisa.

PROPUESTA DE PROYECTO INTERMODULAR

PortfolioQuest – Portfolio Gamificado para Estudiantes

Equipo:

- María García – Frontend + Diseño
- Carlos López – Backend + Bases de datos
- Ana Martínez – Documentación + Testing

IES Tecnológico – Ciclo Formativo DAW

Curso 2026-2027

Fecha: 15 de septiembre de 2026

1. Introducción y justificación

Problema: Los estudiantes de FP no tienen un espacio unificado para recoger sus proyectos, habilidades y logros académicos. Los portfolios existentes son genéricos (GitHub, LinkedIn) y no reflejan la realidad de un estudiante.

Solución: Una aplicación web que permita a los estudiantes crear un portfolio gamificado con insignias, niveles y logros que reflejen su progreso académico.

Beneficios: - Visualización del progreso académico - Motivación mediante gamificación - Portfolio profesional para el mundo laboral - Herramienta transversal para cualquier materia

2. Objetivos

Objetivo general: Desarrollar una aplicación web de portfolio gamificado que permita a los estudiantes gestionar y mostrar sus logros académicos.

Objetivos específicos: 1. Sistema de registro y autenticación de usuarios 2. CRUD de proyectos con categorías y habilidades 3. Sistema de insignias y niveles por logros 4. Panel de progreso con estadísticas 5. Modo oscuro y responsive 6. Exportación del portfolio a PDF

3. Usuario objetivo

- **Quién:** Estudiantes de ciclos formativos (DAW, DAM, ASIR)
- **Qué necesita:** Un espacio para recoger proyectos y logros
- **Cómo lo usa:** Web en navegador, responsive (móvil y escritorio)

- **Por qué lo usará:** Para tener un portfolio profesional y motivador

4. Funcionalidades

#	Funcionalidad	Prioridad	Descripción
1	Registro/Login	● Po	Autenticación con email y contraseña
2	CRUD de proyectos	● Po	Crear, editar, eliminar proyectos
3	Sistema de insignias	● Po	Logros automáticos por acciones
4	Panel de progreso	● P1	Estadísticas y nivel de usuario
5	Búsqueda y filtros	● P1	Filtrar por categoría, habilidad
6	Modo oscuro	● P2	Tema claro/oscuro
7	Exportación PDF	● P2	Generar PDF del portfolio
8	Compartir enlace	● P3	Link público del portfolio

5. Tecnologías y metodología

Capa	Tecnología	Justificación
Frontend	Vue 3 + Composition API	Framework principal, conocido por el equipo
Estilos	Tailwind CSS	Desarrollo rápido, responsive
Build	Vite	Rápido, moderno, bien integrado con Vue
Estado	Pinia	Estado global sencillo
Datos	localStorage / IndexedDB	Sin backend, datos en cliente
Despliegue	GitHub Pages	Gratuito, integrado con Git

Metodología: Scrum Lite con sprints de 1 semana, daily asíncrona por Discord, revisiones semanales.

6. Cronograma

Semana	Fase	Hito
1-2	Propuesta + Diseño	Hito 1: Propuesta aprobada
3-4	Prototipo + wireframes	Hito 2: Diseño validado
5-8	Desarrollo MVP	Hito 3: MVP funcional
9-10	Funcionalidades extra	Hito 4: Versión completa
11-12	Testing + corrección	Hito 5: Versión estable
13	Documentación	Hito 6: Docs completas
14	Presentación	Hito 7: Entrega final

7. Equipo

Miembro	Rol	Responsabilidades
María	Frontend + Diseño	Componentes Vue, estilos, UX
Carlos	Backend + BD	Lógica, estado, persistencia
Ana	Documentación + Testing	README, memoria, tests, bugs

Comunicación: Discord (canal del proyecto), reunión semanal (viernes).

Aclaracion: “¿Puedo usar esta plantilla?”

 ¿Copio la propuesta y cambio los datos?

► ¿Cuáles son las 7 secciones de una propuesta completa?

► ¿Qué diferencia hay entre objetivo general y específicos?

► ¿Por qué justificar cada tecnología?

✓ Resumen en 3 frases

1. Una propuesta completa tiene **7 secciones**: portada, justificación, objetivos, usuario, funcionalidades, tecnología y cronograma.
2. **Justifica cada decisión**: por qué esa tecnología, por qué esas funcionalidades, por qué ese orden.
3. Usa el ejemplo como **referencia, no como copia**: adapta los contenidos a tu proyecto real.

 Volver al [índice de la unidad](#) · Anterior: [o6 — Cronograma](#) · Siguiente: [o8 — Errores de propuesta](#)

o8 — Errores de propuesta

 Estás en:  **UD 1.7 · Propuesta de Proyecto** → o8 · Errores de propuesta

 *Los errores de propuesta se pagan caro — y son evitables*

Una propuesta mal escrita genera malentendidos, expectativas incorrectas y un proyecto desordenado. Los errores más comunes no son de ortografía, sino de **alcance, claridad y realismo**. Conócelos y evítalos.

La idea en una frase

Los errores de propuesta se detectan en la evaluación: mejor prevenir que lamentar.

Los 7 errores más comunes

1. Demasiado ambicioso

“Vamos a hacer una app con 15 funcionalidades, IA, reconocimiento de voz y blockchain.” Suena genial. Pero en 14 semanas no lo harás ni con 3 personas.

Solución: Sé realista. Evalúa tu tiempo, tu equipo y tus habilidades. Menos funcionalidades bien hechas > muchas a medias.

2. Demasiado vago

“Haremos una app de tareas.” ¿Qué tipo de tareas? ¿Para quién? ¿Qué funcionalidades específicas? ¿Qué tecnologías?

Solución: Sé específico. “App de gestión de tareas escolares con CRUD, filtros por materia, prioridad y fecha de entrega, usando Vue 3 + Tailwind.”

3. Sin justificación

“No explicamos por qué hacemos esto.” El tribunal no entiende por qué tu proyecto existe. ¿Qué problema resuelve? ¿Para quién es útil?

Solución: Incluye siempre una sección de justificación con el problema que resuelves y el beneficio que aportas.

4. Copiar el briefing

Repetir textualmente los requisitos del briefing como si fueran tu propuesta. No aporta nada.

Solución: Interpreta el briefing. Añade tu enfoque, tu análisis y tus decisiones. No copies, transforma.

5. Cronograma imposible

“Desarrollamos todo en 2 semanas.” O “La última semana hacemos todo.” Ninguno de los dos es realista.

Solución: Estima por partes, multiplica por 1.5, distribuye el tiempo de forma uniforme y deja margen para imprevistos.

6. Sin equipo definido

“No sabemos quién hace qué.” Resultado: duplicación de trabajo, tareas olvidadas, conflictos.

Solución: Define roles claros desde el principio. Cada persona sabe qué le toca.

7. Sin plan B

“Si algo sale mal, ya improvisaremos.” Los imprevistos siempre llegan. Sin plan B, un problema se convierte en un desastre.

Solución: Piensa qué puede salir mal y qué harás si pasa. Ten alternativas preparadas.

Comparativa: propuesta débil vs sólida

Propuesta débil

Haremos una app de notas. Usaremos Vue. El equipo somos 3. Entregamos en 14 semanas. No hay justificación ni cronograma detallado.

Propuesta sólida

App de gestión de notas escolares con CRUD, filtros y exportación. Vue 3 + Tailwind + Pinia. Equipo con roles definidos. Cronograma con 7 hitos. Justificación: problema real de estudiantes.

Aclaración: “¿Y si mi proyecto es muy sencillo?”

- ▶ 🤔 ¿Necesito una propuesta larga si mi app es simple?

Mini-chequeo

- ▶ ¿Cuál es el error #1 de propuestas?

- ▶ ¿Por qué es malo copiar el briefing?

- ▶ ¿Qué es un plan B en una propuesta?

✓ Resumen en 3 frases

1. Los errores más comunes son: **demasiado ambicioso, demasiado vago, sin justificación y cronograma imposible.**
2. La propuesta sólida es **específica, justificada, realista y con roles definidos.**
3. Piensa en **qué puede salir mal** y prepara un plan B: eso demuestra madurez profesional.

 Volver al [índice de la unidad](#) · Anterior: [07 — Ejemplo de propuesta](#) · Siguiente: [09 — Cierre](#)

09 — Cierre

 Ya tienes todo lo necesario para escribir tu propuesta

Has recorrido toda la unidad: desde qué es una propuesta, hasta cómo evitar errores. Ahora toca **verificar que lo entiendes** y **aplicarlo a tu proyecto real**. La propuesta es el primer documento que entrega tu equipo — hazlo bien.

La idea en una frase

Una propuesta bien escrita es la mitad del proyecto: define todo antes de programar.

Si recuerdas esto, has entendido la unidad.

Mini-chequeo final

Responde estas preguntas antes de seguir:

Preguntas de comprensión

▶ ¿Cuáles son las 5 preguntas que responde una propuesta?

▶ ¿Qué es el briefing y por qué leerlo es el primer paso?

▶ ¿Cuáles son las 3 zonas del alcance?

▶ ¿Cuáles son las 6 fases de un proyecto?

▶ ¿Cuáles son las 7 secciones de una propuesta completa?

Ejercicio práctico

En 90 minutos, escribe tu propuesta completa:

1. **Lee el briefing** (10 min): Extrae requisitos, tecnologías, fechas
2. **Genera ideas** (15 min): Usa SCAMPER o lluvia de ideas
3. **Selecciona la idea** (10 min): Evalúa con los 5 criterios
4. **Define el alcance** (10 min): Lista de funcionalidades Po, P1, P2
5. **Crea el cronograma** (10 min): Fases, hitos, deadlines
6. **Escribe la propuesta** (25 min): Usa la plantilla del ejemplo
7. **Revisa errores** (10 min): ¿Tienes los 7 errores? Corrígelos

Si consigues escribir todo esto → tu propuesta está lista para entregar.

Tabla resumen de la unidad

Concepto	Idea clave
Propuesta	Documento que define qué, por qué, para quién, cómo y cuándo
Briefing	Encargo del profesor: requisitos, tecnologías, evaluación, fechas
Ideas	Genera muchas, juzga pocas. Usa SCAMPER, lluvia de ideas, empatía
Selección	5 criterios: viabilidad, briefing, originalidad, utilidad, esfuerzo
Alcance	Qué entra, qué no, qué queda para después. MVP first
Cronograma	Fases, hitos, deadlines. Estima $\times 1.5$, deja margen
Errores	Demasiado ambicioso, vago, sin justificación, copiar el briefing

Término	Definición
Propuesta	Documento formal que define el alcance, objetivos y plan de un proyecto
Briefing	Encargo o documento de requisitos del proyecto
MVP	Mínimo Viable Product: versión más simple que cumple requisitos mínimos
Alcance	Frontera del proyecto: qué entra, qué no y qué queda para después
Hito	Punto de control donde se verifica que todo va bien
SCAMPER	Técnica de brainstorming: Sustituir, Combinar, Adaptar, Modificar, Poner en otro uso, Eliminar, Revertir
Cronograma	Plan temporal del proyecto con fases, hitos y deadlines
Estimación	Predicción del tiempo necesario para completar una tarea
Plan B	Alternativa preparada para cuando algo sale mal
Viabilidad	Capacidad real de hacer algo con los recursos disponibles

Conexión con las siguientes unidades

Unidad	Qué aprenderás	Cómo se conecta
PI2 Sprints	Cómo ejecutar el proyecto	Tu propuesta es el plan que seguirás en los sprints
Entrega final	Cómo presentar el proyecto	La propuesta es la base de tu memoria y presentación
Scrum	Cómo gestionar el trabajo	El cronograma se ejecuta con Scrum

Si todavía no tienes claro cómo empezar tu propuesta:

1. **Lee el briefing otra vez** — extrae cada requisito
2. **Haz brainstorming** — genera 20 ideas en 10 minutos
3. **Evalúa con la tabla** — puntuación del 1 al 5
4. **Escribe la justificación** — el problema que resuelves
5. **Define 5 funcionalidades Po** — lo mínimo que necesitas

No necesitas una propuesta perfecta. Necesitas una propuesta clara y realista.

✅ Resumen en 3 frases

1. Una propuesta define **qué, por qué, para quién, cómo y cuándo** — en 7 secciones estructuradas.
2. Usa **técnicas de brainstorming** para generar ideas y **criterios objetivos** para seleccionar la mejor.
3. **Sé realista**: menos funcionalidades bien hechas que muchas a medias — y siempre ten un plan B.

 [Volver al índice de la unidad](#) · Anterior: [o8 — Errores de propuesta](#)

01 — Criterios INVEST

 **Estás en:**  **UD 2.1 · Requisitos Avanzado** → [01 · Criterios INVEST](#)

 *No todas las historias de usuario son iguales*

En Proyecto 1 escribiste historias con “Como... quiero... para que...”. Pero ¿cómo saber si una historia es **buena**? Los criterios INVEST son un checklist rápido para evaluar la calidad de cada historia antes de meterla en un sprint.

La idea en una frase

Una buena historia de usuario debe ser Independiente, Negociable, Valiosa, Estimable, Small y Testable (INVEST).

Si una historia falla en algún criterio, necesita ser reescrita antes de estimarla.

¿Qué es INVEST?

INVEST es un acrónimo creado por Bill Wake que define **6 cualidades** de una historia de usuario bien escrita:

Letra	Criterio	Pregunta clave
I	Independiente	¿Puedo hacerla sin dependencias de otras historias?
N	Negociable	¿Se puede discutir el cómo sin cambiar el qué?
V	Valiosa	¿Aporta valor real al usuario o al negocio?
E	Estimable	¿Puedo estimar cuánto esfuerzo requiere?
S	Small	¿Puedo terminarla en un solo sprint?
T	Testable	¿Puedo verificar que funciona con criterios objetivos?

Detalle de cada criterio

La historia no debe depender de otras para completarse.

Dependiente ✗	Independiente ✓
“Como admin, quiero gestionar usuarios para que el profesor pueda ver sus datos”	“Como profesor, quiero ver la lista de alumnos de mi clase”
Requiere que exista la gestión de usuarios primero	Se puede implementar directamente

Consejo: Si dos historias siempre van juntas, considéralo como una sola.

N — Negociable

La historia no es un contrato. El equipo y el producto pueden negociar detalles.

- **Historia negociable:** “Como alumno, quiero guardar mi progreso para no perderlo”
- **Historia no negociable:** “Como alumno, quiero un botón azul de 40px que diga ‘Guardar’ en la esquina superior derecha”

La segunda especifica el **cómo** en vez del **qué**.

V — Valiosa

Cada historia debe aportar valor a alguien: el usuario, el negocio o el equipo.

Sin valor ✗	Con valor ✓
“Como desarrollador, quiero refactorizar el código”	“Como usuario, quiero que la app cargue rápido”
Solo beneficia al equipo técnico	Beneficia al usuario final

Excepción: Historias técnicas (seguridad, rendimiento) pueden ser válidas si explican el valor.

E — Estimable

El equipo debe poder estimar el esfuerzo aproximado.

- Si no puedes estimar, falta información: investiga antes de estimar
- Si es demasiado grande para estimar, divídela en historias más pequeñas

S — Small

La historia debe poder completarse en un sprint (1-2 semanas).

Demasiado grande ✗	Small ✓
“Como alumno, quiero gestionar todo el módulo de tareas”	“Como alumno, quiero crear una tarea con nombre y fecha”

Regla: Si tardas más de 3-4 días, probablemente es demasiado grande.

T — Testable

Deben existir criterios de aceptación objetivos para verificar que funciona.

- **Testable:** “DADO que estoy en tareas, CUANDO pulso crear, ENTONCES la tarea aparece en mi lista”
- **No testable:** “Que la app sea fácil de usar” (subjetivo)

Comparativa: historia INVEST vs historia que falla

Historia INVEST

Como alumno, quiero marcar tareas como completadas para ver mi progreso.
Independiente, clara, estimable en 2 puntos, testeable con 3 criterios.

Historia que falla

Como usuario, quiero que todo funcione bien. No es independiente (depende de todo), no es estimable (¿qué es ‘todo’?), no es testeable (¿qué es ‘bien’?).

Aclaración: “¿Y si una historia no cumple INVEST?”

► 🤔 ¿Qué hago cuando una historia no cumple algún criterio de INVEST?

Mini-chequeo

► ¿Qué significa INVEST?

► ¿Cuál es el error más común con INVEST?

► ¿Qué hacer cuando una historia no es independiente?

✓ Resumen en 3 frases

1. Los criterios **INVEST** evalúan la calidad de una historia de usuario: Independiente, Negociable, Valiosa, Estimable, Small y Testable.
2. Si una historia no cumple algún criterio, **reescribela** antes de estimarla: no la metas en un sprint “a ver qué pasa”.
3. El error más común es crear historias **demasiado grandes** o **demasiado dependientes**: divide y conquista.

 Volver al [índice de la unidad](#) · Siguiente: [02 — MoSCoW avanzado y Kano](#)

02 — MoSCoW avanzado y modelo Kano

 **Estás en:**  **UD 2.1 · Requisitos Avanzado** → 02 · MoSCoW avanzado y Kano

 *No todo es igual de importante. Aprende a priorizar de verdad*

“Esto es todo Must” es la frase más peligrosa en un proyecto. Si todo es urgente, nada es urgente. MoSCoW te da un marco para clasificar. Kano te dice qué genera satisfacción real.

La idea en una frase

MoSCoW clasifica la urgencia. Kano clasifica la satisfacción. Usar ambos juntos es priorizar con cabeza.

No se trata de hacer mucho, sino de hacer lo correcto.

MoSCoW avanzado: más allá de Must/Should/Could/Won't

En Proyecto 1 ya conocías MoSCoW. Ahora vamos un paso más allá:

Las 4 categorías con reglas claras

Categoría	Significado	Regla de oro	% del sprint
Must	Sin esto, la app no funciona	Máximo 60% del sprint	60%
Should	Importante, pero no vital	Máximo 20% del sprint	20%
Could	Deseable si hay tiempo	Máximo 20% del sprint	20%
Won't	No ahora, quizás nunca	No se incluye	0%

La regla del 60/20/20

Si tu sprint tiene 100 puntos de esfuerzo: - **60 puntos** para Must (lo imprescindible) - **20 puntos** para Should (lo importante) - **20 puntos** para Could (lo deseable)

Si no puedes cumplir esta distribución, tu plan es irreal.

Ejemplo aplicado a Proyecto 2

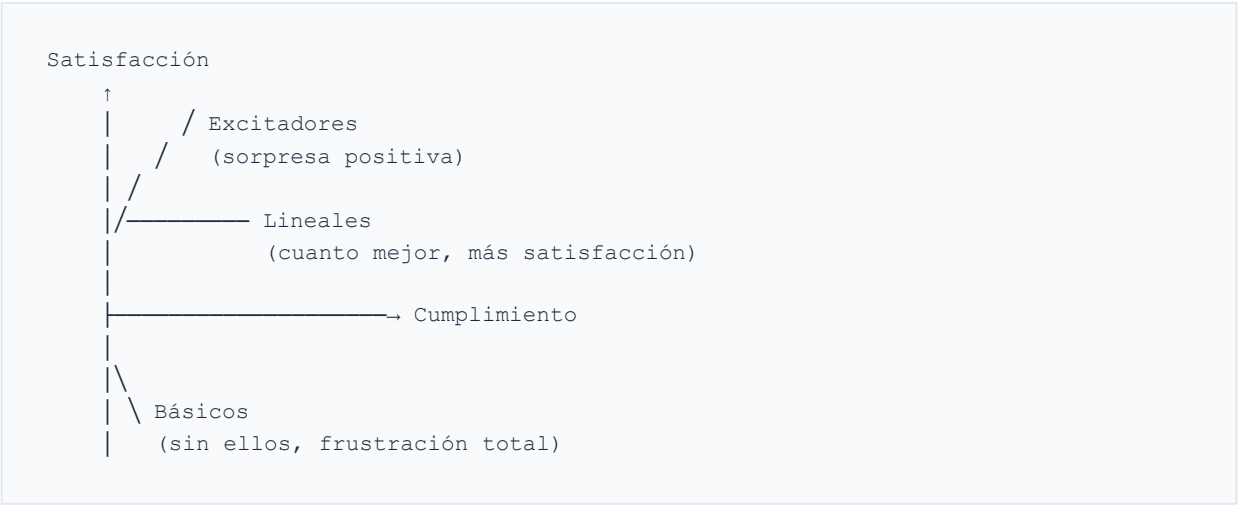
Requisito	MoSCoW	Justificación
Login con email y contraseña	Must	Sin autenticación no hay app
CRUD de tareas completo	Must	Funcionalidad central
Dashboard con estadísticas	Should	Aporta valor pero no es vital
Tema oscuro	Could	Bonito pero no imprescindible
Integración con Google Calendar	Won't	Fuera del alcance del módulo

--

El modelo Kano: ¿qué genera satisfacción real?

Kano clasifica los requisitos según **cómo afectan a la satisfacción del usuario**:

Las 3 categorías de Kano



Categoría	Qué son	Ejemplo	Prioridad
Básicos	Sin ellos, la app no sirve	Login, persistencia de datos	Must siempre
Lineales	Cuanto mejor, más satisfacción	Rendimiento, usabilidad	Should/Could
Excitadores	Generan sorpresa positiva	Atajos de teclado, exportación PDF	Could/Won't

Ejemplo para una app de tareas

Requisito	Tipo Kano	Efecto
Guardar tareas	Básico	Sin esto, la app es inútil
Carga rápida	Lineal	Más rápido = más feliz
Exportar a PDF	Excitador	Sorpresa positiva
Tema oscuro	Excitador	Bonito, pero no esperado
Sincronización en la nube	Básico hoy	Cada vez más usuarios lo dan por sentado

Comparativa: MoSCoW vs Kano

MoSCoW

Clasifica por urgencia y valor para el negocio. Pregunta: ¿es imprescindible para que la app funcione?

Kano

Clasifica por impacto en la satisfacción del usuario. Pregunta: ¿cómo se siente el usuario cuando lo usa?

Aspecto	MoSCoW	Kano
Enfoque	Urgencia / negocio	Satisfacción / usuario
Cuándo usarlo	Al planificar sprints	Al descubrir requisitos
Resultado	Lista priorizada	Mapa de satisfacción
Complemento	Kano te dice qué es emocionante	MoSCoW te dice qué es urgente

Cómo usar ambos juntos: el flujo

1. **Descubre** todos los requisitos posibles (brainstorming)
2. **Clasifica con Kano**: ¿básico, lineal o excitador?
3. **Asigna MoSCoW**: básicos → Must, lineales → Should, excitadores → Could
4. **Ajusta** según el tiempo disponible del sprint
5. **Revisa** al final del sprint: ¿qué quedó pendiente?

Aclaración: “¿Y si todo parece Must?”

- 🤔 ¿Qué pasa si mis compañeros clasifican todo como Must?

Mini-chequeo

- ¿Cuál es la distribución MoSCoW recomendada para un sprint?

- ¿Qué son los excitadores de Kano?

- ¿Cuándo usar MoSCoW y cuándo usar Kano?

✅ Resumen en 3 frases

1. **MoSCoW** clasifica requisitos por urgencia: Must (60%), Should (20%), Could (20%), Won't (0%).
2. **Kano** clasifica por satisfacción: básicos (imprescindibles), lineales (cuanto mejor) y excitadores (sorpresa positiva).
3. Usa **Kano para descubrir** y **MoSCoW para priorizar**: así evitas que todo sea “Must” y haces lo que realmente importa.

03 — Story Points y Planning Poker

 **Estás en:**  **UD 2.1 · Requisitos Avanzado** → 03 · Story Points y Planning Poker

 *Estimar no es adivinar, es comparar*

“¿Cuánto tarda esto?” es la pregunta que más miedo da en un proyecto. Los **story points** no miden tiempo, miden **esfuerzo relativo**. Y el **Planning Poker** es el juego que convierte la estimación en una conversación de equipo en vez de una pelea.

La idea en una frase

Los story points miden el esfuerzo RELATIVO entre historias, no el tiempo exacto. Planning Poker es el juego que usa el equipo para estimar juntos.

No cuentes horas. Compara tamaños.

¿Qué son los Story Points?

Son una medida **abstracta** del esfuerzo, complejidad y trabajo involucrado en una historia. No son horas ni días.

La escala Fibonacci modificada

Puntos	Significado	Referencia
1	Trivial	Un campo de formulario
2	Pequeño	Un CRUD básico con 1 tabla
3	Medio-pequeño	Login con validación
5	Medio	Dashboard con 3 gráficos
8	Medio-grande	Sistema de notificaciones
13	Grande	Autenticación completa con roles
21	Muy grande	Dividir en historias más pequeñas

¿Por qué Fibonacci? Porque a medida que algo crece, la incertidumbre crece también. La diferencia entre 8 y 13 es más borrosa que entre 1 y 2.

¿Por qué no horas?

Horas ✗	Story Points ✓
“Esto tarda 6 horas” → depende de quién lo haga	“Esto vale 5 puntos” → es igual para todos
El novato tarda 3x más que el senior	Los puntos son relativos al equipo
Se convierte en una predicción imposible	Comparas con cosas que ya hiciste

Planning Poker: el juego de la estimación

Cómo funciona

1. El **Product Owner** presenta una historia
2. Cada miembro del equipo **elige una carta** con sus puntos (en secreto)
3. Todos **revelan las cartas** a la vez
4. Si hay **discrepancia grande** (ej: 2 vs 8), los extremos **explican por qué**
5. Se **votan otra vez** hasta llegar a consenso
6. El número que gana es la **estimación oficial**

Las cartas del Planning Poker

1	2	3	5	8	13	21	?	
•	•	•	•	•	•	•	•	

Carta	Significado
1-21	Puntos de esfuerzo
?	No tengo información suficiente
	Necesito una pausa / no puedo estimar

Ejemplo de una ronda

Miembro	Carta	Razón
Ana	5	“He hecho algo parecido, tardé 2 sprints atrás”
Pedro	8	“La parte de validación es más compleja de lo que parece”
Luis	3	“Yo lo veo más sencillo, es solo un formulario”

Discusión: Pedro explica que la validación requiere verificar 3 condiciones. Ana confirma que ella tuvo problemas similares. Luis acepta que no había pensado en eso.

Segunda ronda: Todos votan 5. Consenso alcanzado.

Comparativa: estimar solo vs Planning Poker

Estimar solo

Yo digo 5 puntos. Pero no sé que Pedro no ha usado esa librería antes, ni que Ana ya hizo algo parecido. La estimación es mía, no del equipo.

Planning Poker

Revelamos cartas: 3, 5, 8. Discutimos. Pedro explica la complejidad. Ana aporta experiencia. Llegamos a 5 juntos. La estimación es del equipo.

La velocidad del equipo (velocity)

La **velocidad** es la suma de story points completados en un sprint.

```
Sprint 1: 5 + 3 + 2 + 5 + 3 = 18 puntos
Sprint 2: 8 + 3 + 5 + 2 + 3 = 21 puntos
Sprint 3: 5 + 5 + 8 + 3      = 21 puntos

Velocidad promedio: (18 + 21 + 21) / 3 ≈ 20 puntos/sprint
```

Con esto puedes predecir: “Si tengo 80 puntos pendientes, necesito ~4 sprints”.

Aclaración: “¿Y si no somos expertos en estimar?”

► 🤔 ¿Cómo estimar bien cuando no tenemos experiencia?

Mini-chequeo

► ¿Qué miden los story points?

► ¿Por qué se usa la serie de Fibonacci?

► ¿Qué es la velocidad del equipo?

✓ Resumen en 3 frases

1. Los **story points** miden esfuerzo relativo (no tiempo) usando la escala de Fibonacci: 1, 2, 3, 5, 8, 13, 21.
2. **Planning Poker** es el juego donde el equipo vota en secreto y discute las diferencias: la estimación es del grupo, no de uno solo.
3. La **velocidad** (points/sprint) te permite predecir cuánto durará el proyecto basándote en datos reales, no en suposiciones.

 Volver al [índice de la unidad](#) · Anterior: [02 — MoSCoW avanzado y Kano](#) · Siguiente: [04 — Refinamiento de backlog](#)

04 — Refinamiento de backlog

 Estás en:  **UD 2.1 · Requisitos Avanzado** → 04 · Refinamiento de backlog

 *Refinar no es burocracia, es prevenir desastres*

El **refinamiento** (o backlog grooming) es la sesión donde el equipo revisa las historias pendientes antes de que entren en un sprint. Es como repasar el guion antes de actuar: cuesta 30 minutos y te ahorra un desastre de 2 semanas.

La idea en una frase

El refinamiento es la sesión donde el equipo prepara, aclara y divide las historias ANTES de que entren en un sprint.

Sin refinamiento, entráis a ciegas en cada sprint.

¿Qué es el refinamiento?

Es una reunión **periódica** (semanal o cada sprint) donde el equipo revisa las historias del backlog para que estén **listas para ser estimadas y trabajadas**.

Qué se hace en el refinamiento

Acción	Objetivo	Ejemplo
Revisar	¿La historia cumple INVEST?	“Esta historia no es Small, hay que dividirla”
Aclara	¿Hay dudas sobre la historia?	“¿Qué pasa si el usuario no está logueado?”
Estimar	Asignar story points	Planning Poker después de aclarar
Dividir	Historias demasiado grandes	“Gestionar usuarios” → crear, listar, editar
Añadir criterios	¿Está testeable?	Añadir casos extremos

El checklist de “Definition of Ready” (DoR)

Una historia está **lista** para entrar en un sprint cuando:

Criterio	Pregunta	X	✓
INVEST	¿Cumple los 6 criterios?	“El usuario puede hacer cosas”	“Como alumno, quiero crear tareas”
Criterios de aceptación	¿Tiene Given/When/Then?	Sin criterios	3-5 criterios concretos
Dependencias	¿Depende de algo pendiente?	“Primero hay que hacer el login”	Independiente o la dependencia está hecha
Estimación	¿Se puede estimar?	“No sé cuánto tarda”	5 puntos con consenso
Tamaño	¿Cabe en un sprint?	“Tarda 3 semanas”	“Tarda 3-4 días”
Información	¿Tiene todo lo necesario?	“Habrá que ver el diseño”	Mockup adjunto

Flujo completo del refinamiento

Backlog (todo lo pendiente) ↓ Revisión: ¿es INVEST? → No → Reescribir o dividir ↓ Sí Aclaración: ¿hay dudas? → Sí → Responder y documentar ↓ No Criterios: ¿tiene Given/When/Then? → No → Añadir ↓ Sí Estimación: Planning Poker → Asignar puntos ↓ Listo para sprint planning

Comparativa: con refinamiento vs sin refinamiento

Sin refinamiento

Sprint planning: ‘Esta historia... no sé qué es esto’. Estimáis a ciegas. Mitad del sprint descubrís que no teníais toda la información. Caos.

Con refinamiento

Sprint planning: ‘Esta historia ya la revisamos, tiene criterios, está estimada en 5 puntos’. Empezáis a trabajar el lunes. Calma.

Sin refinamiento	Con refinamiento
Historias vagas entran al sprint	Historias claras y testeadas
Sorpresas a mitad de sprint	Sorpresas mínimas
Estimaciones imprecisas	Estimaciones razonables
El equipo se bloquea frecuentemente	El equipo avanza fluido

Aclaración: “¿Cuánto tiempo dedicar al refinamiento?”

► 🤔 ¿No es perder tiempo reunirse para hablar de historias?

Mini-chequeo

► ¿Qué es el Definition of Ready (DoR)?

► ¿Cuánto tiempo dedicar al refinamiento?

► ¿Quién participa en el refinamiento?

✓ Resumen en 3 frases

1. El **refinamiento** es la sesión donde el equipo revisa, aclara y divide historias **antes** de que entren en un sprint.
2. Usa el **Definition of Ready** (DoR) como checklist: una historia que no cumple el DoR no está lista para ser trabajada.
3. Dedicar el **10% del tiempo** del sprint a refinamiento: 30 minutos de preparación ahorran días de caos.

 Volver al [índice de la unidad](#) · Anterior: [03 — Story points y Planning Poker](#) · Siguiente: [05 — Criterios de aceptación avanzado](#)

05 — Criterios de aceptación avanzado

 **Estás en:**  **UD 2.1 · Requisitos Avanzado** → 05 · Criterios de aceptación avanzado

 *De ‘funciona’ a ‘funciona exactamente así’*

En Proyecto 1 aprendiste el formato Dado/Cuando/Entonces. Ahora toca **dominarlo**: cubrir casos extremos, errores inesperados y escenarios que nunca te imaginaste pero que el usuario sí probará.

La idea en una frase

Un criterio de aceptación avanzado no solo describe el camino feliz: también cubre errores, límites y casos que parecen imposibles pero que pasan.

Given/When/Then: la base sólida

El formato BDD (Behavior-Driven Development) sigue siendo el estándar:

```
GIVEN (DADO) [precondición - el contexto]
WHEN (CUANDO) [acción - lo que hace el usuario]
THEN (ENTONCES) [resultado esperado - qué pasa]
```

Ejemplo básico vs avanzado

Básico	Avanzado
DADO que estoy en login, CUANDO pongo email correcto, ENTONCES entro	DADO que estoy en login con email <code>ana@test.com</code> y contraseña válida, CUANDO pulso “Entrar”, ENTONCES redirijo a <code>/dashboard</code> en menos de 2s y veo “Bienvenida, Ana”

La diferencia: el avanzado es **específico, medible y verificable**.

Los 6 tipos de criterios que debes cubrir

1. Happy path (camino feliz)

Todo va bien, el usuario hace lo esperado:

```
DADO que estoy en la pantalla de registro
CUANDO introduzco un email válido y una contraseña de 8+ caracteres
Y pulso "Registrarme"
ENTONCES se crea mi cuenta y accedo al dashboard
```

2. Validación de errores

Qué pasa cuando algo va mal:

DADO que introduzco un email que ya existe en el sistema
CUANDO pulso "Registrarme"
ENTONCES aparece el error "Este email ya está registrado"
Y el formulario conserva el email introducido

3. Límites y validación

Los bordes del sistema:

DADO que escribo una contraseña de solo 3 caracteres
CUANDO pulso "Registrarme"
ENTONCES aparece "La contraseña debe tener al menos 8 caracteres"
Y el botón se queda deshabilitado

4. Persistencia

Los datos sobreviven al cierre:

DADO que he creado 3 tareas
CUANDO cierro el navegador y lo vuelvo a abrir
ENTONCES las 3 tareas siguen en mi lista con el mismo orden

5. Comportamiento responsive

La app se adapta:

DADO que abro la app en un móvil de 320px de ancho
CUANDO miro la lista de tareas
ENTONCES se muestra en una columna, el texto es legible
Y no hay scroll horizontal

6. Accesibilidad

Todos pueden usarla:

DADO que navego con teclado (sin ratón)
CUANDO pulso Tab para moverme entre campos
ENTONCES cada campo recibe foco visible (borde resaltado)
Y puedo activar el botón con Enter

Historia: “Como alumno, quiero registrarme con email y contraseña”

Criterios de aceptación:

1. Happy path:
DADO que estoy en la pantalla de registro
CUANDO introduzco email 'nuevo@test.com' y contraseña 'MiPass123!'
Y pulso "Registrarme"
ENTONCES se crea mi cuenta y accedo al dashboard
2. Email duplicado:
DADO que el email 'ana@test.com' ya existe
CUANDO intento registrarme con ese email
ENTONCES aparece "Este email ya está registrado"
3. Email inválido:
DADO que escribo 'no-es-email' en el campo de email
CUANDO pulso "Registrarme"
ENTONCES aparece "Introduce un email válido"
4. Contraseña débil:
DADO que escribo '123' como contraseña
CUANDO pulso "Registrarme"
ENTONCES aparece "La contraseña debe tener al menos 8 caracteres"
5. Persistencia:
DADO que me he registrado correctamente
CUANDO cierro el navegador y lo vuelvo a abrir
ENTONCES sigo logueado (la sesión persiste)
6. Responsive:
DADO que abro el registro en un móvil de 320px
CUANDO miro el formulario
ENTONCES todos los campos son visibles sin scroll horizontal

Comparativa: criterios básicos vs avanzados

Criterios básicos

DADO que hago login, CUANDO meto mis datos, ENTONCES funciona. (¿Qué datos?
¿Qué es ‘funciona’? ¿Qué pasa si falla?)

Criterios avanzados

DADO que introduzco email ‘ana@test.com’ y contraseña ‘1234’, CUANDO pulso ‘Entrar’,
ENTONCES accedo al dashboard en <2s. SI el email no existe, ENTONCES aparece
‘Credenciales incorrectas’ y el formulario conserva el email.

Aclaración: “¿Cuántos criterios por historia es demasiados?”

▶ 🤔 ¿No es excesivo poner 6 criterios por historia?

Mini-chequeo

▶ ¿Cuáles son los 6 tipos de criterios de aceptación?

▶ ¿Qué es BDD?

▶ ¿Cómo sé si mis criterios son suficientes?

✅ Resumen en 3 frases

1. Los criterios avanzados usan **Given/When/Then** para ser específicos, medibles y verificables.
2. Debes cubrir **6 tipos**: happy path, errores, límites, persistencia, responsive y accesibilidad.
3. Si una historia necesita **más de 7 criterios**, probablemente es demasiado grande: divídela en historias más pequeñas.

 Volver al [índice de la unidad](#) · Anterior: [04 — Refinamiento de backlog](#) · Siguiente: [06 — Trazabilidad](#)

📖 Estás en: 📄 UD 2.1 · Requisitos Avanzado → 06 · Trazabilidad de requisitos

📖 Si no sabes de dónde viene un requisito, no sabes por qué existe

La **trazabilidad** es la capacidad de seguir un requisito desde su origen (¿quién lo pidió?) hasta el código que lo implementa. Es como un hilo que conecta al usuario con la funcionalidad. Sin ese hilo, pierdes el contexto y tomas decisiones a ciegas.

📌 La idea en una frase

La trazabilidad conecta el origen del requisito (quién lo pidió y por qué) con el código que lo implementa y los tests que lo verifican.

Sin trazabilidad, no sabes si construiste lo que pedían.

¿Qué es la trazabilidad?

Es la relación **rastreable** entre:

Usuario/Necesidad → Requisito → Historia → Código → Test

Ejemplo concreto

Origen	Requisito	Historia	Código	Test
Profesor necesita ver notas de alumnos	RF-003: Consulta de calificaciones	“Como profesor, quiero ver las notas de mis alumnos”	src/components/GradeList.vue	tests/GradeList.spec.js

361 / 743

Si el profesor pregunta “¿por qué se ve así?”, puedes rastrear hasta el código y entender la decisión.

La matriz de trazabilidad

Una tabla que conecta todo:

ID Requisito	Historia	Sprint	Archivo código	Test	Estado
RF-001	US-001: Login	S1	auth/login.ts	auth.test.ts	 Hecho
RF-002	US-002: CRUD tareas	S1-S2	tasks/	tasks.test.ts	 En progreso
RF-003	US-003: Notas	S3	Pendiente	Pendiente	 Pendiente
RNF-001	—	S2	config/performance.ts	perf.test.ts	 Hecho

QuéColumns de la matriz

Columna	Qué indica	Ejemplo
ID Requisito	Identificador único del requisito	RF-001, RNF-003
Historia	User story que lo implementa	US-001
Sprint	En qué sprint se trabaja	S1, S2
Archivo código	Dónde está la implementación	src/auth/login.ts
Test	Dónde se verifica	tests/auth.test.ts
Estado	Hecho / En progreso / Pendiente	  

¿Por qué importa la trazabilidad?

Sin trazabilidad

- El profesor pide un cambio: ¿qué archivos afectan?
- Un test falla: ¿qué requisito viola?
- Un compañero se va: ¿qué hacía él?
- El cliente pregunta “¿por qué hicisteis esto?”: no hay respuesta

Con trazabilidad

- Cambio de requisito: buscas la fila, identificas el código afectado
- Test falla: sabes qué requisito está en riesgo
- Nuevo miembro: entiende qué hace cada parte y por qué
- Auditoría: puedes demostrar que todo lo pedido está implementado

Cómo crear tu matriz (formato ligero)

Para Proyecto 2, no necesitas una herramienta especial. Un archivo Markdown basta:

```
# Matriz de Trazabilidad

| ID | Requisito | Historia | Código | Test | Sprint | Estado |
|----|-----|-----|-----|-----|-----|-----|
| RF-001 | Login email/contraseña | US-001 | src/auth/ | tests/auth/ | S1 |  |
| RF-002 | CRUD tareas | US-002 | src/tasks/ | tests/tasks/ | S1-S2 |  |
| RNF-001 | Carga < 2s | - | src/config/ | tests/perf/ | S2 |  |
```

Guarda esto como `TRAZABILIDAD.md` en la raíz de tu repositorio.

Comparativa: con trazabilidad vs sin trazabilidad

Sin trazabilidad

El profesor: ‘Cambia el login para usar Google’. Tú: ‘¿Qué archivos tengo que tocar?’. Silencio incómodo. Miras el código durante 30 minutos.

Con trazabilidad

El profesor: ‘Cambia el login para usar Google’. Tú: ‘RF-001 - US-001 - src/auth/login.ts - tests/auth.test.ts’. En 5 minutos sabes exactamente qué tocar.

Aclaración: “¿No es mucho trabajo mantener la trazabilidad?”

► 🤔 ¿Merece la pena el esfuerzo de mantener una matriz de trazabilidad?

Mini-chequeo

► ¿Qué conecta la trazabilidad?

► ¿Qué formato usar para la matriz?

► ¿Cuándo actualizar la matriz?

✓ Resumen en 3 frases

1. La **trazabilidad** conecta el origen del requisito con el código y los tests que lo implementan y verifican.
2. Una **matriz simple** en Markdown (`TRAZABILIDAD.md`) es suficiente para proyectos de módulo: actualízala en cada sprint.
3. La trazabilidad te permite **responder rápidamente** a cambios de requisitos y entender el impacto de cada decisión.

07 — Errores avanzados de requisitos

 **Estás en:**  **UD 2.1 · Requisitos Avanzado** → 07 · Errores avanzados de requisitos

 *Los errores de requisitos se pagan caros*

En Proyecto 1 viste errores básicos (requisitos vagos, olvidar RNF). Ahora toca ver los errores **avanzados**: los que parecen correctos pero que lentamente van envenenando tu proyecto hasta que es demasiado tarde para arreglarlos.

La idea en una frase

Los errores avanzados de requisitos no se ven al principio: se detectan cuando ya has programado 2 semanas y descubres que construiste lo que no debías.

Aprenderlos ahora te ahorra dolores de cabeza después.

Los 7 errores avanzados

1. El requisito ambiguo doble

Parece claro, pero admite **dos interpretaciones**:

Requisito ambiguo	Interpretación A	Interpretación B
“El usuario puede ver sus tareas”	Lista simple	Lista con filtros y ordenación
“La app debe ser rápida”	< 1s	< 3s
“El profesor puede evaluar”	Dar notas	Dar notas + comentarios

Solución: Añadir **criterios de aceptación específicos** que eliminen la ambigüedad.

2. El requisito contradictorio

Dos requisitos que **se excluyen mutuamente**:

Requisito A	Requisito B	Conflicto
“Los datos deben estar accesibles sin internet”	“Los datos deben sincronizarse con el servidor en tiempo real”	Offline + tiempo real son incompatibles
“La app debe ser anónima”	“El profesor debe ver quién hizo qué”	Anonimato vs trazabilidad

Solución: Detectar contradictions en el refinamiento y **negociar** con el Product Owner cuál tiene prioridad.

3. El requisito imposible técnicamente

Algo que el usuario pide pero que **no se puede hacer** (o no debería):

Requisito imposible	Por qué no se puede
“Que la app funcione sin navegador”	Es una web, necesita un navegador
“Que detecte si el alumno miente”	No tenemos acceso a la cámara ni IA en frontend-only
“Que envíe emails sin servidor”	Sin backend, no hay envío de emails

Solución: Investigar la viabilidad técnica **antes** de aceptar el requisito. No prometer lo que no puedes cumplir.

4. El requisito que nadie pidió

Algo que el equipo **asumió** sin que nadie lo pidiera:

- “Hicimos un sistema de notificaciones ~~porque~~ parecía útil” → Nadie lo pidió

- “Añadimos modo oscuro porque está de moda” → No era una prioridad
- “Creamos un dashboard con 10 gráficos” → El profesor solo quería una tabla

Solución: Todo requisito debe tener un **origen claro** (¿quién lo pidió y por qué?). Si no tiene origen, no se hace.

5. El requisito que crece (scope creep)

Un requisito que **se expande** sin control:

```
Sprint 1: "Queremos un CRUD de tareas"
Sprint 2: "Y que tenga categorías"
Sprint 3: "Y que tenga etiquetas de color"
Sprint 4: "Y que se pueda exportar"
Sprint 5: "Y que tenga recordatorios"
```

Solución: Cada expansión es una **nueva historia** con su propia estimación. No dejes que un requisito crezca dentro de un sprint.

6. El requisito sin origen

Nadie sabe **quién lo pidió** ni **por qué** existe:

- “Tenemos que hacer un chat” → ¿Quién lo pidió? No lo sé
- “Hay que añadir un buscador” → ¿El profesor lo pidió? No recuerdo
- “Necesitamos integración con Google” → ¿Por qué? No me lo pregunté

Solución: Cada requisito debe tener un **ID** y un **origen**. Si no puedes rastrearlo, no lo hagas.

7. El requisito “para el examen”

Un requisito que **solo sirve para que el profesor vea algo bonito** pero que no aporta valor al usuario:

- “El dashboard debe tener gráficos animados” → El usuario no necesita gráficos animados
- “Hay que hacer una landing page” → El usuario ya está dentro de la app
- “Necesitamos un PDF de 20 páginas” → El usuario quiere la info, no el PDF

Solución: Preguntar siempre: “¿Esto aporta valor al usuario final?” Si la respuesta es “no”, es un requisito para el examen, no para el producto.

Comparativa: errores básicos vs avanzados

Requisitos vagos ('que sea bonita'), olvidar RNF, todo es Must. Se ven fácilmente y se arreglan rápido.

Errores avanzados

Requisitos contradictorios, imposibles, sin origen, que crecen. No se ven hasta que ya has programado y el daño está hecho.

Aclaración: “¿Cómo detectar estos errores a tiempo?”

► 🤔 ¿Qué puedo hacer para detectar errores avanzados antes de programar?

Mini-chequeo

► ¿Cuáles son los 7 errores avanzados de requisitos?

► ¿Qué es el scope creep?

► ¿Cómo evitar requisitos sin origen?

✓ Resumen en 3 frases

1. Los errores avanzados de requisitos son **peor que los básicos**: no se ven al principio pero destruyen el proyecto cuando ya es tarde.
2. Los 7 errores son: ambigüedad, contradicciones, imposibilidad técnica, nadie lo pidió, scope creep, sin origen y “para el examen”.
3. Para detectarlos, hazte **5 preguntas** antes de aceptar cada requisito: quién lo pidió, si es posible, si contradice, si aporta valor y si puede crecer.

 Volver al [índice de la unidad](#) · Anterior: [06 — Trazabilidad](#) · Siguiente: [08 — Template avanzado](#)

08 — Template de requisitos avanzado

 **Estás en:**  **UD 2.1 · Requisitos Avanzado** → [08 · Template de requisitos avanzado](#)

 *El documento que te ahorra semanas de caos*

Este template incluye todo lo aprendido en la unidad: INVEST, Kano, trazabilidad, criterios avanzados y Definition of Ready. Cópialo, rellénalo y tendrás un documento de requisitos profesional para tu Proyecto 2.

La idea en una frase

Copia este template en tu repositorio como `REQUISITOS.md` , rellénalo en 2-3 horas y tendrás la base documental de tu proyecto.

No empieces a programar sin rellenar esto.

Template completo

```
# REQUISITOS — [Nombre de tu app]

## 1. Visión del producto

- **Nombre**: [Nombre de la app]
- **Descripción**: [Una frase que explique qué hace]
- **Usuario principal**: [Quién la usa]
- **Problema que resuelve**: [Qué necesidad satisface]
- **Tecnologías clave**: [Stack principal]

## 2. Requisitos funcionales

| ID | Requisito | Actor | Prioridad (MoSCoW) | Kano | Estado |
|----|-----|-----|-----|-----|-----|
| RF-001 | [Descripción] | [Quién] | Must/Should/Could | Básico/Lineal/Excitador |  |
| RF-002 | [Descripción] | [Quién] | Must/Should/Could | Básico/Lineal/Excitador |  |

## 3. Requisitos no funcionales

| ID | Requisito | Métrica | Prioridad | Estado |
|----|-----|-----|-----|-----|
| RNF-001 | [Carga rápida] | < 2s en móvil gama baja | Must |  |
| RNF-002 | [Accesibilidad] | WCAG 2.1 nivel A | Should |  |

## 4. Historias de usuario

### US-001: [Título]

**Como** [tipo de usuario], **quiero** [acción], **para** [beneficio].

**INVEST**:  Independiente |  Negociable |  Valiosa |  Estimable |  Small |

**Puntos**: [1-21]

**Criterios de aceptación**:
```

DADO que [contexto] CUANDO [acción] ENTONCES [resultado]

```

**Origen**: [Quién lo pidió y por qué]

**Trazabilidad**: [Archivos de código afectados]

**Estado**: 📄 Pendiente / 🔄 En progreso / ✅ Hecho

### US-002: [Título]

[Repetir formato]

## 5. Matriz de trazabilidad

| ID Requisito | Historia | Sprint | Código | Test | Estado |
|-----|-----|-----|-----|-----|-----|
| RF-001 | US-001 | S1 | src/... | tests/... | 📄 |

## 6. Definition of Ready

Una historia está lista para sprint planning cuando:

- [ ] Cumple INVEST
- [ ] Tiene criterios de aceptación (Given/When/Then)
- [ ] No tiene dependencias pendientes
- [ ] Está estimada en story points
- [ ] Cabe en un sprint (≤ 8 puntos)
- [ ] Tiene origen claro (quién lo pidió)

## 7. Decisiones pendientes

| Decisión | Opciones | Fecha límite | Responsable |
|-----|-----|-----|-----|
| [Qué decidir] | [Opción A / Opción B] | [Fecha] | [Quién] |

```

Aclaración: “¿Cuánto tiempo me toma rellenarlo?”

► 🤔 ¿No me va a quitar mucho tiempo rellenar todo esto?

Mini-chequeo

► ¿Qué secciones tiene el template?

► ¿Cuánto tiempo toma rellenarlo?

✓ Resumen en 3 frases

1. El template incluye **7 secciones**: visión, RF, RNF, historias (con INVEST + Kano), trazabilidad, DoR y decisiones pendientes.
2. Cópialo como `REQUISITOS.md` en la raíz de tu repositorio y rellénalo en **2-3 horas**.
3. Actualízalo en cada sprint planning: es un **documento vivo**, no un trámite de una vez.

 Volver al [índice de la unidad](#) · Anterior: [07 — Errores avanzados](#) · Siguiendo: [09 — Cierre](#)

09 — Cierre

 **Estás en:**  **UD 2.1 · Requisitos Avanzado** → 09 · Cierre

 *Ya sabes escribir requisitos como un profesional*

Has recorrido toda la unidad: desde INVEST hasta trazabilidad, pasando por Kano, Planning Poker y los 7 errores avanzados. Ahora toca **verificar que lo entiendes y aplicarlo a tu proyecto**.

La idea en una frase

Los requisitos son el mapa de tu proyecto: sin ellos, estás programando a

ciegas. Con ellos, construyes lo que importa.

Si recuerdas esto, has entendido la unidad.

Mini-chequeo final

Responde estas preguntas antes de seguir:

Preguntas de comprensión

▶ ¿Qué significa INVEST?

▶ ¿Qué diferencia hay entre MoSCoW y Kano?

▶ ¿Qué son los story points?

▶ ¿Qué es Planning Poker?

▶ ¿Qué es el refinamiento de backlog?

▶ ¿Qué tipos de criterios de aceptación debo cubrir?

▶ ¿Qué es la trazabilidad?

▶ ¿Cuáles son los 7 errores avanzados?

Ejercicio práctico

En 3 horas, prepara tu documento de requisitos avanzado:

1. **Visión** (15 min): Nombre, descripción, usuario principal, problema que resuelve

2. **RF + RNF** (30 min): Lista 8-12 RF y 4-6 RNF con IDs, MoSCoW y Kano
3. **Historias** (60 min): Escribe 10-15 historias con INVEST, story points y criterios Given/When/Then
4. **Trazabilidad** (15 min): Crea la matriz con ID, historia, código y test
5. **Revisión** (30 min): Revisa con el equipo que todo cumple INVEST y DoR

Si consigues tener un `REQUISITOS.md` completo → has entendido la unidad.

Tabla resumen de la unidad

Concepto	Idea clave
INVEST	Independiente, Negociable, Valiosa, Estimable, Small, Testable
MoSCoW avanzado	60% Must, 20% Should, 20% Could
Kano	Básicos (imprescindibles), lineales (cuanto mejor), excitadores (sorpresa)
Story points	Esfuerzo relativo con escala Fibonacci (1, 2, 3, 5, 8, 13, 21)
Planning Poker	Vota en secreto, revela, discute, consenso
Refinamiento	Revisar, aclara, dividir historias antes del sprint (10% del tiempo)
Definition of Ready	Checklist que una historia debe cumplir para entrar en sprint
Given/When/Then	Formato BDD para criterios de aceptación
Trazabilidad	Conectar requisito → historia → código → test
Errores avanzados	Ambigüedad, contradicciones, imposibles, sin origen, scope creep

Vocabulario rápido

Término	Definición
INVEST	Acrónimo para evaluar calidad de historias de usuario
MoSCoW	Método de priorización: Must, Should, Could, Won't
Kano	Modelo de satisfacción: básicos, lineales, excitadores
Story Points	Medida abstracta de esfuerzo relativo (escala Fibonacci)
Planning Poker	Juego de estimación colectiva con cartas de puntos
Velocity	Suma de story points completados por sprint
Refinamiento	Sesión de preparación de historias antes del sprint
Definition of Ready	Checklist de "lista para trabajar"
BDD	Behavior-Driven Development: desarrollo guiado por comportamiento
Given/When/Then	Formato estándar para criterios de aceptación
Trazabilidad	Capacidad de rastrear un requisito hasta el código
Scope Creep	Expansión no controlada de requisitos durante el proyecto
Traceability Matrix	Tabla que conecta requisitos con código y tests

Conexión con las siguientes unidades

Unidad	Qué aprenderás	Cómo se conecta
UD 2.2 SCRUM Avanzado	Métricas, burndown, retrospectivas	Usarás los story points y velocity que estimaste aquí
UD 2.3 Git Avanzado	GitFlow, hooks, CI/CD	Los branches seguirán la estructura de sprints
UD 2.4 Patrones de Diseño	MVC, Repository, Observer	Implementarás los requisitos usando patrones
UD 2.5 Diagramas	C4, UML, secuencia	Modelarás la arquitectura de los requisitos

✓ Resumen en 3 frases

1. Los requisitos avanzados usan **INVEST** para calidad, **Kano** para satisfacción y **MoSCoW** para priorizar con criterio.
2. **Planning Poker** estima en equipo, el **refinamiento** prepara historias y la **trazabilidad** conecta requisito con código.
3. Evita los **7 errores avanzados** (ambigüedad, contradicciones, imposibles, sin origen, scope creep, para el examen) y tendrás un proyecto sólido.

 Volver al [índice de la unidad](#) · Anterior: [08 — Template avanzado](#) · Siguiendo: [UD 2.2 — SCRUM Avanzado](#)

01 — Planning Poker

 Estás en: [UD 2.2 · SCRUM Avanzado](#) → 01 · Planning Poker

 *Estimar no es una opinión, es una conversación*

El Planning Poker es la técnica más extendida en equipos SCRUM profesionales. No es solo “votar un número”: es un proceso que obliga al equipo a **compartir conocimiento** antes de comprometerse con una estimación.

La idea en una frase

Planning Poker convierte la estimación individual en una decisión de equipo: cada miembro vota, los extremos explican y el consenso emerge.

El proceso paso a paso

Preparación

1. El **Product Owner** presenta la historia
2. Cada miembro recibe un **mazo de cartas**: 1, 2, 3, 5, 8, 13, 21, ?, 
3. Se acuerda una **referencia**: “Esta historia vale 3 puntos” (historia base)

La ronda

PO presenta: "Como alumno, quiero filtrar tareas por categoría"

Ana	Pedro	Luis	Marta
[8]	[5]	[13]	[5]

- Revelan a la vez: 8, 5, 13, 5
- Discrepancia: Luis dice 13, Ana dice 8
- Luis: "La parte de filtrado con múltiples categorías es compleja"
- Ana: "Yo lo veo más sencillo con un filtro simple"
- PO: "El filtro debe funcionar con AND y OR"
- Luis acepta que con filtro simple es menos
- Segunda ronda: todos votan 8
- Consenso: 8 puntos

Las cartas especiales

Carta	Cuándo usarla
?	No tengo información suficiente para estimar
	Necesito una pausa o estoy fuera de foco
∞ (infinito)	Esta historia es demasiado grande para estimar

Las 3 reglas de oro

1. Silencio al votar

Nadie habla mientras elige su carta. Si hablas, influencias a los demás.

2. Los extremos explican

Cuando hay diferencia grande (ej: 3 vs 13), los que votaron los valores extremos **explican por qué**. Esto genera aprendizaje.

3. No hay imposición

El número que gana es el del **consenso**, no el del jefe ni el del más voz alta.

Comparativa: estimar solo vs Planning Poker

Estimar solo

Yo digo 5. Pedro dice 8. Marta dice 3. Cada uno con su número. La estimación final es un promedio que nadie defiende.

Planning Poker

Revelamos: 3, 5, 8, 5. Pedro explica la complejidad. Marta aporta experiencia previa. Llegamos a 5 juntos. Todos estamos comprometidos.

Errores comunes con Planning Poker

Error	Consecuencia	Solución
Hablar antes de votar	El primero influye a todos	Silencio al elegir carta
El jefe impone su número	El equipo no se compromete	Consenso, no imposición
No explicar los extremos	Se pierde aprendizaje	Siempre explicar por qué
Estimar historias vagas	Estimaciones basura	Refinar primero, estimar después
Usar siempre el mismo número	Pérdida de precisión	Debatir y variar

Aclaración: “¿Y si siempre nos ponemos de acuerdo rápido?”

► 🤔 ¿Es malo que siempre votes lo mismo?

Mini-chequeo

► ¿Cuáles son las 3 reglas de oro del Planning Poker?

► ¿Cuándo se usa la carta “?”

► ¿Qué pasa si no hay consenso?

✓ **Resumen en 3 frases**

1. Planning Poker es la técnica de estimación colectiva donde el equipo **vota en secreto** y **debate las diferencias** hasta llegar a consenso.
2. Las 3 reglas: **silencio** al votar, **extremos explican**, **consenso** (no imposición).
3. Si siempre coincidís rápido, asegúrate de que todos opinan; si nunca coincidís, la historia necesita más refinamiento.



Volver al [índice de la unidad](#) · Siguiendo: [02 — Velocidad del equipo](#)

02 — Velocidad del equipo



Estás en: [UD 2.2 · SCRUM Avanzado](#) → [02 · Velocidad del equipo](#)



La velocidad no es una carrera, es un termómetro

La **velocidad** (velocity) es la suma de story points completados en un sprint. No es un indicador de productividad, es una **herramienta de predicción**: con ella puedes calcular cuántos sprints necesitas para terminar el proyecto.



La idea en una frase

La velocidad es el número de story points completados por sprint. Sirve para predecir cuánto durará el proyecto, no para comparar personas.

No midas personas. Mide el ritmo del equipo.

Sprint 1: US-001 (5) + US-002 (3) + US-003 (2) + US-004 (5) + US-005 (3) = 18 puntos
Sprint 2: US-006 (8) + US-007 (3) + US-008 (5) + US-009 (2) + US-010 (3) = 21 puntos
Sprint 3: US-011 (5) + US-012 (5) + US-013 (8) + US-014 (3) = 21 puntos

Velocidad promedio: $(18 + 21 + 21) / 3 \approx 20$ puntos/sprint

¿Qué cuentas y qué no?

Sí cuenta	No cuenta
Historias completadas (100%)	Historias al 50% (no terminadas)
Puntos de historias hechas	Tareas técnicas sin estimación
Trabajo aceptado por el PO	Trabajo rechazado o sin revisar

Regla clave: Solo cuentan las historias **completadas al 100%**. Una historia al 90% vale 0 puntos para la velocidad.

Cómo usar la velocidad para planificar

Paso 1: Calcular la velocidad promedio

Velocidad promedio = (Suma de puntos de los últimos 3-5 sprints) / (Número de sprints)

Paso 2: Predecir la duración

Puntos pendientes: 80
Velocidad promedio: 20 puntos/sprint
Sprints necesarios: $80 / 20 = 4$ sprints

Paso 3: Planificar el sprint actual

Velocidad promedio: 20 puntos
Sprint actual: acepta como máximo 20 puntos de historias
(Si aceptas más, vas a sobrecargar al equipo)

La tabla de predicción

Puntos pendientes	Velocidad 15	Velocidad 20	Velocidad 25
30	2 sprints	2 sprints	2 sprints
60	4 sprints	3 sprints	3 sprints
90	6 sprints	5 sprints	4 sprints
120	8 sprints	6 sprints	5 sprints
150	10 sprints	8 sprints	6 sprints

Comparativa: velocidad como predicción vs como competición

Velocidad como competición

¡El sprint pasado hicimos 25 puntos! Este sprint tenemos que hacer 30. ¡Más rápido! El equipo se quema, la calidad baja, las historias entran sin terminar.

Velocidad como predicción

Nuestra velocidad es 20 ± 3 . Tenemos 80 puntos. Necesitamos 4 sprints. Aceptemos 20 puntos este sprint y trabajemos con calma.

Qué afecta a la velocidad

Factor	Efecto	Solución
Miembros nuevos	Baja al principio	Onboarding + pair programming
Enfermedades	Baja temporalmente	Sprint con margen
Dependencias externas	Bloquea trabajo	Identificar y planificar
Cambios de requisitos	Rehacer trabajo	Refinamiento + Definition of Ready
Mejora del equipo	Sube gradualmente	Es normal y deseable

Aclaración: “¿La velocidad debe subir siempre?”

- 🤔 ¿Si no sube la velocidad, estamos fallando?

Mini-chequeo

- ¿Qué es la velocidad del equipo?

- ¿Para qué sirve la velocidad?

- ¿La velocidad debe subir siempre?

✅ Resumen en 3 frases

1. La **velocidad** es la suma de story points completados por sprint: solo cuentan historias terminadas al 100%.
2. Úsala para **predecir** la duración del proyecto: puntos pendientes / velocidad promedio = sprints necesarios.
3. La velocidad **no es una competición** entre sprints: debe estabilizarse para indicar un ritmo sostenible.

03 — Burndown y Burnup

 Estás en:  UD 2.2 · SCRUM Avanzado → 03 · Burndown y Burnup

 *Un gráfico dice más que mil palabras en una reunión*

Los gráficos de **burndown** y **burnup** son las herramientas visuales más poderosas de SCRUM. Te muestran en tiempo real si vas por buen camino o si te vas a quedarte corto. Son el semáforo del proyecto.

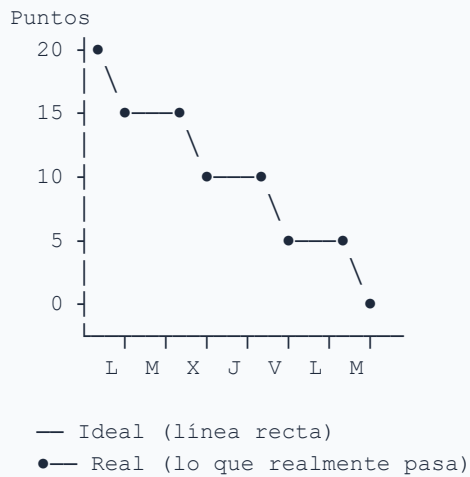
La idea en una frase

El burndown muestra cuánto queda por hacer. El burnup muestra cuánto se ha hecho. Los dos juntos cuentan la historia completa del proyecto.

Si no lo ves en un gráfico, no lo estás midiendo.

El gráfico de Burndown (sprint)

Muestra **cuántos puntos quedan por hacer** cada día del sprint.

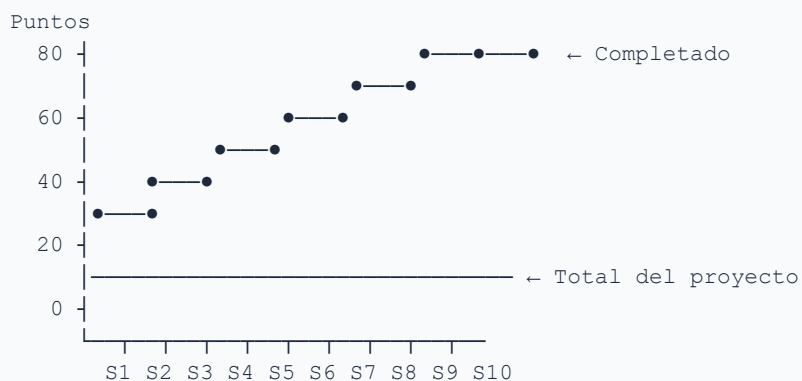


Cómo leerlo

Situación	Significado	Acción
Real por debajo de ideal	Vas bien, o incluso adelantado	Mantener ritmo
Real por encima de ideal	Vas con retraso	Identificar bloqueos
Plano	El equipo está atascado	Intervenir ya
Sube	Se añadieron historias al sprint	Evitar (scope creep)

El gráfico de Burnup (proyecto)

Muestra **cuántos puntos se han completado** a lo largo del tiempo.



Qué te dice

- **Línea superior** (completado): va subiendo con el tiempo

- **Línea inferior** (total): se mantiene estable (o sube si añades requisitos)
- **Distancia entre ambas**: lo que queda por hacer

Burndown vs Burnup

Burndown

Te muestra cuánto queda. Es útil para ver si vas a tiempo en un sprint. Pero si añades trabajo, la línea sube y confunde.

Burnup

Te muestra cuánto has hecho. Es más claro: la línea de completado siempre sube. Si añades trabajo, se ve en la línea de total.

Aspecto	Burndown	Burnup
Qué muestra	Cuánto queda	Cuánto se ha hecho
Útil para	Sprints (1-2 semanas)	Proyecto completo
Problema	Scope creep distorsiona	Menos intuitivo al principio
Claridad	Menos clara con cambios	Más clara siempre

Cómo crear tu propio burndown

Datos que necesitas

Día	Puntos restantes
Lunes	20
Martes	17
Miércoles	14
Jueves	10
Viernes	6

Herramientas sencillas

1. **Google Sheets:** crea una tabla y un gráfico de líneas
2. **GitHub Issues:** usa labels de puntos y cuenta los pendientes
3. **Trello/Jira:** ya generan burndown automáticamente

Aclaración: “¿Y si mi burndown sube en vez de bajar?”

► 🤔 ¿Qué significa que la línea de burndown suba?

Mini-chequeo

► ¿Qué muestra el burndown?

► ¿Qué muestra el burnup?

► ¿Qué hago si mi burndown sube?

1. El **burndown** muestra cuánto queda por hacer cada día: si la línea real está por encima de la ideal, vas con retraso.
2. El **burnup** muestra cuánto se ha completado: es más claro cuando hay cambios de alcance porque se ve en la línea de total.
3. Usa burndown para **sprints** y burnup para el **proyecto completo**: así siempre sabes dónde estás.

 Volver al [índice de la unidad](#) · Anterior: [02 — Velocidad del equipo](#) · Siguiendo: [04 — Bloqueos y dependencias](#)

04 — Bloqueos y dependencias

 Estás en:  **UD 2.2 · SCRUM Avanzado** → 04 · Bloqueos y dependencias

 *Un bloqueo a tiempo salva un sprint entero*

Todo sprint tiene obstáculos. La diferencia entre un equipo bueno y uno excelente es **cuándo detecta** los bloqueos. Un bloqueo descubierto el lunes se resuelve en horas. Uno descubierto el viernes destruye el sprint.

La idea en una frase

Un bloqueo es cualquier cosa que impida a un miembro del equipo avanzar en su trabajo. Detectarlo antes de que te destruya es responsabilidad de todo el equipo.

No esperes a la retrospectiva para hablar de problemas.

Tipos de bloqueos

Bloqueos internos (dentro del equipo)

Tipo	Ejemplo	Solución
Técnico	“No sé cómo usar esta librería”	Pair programming, investigar juntos
Dependencia interna	“Necesito que Ana termine la API antes”	Reorganizar tareas, ayudar a Ana
Decisión pendiente	“No nos ponemos de acuerdo en el diseño”	Votar, decidir rápido, avanzar
Falta de información	“No sé qué formato tiene el JSON”	Preguntar al PO, revisar documentación

Bloqueos externos (fuera del equipo)

Tipo	Ejemplo	Solución
Dependencia externa	“Esperando la API del instituto”	Identificar y planificar con antelación
Herramienta	“El servidor de pruebas está caído”	Tener alternativas, testear en local
Persona	“El PO no responde a mis preguntas”	Establecer horarios de respuesta
Requisito	“El profesor cambió los requisitos”	Documentar y priorizar de nuevo

El tablero de bloqueos

Añade una columna visible en tu Kanban:

Backlog	To Do	In Prog	Bloqueado	Done
		US-003	US-002 ● API externa	US-001

Regla de oro

Si una tarea lleva más de 1 día bloqueada, hay que actuar. No esperar.

Cómo gestionar dependencias

Paso 1: Identificar antes del sprint

En el sprint planning, pregunta: “¿Esta historia depende de algo que no está hecho?”

Dependencia	Riesgo	Plan
API externa	Alta	Testear con mock, luego conectar
Diseño del PO	Media	Pedir diseño antes del sprint
Otro miembro	Baja	Ayudar al compañero primero

Paso 2: Visualizar en el tablero

Marca las dependencias con colores o etiquetas:

- **Rojo:** dependencia externa (no controlas cuándo se resuelve)
- **Amarillo:** dependencia interna (puedes ayudar a resolverla)
- **Verde:** sin dependencias

Paso 3: Resolver proactivamente

No esperes a que tu compañero termine. Si puedes **ayudar**, ayuda. Si puedes **investigar**, investiga. El equipo avanza junto o no avanza.

Comparativa: gestionar bloqueos vs ignorarlos

Ignorar bloqueos

El sprint planning: ‘Esto depende de la API, ya se resolverá’. Sprint day 3: ‘La API no está’. Sprint day 7: ‘No hemos avanzado en esta historia’. Retrospectiva: sorpresa.

Gestionar bloqueos

El sprint planning: ‘Esta historia depende de la API. Plan: usar mock el lunes, conectar el miércoles’. Sprint day 3: ‘Mock funcionando, avanzando’. Sprint day 7: ‘Historia terminada con mock + API real’.

Aclaración: “¿Y si el bloqueo no depende de mí?”

- ▶ 🤔 ¿Qué hago si estoy bloqueado por algo que no controlo?

Mini-chequeo

- ▶ ¿Cuáles son los 4 tipos de bloqueos internos?

- ▶ ¿Cuándo actuar sobre un bloqueo?

- ▶ ¿Qué hacer si estoy bloqueado y no depende de mí?

✅ Resumen en 3 frases

1. Los bloqueos son **cualquier cosa que impida avanzar**: técnicos, dependencias, decisiones pendientes, falta de información.
2. **Identifícalos en el sprint planning** y visualízalos en el tablero con una columna “Bloqueado”.
3. Si un bloqueo lleva **más de 1 día**, actúa: busca alternativas, pide ayuda o cambia de tarea.

05 — Retrospectivas avanzadas

 **Estás en:**  **UD 2.2 · SCRUM Avanzado** → 05 · Retrospectivas avanzadas

 *La retrospectiva es la ceremonia más importante de SCRUM*

La retrospectiva no es una reunión para quejarse. Es la herramienta que hace que el equipo **mejore cada sprint**. Si tu retrospectiva es siempre “¿qué salió bien? ¿qué salió mal?”, es hora de cambiar de técnica.

La idea en una frase

Una retrospectiva avanzada no solo identifica problemas: genera acciones concretas que se implementan en el siguiente sprint.

Hablar no es suficiente. Hay que cambiar algo.

Por qué las retrospectivas básicas no funcionan

Retrospectiva básica	Problema
“¿Qué salió bien? ¿Qué salió mal?”	Siempre las mismas respuestas
“Alguien tiene que mejorar”	Nadie asume responsabilidad
“Hablamos de todo”	Se habla de nada en concreto
“Lo apuntamos para el próximo sprint”	Nunca se revisa si se hizo

5 técnicas avanzadas

1. Start / Stop / Continue

Cada miembro escribe en 3 columnas:

Start (empezar)	Stop (dejar)	Continue (mantener)
“Empezar a hacer pair programming”	“Dejar de estimar sin refinar”	“Continuar con las daily standups”
“Usar mocks para APIs externas”	“Dejar tareas bloqueadas más de 1 día”	“Continuar documentando decisions”

Por qué funciona: Es concreto y cada acción es asignable.

2. Mad / Sad / Glad

Cada miembro expresa cómo se sintió:

Mad (enfadado)	Sad (triste)	Glad (contento)
“Me enfadé porque la API cambió sin avisar”	“Triste que no terminamos la historia”	“Contento porque aprendimos GitFlow”

Por qué funciona: Las emociones revelan problemas que la lógica oculta.

3. 4Ls: Liked / Learned / Lacked / Longed For

Liked (gustó)	Learned (aprendí)	Lacked (faltó)	Longed For (quería)
“Me gustó el refinamiento”	“Aprendí Planning Poker”	“Faltó comunicación con el PO”	“Quería más tiempo para testing”

Por qué funciona: Cubre aspectos positivos, aprendizajes, carencias y deseos.

4. Sailboat (barco)

Visual: el equipo dibuja un barco con 4 elementos:

- **Islas** (meta): ¿a dónde queremos llegar?
- **Viento** (impulso): ¿qué nos ayuda?
- **Anclas** (freno): ¿qué nos frena?
- **Rocas** (riesgos): ¿qué puede salir mal?

Por qué funciona: Es visual, participativo y revela perspectivas diferentes.

5. Timer Toolbox

Cada tema tiene **5 minutos**: 1. **2 min** individual: cada uno escribe sus puntos 2. **3 min** discusión: solo los puntos más votados 3. **1 min** acuerdos: ¿qué hacemos concretamente?

Por qué funciona: Evita que una persona monopolice la conversación.

La fórmula STAR: estructura tus respuestas

Para que las aportaciones de la retrospectiva sean útiles, estructura cada una con la fórmula STAR:

- **Situación**: qué pasó
- **Task**: qué teníamos que hacer
- **Action**: qué hicimos
- **Result**: qué resultado obtuvimos

Ejemplo: “En el sprint 2 (S), teníamos que entregar la pantalla de login (T). No nos reunimos y cada uno fue por libre (A). El login quedó incompleto y perdimos un día arreglando conflictos (R). La próxima vez nos reuniremos 15 minutos antes de empezar.”

Por qué funciona: obliga a hablar de **hechos**, no de personas. Una aportación STAR convierte una queja en una mejora accionable.

Comparativa: retrospectiva básica vs avanzada

Básica

¿Qué salió bien? ‘El diseño’. ¿Qué salió mal? ‘Todo’. ¿Alguna acción? ‘Comunicarse mejor’.
Resultado: mismo problema el próximo sprint.

Avanzada

Start/Stop/Continue: Empezar pair programming, dejar de estimar sin refinar, continuar con daily. Acción concreta: lunes 10:00 primer pair programming. Resultado: mejora medible.

El checklist de la retrospectiva efectiva

Criterio	✓
¿Todos hablan (no solo el más ruidoso)?	☐
¿Hay acciones concretas asignadas?	☐
¿Se revisaron las acciones del sprint anterior?	☐
¿Se mantiene un tiempo límite?	☐
¿El ambiente es seguro (sin culpas)?	☐

Aclaración: “¿Cómo hacer que la gente hable?”

► 🤔 ¿Qué hago si en la retrospectiva nadie dice nada?

Mini-chequeo

► ¿Cuáles son las 5 técnicas avanzadas?

► ¿Cuánto dura una retrospectiva?

► ¿Qué hacer después de la retrospectiva?

► ¿Qué es la fórmula STAR?

✓ Resumen en 3 frases

1. Una retrospectiva avanzada usa **técnicas concretas** (Start/Stop/Continue, Mad/Sad/Glad, 4Ls, Sailboat, Timer Toolbox) y estructura cada aportación con la **fórmula STAR** en vez de preguntas genéricas.
2. El objetivo no es hablar: es generar **acciones asignables** que se implementen en el siguiente sprint.
3. Revisa las **acciones del sprint anterior** al inicio de cada retrospectiva: si nunca se revisan, no sirven de nada.

 Volver al [índice de la unidad](#) · Anterior: [04 — Bloqueos y dependencias](#) · Siguiente: [06 — Métricas y KPIs](#)

06 — Métricas y KPIs de SCRUM

 Estás en:  **UD 2.2 · SCRUM Avanzado** → 06 · Métricas y KPIs de SCRUM

 Sin métricas, estás adivinando. Con métricas, estás decidiendo

Las métricas no son para castigar: son para **entender**. Un equipo que mide sabe dónde está y hacia dónde va. Un equipo que no mide repite los mismos errores cada sprint sin saber por qué.

La idea en una frase

Las métricas de SCRUM te dicen la salud del proyecto: velocity mide ritmo, lead time mide velocidad de entrega, defect rate mide calidad.

Mide lo correcto, no todo.

Las 6 métricas esenciales

1. Velocity (velocidad)

Qué mide: Story points completados por sprint.

```
Sprint 1: 18 pts | Sprint 2: 21 pts | Sprint 3: 20 pts  
Velocidad promedio: 20 pts/sprint
```

Para qué sirve: Predecir cuándo terminarás.

2. Lead Time (tiempo de entrega)

Qué mide: Días desde que se acepta una historia hasta que está terminada.

```
US-001: aceptada lunes → terminada jueves = 4 días  
US-002: aceptada lunes → terminada miércoles = 3 días  
Lead time promedio: 3.5 días
```

Para qué sirve: Saber cuánto tarda una historia en llegar al usuario.

3. Cycle Time (tiempo de ciclo)

Qué mide: Días desde que se empieza a trabajar hasta que se termina.

```
US-001: empezada martes → terminada jueves = 3 días  
(Ciclo = Lead time - tiempo esperando en cola)
```

Para qué sirve: Detectar cuellos de botella en el proceso.

4. Defect Rate (tasa de defectos)

Qué mide: Historias que vuelven a “To Do” después de marcarse como “Done”.

```
Sprint 2: 20 historias hechas, 2 vuelven = 10% defect rate
```

Para qué sirve: Medir la calidad del trabajo.

5. Sprint Goal Achievement (cumplimiento del objetivo)

Qué mide: ¿Se cumplió el objetivo del sprint?

```
Sprint 1: ✅ Objetivo cumplido  
Sprint 2: ⚠️ Objetivo parcial (70%)  
Sprint 3: ✅ Objetivo cumplido
```

Para qué sirve: Saber si el equipo entrega lo que promete.

6. WIP (Work In Progress)

Qué mide: Número de tareas en progreso simultáneamente.

```
Recomendado: máximo 2-3 tareas por persona  
Si hay 5 tareas en "In Progress" con 3 personas → problema
```

Para qué sirve: Detectar multitasking excesivo.

Métricas que NO debes medir

Métrica mala	Por qué es mala
Horas trabajadas	No mide productividad, mide presencia
Líneas de código	Más código ≠ mejor código
Número de commits	Muchos commits pequeños ≠ productividad
Velocidad entre equipos	Cada equipo tiene su contexto

Comparativa: métricas útiles vs métricas tóxicas

Métrica tóxica

Pedro hizo 15 commits esta semana. Ana solo 5. Pedro es mejor. - Error: los commits no miden calidad ni valor. Ana puede haber hecho 1 commit con 200 líneas perfectas.

Métrica útil

El lead time promedio es 4 días. Si bajamos a 3, entregamos antes al usuario. - Correcto: mide valor al usuario, no actividad individual.

El dashboard del equipo

Un lugar donde ver todas las métricas de un vistazo:

```
# Dashboard Sprint 3

| Métrica | Valor | Tendencia |
|-----|-----|-----|
| Velocity | 20 pts | 🔄 Estable |
| Lead time | 3.5 días | ↓ Mejorando |
| Defect rate | 5% | ↓ Mejorando |
| Sprint goal | ✅ Cumplido | ↑ |
| WIP | 2.5 tareas/persona | 🔄 OK |

## Acciones del sprint
- Reducir WIP a 2 tareas/persona
- Mantener pair programming en historias complejas
```

Aclaración: “¿No es demasiado medir todo esto?”

▶ 🤔 ¿Necesito medir las 6 métricas?

Mini-chequeo

▶ ¿Cuáles son las 3 métricas más importantes?

▶ ¿Qué métrica NO debo medir?

▶ ¿Qué es el WIP?

✅ Resumen en 3 frases

1. Las 6 métricas esenciales son **velocity**, **lead time**, **cycle time**, **defect rate**, **sprint goal** y **WIP**.
2. Empieza con **3**: velocity (predecir), lead time (tiempo de entrega) y sprint goal (cumplimiento). Añade las otras después.
3. **Nunca midas** horas, líneas de código o commits: miden actividad, no valor. Mide para decidir, no para castigar.

 Volver al [índice de la unidad](#) · Anterior: [05 — Retrospectivas avanzadas](#) · Siguiente: [07 — SCRUM Master estudiante](#)

 Estás en:  UD 2.2 · SCRUM Avanzado → 07 · SCRUM Master estudiante

 *El SCRUM Master no manda. El SCRUM Master facilita*

En Proyecto 2, uno de vosotros será SCRUM Master. No es el jefe del equipo. Es la persona que **se asegura de que SCRUM funcione**: que se cumplan las ceremonias, que los bloqueos se resuelvan y que el equipo mejore cada sprint.

La idea en una frase

El SCRUM Master es facilitador, no jefe: su trabajo es que el equipo sea autónomo, no dar órdenes.

Si el equipo no puede funcionar sin ti, has fallado como SCRUM Master.

¿Qué hace un SCRUM Master?

Antes del sprint

Acción	Cómo	Ejemplo
Preparar el sprint planning	Asegurar que las historias están refinadas	Revisar el DoR antes de la reunión
Eliminar impedimentos	Buscar lo que el equipo necesita	Pedir acceso al servidor de pruebas
Facilitar la estimación	Que todos participen en Planning Poker	Preguntar al que menos habla

Durante el sprint

Acción	Cómo	Ejemplo
Protector	Evitar que outsiders interrumpen al equipo	“Ahora mismo el equipo está enfocado, ¿puedes esperar a la daily?”
Facilitador de daily	Que la daily dure 15 min y sea útil	Usar el formato: ¿qué hice? ¿qué haré? ¿estoy bloqueado?
Detector de bloqueos	Identificar y resolver impedimentos	“Veo que US-003 lleva 2 días bloqueada, ¿qué necesitáis?”

Después del sprint

Acción	Cómo	Ejemplo
Facilitar la retrospectiva	Usar técnicas avanzadas, no solo preguntar	Aplicar Start/Stop/Continue
Documentar acuerdos	Que las acciones tengan responsable y fecha	“Acción: pair programming los lunes. Responsable: Ana. Fecha: próximo sprint”
Revisar acciones anteriores	Ver si se cumplieron los compromisos	“El sprint pasado dijimos que haríamos pair programming, ¿se hizo?”

Lo que un SCRUM Master NO hace

NO hace	POR QUÉ
Dar órdenes	El equipo es autónomo
Tomar decisiones técnicas	El equipo decide cómo implementar
Hacer el trabajo del equipo	Facilita, no ejecuta
Culpar a individuos	Los problemas son del equipo, no de uno
Saltarse las ceremonias	Las ceremonias son para el equipo, no para el SM

Jefe de proyecto

Tú haces esto, tú haces lo otro. La reunión es a las 10. El deadline es el viernes.

SCRUM Master

¿Qué necesitáis para avanzar? ¿Hay algún bloqueo? La retrospectiva es a las 10, ¿estáis preparados?

Aspecto	Jefe de proyecto	SCRUM Master
Poder	Decide qué hacer	Facilita cómo hacerlo
Comunicación	Vertical (arriba-abajo)	Horizontal (equipo-equipos)
Responsabilidad	Del resultado	Del proceso
Enfoque	Plazos y entregables	Mejora continua

Los 5 antipatrones del SCRUM Master

Antipatrón	Problema	Solución
El SM jefe	Da órdenes, el equipo depende de él	Dejar que el equipo decida
El SM ausente	No facilita nada, solo tiene el título	Comprometerse con el rol
El SM multitasca	Es SM y desarrollador a la vez	Priorizar el rol de SM
El SM perfeccionista	Obliga a seguir SCRUM al 100%	Adaptar al contexto del aula
El SM siervo	Hace el trabajo de todos	Decir “no” y que el equipo asuma

Comunicación y conflictos: las herramientas del SM

El 70% de los fracasos de proyectos se deben a problemas de **comunicación**, no a problemas técnicos. El SM no resuelve los conflictos: **facilita que el equipo los resuelva**.

La fórmula SBI para dar feedback

- **Situación:** “En la daily de hoy...”
- **Behavior:** “...dijiste que terminarías la tarea y llevas 3 días con ella...”
- **Impact:** “...y eso retrasa al equipo entero.”

No es “eres un vago que no cumple”. Es “hay un problema con el plazo, ¿cómo lo solucionamos?”.

Las 5 reglas de comunicación que el SM garantiza

1. **Hablad pronto:** un problema detectado al principio se arregla en 5 minutos; al final puede echar abajo el proyecto
2. **Hablad claro:** “no funciona” no es comunicación; “el login falla en Firefox con Google” sí lo es
3. **Hablad con respeto:** “esto está mal” es una crítica; “¿has probado a hacerlo así?” es una sugerencia
4. **Escuchad:** escuchar no es esperar tu turno para hablar
5. **Documentad:** la memoria es volátil, los documentos no

Resolución de conflictos en 5 pasos

1. **Identificar:** no barrer bajo la alfombra
2. **Escuchar** a ambas partes sin interrumpir
3. **Encontrar el interés común:** casi siempre ambos quieren que el proyecto funcione
4. **Acordar una solución:** puede ser una tercera opción que nadie había pensado
5. **Documentar** la decisión y no reabrir el tema hasta el próximo sprint

► 🚨 ¿Qué hago si alguien del equipo no trabaja?

Aclaración: “¿Cómo ser SM sin ser el jefe?”

► 🤔 ¿Cómo facilito sin imponer?

Mini-chequeo

► ¿Cuál es el rol principal del SCRUM Master?

► ¿Qué NO debe hacer un SCRUM Master?

► ¿Cómo sé que soy un buen SM?

► ¿Qué es la fórmula SBI?

✓ Resumen en 3 frases

1. El SCRUM Master **facilita**, no manda: su trabajo es que el equipo sea autónomo y mejore cada sprint.
2. Sus 3 responsabilidades: **preparar** ceremonias, **eliminar** bloqueos, **documentar** acuerdos.
3. El indicador de éxito: cuando el equipo **funciona sin ti**, has hecho bien tu trabajo.

 Volver al [índice de la unidad](#) · Anterior: [o6 — Métricas y KPIs](#) · Siguiente: [o8 — Template SCRUM avanzado](#)

o8 — Template SCRUM avanzado

 Estás en:  UD 2.2 · SCRUM Avanzado → o8 · Template SCRUM avanzado

Este template incluye los formatos esenciales de SCRUM avanzado: sprint planning, daily standup, retrospectiva y dashboard de métricas. Cópialos, adapta los tiempos a tu proyecto y úsalos cada sprint.

La idea en una frase

Copia estos formatos en tu repositorio, rellénalos en cada sprint y tendrás un registro completo de la evolución de tu proyecto.

La organización no es burocracia. Es la base del trabajo profesional.

Template 1: Sprint Planning

```
# Sprint Planning – Sprint [N]

**Fecha**: [DD/MM/YYYY]
**Duración**: 2 horas
**Participantes**: [Nombres]

## Objetivo del sprint

¿Qué queremos conseguir este sprint? (1 frase concreta)

> [Escribir el objetivo aquí]

## Historias aceptadas

| ID | Historia | Puntos | Responsable | Estado |
|----|-----|-----|-----|-----|
| US-001 | [Título] | 5 | [Nombre] |  |
| US-002 | [Título] | 3 | [Nombre] |  |
| US-003 | [Título] | 8 | [Nombre] |  |
| **Total** | | **16** | | |

## Velocidad del equipo

| Sprint anterior | Velocidad | Puntos aceptados este sprint |
|-----|-----|-----|
| Sprint [N-1] | [X] pts | [Y] pts (máximo: velocidad promedio) |

## Dependencias y bloqueos conocidos

| Historia | Dependencia | Plan |
|-----|-----|-----|
| US-003 | API externa | Mock hasta día 3, conectar día 4 |

## Definition of Ready (checklist)

- [ ] Cumple INVEST
- [ ] Tiene criterios de aceptación (Given/When/Then)
- [ ] No tiene dependencias pendientes
- [ ] Está estimada en story points
- [ ] Cabe en un sprint (≤ 8 puntos)
```

Template 2: Daily Standup

```
# Daily Standup – Sprint [N] – Día [X]

**Fecha**: [DD/MM/YYYY]
**Hora**: [HH:MM] (máximo 15 minutos)

| Miembro | Ayer hice | Hoy haré | ¿Estoy bloqueado? |
|-----|-----|-----|-----|
| [Nombre 1] | [Tarea] | [Tarea] | No / [Descripción] |
| [Nombre 2] | [Tarea] | [Tarea] | No / [Descripción] |
| [Nombre 3] | [Tarea] | [Tarea] | No / [Descripción] |

## Bloqueos activos

| Bloqueo | Responsable | Acción | Fecha límite |
|-----|-----|-----|-----|
| [Descripción] | [Nombre] | [Qué hacer] | [Fecha] |

## Acciones de la daily

- [ ] [Acción concreta] – Responsable: [Nombre]
```

Template 3: Retrospectiva (Start/Stop/Continue)

```
# Retrospectiva – Sprint [N]

**Fecha**: [DD/MM/YYYY]
**Duración**: 60 minutos
**Técnica**: Start / Stop / Continue

## Start (empezar a hacer)

| Acción | Responsable | Sprint objetivo |
|-----|-----|-----|
| [Acción] | [Nombre] | Sprint [N+1] |

## Stop (dejar de hacer)

| Acción | Responsable | Sprint objetivo |
|-----|-----|-----|
| [Acción] | [Nombre] | Sprint [N+1] |

## Continue (mantener)

| Acción | ¿Por qué funciona? |
|-----|-----|
| [Acción] | [Razón] |

## Revisión de acciones del sprint anterior

| Acción del sprint [N-1] | ¿Se hizo? | Notas |
|-----|-----|-----|
| [Acción] |  /  | [Comentario] |
```

Template 4: Dashboard de métricas

```
# Dashboard – Sprint [N]

## Métricas clave

| Métrica | Valor | Tendencia | Objetivo |
|-----|-----|-----|-----|
| Velocity | [X] pts | ↑↓↔ | Estable |
| Lead time | [X] días | ↑↓↔ | < 5 días |
| Sprint goal | ☒/☒ | ↑↓ | ☒ siempre |
| WIP | [X] tareas/persona | ↑↓↔ | ≤ 2 |
| Defect rate | [X]% | ↑↓ | < 10% |

## Resumen del sprint

### Lo que salió bien
- [Punto 1]
- [Punto 2]

### Lo que mejorar
- [Punto 1]
- [Punto 2]

### Acciones para el próximo sprint
- [Acción 1] – Responsable: [Nombre]
- [Acción 2] – Responsable: [Nombre]
```

Aclaración: “¿No es mucho paperwork?”

► 🤔 ¿Necesito rellenar todo esto cada sprint?

Mini-chequeo

► ¿Cuántos templates hay?

► ¿Cuál es el mínimo necesario?

► ¿Dónde guardo los templates?

✓ Resumen en 3 frases

1. El template incluye **4 formatos**: Planning, Daily, Retrospectiva y Dashboard de métricas.
2. El **mínimo necesario** es Planning + Retrospectiva: las dos ceremonias que más valor aportan.
3. Adapta los templates a tu proyecto: no rellenes lo que no necesites, pero **no te saltes el Planning ni la Retrospectiva**.

 Volver al [índice de la unidad](#) · Anterior: [07 — SCRUM Master estudiante](#) · Siguiente: [09 — Cierre](#)

09 — Cierre

 Estás en:  **UD 2.2 · SCRUM Avanzado** → 09 · Cierre

 *SCRUM no es un conjunto de reglas, es una mentalidad de mejora*

Has recorrido toda la unidad: desde Planning Poker hasta las retrospectivas avanzadas, pasando por velocidad, burndown y el rol del SCRUM Master. Ahora toca **verificar que lo entiendes y aplicarlo a tu proyecto**.

La idea en una frase

SCRUM funciona si lo practicas. No es suficiente leerlo: hay que vivirlo en cada sprint, con sus aciertos y sus errores.

La teoría sin práctica es solo opinión.

Responde estas preguntas antes de seguir:

Preguntas de comprensión

► ¿Cuáles son las 3 reglas de oro del Planning Poker?

► ¿Qué es la velocidad del equipo?

► ¿Qué muestra el burndown?

► ¿Cuándo actuar sobre un bloqueo?

► ¿Qué es una retrospectiva avanzada?

► ¿Cuáles son las 3 métricas más importantes?

► ¿Qué hace un SCRUM Master?

► ¿Qué haces si el equipo no se pone de acuerdo?

Ejercicio práctico

En 2 horas, prepara tu primer sprint con SCRUM avanzado:

1. **Planning** (45 min): Presenta historias, vota con Planning Poker, acepta historias según velocidad
2. **Kanban** (15 min): Crea la columna “Bloqueado” y visualiza dependencias
3. **Dashboard** (15 min): Crea tu tabla de métricas con velocity, lead time y sprint goal
4. **Retrospectiva** (30 min): Usa Start/Stop/Continue con tu equipo
5. **Documenta** (15 min): Guarda todo en `SCRUM/` de tu repositorio

Si consigues tener un Planning + Dashboard + Retrospectiva documentados → has entendido la unidad.

Ejercicio: decisiones difíciles en el sprint

En el día a día del sprint también hay que decidir. Ante cualquier dilema, aplica la **matriz Value/Effort**:

	Bajo esfuerzo	Alto esfuerzo
Alto valor	HACERLO AHORA (quick win)	PLANIFICARLO (necesita tiempo)
Bajo valor	HACERLO SI SOBRA TIEMPO	NO HACERLO (descartar)

Ana — ‘Trabajemos el finde’

Si no terminamos la funcionalidad principal, el proyecto queda incompleto. Un fin de semana más y lo sacamos.

Pablo — ‘Recortemos features’

Si llegamos agotados al examen, peor. Mejor entregar menos funcionalidades bien hechas que muchas a medias.

Siempre recorta funcionalidades: el tribunal prefiere un proyecto completo con menos funciones que uno lleno de bugs. Y evita el **bikeshedding**: si la decisión no afecta al funcionamiento (el color de los botones), resólvvela en 2 minutos. La arquitectura sí merece debate.

Regla si el equipo no se pone de acuerdo: debatid 5 minutos, votad por mayoría simple, y quien pierde acata. Una decisión imperfecta ejecutada con convicción es mejor que una perfecta que nunca se tomó.



Tabla resumen de la unidad

Concepto	Idea clave
Planning Poker	Estimación colectiva: silencio, extremos explican, consenso
Velocity	Story points completados por sprint (solo 100%)
Burndown	Puntos que quedan por hacer cada día
Burnup	Puntos completados a lo largo del tiempo
Bloqueos	Cualquier cosa que impida avanzar (actuar en 1 día)
Retrospectiva avanzada	Técnicas concretas → acciones asignables
Métricas	Velocity, lead time, sprint goal como mínimo
SCRUM Master	Facilitador, no jefe: hace que el equipo funcione solo
WIP	Máximo 2-3 tareas por persona en progreso
Definition of Ready	Checklist para que una historia entre al sprint

Vocabulario rápido

Término	Definición
Planning Poker	Juego de estimación colectiva con cartas de puntos
Velocity	Suma de story points completados por sprint
Lead Time	Días desde que se acepta una historia hasta que está terminada
Cycle Time	Días desde que se empieza a trabajar hasta que se termina
Burndown	Gráfico de puntos restantes por día del sprint
Burnup	Gráfico de puntos completados a lo largo del tiempo
WIP	Work In Progress: tareas en progreso simultáneamente
Defect Rate	Porcentaje de historias que vuelven a “To Do” tras marcarse “Done”
SCRUM Master	Facilitador del proceso SCRUM, no jefe del equipo
Retrospectiva	Reunión de mejora continua al final de cada sprint
Value/Effort	Matriz para priorizar decisiones: valor (impacto) vs esfuerzo (tiempo/energía)
MoSCoW	Clasificación de requisitos: Must / Should / Could / Won't have
Bikeshedding	Gastar tiempo en decisiones triviales en vez de importantes
Definition of Ready	Checklist de “lista para entrar en sprint”
Scope Creep	Expansión no controlada de requisitos durante el sprint
Sprint Goal	Objetivo concreto del sprint que el equipo se compromete a cumplir

 **Conexión con las siguientes unidades**

Unidad	Qué aprenderás	Cómo se conecta
UD 2.3 Git Avanzado	GitFlow, hooks, CI/CD	Los branches seguirán la estructura de sprints que organizaste
UD 2.4 Patrones de Diseño	MVC, Repository, Observer	Implementarás las historias del sprint usando patrones
UD 2.5 Diagramas	C4, UML, secuencia	Modelarás la arquitectura que planificaste en el sprint
UD 2.6 IA como Copiloto	LLMs, prompt engineering	Usarás IA para acelerar las tareas del sprint

✓ Resumen en 3 frases

1. **Planning Poker** estima en equipo, la **velocidad** predice la duración y el **burndown** muestra el progreso real del sprint.
2. Los **bloqueos** se actúan en 1 día, las **retrospectivas** generan acciones concretas y las **métricas** dicen la salud del proyecto.
3. El **SCRUM Master** facilita, no manda: su éxito se mide por cuán poco le necesita el equipo.

 Volver al [índice de la unidad](#) · Anterior: [o8 — Template SCRUM avanzado](#) · Siguiente: [UD 2.3 — Git Avanzado](#)

o1 — GitFlow

 Estás en:  **UD 2.3 · Git Avanzado** → [o1 · GitFlow](#)

 Sin una estrategia de branches, Git se convierte en caos

GitFlow es la estrategia de branching más popular para proyectos con **versiones y releases definidos**. Define qué branch usar para cada cosa: features, hotfixes, releases. Sin ella, tu repositorio se convierte en un laberinto de ramas que nadie entiende.

La idea en una frase

GitFlow define 5 tipos de branches: main, develop, feature, release y hotfix. Cada uno tiene un propósito y un flujo claro.

No todas las ramas son iguales. Cada tipo de cambio tiene su branch.

Las 5 ramas de GitFlow

```
main (producción estable)
  ↑
  └─ release/v1.0 (preparando release)
      └─ develop (desarrollo integrado)
          ↑
          └─ feature/login (nueva funcionalidad)
              └─ feature/dashboard (nueva funcionalidad)
                  └─ feature/api (nueva funcionalidad)
                      └─ hotfix/bug-login (arreglo urgente)
```

Definición de cada branch

Branch	Propósito	Quién usa	Se fusiona con
main	Código en producción estable	Nadie trabaja directamente	—
develop	Última versión funcional	Todo el equipo	→ main (en release)
feature/	Nueva funcionalidad	1 desarrollador	→ develop
release/	Preparar una release	SM / tech lead	→ main Y develop
hotfix/	Arreglo urgente en producción	Quien lo detecta	→ main Y develop

Flujo completo de GitFlow

Feature (funcionalidad nueva)

```
# 1. Crear feature desde develop
git checkout develop
git checkout -b feature/login

# 2. Trabajar y hacer commits
git add .
git commit -m "feat: add login form"

# 3. Merge a develop
git checkout develop
git merge --no-ff feature/login
git push origin develop

# 4. Eliminar el branch de feature
git branch -d feature/login
```

Release (preparar versión)

```
# 1. Crear release desde develop
git checkout develop
git checkout -b release/v1.0

# 2. Últimos ajustes (sin features nuevas)
git commit -m "chore: bump version to 1.0.0"

# 3. Merge a main
git checkout main
git merge --no-ff release/v1.0
git tag -a v1.0.0 -m "Release v1.0.0"
git push origin main --tags

# 4. Merge a develop
git checkout develop
git merge --no-ff release/v1.0

# 5. Eliminar release
git branch -d release/v1.0
```

Hotfix (arreglo urgente)

```
# 1. Crear hotfix desde main
git checkout main
git checkout -b hotfix/bug-login

# 2. Arreglar el bug
git commit -m "fix: correct login validation"

# 3. Merge a main
git checkout main
git merge --no-ff hotfix/bug-login
git tag -a v1.0.1 -m "Hotfix v1.0.1"

# 4. Merge a develop
git checkout develop
git merge --no-ff hotfix/bug-login
```

Comparativa: sin GitFlow vs con GitFlow

Sin GitFlow

Todos trabajamos en main. Alguien hace merge de algo roto. Nadie sabe qué es producción y qué es desarrollo. El profesor pregunta '¿qué está desplegado?' y nadie responde.

Con GitFlow

Features en feature/, producción en main. El release es un branch de preparación. Si hay un bug urgente, hotfix desde main. Todo claro, todo trazable.

¿Cuándo usar GitFlow vs no usarlo?

Situación	GitFlow	Trunk-Based
Proyecto con versiones (v1.0, v2.0)	✅ Ideal	⚠️ Posible
Deploy continuo (varias veces al día)	❌ Pesado	✅ Ideal
Equipo pequeño (2-4 personas)	⚠️ OK pero pesado	✅ Más ligero
Proyecto de módulo (6 semanas)	✅ Recomendado	⚠️ Posible

Aclaración: “¿GitFlow no es demasiado para un proyecto de módulo?”

► 😬 ¿Necesito 5 tipos de branches para un proyecto de 6 semanas?

Mini-chequeo

► ¿Cuáles son las 5 ramas de GitFlow?

► ¿Desde dónde se crea un branch de feature?

► ¿Cuándo usar GitFlow?

✓ Resumen en 3 frases

1. GitFlow define **5 tipos de branches**: main, develop, feature, release y hotfix, cada uno con un propósito claro.
2. **Nunca trabajas directamente en main**: siempre crea un branch (feature, release o hotfix) y mézclalo cuando esté listo.
3. Para proyectos pequeños, **simplifica** a main + develop + feature: lo importante es separar producción de desarrollo.

 Volver al [índice de la unidad](#) · Siguiendo: [02 — Trunk-Based Development](#)

02 — Trunk-Based Development

 Estás en:  **UD 2.3 · Git Avanzado** → 02 · Trunk-Based Development

 *Menos branches, más integración, menos dolor*

Trunk-Based Development es la estrategia opuesta a GitFlow: todos trabajan en **main** (o en branches de muy corta vida) y se integra constantemente. Es la base del **deploy continuo** y de equipos como Google, Facebook o Netflix.

La idea en una frase

Trunk-Based Development significa integrar en main cada pocas horas:

branches cortos, integración frecuente, problemas pequeños.

Si tu branch vive más de 2 días, algo va mal.

¿Cómo funciona?

```
main ← todos integran aquí cada pocas horas
↑
├─ feature/login (1-2 commits, merge en 1 día)
├─ feature/dashboard (1-2 commits, merge en 1 día)
└─ feature/api (1-2 commits, merge en 1 día)
```

Las reglas

1. **Branches de vida corta:** máximo 1-2 días, idealmente horas
2. **Integración frecuente:** merge a main al menos 1 vez al día
3. **Tests automáticos:** CI debe pasar antes de merge
4. **Feature flags:** features incompletas se ocultan con flags, no con branches

Trunk-Based vs GitFlow

GitFlow

Mi branch 'feature-completa' tiene 50 commits y 2 semanas de vida. Cuando hago merge, conflicto con 3 archivos. Resuelvo conflictos durante 2 horas.

Trunk-Based

Mi branch tiene 2 commits y 4 horas de vida. Merge limpio, CI pasa, deploy automático. Si algo falla, es pequeño y fácil de arreglar.

Aspecto	GitFlow	Trunk-Based
Vida del branch	Días/semanas	Horas
Frecuencia de merge	Por feature completa	Varias veces al día
Conflictos	Grandes y dolorosos	Pequeños y frecuentes
Deploy	Manual, por versiones	Automático, continuo
Feature flags	No necesarios	Necesarios

Feature flags: la clave de Trunk-Based

Si una feature tarda 1 semana pero debes integrar cada pocas horas, ¿cómo lo haces? Con **feature flags**:

```
// En tu código
if (featureFlags.isEnabled('dark-mode')) {
  renderDarkMode();
} else {
  renderLightMode();
}
```

```
// feature-flags.json
{
  "dark-mode": false,
  "new-dashboard": true,
  "beta-api": false
}
```

La feature está en el código pero **desactivada**. Cuando esté lista, activas el flag.

¿Cuándo usar Trunk-Based?

Situación	Trunk-Based	GitFlow
Deploy continuo	✅ Ideal	❌ Pesado
Equipo maduro (con CI/CD)	✅ Ideal	⚠️ OK
Equipo junior	⚠️ Arriesgado	✅ Más seguro
Proyecto con versiones	⚠️ Posible con feature flags	✅ Natural

Aclaración: “¿Trunk-Based es demasiado arriesgado para nosotros?”

► 🤔 ¿Podemos usar Trunk-Based si somos un equipo junior?

Mini-chequeo

► ¿Cuál es la regla principal de Trunk-Based?

► ¿Qué son los feature flags?

► ¿Cuándo usar Trunk-Based vs GitFlow?

✅ Resumen en 3 frases

1. Trunk-Based Development integra en **main cada pocas horas**: branches de vida corta, integración frecuente, conflictos pequeños.
2. Los **feature flags** permiten integrar features incompletas sin que estén visibles: la clave para hacer Trunk-Based.
3. Es ideal para **deploy continuo** pero requiere **tests automáticos** y CI/CD configurado: sin eso, es arriesgado.

 Volver al [índice de la unidad](#) · Anterior: [01 — GitFlow](#) · Siguiente: [03 — Conflictos de merge](#)

03 — Conflictos de merge

 Estás en:  UD 2.3 · Git Avanzado → 03 · Conflictos de merge

 *Los conflictos no son errores. Son señales de que el equipo trabaja junto*

Un conflicto de merge aparece cuando **dos personas cambiaron la misma línea** de un archivo. No es dramático: Git te dice exactamente dónde está el problema y tú decides qué versión es la correcta.

La idea en una frase

Un conflicto de merge es Git diciéndote: “Dos personas tocaron esto. Necesito que decidas cuál es la versión correcta.”

No le tengas miedo. Léelo, entiéndelo, resuélvelo.

¿Cuándo aparecen conflictos?

```
Ana modifica: src/App.vue (línea 10)
Pedro modifica: src/App.vue (línea 15)

Ambos hacen push
Git: "¡Choque! No sé cuál es la versión correcta"
```

Qué ves en el archivo

```
<<<<<<< HEAD
function loginUser(email, password) {
  // Código de Ana
}
=====
function authenticateUser(email, password) {
  // Código de Pedro
}
>>>>>>> feature/login
```

Símbolo	Significado
<<<<<<< HEAD	Inicio de tu versión (la que tienes local)
=====	Separador
>>>>>>> feature/login	Fin de la versión que intentas mergear

Paso a paso para resolver un conflicto

Paso 1: Identificar el conflicto

```
git status
# M  src/App.vue (both modified)
```

Paso 2: Abrir el archivo y ver los marcadores

Busca las líneas <<<<<<< , ===== y >>>>>>> .

Paso 3: Decidir qué versión usar

Tres opciones:

Opción	Cuándo usarla
Quitar tu versión	La otra es mejor
Quitar la otra versión	La tuya es mejor
Combinar ambas	Las dos aportan algo

Paso 4: Guardar y hacer commit

```
git add src/App.vue
git commit -m "fix: resolve merge conflict in App.vue"
```

Ejemplo resuelto

Antes (con conflicto):

```
<<<<<< HEAD
function loginUser(email, password) {
=====
function authenticateUser(email, password) {
>>>>>> feature/login
```

Decisión: Usar `loginUser` (nombre más claro) pero con la lógica de validación de Pedro.

Después (resuelto):

```
function loginUser(email, password) {
  // Lógica de validación de Pedro aquí
}
```

Comparativa: conflictos frecuentes vs infrecuentes

Conflictos frecuentes

Cada merge tiene 5 conflictos. Resolvemos 30 minutos. Nadie sabe qué versión es la buena. A veces perdemos código.

Conflictos infrecuentes

Un conflicto al mes. Lo resolvemos en 5 minutos. Siempre sabemos qué versión elegir porque el equipo se comunica.

Cómo prevenir conflictos

Estrategia	Cómo	Efecto
Branches cortos	Merge cada pocas horas	Menos tiempo para chocar
Comunicación	“Voy a tocar App.vue” en la daily	Evita que dos personas toquen lo mismo
Archivos pequeños	Dividir en componentes	Menos probabilidad de choque
Integración frecuente	Pull antes de empezar	Tienes los cambios de los demás

Conflictos de personas: el protocolo humano

Los conflictos técnicos se resuelven con procedimiento. Los **conflictos entre personas** (distinto criterio, alguien que no avanza, tensiones) necesitan conversación honesta:

1. Habladlo en persona, no por chat

Quedad 15 min (presencial o videollamada, cámara encendida). Cada uno expone su punto **sin interrumpir** (2 min cada uno). Objetivo: entender al otro, no ganar. El chat empeora los malentendidos; la voz y la cara permiten empatía y matices.

2. Buscad un acuerdo o solución intermedia

- “Probamos tu enfoque en esta feature y el mío en la siguiente”.
- “Dividimos el módulo: tú haces X, yo Y, y revisamos juntos el contrato (interfaz)”.
- “Pedimos opinión al profesor como tie-break técnico”.

3. Si no hay acuerdo → tercera persona

Profesor (alcance, plazos, criterio de evaluación), compañero neutral (cuestiones técnicas puras) o mediador externo (si es relacional).

⚠ **No dejéis que un conflicto personal se enquisté.** Una semana de silencio = dos semanas de arreglo. La daily y la retro son los sitios naturales para sacar temas antes de que exploten.

El caso del “free rider” (quien no aporta)

Patrón: una persona no entrega, no responde, no viene a las dailies. Protocolo:

1. **Daily:** se nota (“ayer no avancé”, “hoy tampoco sé qué hacer”).
2. **Privado:** un compañero (no el grupo) le pregunta: “¿qué pasa? ¿te bloquea algo? ¿podemos ayudar?”.
3. **Planning:** se redistribuye su carga **visiblemente en el tablero** y se le asigna una tarea concreta y medible (docs, tests, despliegue, investigación).
4. **Seguimiento:** daily + check-in a medio día.
5. **Si persiste:** se informa al profesor **con datos** (tablero, commits, actas), no con quejas.

Aclaración: “¿Y si no sé qué versión es la correcta?”

► 🤔 ¿Qué hago si no sé si quedarme con mi versión o con la del otro?

Mini-chequeo

► ¿Qué significan los marcadores de conflicto?

► ¿Cuántas opciones tengo para resolver un conflicto?

► ¿Cómo prevenir conflictos?

► ¿Cuál es el primer paso ante un conflicto interpersonal?

► ¿Cómo se gestiona a un “free rider”?

1. Un conflicto de merge es Git diciéndote que **dos personas cambiaron la misma línea**: lee los marcadores, decide y resuelve.
2. Tres opciones: **quitar tu versión, quitar la otra o combinar ambas**. Si no sabes, pregunta al autor.
3. **Prevención**: branches cortos, comunicación, archivos pequeños e integración frecuente = menos conflictos. Y recuerda: los conflictos entre personas se resuelven **cara a cara**, no por chat.

 Volver al [índice de la unidad](#) · Anterior: [02 — Trunk-Based](#) · Siguiente: [04 — Git Hooks](#)

04 — Git Hooks

 Estás en:  **UD 2.3 · Git Avanzado** → 04 · Git Hooks

 *Los hooks son el guardián automático de tu código*

Los **Git hooks** son scripts que se ejecutan automáticamente antes o después de ciertas acciones de Git. El más útil es **pre-commit**: ejecuta linting, tests o formateo ANTES de permitirte hacer commit. Si algo falla, no puedes commitear. Punto.

La idea en una frase

Los Git hooks ejecutan scripts automáticamente en momentos clave: pre-commit verifica calidad, commit-msg valida el formato del mensaje.

Si no pasa el hook, no pasa el commit.

¿Qué son los Git hooks?

Son scripts en la carpeta `.git/hooks/` que Git ejecuta automáticamente:

```
.git/hooks/
├─ pre-commit      ← Se ejecuta ANTES de cada commit
├─ commit-msg      ← Se ejecuta DESPUÉS de escribir el mensaje
├─ pre-push        ← Se ejecuta ANTES de cada push
└─ post-merge      ← Se ejecuta DESPUÉS de cada merge
```

Hook 1: pre-commit (verificar calidad)

Este hook se ejecuta **antes** de cada commit. Si falla, el commit no se crea.

Archivo `.git/hooks/pre-commit` :

```
#!/bin/sh
# pre-commit hook: verifica calidad antes de permitir commit

echo "🔍 Ejecutando pre-commit checks..."

# 1. Verificar que no hay console.log en el código
if git diff --cached --name-only | xargs grep -l "console.log" 2>/dev/null; then
    echo "❌ Error: Hay console.log en el código. Elimínalo antes de commitear."
    exit 1
fi

# 2. Ejecutar linter (si existe)
if [ -f "package.json" ]; then
    echo "📄 Ejecutando linter..."
    npm run lint 2>/dev/null
    if [ $? -ne 0 ]; then
        echo "❌ Error: El linter ha encontrado problemas. Arréglalos antes de commitear."
        exit 1
    fi
fi

# 3. Verificar que no hay archivos de configuración con secrets
if git diff --cached --name-only | xargs grep -l "password\|secret\|api_key" 2>/dev/null; then
    echo "❌ Error: Parece que hay secrets en el código. No los commitees."
    exit 1
fi

echo "✅ Pre-commit checks pasados."
exit 0
```

Cómo activarlo:

```
# Hacer el script ejecutable
chmod +x .git/hooks/pre-commit
```

Hook 2: commit-msg (validar formato)

Este hook verifica que el mensaje de commit sigue la convención.

Archivo `.git/hooks/commit-msg` :

```
#!/bin/sh
# commit-msg hook: verifica que el mensaje sigue Conventional Commits

commit_msg=$(cat "$1")

# Patrón: tipo(alcance): descripción
# Tipos válidos: feat, fix, docs, style, refactor, test, chore, perf
pattern="^(feat|fix|docs|style|refactor|test|chore|perf) (\(.+\))?: .{1,72}"

if ! echo "$commit_msg" | grep -qE "$pattern"; then
    echo "❌ Error: El mensaje de commit no sigue Conventional Commits."
    echo ""
    echo "Formato esperado: tipo(alcance): descripción"
    echo ""
    echo "Ejemplos:"
    echo "  feat: add login form"
    echo "  fix: correct password validation"
    echo "  docs: update README"
    echo "  feat(auth): add Google login"
    echo ""
    echo "Tu mensaje: $commit_msg"
    exit 1
fi

echo "✅ Formato de commit válido."
exit 0
```

Hook 3: pre-push (verificar antes de enviar)

```
#!/bin/sh
# pre-push hook: ejecuta tests antes de push

echo "🔧 Ejecutando tests antes de push..."

if [ -f "package.json" ]; then
    npm test 2>/dev/null
    if [ $? -ne 0 ]; then
        echo "❌ Error: Hay tests fallando. Arréntalos antes de push."
        exit 1
    fi
fi

echo "✅ Tests pasados. Push permitido."
exit 0
```

Comparativa: con hooks vs sin hooks

Sin hooks

Hago commit con console.log. Push. El linter falla en CI. El profesor ve mi código con console.log. Vergüenza.

Con hooks

Hago commit. El hook detecta console.log. No me deja commitear. Lo elimino, commiteo limpio. CI pasa.

Instalación con Husky (alternativa moderna)

En vez de scripts en `.git/hooks/`, puedes usar **Husky** (para proyectos Node.js):

```
# Instalar Husky
npm install husky --save-dev

# Inicializar Husky
npx husky init

# Crear hook pre-commit
npx husky add .husky/pre-commit "npm run lint"

# Crear hook commit-msg
npx husky add .husky/commit-msg "npx commitlint --edit $1"
```

Archivo `.husky/pre-commit` :

```
npm run lint
```

Archivo `.husky/commit-msg` :

```
npx commitlint --edit $1
```

El kit completo: ESLint + Prettier + lint-staged

Los hooks manuales son útiles, pero el combo moderno para Node.js es **ESLint** (calidad: errores, malas prácticas) + **Prettier** (formato: espacios, comillas, comas) + **lint-staged** (solo archivos staged), disparados por un hook `pre-commit`. El resultado: el código de todo el equipo parece escrito por la misma persona, sin discusiones de estilo.

1. Instalar ESLint y Prettier

```
npm init @eslint/config@latest
# Elegir: TypeScript, Vue/React, Browser/Node, JSON config

npm install -D prettier eslint-config-prettier eslint-plugin-prettier
```

- `eslint-config-prettier` : **apaga** las reglas de ESLint que chocan con Prettier.
- `eslint-plugin-prettier` : hace que Prettier corra como regla de ESLint.

2. Configuración compartida

`.eslintrc.json` :

```
{
  "extends": [
    "eslint:recommended",
    "plugin:@typescript-eslint/recommended",
    "plugin:prettier/recommended",
    "prettier"
  ],
  "plugins": ["prettier"],
  "rules": {
    "prettier/prettier": "error",
    "@typescript-eslint/no-unused-vars": ["warn", { "argsIgnorePattern": "^_" }]
  }
}
```

`.prettierrc` :

```
{
  "semi": true,
  "singleQuote": true,
  "tabWidth": 2,
  "trailingComma": "es5",
  "printWidth": 100,
  "bracketSpacing": true,
  "arrowParens": "avoid"
}
```

💡 El estilo no es gusto personal: es **contrato de equipo**. Se acuerda una vez (15 min al empezar), se automatiza y se olvida. Escribid lo acordado en `docs/estilo-codigo.md` para el que llegue nuevo.

3. Husky + lint-staged (formatea y lintea antes del commit)

```
npm install -D husky lint-staged
npx husky install
npx husky add .husky/pre-commit "npx lint-staged"
```

`package.json` :

```
{
  "lint-staged": {
    "*.{js,ts,jsx,tsx,vue,json,css,md}": ["prettier --write", "eslint --fix"]
  }
}
```

Al hacer `git commit`, `lint-staged` pasa Prettier y ESLint `--fix` **solo a los archivos staged**. Si hay errores que no se auto-arreglan, el commit se cancela. Cero excusas.

4. Opcional: `.editorconfig` para el IDE

```
# .editorconfig
root = true
[*]
indent_style = space
indent_size = 2
end_of_line = lf
charset = utf-8
trim_trailing_whitespace = true
insert_final_newline = true
```

Configurar al principio

30 min una vez. Luego cero fricción: commits limpios, PRs legibles, reviews centrados en lógica. El CI pasa a la primera.

Arreglar después

Semanas de ‘formateo masivo’ antes de la entrega, conflictos en cada PR por espacios/comillas, reviews que se pierden en ruido. Horas perdidas.

Aclaración: “¿Los hooks son obligatorios?”

► 🤔 ¿Puedo saltarme un hook?

Mini-chequeo

► ¿Qué es un Git hook?

► ¿Qué verifica el hook pre-commit?

► ¿Qué es Husky?

► ¿Qué hace `lint-staged` en el hook pre-commit?

✅ Resumen en 3 frases

1. Los **Git hooks** ejecutan scripts automáticamente: pre-commit verifica calidad, commit-msg valida el formato, pre-push ejecuta tests.
2. Un **pre-commit** que busque `console.log`, ejecute linter y detecte secrets te ahorra problemas en CI y en la revisión del profesor.
3. Usa **Husky** si tu proyecto es Node.js: gestiona los hooks de forma más sencilla que los scripts manuales, y combínalo con **ESLint + Prettier + lint-staged** para que el estilo sea un contrato de equipo automatizado.

05 — CI/CD con GitHub Actions

 Estás en:  UD 2.3 · Git Avanzado → 05 · CI/CD con GitHub Actions

 CI/CD es como tener un asistente que verifica todo por ti

CI (Continuous Integration) ejecuta tests automáticamente en cada push. **CD** (Continuous Deployment) despliega automáticamente cuando todo pasa. **GitHub Actions** es la herramienta que lo hace todo: un archivo YAML y tienes un pipeline completo.

La idea en una frase

CI/CD con GitHub Actions significa: haces push, GitHub ejecuta tests, linting y build automáticamente. Si todo pasa, se despliega. Si falla, te avisa.

No hagas manualmente lo que puede automatizarse.

¿Qué es un pipeline CI/CD?

```
git push → GitHub Actions ejecuta:  
1. Install dependencies (npm install)  
2. Run linter (npm run lint)  
3. Run tests (npm test)  
4. Build (npm run build)  
5. Deploy (subir a GitHub Pages)
```

Si todo pasa →  Desplegado

Si algo falla →  Notificación

Ejemplo 1: Pipeline básico (lint + test + build)


```
name: CI

# Se ejecuta en cada push y pull request
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  ci:
    runs-on: ubuntu-latest

    steps:
      # 1. Checkout del código
      - name: Checkout
        uses: actions/checkout@v4

      # 2. Configurar Node.js
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      # 3. Instalar dependencias
      - name: Install dependencies
        run: npm ci

      # 4. Ejecutar linter
      - name: Run linter
        run: npm run lint

      # 5. Ejecutar tests
      - name: Run tests
        run: npm test

      # 6. Build
      - name: Build
        run: npm run build
```

Ejemplo 2: Deploy a GitHub Pages

Archivo `.github/workflows/deploy.yml` :

```
name: Deploy to GitHub Pages

on:
  push:
    branches: [main]

permissions:
  contents: read
  pages: write
  id-token: write

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Build
        run: npm run build

      - name: Upload artifact
        uses: actions/upload-pages-artifact@v3
        with:
          path: ./dist

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${ steps.deployment.outputs.page_url }
    steps:
      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```

Ejemplo 3: CI con cache para ser más rápido

```
name: CI with Cache

on:
  push:
    branches: [main]

jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Lint
        run: npm run lint

      - name: Test
        run: npm test

      - name: Build
        run: npm run build

      # Subir build como artifact
      - name: Upload build
        uses: actions/upload-artifact@v4
        with:
          name: build
          path: ./dist
```

Comparativa: manual vs CI/CD

Manual

Hago push. Olvido ejecutar tests. Subo código roto. El profesor lo ve. Me pide que lo arregle. Pierdo 2 horas.

CI/CD

Hago push. GitHub Actions ejecuta tests. Falla. Me avisa. Arreglo antes de que nadie lo vea. Ahorro 2 horas.

Aspecto	Manual	CI/CD
Tests	Los ejecuto cuando me acuerdo	Se ejecutan siempre
Linter	A veces lo salto	Obligatorio
Build	Lo hago manualmente	Automático
Deploy	Subo archivos a mano	Automático
Errores	Los descubro tarde	Los descubro al instante

Aclaración: “¿Necesito CI/CD para un proyecto de módulo?”

► 🤖 ¿Merece la pena configurar GitHub Actions para un proyecto de 6 semanas?

Mini-chequeo

► ¿Qué es CI/CD?

► ¿Qué es GitHub Actions?

► ¿Cuántos archivos necesito para CI/CD?

✅ Resumen en 3 frases

1. **CI/CD** ejecuta automáticamente tests, linting y build en cada push: si falla, te avisa; si pasa, despliega.
2. **GitHub Actions** se configura con **1-2 archivos YAML** en `.github/workflows/`: es más fácil de lo que parece.

3. CI/CD te ahorra errores públicos, olvidos de testing y despliegues manuales: es una inversión que se paga sola.

 Volver al [índice de la unidad](#) · Anterior: [04 — Git Hooks](#) · Siguiente: [06 — Convenciones de commit](#)

06 — Convenciones de commit

 Estás en:  **UD 2.3 · Git Avanzado** → [06 · Convenciones de commit](#)

 *Un commit sin convención es un mensaje inútil*

“arreglo”, “cosas”, “funciona”, “WIP”... Estos mensajes no le dicen a nadie qué cambió ni por qué. **Conventional Commits** es un estándar que hace tu historial de Git **legible, buscable y profesional**.

La idea en una frase

Conventional Commits usa un formato fijo: tipo(alcance): descripción. Así el historial de Git cuenta la historia de tu proyecto.

Tu historial de Git es documentación. Trátalo como tal.

El formato

tipo(alcance): descripción corta

[corpo opcional]

[feat opcional]

Ejemplos reales

```
feat: add login form with email validation
fix: correct password length validation
docs: update README with setup instructions
style: format code with Prettier
refactor: extract auth logic to separate module
test: add unit tests for login component
chore: update dependencies to latest versions
perf: optimize image loading with lazy loading
```

Los 8 tipos de commit

Tipo	Significado	Cuándo usarlo	Ejemplo
feat	Nueva funcionalidad	Algo nuevo que antes no existía	<code>feat: add task filtering</code>
fix	Corrección de bug	Algo que no funcionaba bien	<code>fix: correct date validation</code>
docs	Documentación	Cambios en README, docs, comentarios	<code>docs: add API documentation</code>
style	Formato	Espacios, sangría, Prettier (no cambia lógica)	<code>style: format with Prettier</code>
refactor	Refactorización	Reestructurar código sin cambiar comportamiento	<code>extract auth to module</code>
test	Tests	Añadir o modificar tests	<code>test: add login tests</code>
chore	Mantenimiento	Dependencias, CI, configuración	<code>chore: update npm packages</code>
perf	Rendimiento	Mejoras de rendimiento	<code>perf: add image lazy loading</code>

El alcance (opcional)

El alcance indica qué parte del código afecta:

```
feat(auth): add Google login
fix(api): correct user endpoint
docs(readme): update setup instructions
feat(ui): add dark mode toggle
```

El footer (opcional)

Parareferencias a issues o breaking changes:

```
feat: add password reset
```

Closes #42

```
feat!: change authentication API
```

BREAKING CHANGE: the login endpoint now requires email instead of username

Comparativa: commits sin convención vs con convención

Sin convención

git log: 'arreglo', 'cosas', 'funciona', 'WIP', 'update', 'fix', 'more changes'. No sé qué hizo nadie en qué sprint.

Con convención

git log: 'feat: add login', 'fix: correct validation', 'docs: update README'. Sé exactamente qué se hizo, en qué parte y por qué.

Generadores de commit messages

Commitlint (automático)

```
# Instalar
npm install @commitlint/cli @commitlint/config-conventional --save-dev

# Crear commitlint.config.js
module.exports = {
  extends: ['@commitlint/config-conventional']
};
```

Git-cz (interactive)


```
# Instalar
npm install commitizen cz-conventional-changelog --save-dev

# Usar
npx cz
# Te hace preguntas y genera el mensaje automáticamente
```

Aclaración: “¿Es obligatorio usar Conventional Commits?”

► 🤔 ¿Tengo que usar Conventional Commits en mi proyecto?

Mini-chequeo

► ¿Cuál es el formato de Conventional Commits?

► ¿Cuáles son los 8 tipos de commit?

► ¿Qué es Commitlint?

✅ Resumen en 3 frases

1. **Conventional Commits** usa el formato `tipo(alcance): descripción` para hacer el historial de Git legible y buscable.
2. Los 8 tipos son **feat**, **fix**, **docs**, **style**, **refactor**, **test**, **chore**, **perf**: cada uno tiene un propósito claro.
3. Usa **Commitlint** para verificar automáticamente los mensajes: si no cumple el formato, no deja committear.

07 — Code Review

 Estás en:  **UD 2.3 · Git Avanzado** → 07 · Code Review

 *Code review no es buscar errores: es aprender juntos*

El **code review** (revisión de código) es cuando un compañero revisa tu código antes de que se fusion con main. No es un examen: es una oportunidad para **aprender, detectar errores antes** de producción y mejorar la calidad del código del equipo.

La idea en una frase

El code review es una conversación sobre el código: el autor explica por qué, el revisor pregunta por qué no de otra manera. Ambos aprenden.

No es juzgar. Es mejorar juntos.

¿Cómo funciona un code review?

Flujo básico

1. Autor crea un Pull Request (PR)
2. Revisor(es) revisan el código
3. Comentarios → Autor responde/cambia
4. Aprobación → Merge a main

```
# 1. Crear branch y trabajar
git checkout -b feature/login
# ... hacer cambios ...
git push origin feature/login

# 2. En GitHub: crear Pull Request
# → Seleccionar main como destino
# → Añadir descripción
# → Asignar revisor

# 3. Revisor comenta en el PR
# → Comentarios inline en líneas específicas
# → Sugerencias de cambios
# → Aprobación o solicitud de cambios

# 4. Autor responde y hace cambios
# → Push de nuevos commits
# → Revisor re-ve
# → Merge cuando esté listo
```

Qué buscar en un code review

Lo que SÍ revisar

Categoría	Qué mirar	Ejemplo
Correctitud	¿Hace lo que debe?	“Esta función no valida email vacío”
Legibilidad	¿Se entiende el código?	“Este nombre de variable no es claro”
Seguridad	¿Expone datos?	“Esto tiene un console.log con la contraseña”
Rendimiento	¿Es eficiente?	“Este bucle carga todo en memoria”
Mantenibilidad	¿Es fácil de cambiar?	“Esto debería ser una función separada”

Lo que NO revisar

Categoría	Por qué no
Estilo personal	“A mí me gusta con espacios” → Usa Prettier
Preferencias	“Yo lo habría hecho así” → Si funciona, no toques
Cosas triviales	“Falta un punto y coma” → Usa linter automático

Cómo dar feedback efectivo

El modelo SBI (Situación-Comportamiento-Impacto)

Parte	Qué decir	Ejemplo
Situación	Dónde está el problema	“En la función login()...”
Comportamiento	Qué hace mal	“...estás guardando la contraseña en texto plano...”
Impacto	Por qué importa	“...esto es un problema de seguridad”

Formas de commentar

Malo ✗	Bueno ✓
“Esto está mal”	“¿Por qué no usas validación aquí?”
“Cambia esto”	“Podrías considerar extraer esto a una función”
“No entiendo”	“¿Puedes explicarme por qué usas estalibrería?”
“Esto es horrible”	“Hay una oportunidad de mejorar la legibilidad aquí”

Request changes vs Comment: la regla de los dos comentarios

En GitHub, cada comentario de un PR tiene un peso distinto. Saber cuándo usar cada uno evita bloquear por nimiedades y no dejar pasar lo importante:

Tipo	Cuándo usarlo	Ejemplo
Request changes	Objetivo: bug, test faltante, rompe la arquitectura	“Falta validación de email vacío en login()”
Comment	Subjetivo/opinión: estilo, sugerencia, pregunta	“¿Por qué esta variable se llama <code>data</code> y no <code>usuario</code> ?”

- Si es **objetivo** → Request changes con explicación clara (bloquea el merge).
- Si es **opinión** → Comment (no bloquea): conversad en el hilo o en la daily.

💡 Un nombre de variable, aunque sea mejor, es subjetivo: se comenta, se habla, no se bloquea. Un bug real, sí bloquea.

El PR Template: guía al autor y al revisor

Una plantilla de Pull Request obliga al autor a **explicar qué hace** y al revisor a **saber qué mirar**.

Se pega en `.github/pull_request_template.md` :

```
## Qué hace este PR
<!-- Descripción breve + link al issue -->
Closes #123

## Cómo probar
1. Pasos para verificar manualmente
2. Tests nuevos: `npm test`

## Capturas / logs (si aplica)
<!-- Pantallas, respuesta API, logs -->

## Checklist del autor
- [ ] Tests pasan en local
- [ ] Lint/Prettier OK
- [ ] Docs actualizadas (si toca)
- [ ] Sin `console.log` / `debugger` / código comentado

## Notas para el revisor
<!-- Qué mirar con lupa, dudas, decisiones técnicas -->
```

Comparativa: code review como ataque vs como aprendizaje

Como ataque

¿Por qué has hecho esto así? Está mal. Cambia todo. No sé por qué usaste esta librería. -> El autor se defiende, el ambiente se tense, nadie aprende.

Como aprendizaje

¿Por qué elegiste esta librería? Ah, no conocía esa opción. Podrías considerar X porque Y. -> El autor aprende, el revisor aprende, el código mejora.

Aclaración: “¿Y si me da vergüenza que revisen mi código?”

▶ 🤔 ¿Cómo superar la vergüenza del code review?

Mini-chequeo

▶ ¿Qué es un Pull Request?

▶ ¿Qué buscar en un code review?

▶ ¿Cómo dar feedback constructivo?

▶ ¿Cuándo uso Request changes y cuándo Comment?

▶ ¿Para qué sirve un PR Template?

✓ Resumen en 3 frases

1. El **code review** es cuando un compañero revisa tu código antes de merge: no es un examen, es una oportunidad de aprender juntos.
2. Revisa **correctitud, legibilidad, seguridad, rendimiento y mantenibilidad**. No revises estilo personal ni preferencias.
3. Da feedback con el modelo **SBI** (Situación-Comportamiento-Impacto): pregunta en vez de ordenar, sugiere en vez de imponer. Y usa la regla de los **dos comentarios**: Request changes para lo objetivo, Comment para opiniones.

 Volver al [índice de la unidad](#) · Anterior: [o6 — Convenciones de commit](#) · Siguiendo: [o8 — Template Git](#)

o8 — Template de workflow Git

 Estás en:  **UD 2.3 · Git Avanzado** → [o8 · Template de workflow Git](#)

 *Tu workflow de Git en un solo documento*

Este template incluye todo lo aprendido: GitFlow simplificado, convenciones de commit, hooks y la configuración de CI/CD. Cópialo, adapta-lo a tu proyecto y trabaja como un equipo profesional.

La idea en una frase

Guarda este template como `GIT-WORKFLOW.md` en la raíz de tu repositorio para que todo el equipo siga las mismas reglas.

Template completo

```
# Git Workflow - [Nombre del proyecto]

## Estructura de branches

- **main**: código en producción (desplegado)
- **develop**: última versión funcional (integrada)
- **feature/**: nueva funcionalidad (desde develop)
- **hotfix/**: arreglo urgente (desde main)

## Reglas

### Crear un branch de feature

```bash
git checkout develop
git checkout -b feature/nombre-descriptivo
Ejemplo: feature/login-form, feature/task-filter
```

## Hacer commits

Usar Conventional Commits:

```
git commit -m "feat: add login form"
git commit -m "fix: correct email validation"
git commit -m "docs: update README"
```

Tipos válidos: feat, fix, docs, style, refactor, test, chore, perf

## Merge a develop

```
git checkout develop
git merge --no-ff feature/nombre
git push origin develop
git branch -d feature/nombre
```

## Crear release



```
git checkout develop
git checkout -b release/v1.0
Últimos ajustes
git checkout main
git merge --no-ff release/v1.0
git tag -a v1.0.0 -m "Release v1.0.0"
git checkout develop
git merge --no-ff release/v1.0
```

## Hotfix

```
git checkout main
git checkout -b hotfix/bug-descriptivo
Arreglar el bug
git checkout main
git merge --no-ff hotfix/bug-descriptivo
git tag -a v1.0.1 -m "Hotfix v1.0.1"
git checkout develop
git merge --no-ff hotfix/bug-descriptivo
```

## Hooks (pre-commit)

El pre-commit hook verifica: - No hay console.log en el código - El linter pasa - No hay secrets expuestos

## CI/CD (GitHub Actions)

En cada push a main o develop: 1. Install dependencies 2. Run linter 3. Run tests 4. Build 5. Deploy automático (solo main)

---

## Aclaración: "¿Necesito todo esto para un proyecto de módulo?"

<details>

<summary>🤔 ¿No es excesivo este workflow para 6 semanas?</summary>

**\*\*Adáptalo a tu realidad\*\***. Para un proyecto de módulo, simplifica:

Completo	Simplificado
GitFlow completo (5 branches)	main + develop + feature
Husky + Commitlint	Solo pre-commit con lint
CI/CD completo	Solo CI (lint + test + build)

Lo importante es que **\*\*tengas reglas\*\*** y que **\*\*todo el equipo las siga\*\***. No necesitas .

</details>

---

## Mini-chequeo

<details>

<summary>¿Cuáles son las ramas mínimas necesarias?</summary>

**\*\*main\*\*** (producción) + **\*\*develop\*\*** (desarrollo) + **\*\*feature/\*\*** (cada funcionalidad). Es

</details>

<details>

<summary>¿Qué verifica el pre-commit?</summary>

Depende de tu configuración: puede verificar **\*\*console.log\*\***, ejecutar **\*\*linter\*\***, detec

</details>

<details>

<summary>¿Cuándo usar hotfix?</summary>

Cuando hay un **\*\*bug urgente en producción\*\*** que necesita arreglo inmediato. Se crea desc

</details>

---

## ✅ Resumen en 3 frases

1. El workflow define **\*\*ramas\*\*** (main, develop, feature), **\*\*convenciones\*\*** (Conventional
2. **\*\*Simplifica\*\*** para proyectos pequeños: main + develop + feature es suficiente. No ne
3. Guarda el template como `GIT-WORKFLOW.md` para que **\*\*todo el equipo\*\*** siga las mismas

---

> 🗺️ Volver al [índice de la unidad](../u2-3-git-avanzado) · Anterior: [07 – Code Review

---

## 09 – Cierre

> 🗺️ **\*\*Estás en:\*\*** 🌿 **\*\*UD 2.3 · Git Avanzado\*\*** → 09 · Cierre

> 📖 Git ya no es un misterio: es tu herramienta de trabajo diario\_

>

> Has recorrido toda la unidad: desde GitFlow hasta CI/CD, pasando por

> hooks, convenciones de commit y code review. Ahora toca **\*\*verificar que**

> lo entiendes\*\* y **\*\*aplicarlo a tu proyecto\*\***.

## 🗨 La idea en una frase

> **Git** no es solo guardar código: es una estrategia de trabajo que, bien usada, te ahorra dolores de cabeza.  
Si tu historial de Git es "cosas", algo va mal.

---

## 🔄 Mini-chequeo final

### Preguntas de comprensión

<details>

<summary>¿Cuáles son las 5 ramas de GitFlow?</summary>

**main** (producción), **develop** (desarrollo integrado), **feature** (nueva funcionalidad), **release** (preparación para producción), **hotfix** (corrección rápida de producción).  
</details>

<details>

<summary>¿Cuándo usar Trunk-Based vs GitFlow?</summary>

**Trunk-Based** para deploy continuo y equipos maduros. **GitFlow** para proyectos con ciclos de desarrollo complejos.  
</details>

<details>

<summary>¿Qué significan los marcadores de conflicto?</summary>

`<<<<<< HEAD` = tu versión, `=====` = separador, `>>>>>> feature` = la otra versión.  
</details>

<details>

<summary>¿Qué verifica un hook pre-commit?</summary>

Depende de la configuración: **console.log**, **linter**, **secrets**, tests... Lo que quieras validar antes de commit.  
</details>

<details>

<summary>¿Qué es CI/CD?</summary>

**CI**: ejecuta tests y build automáticamente en cada push. **CD**: despliega automáticamente a producción.  
</details>

<details>

<summary>¿Cuál es el formato de Conventional Commits?</summary>

`tipo(alcance): descripción`. Tipos: feat, fix, docs, style, refactor, test, chore, perf, hotfix, revert.  
</details>

<details>

<summary>¿Qué buscar en un code review?</summary>

**Correctitud** (¿hace lo que debe?), **legibilidad**, **seguridad**, **rendimiento** y buenas prácticas.  
</details>

---

### Ejercicio práctico

**En 2 horas, configura tu workflow de Git:**

- Estructura** (15 min): Crea main y develop. Crea tu primer branch de feature.
- Convenciones** (10 min): Añade un `GIT-WORKFLOW.md` con las reglas.
- Hook** (20 min): Configura un pre-commit que ejecute el linter.
- CI** (30 min): Crea `.github/workflows/ci.yml` con lint + test + build.
- Code review** (15 min): Crea un PR y pide revisión a un compañero.
- Prueba** (30 min): Haz push, verifica que CI funciona, merge el PR.

**Si consigues tener CI ejecutándose en cada push → has entendido la unidad.**

## ## 📄 Tabla resumen de la unidad

Concepto	Idea clave
----- -----	
<b>GitFlow</b>	5 branches: main, develop, feature, release, hotfix
<b>Trunk-Based</b>	Branches cortos, integración frecuente, deploy continuo
<b>Conflictos</b>	Dos personas cambiaron lo mismo: lee, decide, resuelve
<b>Git Hooks</b>	Scripts automáticos: pre-commit, commit-msg, pre-push
<b>CI/CD</b>	Automatizar: tests → build → deploy en cada push
<b>Conventional Commits</b>	`tipo(alcance): descripción` para historial legible
<b>Code Review</b>	Revisión de código: correctitud, legibilidad, seguridad
<b>Pull Request</b>	Solicitud de merge: código revisado antes de integrar

## ## 📖 Vocabulario rápido

Término	Definición
----- -----	
<b>GitFlow</b>	Estrategia de branching con 5 tipos de ramas
<b>Trunk-Based</b>	Desarrollo en main con branches de vida corta
<b>Feature branch</b>	Branch para una nueva funcionalidad
<b>Hotfix</b>	Branch para arreglar un bug urgente en producción
<b>Merge</b>	Integrar los cambios de un branch en otro
<b>Conflict</b>	Choque cuando dos personas cambiaron la misma línea
<b>Git Hook</b>	Script que Git ejecuta automáticamente
<b>Pre-commit</b>	Hook que se ejecuta antes de cada commit
<b>CI</b>	Continuous Integration: tests y build automáticos
<b>CD</b>	Continuous Deployment: deploy automático
<b>GitHub Actions</b>	Herramienta de GitHub para pipelines CI/CD
<b>Conventional Commits</b>	Estándar de mensajes de commit: tipo(alcance): descripción
<b>Pull Request</b>	Solicitud de merge en GitHub con revisión de código
<b>Code Review</b>	Revisión de código por un compañero antes de merge
<b>Changelog</b>	Registro automático de cambios por versión

## ## 🔗 Conexión con las siguientes unidades

Unidad	Qué aprenderás	Cómo se conecta
----- ----- -----		
<b>UD 2.4 Patrones de Diseño</b>	MVC, Repository, Observer	Implementarás features usando
<b>UD 2.5 Diagramas</b>	C4, UML, secuencia	Documentarás la arquitectura que estás co
<b>UD 2.6 IA como Copiloto</b>	LLMs, prompt engineering	Usarás IA para acelerar el de
<b>UD 2.7 Despliegue</b>	Docker, CI/CD avanzado	Tu pipeline CI/CD se extenderá con D

## ## ✅ Resumen en 3 frases

- GitFlow** separa producción de desarrollo con 5 tipos de branches; **Trunk-Based**
- Los **hooks** verifican calidad antes de commit, **Conventional Commits** hace el hi
- El **code review** no es un examen: es una conversación donde el autor explica y el r

> 📄 Volver al [índice de la unidad](../u2-3-git-avanzado) · Anterior: [08 – Template G

## ## 01 – ¿Por qué hacer tests?

> 📖 **Estás en:** 🧪 **UD 2.4 · Testing** → 01 · ¿Por qué hacer tests?

> 📖 Los tests son el seguro que todos necesitan pero nadie quiere contratar\_

>

> Tu proyecto funciona en tu ordenador. Pero ¿y mañana? ¿Y cuando

```

> alguien toque una línea de código? Los tests son verificaciones
> automatizadas que te dicen **pass** o **fail**. No adivinan: comprueban.
> Si algo se rompe, te avisan antes de que el profesor lo descubra.

📌 La idea en una frase

> **Un test es una verificación automatizada de que una parte de tu aplicación hace lo c

Si no puedes probarlo, no puedes confiar en ello.

¿Qué es un test?

Un test es un **pequeño programa que comprueba otro programa**. Le da una entrada, espe

```javascript
function calcularIVA(precio) {
  return precio * 1.21;
}

test("el IVA del 21% se calcula correctamente", () => {
  expect(calcularIVA(100)).toBe(121);
  expect(calcularIVA(50)).toBe(60.5);
  expect(calcularIVA(0)).toBe(0);
});

```

Si alguien cambia la función y rompe el cálculo, el test falla automáticamente. No necesitas que un usuario lo descubra.

Las 4 razones para hacer tests

1. Detectar errores antes de producción

Cada bug que atrapas con un test es un bug que el tribunal nunca verá.

2. Prevenir regresiones

Cuando refactoras o añades funcionalidad, los tests verifican que lo que antes funcionaba sigue funcionando. **Regresión** = volver a un estado roto que ya estaba arreglado.

3. Documentar comportamiento

Un test es documentación viva. Si quieres saber qué hace una función, lee su test. Es más fiable que un comentario porque se ejecuta y se verifica solo.

4. Dar confianza al entregar

El estrés antes de la entrega es real. Los tests te dan seguridad de que lo crítico funciona.

¿Cuándo testear?

Momento	Qué testear
Al escribir una función	Test unitario de esa función
Al integrar dos partes	Test de integración
Al entregar	Batería completa + manual
Al hacer un cambio grande	Tests de regresión
Antes de un merge	Todos los tests del proyecto

Test-first vs Test-after

Enfoque	Cómo funciona	Cuándo usarlo
Test-first (TDD)	Escribes el test antes del código	Cuando sabes qué quieres que haga la función
Test-after	Escribes el código, luego los tests	Cuando estás explorando o aprendiendo

Ambos son válidos. Lo importante es **que los tests existan**.

Tests como documentación viva

```
// Este test documenta que validarEmail acepta emails válidos
test("acepta emails válidos", () => {
  expect(validarEmail("usuario@ejemplo.com")).toBe(true);
  expect(validarEmail("a@b.co")).toBe(true);
});

// Y también documenta que rechaza emails inválidos
test("rechaza emails inválidos", () => {
  expect(validarEmail("")).toBe(false);
  expect(validarEmail("no-es-email")).toBe(false);
  expect(validarEmail("@falta-usuario.com")).toBe(false);
});
```

Sin este test, un compañero tendría que leer la función para entender qué acepta y qué rechaza. Con el test, lo sabe en 2 segundos.

Comparativa: sin tests vs con tests

Sin tests

¿Funcionará mañana? No lo sé. Esperemos que sí. Voy a probar a mano otra vez a las 3 de la madrugada.

Con tests

Los 47 tests pasan. Si mañana funciona, funciona. Si no, el test me dirá exactamente qué está roto.

Aclaración: “¿Y si no tengo tiempo?”

► 🕒 No me da tiempo de escribir tests, ¿qué hago?

Mini-chequeo

► ¿Qué es un test?

► ¿Cuáles son las 4 razones para hacer tests?

► ¿Test-first o test-after?

1. Un test es una **verificación automatizada** que comprueba input + output esperado + comparación: si falla, te avisa antes que un usuario.
2. Las 4 razones son: **detectar errores, prevenir regresiones, documentar comportamiento y dar confianza** al entregar.
3. No necesitas cobertura del 100%: **cubre lo crítico** primero y amplía cuando puedas.

 Volver al [índice de la unidad](#) · Siguiendo: [02 — Tipos de test](#)

02 — Tipos de test

 **Estás en:**  **UD 2.4 · Testing** → 02 · Tipos de test

 *Los tests son como las capas de una cebolla: cada una protege algo distinto*

Un equipo tenía 200 tests. El profesor preguntó: “¿Qué prueban?”. “Pues... todo”. Pero 180 eran de integración de base de datos y solo 5 probaban la lógica de negocio. Cuando el frontend se rompió, ningún test lo detectó.

Tener muchos tests no es lo mismo que tener los **tests adecuados**. Cada tipo cumple una función distinta.

La idea en una frase

Los tests se organizan en pirámide: muchos unitarios (base), algunos de integración (medio) y pocos E2E (cima). Si la inviertes, tus tests serán lentos y frágiles.

La pirámide de tests

E2E / UI (poco)	← costosos, lentos, frágiles
Integración (medio)	← verifican la unión de partes
Unitario (mucho)	← rápidos, baratos, precisos

En la base, **muchos tests unitarios**. En el medio, **algunos de integración**. En la cima, **pocos E2E**. Si inviertes la pirámide (muchos E2E y pocos unitarios), tus tests serán lentos y frágiles.

Test unitario

Un test unitario verifica **una función o método aislado**, sin dependencias externas. Es la pieza más pequeña del testing.

Qué prueba

- Una función de cálculo
- Una validación
- Una transformación de datos
- Una lógica condicional

Ejemplo

```
function esMayorDeEdad(edad) {  
  return edad >= 18;  
}  
  
test("devuelve true para 18 o más", () => {  
  expect(esMayorDeEdad(18)).toBe(true);  
  expect(esMayorDeEdad(25)).toBe(true);  
});  
  
test("devuelve false para menos de 18", () => {  
  expect(esMayorDeEdad(17)).toBe(false);  
  expect(esMayorDeEdad(0)).toBe(false);  
});
```

Características

Característica	Valor
Velocidad	Milisegundos
Coste	Bajo
Fragilidad	Muy baja
Alcance	Una sola unidad de código

Test de integración

Un test de integración verifica que **varias partes trabajan juntas** correctamente. No prueba una función aislada, sino la unión de dos o más componentes.

Qué prueba

- Comunicación entre frontend y backend
- Integración con una base de datos
- Llamadas a una API externa
- Flujo de datos entre capas

Ejemplo

```
test("crear usuario devuelve 201 y guarda en BD", async () => {
  const response = await request(app)
    .post("/api/usuarios")
    .send({ nombre: "Ana", email: "ana@test.com" });

  expect(response.status).toBe(201);
  expect(response.body.nombre).toBe("Ana");

  const usuario = await db.obtenerPorEmail("ana@test.com");
  expect(usuario).not.toBeNull();
  expect(usuario.nombre).toBe("Ana");
});
```

Características

Característica	Valor
Velocidad	Segundos
Coste	Medio
Fragilidad	Media
Alcance	Interacción entre componentes

Test E2E (End-to-End)

Un test E2E simula **un flujo completo del usuario**, desde la interfaz hasta la base de datos. Es como un usuario real usando la aplicación.

Qué prueba

- Flujo completo: registro → login → acción principal
- Formularios completos con validación
- Navegación entre páginas
- Comportamiento en el navegador

Ejemplo (Playwright)

```
test("permite registrarse y ver el perfil", async ({ page }) => {
  await page.goto("/registro");
  await page.fill('input[name="nombre"]', "Ana García");
  await page.fill('input[name="email"]', "ana@test.com");
  await page.fill('input[name="password"]', "123456");
  await page.click('button[type="submit"]');

  await expect(page).toHaveURL(/\/perfil/);
  await expect(page.locator("body")).toContainText("Ana García");
});
```

Características

Característica	Valor
Velocidad	Segundos a minutos
Coste	Alto
Fragilidad	Alta (cambios en UI los rompen)
Alcance	Flujo completo del usuario

Test manual

El test manual es cuando **una persona prueba la aplicación** haciendo clic, escribiendo y navegando. No es automatizado, pero es insustituible.

Qué prueba

- Aspecto visual (¿se ve bien?)
- Experiencia de usuario (¿es fácil?)
- Casos borde que un test automatizado no contempla
- Compatibilidad con distintos dispositivos

Cuándo usarlo

- **Siempre** antes de una entrega
- Cuando el aspecto visual importa
- Para descubrir problemas que la automatización no ve

► 🤖 ¿El test manual es inferior al automatizado?

Tabla comparativa

Tipo	Qué prueba	Velocidad	Coste	Cuándo usar
Unitario	Una función aislada	<1ms	Bajo	Siempre, en gran cantidad
Integración	Varias partes juntas	Segundos	Medio	Flujo principal, API, BD
E2E	Flujo completo	Minutos	Alto	Flujos críticos, pocas
Manual	Experiencia del usuario	Variable	Alto	Siempre antes de entregar

Ejemplo: pirámide en la práctica

Para una app de reservas de restaurantes:

Unitarios (15 tests): - Validar fecha de reserva (no pasada, no muy lejana) - Calcular precio total con IVA y descuentos - Verificar disponibilidad de mesa - Generar código de confirmación único

Integración (5 tests): - Crear reserva → guardar en BD → devolver confirmación - Login → obtener perfil → mostrar reservas del usuario - Cancelar reserva → actualizar disponibilidad de mesa

E2E (3 tests): - Flujo completo: login → buscar restaurante → reservar → confirmar - Flujo de cancelación: login → ver reserva → cancelar - Flujo de error: intentar reservar mesa ocupada → mostrar error

Manual: - Abrir en Chrome, Firefox, Safari - Probar en móvil y tablet - Verificar emails de confirmación - Comprobar mensajes de error comprensibles

Mini-chequeo

► ¿Qué tipo de test es más rápido?

► ¿Cuántos tests E2E debería tener comparado con unitarios?

► ¿Qué es un test de integración?

✓ Resumen en 3 frases

1. La pirámide de tests indica: **muchos unitarios** (base), **algunos de integración** (medio) y **pocos E2E** (cima).
2. Cada tipo cumple una función distinta: **unitario** para lógica aislada, **integración** para unión de partes, **E2E** para flujos completos.
3. Los tests **manuales** son complementarios, no inferiores: detectan problemas de experiencia que la automatización no ve.

 Volver al [índice de la unidad](#) · Anterior: [01 — ¿Por qué hacer tests?](#) · Siguiendo: [03 — Qué probar en tu proyecto](#)

03 — Qué probar en tu proyecto

 Estás en:  **UD 2.4 · Testing** → 03 · Qué probar en tu proyecto

 *No testees todo: testea lo que importa*

Un equipo llevaba 3 semanas escribiendo tests. Llevaban 40 tests... para las 3 pantallas menos importantes de la app. Las pantallas críticas seguían sin tests porque “ya las probaremos después”.

El profesor les dijo: “Si solo tenéis tiempo para 10 tests, ¿cuáles escribiríais?”. Se quedaron en silencio. No lo habían pensado.

La idea en una frase

No hace falta probarlo todo. Hace falta probar lo que importa: lo que más usa el usuario, lo que más rompe y lo que más cambia.

Las 3 reglas de priorización

1. Lo que más usa el usuario

Si el 80% de los usuarios hace el 20% de las funcionalidades, ese 20% merece el 80% de tus tests.

Pregúntate: ¿Cuáles son las 3 pantallas que todo usuario abre?

En una app de reservas: - Buscar un restaurante disponible - Crear una reserva - Ver mis reservas

En una app de tareas: - Crear una tarea - Marcar tarea como hecha - Ver lista de tareas pendientes

2. Lo que más rompe (fragilidad)

Algunas partes del código son más frágiles que otras:

- **Validaciones:** email inválido, fecha pasada, campos vacíos
- **Cálculos:** precios, descuentos, fechas
- **Condiciones borde:** lista vacía, usuario sin datos, conexión lenta
- **Integraciones:** llamadas a APIs externas, base de datos

3. Lo que más ha cambiado

Si acabas de refactorizar un módulo, esos cambios necesitan tests. Si escribiste código nuevo, necesita tests. Si arreglaste un bug, necesita un test que verifique que no vuelve a aparecer.

Mapa de riesgo

Riesgo ALTO Impacto ALTO → Testear	Riesgo BAJO Impacto ALTO → Testear
Riesgo ALTO Impacto BAJO → Testear	Riesgo BAJO Impacto BAJO → Ignorar

Las funcionalidades con **alto riesgo y alto impacto** son las que más tests necesitan. Fallan a menudo y si fallan, el proyecto sufre.

Ejemplo práctico: qué testear en una app de reservas

Funcionalidad	Uso	Fragilidad	Prioridad
Login	Alto	Baja	Media
Buscar restaurante	Alto	Media	Alta
Crear reserva	Alto	Alta	Muy alta
Cancelar reserva	Medio	Alta	Alta
Ver perfil	Bajo	Baja	Baja
Cambiar contraseña	Bajo	Media	Media
Ver historial	Bajo	Baja	Baja

Los tests deberían enfocarse en **crear reserva** y **cancelar reserva**: alto uso y alta fragilidad.

Nivel mínimo viable de testing

Como mínimo, tu proyecto debería tener:

Tests unitarios del core

Las funciones de lógica de negocio que son la esencia de tu aplicación:

- Validaciones de datos de entrada
- Cálculos (precios, fechas, puntuaciones)
- Transformaciones de datos
- Lógica de negocio principal

Un test de integración del flujo principal

El camino que sigue el usuario para completar la tarea principal:

- Registro → login → acción principal
- Búsqueda → selección → confirmación
- Formulario → envío → respuesta

Estrategia de testing por fases

Fase 1: Core (semanas 1-2)

Tests unitarios de las funciones de lógica de negocio. Un test de integración del flujo principal.

Fase 2: Estabilidad (semanas 3-4)

Tests de integración de las funcionalidades secundarias. Tests de validación y casos borde.

Fase 3: Robustez (semana 5)

Tests E2E de los flujos críticos. Pruebas manuales en múltiples dispositivos.

Fase 4: Entrega (semana 6)

Batería completa. Prueba manual exhaustiva.

Ejemplo: app de reservas de restaurantes

Sin priorizar

Escribimos 35 tests: login, perfil, configuración, idioma, historial... Pero crear reserva no tiene ni uno. 'Ya lo probaremos a mano'.

Priorizando

Escribimos 12 tests enfocados: crear reserva (5), cancelar reserva (3), buscar restaurante (2), login (2). Lo crítico está cubierto.

Aclaración: “¿Qué pasa si solo puedo testear una parte?”

► 🤔 No me da tiempo de testear todo, ¿qué priorizo?

Mini-chequeo

► ¿Cuáles son las 3 reglas de priorización?

► ¿Cuál es el nivel mínimo viable de testing?

► ¿Qué es el mapa de riesgo?

✅ Resumen en 3 frases

1. No testes todo: **prioriza por uso, fragilidad y cambio reciente** con las 3 reglas de priorización.
2. El nivel mínimo son **tests unitarios del core, integración del flujo principal y manual antes de entregar**.
3. Usa el **mapa de riesgo** para decidir qué función merece más tests y aplica la estrategia por fases del sprint.

 Volver al [índice de la unidad](#) · Anterior: [02 — Tipos de test](#) · Siguiente: [04 — Tests en tu flujo](#)

📖 El commit que rompió todo (y el test que lo evitó)

Martes por la mañana. Luis sube un cambio: “Refactorizo la función de validación para que sea más clara”. A las 11:00, Pedro hace pull y la app falla. “¿Qué has hecho, Luis?”. “En mi máquina funciona”.

Si los tests se hubieran ejecutado automáticamente antes del commit, el error se habría detectado en el momento. Los tests no son solo seguridad: son **comunicación** entre el equipo.

📌 La idea en una frase

Los tests no sirven si no se ejecutan. Integrarlos en tu flujo de trabajo es lo que los convierte en un seguro real.

El flujo básico: test antes de commit

La regla es simple: **no hagas commit si los tests fallan.**

```
Escribir código → Ejecutar tests → ¿Pasan? → Commit
                        ↓
                        No → Arreglar → Ejecutar tests → Commit
```

En la práctica

1. Antes de `git add`, ejecuta la batería de tests
2. Si todos pasan, haz commit
3. Si alguno falla, arregla el error primero
4. No hagas commit con tests fallidos “porque ya lo arreglo después”

Comandos útiles

```
# JavaScript (con npm)
npm test

# Java (con Maven)
mvn test

# Ver solo los tests que fallan
npm test -- --verbose
```

El flujo del equipo: evitar roturas

Cuando trabajáis varios en el mismo repositorio, los tests evitan que el código de uno rompa el de otro.

Antes de mergear una rama

1. Hacer pull de la rama principal
2. Ejecutar todos los tests
3. Si pasan, mergear tu rama
4. Si no pasan, arreglar antes de mergear

Sin CI

Confío en que mis compañeros ejecuten los tests antes de subir. A veces lo hacen, a veces no.

Con CI

GitHub ejecuta los tests automáticamente. Si fallan, el merge se bloquea. No hay forma de saltárselo.

CI: los tests se ejecutan solos

La **integración continua** (CI) ejecuta los tests automáticamente cuando alguien hace push. Si fallan, nadie puede mergear.

```
# .github/workflows/test.yml
name: Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm install
      - run: npm test
```

Si tu proyecto ya tiene CI para el despliegue, **añade este job de tests** antes del build. Así solo se despliega lo que pasa los tests.

Pre-commit hooks: la barrera local

Un **pre-commit hook** ejecuta los tests automáticamente antes de cada commit local:

```
# Instalar husky (JavaScript)
npm install --save-dev husky
npx husky init

# Configurar pre-commit
echo "npm test" > .husky/pre-commit
```

Ahora, cada vez que hagas `git commit`, Husky ejecutará `npm test`. Si falla, **el commit se bloquea** hasta que lo arregles.

► 🤔 ¿Y si no sé configurar Husky o CI?

Test-first: el enfoque TDD

TDD (Test-Driven Development) significa escribir el test **antes** del código:



Ejemplo

```
// 1. ROJO: el test falla porque la función no existe
test("sumar dos números", () => {
  expect(sumar(2, 3)).toBe(5);
});

// 2. VERDE: implementas la función mínima
function sumar(a, b) {
  return a + b;
}

// 3. REFACTOR: aquí no hace falta
```

Ventajas del TDD

- Cada función tiene al menos un test desde el primer momento
- El código crece respaldado por verificación continua
- Refactorizar es seguro: los tests te avisan si algo se rompe

Cuándo NO usar TDD

- Interfaces visuales y prototipos rápidos
- Cuando estás explorando cómo hacer algo
- Cuando el tiempo apremia y el test sería más lento que la función

Qué ejecutar y cuándo

Momento	Qué ejecutar
Antes de cada commit	Tests unitarios del código que has tocado
Antes de mergear	Todos los tests del proyecto
En CI (push/PR)	Todos los tests + lint
Antes de cada entrega	Batería completa + pruebas manuales

Mini-chequeo

► ¿Cuál es la regla básica antes de hacer commit?

► ¿Qué es un pre-commit hook?

► ¿Cuáles son los 3 pasos del ciclo TDD?

► ¿Qué hace la CI en GitHub Actions?

✓ Resumen en 3 frases

1. **No hagas commit si los tests fallan:** ejecuta la batería antes de `git add` y arregla antes de subir.
2. **Automatiza lo que puedas:** un pre-commit hook (Husky) protege en local y la CI (GitHub Actions) protege en el repositorio.
3. **TDD es una opción, no una obligación:** escribir el test antes del código funciona muy bien en lógica de negocio y mal en interfaces visuales.

 Volver al [índice de la unidad](#) · Anterior: [03 — Qué probar en tu proyecto](#) · Siguiente: [05 — Pruebas manuales](#)

05 — Pruebas manuales

 Estás en:  **UD 2.4 · Testing** → 05 · Pruebas manuales

El equipo de EventApp tenía 45 tests automatizados que pasaban todos. El día de la entrega, el profesor abrió la app en su portátil: el formulario de registro no se veía en la pantalla del proyector, el botón de “Crear evento” quedaba debajo de un banner que solo se veía en pantallas anchas, y un email con caracteres especiales rompía la app sin mensaje de error.

Los 45 tests no detectaron nada. **Los tests automatizados no ven pantallas. Las pruebas manuales son la red de seguridad que no pueden sustituir.**

La idea en una frase

Los tests automatizados verifican que el código es correcto. Las pruebas manuales verifican que la experiencia es buena.

Por qué los tests automatizados no bastan

Los tests automatizados comprueban resultados concretos: “esta función devuelve 5”, “esta API devuelve 201”. Pero no pueden comprobar:

- ¿El botón se ve en la pantalla?
- ¿El color del texto tiene suficiente contraste?
- ¿El formulario es fácil de rellenar?
- ¿El usuario sabe qué hacer después de ver cada pantalla?

Esas preguntas solo las responde una persona mirando la pantalla.

Solo automatizado

Los tests pasan, pero nadie ha abierto la app en un móvil, nadie ha probado el registro con un email raro y nadie sabe si los botones se ven en el proyector.

Automatizado + manual

Los tests pasan y además he recorrido la app como un usuario real: flujo principal, casos raros, móvil y navegador limpio. Sé que funciona de verdad.

Protocolo de pruebas manuales

1. Navegador limpio

Abre la aplicación en un navegador **sin datos de sesión**:

- Usa modo incógnito o privado
- Borra cookies y caché antes de empezar
- No uses el usuario que ya tienes guardado

¿Por qué? Si tienes la sesión iniciada, no estás probando el flujo real de un usuario que llega por primera vez.

2. Flujo principal (happy path)

Recorre el camino que **todo usuario** hace:

```
Registro → Login → Acción principal → Confirmación → Cerrar sesión
```

Marca en una lista cada paso completado. Si el flujo principal no funciona, nada más importa.

3. Casos borde

Ahora intenta **romperla** con entradas raras:

- Campos vacíos y formularios incompletos
- Emails con caracteres especiales (ñ, +, guiones)
- Texto muy largo o muy corto
- Números negativos, cero, decimales
- Fechas pasadas y fechas lejanas

4. Otros dispositivos y tamaños

- Redimensiona la ventana (o usa las herramientas del navegador)
- Prueba en móvil y tablet (vista responsive)
- Verifica que no haya desbordamientos ni elementos cortados

5. Fuera de línea y conexión lenta

- Prueba con la red desactivada: ¿la app se rompe o muestra un error claro?
- Simula una conexión lenta: ¿las pantallas de carga son aceptables?

Exploratory testing: rompe tu propia app

El **exploratory testing** es probar la app sin guion, buscando fallos como lo haría un usuario descontento:

Mente	Qué buscar
Usuario con prisa	Clicks rápidos, doble submit, volver atrás
Usuario confundido	¿Hay mensajes que no se entienden?
Usuario curioso	¿Qué pasa si hago clic en sitios inesperados?
Usuario con datos raros	Nombres con tildes, espacios, símbolos

▶ 📅 ¿Cuándo hacer pruebas manuales?

Mini-chequeo

▶ ¿Por qué los tests automatizados no detectan problemas de pantalla?

▶ ¿Cuáles son los 5 pasos del protocolo de pruebas manuales?

▶ ¿Qué es el exploratory testing?

✓

Resumen en 3 frases

1. Los tests automatizados no ven pantallas: **las pruebas manuales son la red de seguridad** que detecta problemas de experiencia, visuales y de dispositivo.
2. Sigue el **protocolo de 5 pasos** antes de cada entrega: navegador limpio, happy path, casos borde, dispositivos y conexión.
3. **Prueba siempre en el entorno real del tribunal:** proyector, conexión y resolución del aula, no solo en tu portátil.

06 — Herramientas de testing

 Estás en:  UD 2.4 · Testing → 06 · Herramientas

La herramienta equivocada para el trabajo correcto

El equipo de TaskManager usaba JUnit para testear su frontend en React. Los tests eran lentos, frágiles y cada cambio de CSS los rompía.

“¿Por qué usáis JUnit para el frontend?”, preguntó el profesor. “Porque es lo que conocemos”, respondieron.

Cambiaron a Testing Library para el frontend y JUnit solo para el backend. Los tests se volvieron rápidos y estables. A veces no es cuestión de esfuerzo, sino de **usar la herramienta adecuada**.

La idea en una frase

No todas las herramientas sirven para todo. Elegir la correcta para cada capa te ahorra horas de frustración.

Herramientas para JavaScript

El framework de testing **moderno y rápido**, nativo de Vite. Si tu proyecto usa Vite, es la opción natural.

```
// Instalación
npm install --save-dev vitest

// package.json
{
  "scripts": {
    "test": "vitest"
  }
}
```

Características: - API compatible con Jest (migración fácil) - Más rápido que Jest en proyectos con Vite - Soporte nativo de TypeScript - Watch mode y modo UI integrados

Jest

El framework **más popular y maduro** de JavaScript. Creado por Meta para React, funciona con cualquier proyecto JS.

```
// Instalación
npm install --save-dev jest

// package.json
{
  "scripts": {
    "test": "jest"
  }
}
```

Características: - Incluye todo: aserciones, mocks y cobertura de código - Ejecución paralela automática - Gran comunidad y documentación

Jest

Soy maduro, tengo documentación para todo y funciona con cualquier proyecto JavaScript. Si no sabes cuál elegir, empieza por mí.

Vitest

Soy rápido, moderno y si ya usas Vite, me configuras en 2 minutos. Y mi API es la misma que la de Jest.

Testing Library

No es un framework: es un conjunto de bibliotecas para testear interfaces **desde la perspectiva del usuario final**.

```
// React Testing Library
test("el usuario puede escribir en el buscador", () => {
  render();
  const input = screen.getByPlaceholderText("Buscar...");
  fireEvent.change(input, { target: { value: "hola" } });
  expect(input.value).toBe("hola");
});
```

Filosofía: testea lo que el usuario ve y hace, no los detalles internos del componente.

Herramientas para Java

JUnit 5

El **estándar** para testing en Java. Si has dado Programación, ya lo conoces.

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculadoraTest {

    @Test
    void sumarDosNumeros() {
        Calculadora calc = new Calculadora();
        assertEquals(5, calc.sumar(2, 3));
    }

    @Test
    void dividirPorCeroLanzaExcepcion() {
        Calculadora calc = new Calculadora();
        assertThrows(ArithmeticException.class,
            () -> calc.dividir(10, 0));
    }
}
```

Características: - Anotaciones claras: `@Test` , `@BeforeEach` , `@AfterEach` - Integración con Maven y Gradle - Assertions completas: `assertEquals` , `assertTrue` , `assertThrows`

Mockito

Para crear **mocks** (objetos falsos) en Java. Útil cuando tu código depende de una base de datos o un servicio externo.

```
@ExtendWith(MockitoExtension.class)
class ReservaServiceTest {

    @Mock
    ReservaRepository repository;

    @InjectMocks
    ReservaService service;

    @Test
    void crearReservaGuardaEnBD() {
        Reserva reserva = new Reserva("Sala A", LocalDate.now());
        when(repository.save(any())).thenReturn(reserva);

        Reserva resultado = service.crear(reserva);

        verify(repository).save(reserva);
        assertEquals("Sala A", resultado.getSala());
    }
}
```

Herramientas E2E

Playwright

La opción **moderna** para tests de extremo a extremo. Navegadores reales, multi-pestaña y grabación visual.

```
// Playwright
test("registro y perfil", async ({ page }) => {
    await page.goto("/registro");
    await page.fill('input[name="email"]', "ana@test.com");
    await page.click('button[type="submit"]');
    await expect(page).toHaveURL(/\/perfil/);
});
```

Cypress

El framework E2E **más popular** para aplicaciones web. Ejecuta las pruebas directamente en el navegador con una interfaz visual muy cuidada.

Cuándo elegir cada uno: ambos valen. Si ya usas la suite de Playwright en clase, quédate con Playwright; si te llama más el flujo visual de Cypress, úsalo. Lo importante es que **tengas al menos 1-2 tests E2E de tus flujos críticos**.

Tabla de decisión según tu stack

Stack	Tests unitarios	Tests de UI	Tests E2E
JavaScript/TypeScript + Vite	Vitest	Testing Library	Playwright o Cypress
JavaScript (cualquier proyecto)	Jest	Testing Library	Playwright o Cypress
Java + Spring	JUnit 5 + Mockito	—	Playwright (contra la API)
JavaScript + Java (front/back)	Vitest (front) + JUnit (back)	Testing Library	Playwright

►  ¿Cuál elijo para mi proyecto?

Mini-chequeo

► ¿Qué herramienta elegir para un proyecto JavaScript con Vite?

► ¿Y para un backend en Java?

► ¿Cuál es la filosofía de Testing Library?

► ¿Puedo mezclar Jest y Vitest en el mismo proyecto?

✓ Resumen en 3 frases

1. **JavaScript:** Vitest (o Jest) para unitarios, Testing Library para la interfaz y Playwright para E2E.
2. **Java:** JUnit 5 para unitarios y Mockito para mocks de dependencias.

3. **Elige la misma herramienta que tu stack** y no mezcles runners: es mejor pocos tests bien hechos que un ecosistema que nadie domina.

 Volver al [índice de la unidad](#) · Anterior: [05 — Pruebas manuales](#) · Siguiente: [07 — Testing en equipo](#)

07 — Testing en equipo

 Estás en:  **UD 2.4 · Testing** → 07 · Testing en equipo

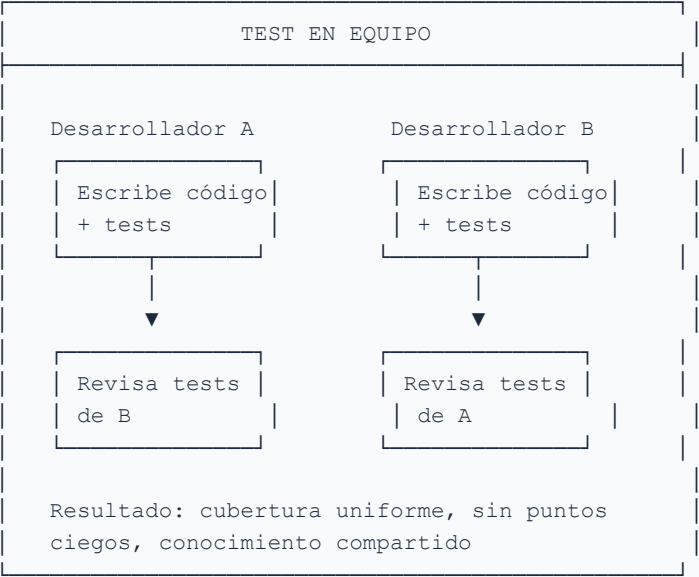
 *El testing es responsabilidad de todos, no solo de quien escribió el código*

En un proyecto real, **cada miembro del equipo escribe y revisa tests**. Si solo una persona testea, el proyecto tiene un punto ciego: la persona que se va y deja código sin cubrir. Testing en equipo significa **responsabilidad compartida**, acuerdos claros y code review con foco en calidad.

La idea en una frase

El mejor test no es el que escribes tú: es el que tu compañero entiende, revisa y mantiene contigo.

Responsabilidad compartida



Reglas básicas

Regla	Por qué
Cada uno escribe tests para su código	Si no los escribes, nadie lo hace por ti
Cada uno revisa tests de los demás	Segundo par de ojos detecta más bugs
Los tests se commitean junto al código	Sin tests, el PR no se aprueba
Se usan los mismos frameworks	Consistencia: mismo runner, misma configuración
No se saltan tests para “ir más rápido”	La deuda técnica se paga después, con intereses

Code review con foco en testing

Al revisar un PR, estos son los **5 aspectos** que debes comprobar:

1. Cobertura de tests

¿El código nuevo tiene tests? ¿Cubre el happy path y los casos límite?

CODE REVIEW: COBERTURA

PR: "Añadir validación de fecha"

- ✓ Test happy path
- ✓ Test fecha pasada
- ✓ Test fecha futura lejana
- ✓ Test formato incorrecto
- ✗ Test string vacío

→ Pedir: test para string vacío

2. Casos límite

¿Se testean entradas vacías, null, valores extremos, errores de red?

3. Nombres descriptivos

¿Los tests explican qué se verifica? Sigue la convención:

```
should [comportamiento esperado] when [condición]
```

Ejemplo:

```
should return error when date is in the past
should create task when all fields are valid
should show message when user has no tasks
```

4. Aislamiento

¿Los tests dependen de datos externos, estado global o ejecución en orden? Un test que falla solo “a veces” es peor que no tener test.

5. Legibilidad

¿Un compañero entiende qué verifica el test sin leer el código fuente? Si necesitas comentarios para explicar un test, el nombre del test está mal.

Comparativa: dos enfoques de testing

Enfoque A — Testing individual

Cada uno escribe sus tests y nadie los revisa. El testing es ‘mi responsabilidad’. Resultado: tests inconsistentes, algunos buenos otros inexistentes, conocimiento fragmentado.

Enfoque B — Testing colaborativo

Cada uno escribe tests para su código, pero el compañero los revisa. Se comparten convenciones y herramientas. Resultado: cobertura uniforme, aprendizaje mutuo, menos puntos ciegos.

Acuerdos de testing del equipo

Estos acuerdos se definen al inicio del proyecto y se documentan en `TESTING.md` :

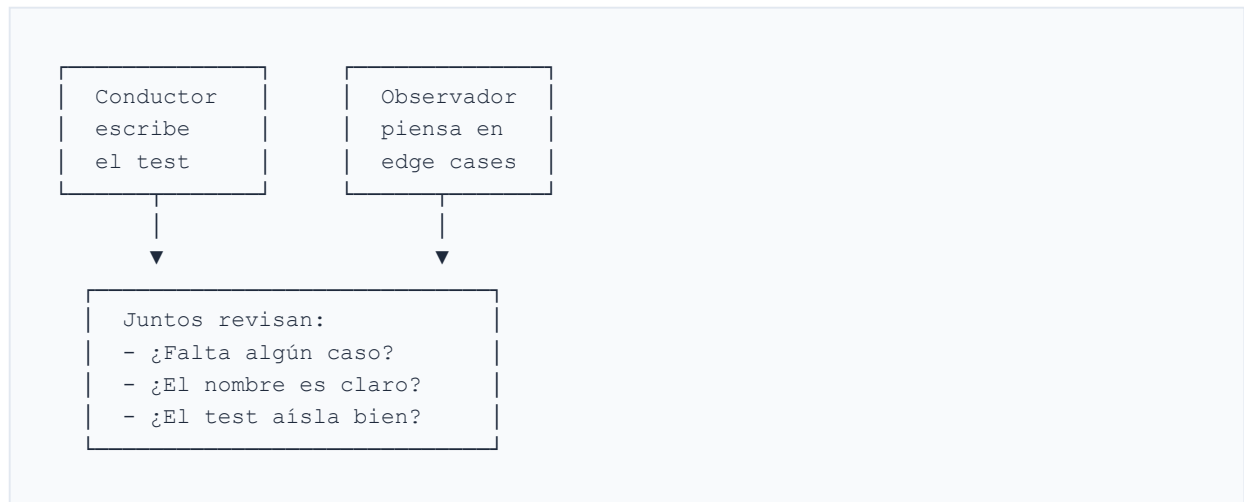
Acuerdo	Descripción
Framework	Vitest (frontend), JUnit (backend)
Cobertura mínima	60% en funciones core
Convención de nombres	<code>should [behavior] when [condition]</code>
Dónde van los tests	Junto al componente: <code>TaskList.test.js</code>
Antes de merge	Todos los tests pasan, sin excepción
Tests pendientes	Se crean issues en GitHub con etiqueta <code>testing-debt</code>
Revisión	Cada PR incluye revisión de tests

Pair testing

El **pair testing** es un método donde dos personas trabajan juntas en los tests de una función:

Rol	Quién	Qué hace
Conductor	Escribe el código y los tests	Decide la implementación
Observador	Revisa, sugiere, piensa en casos límite	Detecta huecos en la cobertura

Flujo del pair testing



Ventaja: dos personas detectan más problemas que una. El observador piensa en casos que el conductor no ve.

► 💡 ¿Y si mi compañero no escribe tests?

Mini-chequeo

► ¿Por qué el testing no debe ser solo responsabilidad de una persona?

► ¿Qué 5 aspectos se revisan en un code review con foco en testing?

► ¿Qué es el pair testing y por qué ayuda?

✓ Resumen en 3 frases

1. El **testing es responsabilidad de todo el equipo**: cada uno escribe tests para su código y revisa los tests de los demás en los PRs.
2. El **code review con foco en testing** comprueba 5 aspectos: cobertura, casos límite, nombres, aislamiento y legibilidad.
3. Los **acuerdos de testing** se documentan en `TESTING.md` al inicio del proyecto: framework, cobertura, convención de nombres y reglas de merge.

 Volver al [índice de la unidad](#) · Anterior: [o6 — Herramientas](#) · Siguiente: [o8 — Checklist de calidad](#)

o8 — Checklist de calidad

 Estás en:  UD 2.4 · Testing → o8 · Checklist de calidad

 *El checklist es tu red de seguridad antes de entregar*

Antes de cada entrega al tribunal, necesitas verificar que todo funciona: tests pasan, la app carga, el responsive funciona y no hay secrets en el repositorio. Este checklist es la **última revisión** antes de hacer push definitivo. Imprímelo, márcalo paso a paso, y no te saltes ninguno.

La idea en una frase

Un checklist no es burocracia: es la diferencia entre entregar con confianza y entregar con miedo.

Checklist completo de calidad

1. Tests unitarios

Verificación	Estado
Todas las funciones core tienen tests	☐
Los casos límite están testeados (vacío, null, extremos)	☐
Todos los tests pasan: <code>npm test</code>	☐
No hay tests pendientes o skipped sin justificación	☐

2. Tests de integración

Verificación	Estado
Los endpoints de la API responden correctamente	☐
Las operaciones de base de datos funcionan	☐
El flujo completo login → acción → guardar funciona	☐
Los errores de red se manejan correctamente	☐

3. Pruebas manuales

Verificación	Estado
Happy path completado sin errores	☐
Casos límite verificados manualmente	☐
Layout responsive en móvil, tablet y desktop	☐
Formularios validan y muestran errores	☐
Navegación entre páginas funciona	☐

4. Compatibilidad entre navegadores

Verificación	Estado
Chrome: todo funciona	☐
Firefox: todo funciona	☐
Safari: todo funciona (si tienes Mac)	☐
Móvil: layout correcto, sin overflow	☐

5. Repositorio

Verificación	Estado
README actualizado con instrucciones claras	☐
<code>.gitignore</code> excluye <code>node_modules</code> , <code>.env</code> , build	☐
No hay secrets hardcodeados en el código	☐
Dependencias innecesarias eliminadas	☐
<code>package-lock.json</code> commiteado	☐

Cuándo usar el checklist



Momento	Qué comprobar
Antes de commit	Tests pasan, código sin errores
Antes de sprint	Tests + responsive + docs actualizadas
Antes de entrega final	Checklist completo: tests, manual, browsers, repo

Qué mira el tribunal

El tribunal tiene un criterio de evaluación específico para testing. Esto es lo que comprueban:

Criterio del tribunal	Qué esperan ver
Tests ejecutándose	<code>npm test</code> pasa sin errores
Datos de prueba	Datos de demo cargados, no vacíos
Cobertura visible	Informe de cobertura o tests documentados
Documentación	README explica cómo ejecutar tests
Funcionamiento	La app no crashea durante la demo

Errores comunes del tribunal

Error	Cómo evitarlo
“Los tests fallan en otra máquina”	Ejecuta tests en un directorio limpio antes de entregar
“No hay datos de demo”	Crea un script <code>seed.js</code> que cargue datos de ejemplo
“La app crashea al cargar”	Verifica la consola del navegador antes de presentar
“El README no explica nada”	Incluye secciones: instalación, tests, demo

Versión imprimible

CHECKLIST DE CALIDAD – ENTREGA FINAL

=====

Proyecto: _____ Sprint: _____

TESTS

- ☐ Todos los tests pasan (npm test)
- ☐ Casos límite testeados
- ☐ Cobertura $\geq 60\%$

RESPONSIVE

- ☐ Móvil correcto
- ☐ Tablet correcto
- ☐ Desktop correcto

NAVEGACIÓN

- ☐ Login funciona
- ☐ CRUD principal funciona
- ☐ Formularios validan
- ☐ Errores se muestran

REPOSITORIO

- ☐ README actualizado
- ☐ No hay secrets
- ☐ .gitignore correcto
- ☐ Dependencias limpias

TRIBUNAL

- ☐ Datos de demo cargados
- ☐ App no crashea
- ☐ Plan B preparado

Firma: _____

► 💡 ¿Necesito comprobar todo esto cada vez?

Mini-chequeo

► ¿Cuándo usar el checklist de calidad?

► ¿Qué comprueba el tribunal sobre testing?

► ¿Qué hacer si el checklist es demasiado largo?

✓ Resumen en 3 frases

1. El checklist comprueba **tests, responsive, navegación, repositorio y preparación para el tribunal** antes de cada entrega.
2. Se usa en **tres momentos**: antes de commit (básico), antes de sprint (intermedio) y antes de entrega final (completo).
3. El tribunal comprueba que **los tests pasan, hay datos de demo y la app no crashea**: prepárate para eso y tendrás un plan B.

 Volver al [índice de la unidad](#) · Anterior: [07 — Testing en equipo](#) · Siguiente: [09 — Cierre de la unidad](#)

09 — Cierre de la unidad

 **Estás en:**  **UD 2.4 · Testing** → 09 · Cierre de la unidad

 *Hora de poner en práctica todo lo que has aprendido*

Has recorrido toda la unidad: desde por qué hacer tests hasta las herramientas y el testing en equipo. Ahora toca **verificar que lo entiendes** con ejercicios interactivos que te harán pensar, no solo recordar. No hay código: hay lógica.

La idea en una frase

Testing no es escribir tests: es pensar en cómo puede fallar tu código antes de que lo haga.

Ejercicio 1 — “Sé el Testing”

Tres dilemas de equipo sobre prioridades de testing. Piensa qué harías tú:

► Dilema 1: Solo tienes tiempo para testear una cosa

► Dilema 2: Un test falla “a veces”

► Dilema 3: Tu compañero no escribe tests

Ejercicio 2 — “Dilema de equipo”

Dos enfoques de testing en conflicto:

Enfoque A — Test Driven Development

Escribo el test ANTES del código. Más lento al principio, pero detecto bugs antes y el código final es más limpio. Al final del sprint, ahorro tiempo en debugging.

Enfoque B — Testing después

Primero hago que funcione, luego testeo. Más rápido al principio, pero al final del sprint descubro bugs que cuestan más de arreglar. A veces no testeo por falta de tiempo.

► ¿Cuál es mejor para un proyecto DAW?

Ejercicio 3 — “¿Quién Soy?”

Cuatro adivinanzas de conceptos de testing. Intenta adivinar antes de abrir:

► Adivinanza 1

► Adivinanza 2

► Adivinanza 3

► Adivinanza 4

Ejercicio 4 — “Laboratorio de Tortura”

Encuentra **3 o más bugs** en esta función de validación de fechas de reserva:

```
function validarFechaReserva(fecha, horaInicio, horaFin) {  
  const fechaActual = new Date()  
  const fechaReserva = new Date(fecha)  
  
  if (fechaReserva < fechaActual) {  
    return { valido: false, error: 'Fecha en el pasado' }  
  }  
  
  if (horaInicio >= horaFin) {  
    return { valido: false, error: 'Hora inicio debe ser antes que fin' }  
  }  
  
  return { valido: true }  
}
```

► Pista 1: Compara fechas con horas

► Pista 2: ¿Qué pasa con string vacío o null?

► Pista 3: ¿Y las horas inválidas?

► Pista 4: ¿Reservas en otro huso horario?

► Bugs encontrados (resumen)

Ejercicio 5 — “Atrévete a Pensar”

Tres preguntas abiertas sobre filosofía del testing:

► **Pregunta 1: ¿Puede un proyecto tener 100% de cobertura y seguir teniendo bugs?**

► **Pregunta 2: ¿Qué es más costoso: testear o arreglar bugs en producción?**

► **Pregunta 3: ¿Los tests documentan el código?**

Ejercicio 6 — “Post-Créditos”

► **¿Qué viene después de Testing?**

Resumen en 3 frases

1. **Testing es pensar en fallos:** antes de escribir código, piensa en cómo puede fallar y diseña tests que lo detecten.
2. **El testing es responsabilidad del equipo:** acuerdos claros, code review con foco en testing y pair testing mejoran la calidad.
3. **El checklist es tu red de seguridad:** úsalo antes de cada entrega para no olvidar nada crítico.

 [Volver al índice de la unidad](#) · Anterior: [o8 — Checklist de calidad](#)

📖 **Estás en:** 🏠 **UD 2.5 · Patrones de Diseño** → 01 · ¿Qué son los patrones de diseño?

📖 *Los patrones son recetas de cocina para programadores*

Los patrones de diseño no son código copiado. Son **soluciones probadas** a problemas que aparecen una y otra vez. Es como si alguien ya hubiera resuelto tu problema 1000 veces y te dejara la receta.

📖 **La idea en una frase**

Un patrón de diseño es una solución reutilizable a un problema de diseño que aparece frecuentemente en distintos contextos.

No reinventes la rueda. Usa la que ya funciona.

¿Por qué existen los patrones?

Porque los programadores se enfrentan a los **mismos problemas** una y otra vez:

Problema	Patrón que lo resuelve
“Solo quiero UNA instancia de configuración”	Singleton
“Necesito crear objetos sin acoplar el código”	Factory
“Quiero construir objetos complejos paso a paso”	Builder
“Necesito adaptar una interfaz incompatible”	Adapter
“Quiero simplificar una interfaz compleja”	Facade
“Necesito notificar cambios a múltiples objetos”	Observer

Los 3 grupos de patrones

Creacionales (cómo se crean los objetos)

Se enfocan en **cómo instanciar** objetos de forma flexible y controlada.

- **Singleton**: una sola instancia global
- **Factory**: crear objetos sin especificar la clase exacta
- **Builder**: construir objetos complejos paso a paso

Estructurales (cómo se combinan los objetos)

Se enfocan en **cómo se componen** clases y objetos para formar estructuras más grandes.

- **Adapter**: convertir una interfaz en otra
- **Facade**: simplificar una interfaz compleja
- **Composite**: tratar objetos individuales y compuestos igual

Comportamiento (cómo interactúan los objetos)

Se enfocan en **cómo se comunican** y distribuyen responsabilidades entre objetos.

- **Observer**: notificar cambios a múltiples objetos
- **Strategy**: intercambiar algoritmos en tiempo de ejecución
- **Command**: encapsular acciones como objetos

Comparativa: código sin patrón vs con patrón

Sin patrón

En 5 archivos diferentes tengo: 'new Logger()', 'new Logger()', 'new Logger()'. Cada uno crea su propia instancia. Los logs se pierden.

Con patrón (Singleton)

Logger.getInstance().log('mensaje')'. Todos usan la misma instancia. Los logs están centralizados. Un solo punto de control.

Aclaración: “¿Necesito memorizar todos los patrones?”

▶ 🤔 ¿Tengo que memorizar los 23 patrones de GoF?

Mini-chequeo

▶ ¿Qué son los patrones de diseño?

▶ ¿Cuáles son los 3 grupos?

▶ ¿Cuántos patrones debo conocer?

✅ Resumen en 3 frases

1. Los patrones de diseño son **soluciones probadas** a problemas que aparecen una y otra vez en el desarrollo de software.
2. Se agrupan en **3 grupos**: creacionales (cómo se crean), estructurales (cómo se combinan) y comportamiento (cómo interactúan).
3. No necesitas memorizar los 23 patrones: **entiende el problema que resuelven** y úsalos cuando lo necesites.

02 — Patrones creacionales

 Estás en:  UD 2.5 · Patrones de Diseño → 02 · Patrones creacionales

 *Crear objetos es fácil. Crearlos bien es otro cuento*

Los patrones creacionales controlan **cómo se crean** los objetos. En vez de hacer `new Clase()` por todas partes (acoplamiento total), usas patrones que te dan flexibilidad, control y código más limpio.

La idea en una frase

Los patrones creacionales te dan control sobre la creación de objetos: Singleton para una sola instancia, Factory para crear diferentes tipos, Builder para construir paso a paso.

No crees objetos a lo loco. Crea con estrategia.

Singleton: una sola instancia

El problema

Necesitas que solo **exista UNA instancia** de una clase (configuración, logger, conexión a base de datos).

La solución

```
// Singleton en JavaScript
class Config {
  constructor() {
    if (Config.instance) {
      return Config.instance;
    }
    this.settings = {};
    Config.instance = this;
  }

  set(key, value) {
    this.settings[key] = value;
  }

  get(key) {
    return this.settings[key];
  }
}

// Uso
const config1 = new Config();
const config2 = new Config();

config1.set('theme', 'dark');
console.log(config2.get('theme')); // 'dark' (misma instancia)
console.log(config1 === config2); // true
```

Cuándo usarlo

Caso	Ejemplo
Configuración global	<code>Config.getInstance()</code>
Logger	<code>Logger.getInstance()</code>
Cache	<code>Cache.getInstance()</code>

Cuándo NO usarlo

- Cuando no necesitas una sola instancia
- Cuando dificulta los tests (estado global)

Factory: crear objetos sin especificar la clase

El problema

Necesitas crear diferentes tipos de objetos según la entrada, pero no quieres hacer `if/else` por todas partes.

La solución

```
// Factory en JavaScript
class NotificationFactory {
  static create(type, message) {
    switch (type) {
      case 'success':
        return new SuccessNotification(message);
      case 'error':
        return new ErrorNotification(message);
      case 'warning':
        return new WarningNotification(message);
      default:
        return new Notification(message);
    }
  }
}

class Notification {
  constructor(message) {
    this.message = message;
  }
  render() {
    return `
    ${this.message} → `;
  }
}

class SuccessNotification extends Notification {
  render() {
    return `
    ${this.message} → `;
  }
}

class ErrorNotification extends Notification {
  render() {
    return `
    ${this.message} → `;
  }
}

// Uso
const notif = NotificationFactory.create('success', 'Guardado correctamente');
notif.render(); //
Guardado correctamente →
```

Cuándo usarlo

Caso	Ejemplo
Diferentes tipos de notificaciones	Success, Error, Warning
Diferentes formatos de export	PDF, CSV, JSON
Diferentes tipos de usuario	Admin, Profesor, Alumno

Builder: construir objetos paso a paso

El problema

Un objeto tiene **muchos parámetros opcionales** y el constructor se vuelve ilegible.

La solución

```
// Builder en JavaScript
class QueryBuilder {
  constructor() {
    this.table = '';
    this.conditions = [];
    this.orderBy = '';
    this.limit = null;
  }

  from(table) {
    this.table = table;
    return this; // permite encadenar
  }

  where(condition) {
    this.conditions.push(condition);
    return this;
  }

  order(field) {
    this.orderBy = field;
    return this;
  }

  take(limit) {
    this.limit = limit;
    return this;
  }

  build() {
    let query = `SELECT * FROM ${this.table}`;
    if (this.conditions.length) {
      query += ` WHERE ${this.conditions.join(' AND ')} `;
    }
    if (this.orderBy) {
      query += ` ORDER BY ${this.orderBy}`;
    }
    if (this.limit) {
      query += ` LIMIT ${this.limit}`;
    }
    return query;
  }
}

// Uso (encadenable)
const query = new QueryBuilder()
  .from('users')
  .where('age > 18')
  .where('active = true')
  .order('name')
  .take(10)
  .build();

// "SELECT * FROM users WHERE age > 18 AND active = true ORDER BY name LIMIT 10"
```

Cuándo usarlo

Caso	Ejemplo
Queries SQL	Construir consultas complejas
Configuración compleja	Objetos con muchos parámetros opcionales
Objetos inmutables	Crear y no modificar después

Comparativa: los 3 creacionales

Singleton

Solo UNA instancia. Todos acceden al mismo objeto. Útil para configuración global, logs o cache.

Factory

Creo diferentes tipos de objetos según la entrada. Sin if/else por todas partes. Flexible y extensible.

Builder

Construyo un objeto complejo paso a paso, encadenando métodos. El código queda limpio y legible.

Patrón	Problema	Solución
Singleton	“Necesito una sola instancia global”	Controlar la creación
Factory	“Necesito crear diferentes tipos”	Un factory que elige la clase
Builder	“El constructor tiene demasiados parámetros”	Construir paso a paso

Aclaración: “¿Cuándo uso Factory y cuándo Builder?”

▶ 🤔 ¿No hacen lo mismo Factory y Builder?

Mini-chequeo

▶ ¿Qué hace Singleton?

▶ ¿Qué hace Factory?

▶ ¿Qué hace Builder?

✓ Resumen en 3 frases

1. **Singleton** asegura una sola instancia global: úsalo para configuración, logger o cache.
2. **Factory** crea diferentes tipos de objetos según la entrada: úsalo para notificaciones, exportaciones o tipos de usuario.
3. **Builder** construye objetos complejos paso a paso con encadenamiento: úsalo para queries, configuraciones o objetos con muchos parámetros opcionales.

 Volver al [índice de la unidad](#) · Anterior: [01 — ¿Qué son los patrones?](#) · Siguiente: [03 — Patrones estructurales](#)

03 — Patrones estructurales

📖 *Los patrones estructurales organizan la arquitectura de tu código*

Los patrones estructurales se enfocan en **cómo se componen** las clases y objetos para formar estructuras más grandes. Son el pegamento que conecta las piezas de tu aplicación de forma limpia y flexible.

📖 La idea en una frase

Los patrones estructurales adaptan, simplifican o componen interfaces: Adapter convierte una interfaz en otra, Facade simplifica una compleja, Composite trata individuos y compuestos igual.

Organiza tus piezas para que encajen bien.

Adapter: convertir una interfaz en otra

El problema

Tienes una clase que no tiene la interfaz que necesitas. No puedes modificarla (es de terceros o muy estable).

La solución

```
// Adapter en JavaScript

// Interfaz antigua (API vieja)
class OldWeatherAPI {
  获取Weather(city) {
    return { temp: 25, condicion: 'soleado' };
  }
}

// Interfaz que necesitamos (API nueva)
class WeatherAdapter {
  constructor(oldAPI) {
    this.oldAPI = oldAPI;
  }

  getTemperature(city) {
    const data = this.oldAPI.获取Weather(city);
    return data.temp;
  }

  getCondition(city) {
    const data = this.oldAPI.获取Weather(city);
    return data.condicion;
  }
}

// Uso
const oldAPI = new OldWeatherAPI();
const adapter = new WeatherAdapter(oldAPI);

adapter.getTemperature('Madrid'); // 25
adapter.getCondition('Madrid');   // 'soleado'
```

Cuándo usarlo

Caso	Ejemplo
API de terceros cambia	Adaptar la interfaz vieja a la nueva
Datos en formato diferente	Convertir JSON de un formato a otro
Compatibilidad con versiones	Mantener código viejo funcionando

Facade: simplificar una interfaz compleja

El problema

Tienes múltiples clases con interfaces complejas. El cliente no necesita知道 todo eso.

La solución


```
// Facade en JavaScript

// Clases complejas (subsistemas)
class Auth {
  login(user, pass) { /* ... */ }
  logout() { /* ... */ }
  checkSession() { /* ... */ }
}

class Database {
  connect() { /* ... */ }
  query(sql) { /* ... */ }
  disconnect() { /* ... */ }
}

class Cache {
  get(key) { /* ... */ }
  set(key, value) { /* ... */ }
  clear() { /* ... */ }
}

// Facade: simplifica todo
class AppFacade {
  constructor() {
    this.auth = new Auth();
    this.db = new Database();
    this.cache = new Cache();
  }

  // Un solo método que coordina todo
  async getUserProfile(userId) {
    // 1. Ver cache
    const cached = this.cache.get(`user_${userId}`);
    if (cached) return cached;

    // 2. Si no hay cache, consultar BD
    this.db.connect();
    const user = this.db.query(`SELECT * FROM users WHERE id = ${userId}`);
    this.db.disconnect();

    // 3. Guardar en cache
    this.cache.set(`user_${userId}`, user);

    return user;
  }
}

// Uso: una sola llamada
const app = new AppFacade();
const profile = await app.getUserProfile(123);
```

Cuándo usarlo

Caso	Ejemplo
Simplificar API compleja	Un facade que coordina auth + DB + cache
Ocultar complejidad interna	El cliente no necesita知道 los subsistemas
Punto de entrada único	Una sola interfaz para todo
509 / 743	

Composite: tratar individuos y compuestos igual

El problema

Tienes objetos individuales y grupos de objetos, y quieres tratarlos de la misma manera.

La solución

```

// Composite en JavaScript

// Componente base
class Task {
  constructor(name, hours) {
    this.name = name;
    this.hours = hours;
  }

  getHours() {
    return this.hours;
  }

  describe() {
    return `${this.name}: ${this.hours}h`;
  }
}

// Composite: grupo de tareas
class TaskGroup {
  constructor(name) {
    this.name = name;
    this.tasks = [];
  }

  add(task) {
    this.tasks.push(task);
    return this;
  }

  getHours() {
    return this.tasks.reduce((sum, task) => sum + task.getHours(), 0);
  }

  describe() {
    return `${this.name} (${this.getHours()}h total):\n` +
      this.tasks.map(t => '  - ' + t.describe()).join('\n');
  }
}

// Uso
const login = new Task('Login', 8);
const crud = new Task('CRUD tareas', 16);
const tests = new Task('Tests', 8);

const sprint1 = new TaskGroup('Sprint 1')
  .add(login)
  .add(crud);

const proyecto = new TaskGroup('Proyecto')
  .add(sprint1)
  .add(tests);

proyecto.getHours(); // 32
proyecto.describe(); // "Proyecto (32h total):\n  - Sprint 1 (24h total):\n    - Log:

```

Cuándo usarlo

Caso	Ejemplo
Árboles de archivos	Carpetas y archivos tratados igual
Menús jerárquicos	Menús con submenús
Organización de tareas	Sprints con subtareas

Comparativa: los 3 estructurales

Adapter

Tengo una interfaz incompatible. La adapto para que funcione con mi código. No cambio la original, añado un adaptador.

Facade

Tengo 5 clases complejas. Creo un facade que las coordina. El cliente llama a un solo método y todo funciona.

Composite

Tengo objetos individuales y grupos. Los trato igual: cada uno tiene getHours(). El grupo suma los de sus hijos.

Patrón	Problema	Solución
Adapter	Interfaz incompatible	Añadir una capa de traducción
Facade	Complejidad excesiva	Un punto de entrada simplificado
Composite	Individuos + grupos	Tratar ambos de la misma manera

Aclaración: “¿Adapter y Facade no son lo mismo?”

▶ 🤔 ¿Cuál es la diferencia entre Adapter y Facade?

Mini-chequeo

▶ ¿Qué hace Adapter?

▶ ¿Qué hace Facade?

▶ ¿Qué hace Composite?

✅ Resumen en 3 frases

1. **Adapter** traduce una interfaz incompatible en otra que funciona: no cambia la original, añade una capa de traducción.
2. **Facade** simplifica una interfaz compleja con un punto de entrada único: el cliente llama a un método y el facade coordina todo.
3. **Composite** trata individuos y compuestos de la misma manera: cada uno tiene los mismos métodos, el grupo suma los de sus hijos.

 [Volver al índice de la unidad](#) · Anterior: [02 — Patrones creacionales](#) · Siguiente: [04 — Patrones de comportamiento](#)

📖 *Los patrones de comportamiento distribuyen el trabajo entre objetos*

Los patrones de comportamiento se enfocan en **cómo se comunican** los objetos y cómo se distribuyen las responsabilidades. Son los que hacen que tu código **reaccione** a cambios, **intercambie** algoritmos y **encapsule** acciones.

📖 La idea en una frase

Observer notifica cambios, Strategy intercambia algoritmos y Command encapsula acciones: los 3 hacen que tu código sea más flexible y desacoplado.

Cada uno resuelve un problema de comunicación diferente.

Observer: notificar cambios a múltiples objetos

El problema

Cuando algo cambia, necesitas **notificar** a múltiples objetos sin acoplarlos.

La solución

```
// Observer en JavaScript

class EventBus {
  constructor() {
    this.listeners = {};
  }

  on(event, callback) {
    if (!this.listeners[event]) {
      this.listeners[event] = [];
    }
    this.listeners[event].push(callback);
  }

  off(event, callback) {
    if (!this.listeners[event]) return;
    this.listeners[event] = this.listeners[event].filter(cb => cb !== callback);
  }

  emit(event, data) {
    if (!this.listeners[event]) return;
    this.listeners[event].forEach(cb => cb(data));
  }
}

// Uso
const bus = new EventBus();

// Observador 1: actualiza la UI
bus.on('taskCreated', (task) => {
  document.getElementById('task-list').innerHTML += `<li>${task.name}</li>`;
});

// Observador 2: guarda en localStorage
bus.on('taskCreated', (task) => {
  const tasks = JSON.parse(localStorage.getItem('tasks') || '[]');
  tasks.push(task);
  localStorage.setItem('tasks', JSON.stringify(tasks));
});

// Observador 3: muestra notificación
bus.on('taskCreated', (task) => {
  showNotification(`Tarea "${task.name}" creada`);
});

// Emitir evento (todos los observadores se activan)
bus.emit('taskCreated', { name: 'Nueva tarea' });
```

Cuándo usarlo

Caso	Ejemplo
Eventos de UI	Click, cambio de formulario
Notificaciones	Alertar a múltiples componentes
Sincronización	Actualizar varias vistas con los mismos datos

Strategy: intercambiar algoritmos en tiempo de ejecución

El problema

Tienes **múltiples algoritmos** para hacer lo mismo y quieres poder cambiar entre ellos.

La solución

```
// Strategy en JavaScript

// Estrategias de ordenación
const sortStrategies = {
  byName: (a, b) => a.name.localeCompare(b.name),
  byDate: (a, b) => new Date(a.date) - new Date(b.date),
  byPriority: (a, b) => a.priority - b.priority,
};

// Contexto que usa la estrategia
class TaskList {
  constructor(tasks) {
    this.tasks = tasks;
    this.sortStrategy = sortStrategies.byName; // por defecto
  }

  setSortStrategy(strategy) {
    this.sortStrategy = strategy;
  }

  sort() {
    return [...this.tasks].sort(this.sortStrategy);
  }
}

// Uso
const tasks = [
  { name: 'B', date: '2024-01-02', priority: 2 },
  { name: 'A', date: '2024-01-01', priority: 1 },
  { name: 'C', date: '2024-01-03', priority: 3 },
];

const list = new TaskList(tasks);

list.setSortStrategy(sortStrategies.byName);
list.sort(); // [{name:'A'}, {name:'B'}, {name:'C'}]

list.setSortStrategy(sortStrategies.byPriority);
list.sort(); // [{priority:1}, {priority:2}, {priority:3}]
```

Cuándo usarlo

Caso	Ejemplo
Filtros de búsqueda	Por nombre, fecha, categoría
Ordenación	Por fecha, nombre, prioridad
Validación	Reglas diferentes según el tipo

Command: encapsular acciones como objetos

El problema

Necesitas **desacoplar** quién ejecuta una acción de quién la solicita. Útil para deshacer/rehacer.

La solución

```

// Command en JavaScript

class Task {
  constructor(name) {
    this.name = name;
    this.done = false;
  }
}

// Comandos
class CreateTaskCommand {
  constructor(taskList, task) {
    this.taskList = taskList;
    this.task = task;
  }

  execute() {
    this.taskList.push(this.task);
  }

  undo() {
    const index = this.taskList.indexOf(this.task);
    if (index > -1) this.taskList.splice(index, 1);
  }
}

class CompleteTaskCommand {
  constructor(task) {
    this.task = task;
  }

  execute() {
    this.task.done = true;
  }

  undo() {
    this.task.done = false;
  }
}

// Historial de comandos
class CommandHistory {
  constructor() {
    this.history = [];
  }

  execute(command) {
    command.execute();
    this.history.push(command);
  }

  undo() {
    const command = this.history.pop();
    if (command) command.undo();
  }
}

// Uso
const tasks = [];
const history = new CommandHistory();

history.execute(new CreateTaskCommand(tasks, new Task('Login')));
history.execute(new CreateTaskCommand(tasks, new Task('Dashboard')));

tasks.length; // 2

```

```
history.undo();
tasks.length; // 1 (se eliminó 'Dashboard')
```

Cuándo usarlo

Caso	Ejemplo
Deshacer/rehacer	Editor de texto, dibujo
Cola de acciones	Procesamiento asíncrono
Transacciones	Guardar/deshacer cambios

Comparativa: los 3 de comportamiento

Observer

Cuando pasa algo, notifico a todos los que estén escuchando. No sé quiénes son, solo emito el evento.

Strategy

Tengo múltiples formas de hacer algo. Cambio la estrategia en tiempo de ejecución sin modificar el contexto.

Command

Encapsulo cada acción como un objeto. Puedo ejecutarla, deshacerla, guardarla en un historial.

Patrón	Problema	Solución
Observer	“Necesito notificar a múltiples objetos”	Emitir eventos, suscribirse
Strategy	“Necesito intercambiar algoritmos”	Injectar la estrategia
Command	“Necesito deshacer/rehacer acciones”	Encapsular como objetos

Aclaración: “¿Cuándo uso Observer y cuándo Strategy?”

► 🤔 ¿No son similares Observer y Strategy?

Mini-chequeo

► ¿Qué hace Observer?

► ¿Qué hace Strategy?

► ¿Qué hace Command?

✅ Resumen en 3 frases

1. **Observer** notifica cambios a múltiples objetos: emite eventos y los suscriptores reaccionan sin acoplarse al emisor.
2. **Strategy** intercambia algoritmos en tiempo de ejecución: cambias la estrategia (ordenación, filtrado) sin modificar el contexto.
3. **Command** encapsula acciones como objetos: cada acción tiene `execute()` y `undo()`, permitiendo deshacer y guardar historial.

05 — MVC (Model-View-Controller)

 Estás en:  UD 2.5 · Patrones de Diseño → 05 · MVC

 *MVC separa tu app en 3 capas claras: datos, interfaz y lógica*

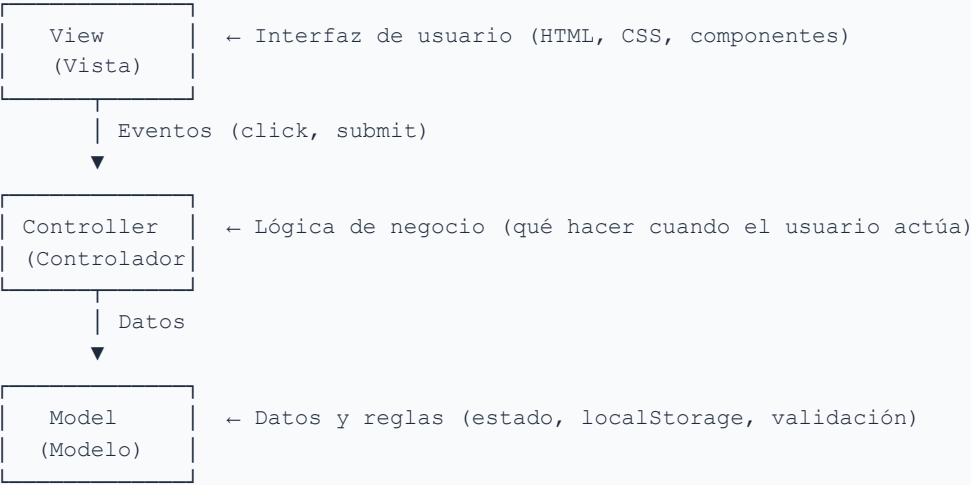
MVC (Model-View-Controller) es el patrón arquitectónico más usado en desarrollo web. Separa la aplicación en 3 capas: **Model** (datos), **View** (interfaz) y **Controller** (lógica). Cada capa tiene su responsabilidad y no se mezcla con las demás.

La idea en una frase

MVC separa tu app en 3 capas: Model gestiona los datos, View muestra la interfaz, Controller coordina entre ambos. Cada capa hace una cosa y la hace bien.

No mezcles lógica con interfaz. Separa y conquista.

Las 3 capas de MVC



Cada capa en detalle

Capa	Responsabilidad	Ejemplo
Model	Gestiona datos y estado	Lista de tareas, usuario actual, validación
View	Muestra la interfaz	Componente de lista, formulario, botones
Controller	Coordina Model y View	Maneja clicks, llama al Model, actualiza la View

Ejemplo: Lista de tareas con MVC

Model (datos)

```
// model/TaskModel.js
class TaskModel {
  constructor() {
    this.tasks = JSON.parse(localStorage.getItem('tasks') || '[]');
  }

  addTask(name) {
    const task = { id: Date.now(), name, done: false };
    this.tasks.push(task);
    this.save();
    return task;
  }

  toggleTask(id) {
    const task = this.tasks.find(t => t.id === id);
    if (task) {
      task.done = !task.done;
      this.save();
    }
  }

  deleteTask(id) {
    this.tasks = this.tasks.filter(t => t.id !== id);
    this.save();
  }

  save() {
    localStorage.setItem('tasks', JSON.stringify(this.tasks));
  }

  getAll() {
    return this.tasks;
  }
}
```

View (interfaz)

```

// view/TaskView.js
class TaskView {
  constructor(container) {
    this.container = container;
    this.onAdd = null;
    this.onToggle = null;
    this.onDelete = null;
  }

  render(tasks) {
    this.container.innerHTML = `

      <input type="text" id="task-input" placeholder="Nueva tarea">
      <button id="add-btn">Añadir</button>

      →
      <ul class="task-list">
        ${tasks.map(task => `
          <li class="${task.done ? 'done' : ''}">
            <input type="checkbox" ${task.done ? 'checked' : ''} data-id="${task.id}">
            <span>${task.name}</span>
            <button data-delete="${task.id}">x</button>
          </li>
        `).join('')}
      </ul>
    `;

    this.bindEvents();
  }

  bindEvents() {
    document.getElementById('add-btn').addEventListener('click', () => {
      const input = document.getElementById('task-input');
      if (input.value.trim()) {
        this.onAdd(input.value.trim());
        input.value = '';
      }
    });

    document.querySelectorAll('[data-id]').forEach(el => {
      el.addEventListener('change', () => {
        this.onToggle(parseInt(el.dataset.id));
      });
    });

    document.querySelectorAll('[data-delete]').forEach(el => {
      el.addEventListener('click', () => {
        this.onDelete(parseInt(el.dataset.delete));
      });
    });
  }
}

```

Controller (lógica)


```
// controller/TaskController.js
class TaskController {
  constructor(model, view) {
    this.model = model;
    this.view = view;

    // Conectar eventos de la View con el Model
    this.view.onAdd = (name) => this.addTask(name);
    this.view.onToggle = (id) => this.toggleTask(id);
    this.view.onDelete = (id) => this.deleteTask(id);

    // Render inicial
    this.render();
  }

  render() {
    this.view.render(this.model.getAll());
  }

  addTask(name) {
    this.model.addTask(name);
    this.render();
  }

  toggleTask(id) {
    this.model.toggleTask(id);
    this.render();
  }

  deleteTask(id) {
    this.model.deleteTask(id);
    this.render();
  }
}

// Uso
const model = new TaskModel();
const view = new TaskView(document.getElementById('app'));
const controller = new TaskController(model, view);
```

Comparativa: con MVC vs sin MVC

Sin MVC

Todo en un solo archivo: HTML + lógica + datos mezclados. Para cambiar el diseño, miedo a romper la lógica. Para cambiar la lógica, miedo a romper el diseño.

Con MVC

Model solo datos, View solo interfaz, Controller solo coordinación. Cambio el diseño sin tocar la lógica. Cambio la lógica sin tocar el diseño.

MVC en frameworks modernos

Framework	Model	View	Controller
Vue	Composition API / data	Template / SFC	Methods / computed
React	State / hooks	JSX components	Event handlers
Angular	Services	Templates	Components
Laravel	Eloquent models	Blade templates	Controllers

En todos los casos, la **idea** es la misma: separar datos, interfaz y lógica.

Aclaración: “¿Vue ya usa MVC?”

► 🤔 ¿Si uso Vue, ya estoy usando MVC?

Mini-chequeo

► ¿Qué son las 3 capas de MVC?

► ¿Por qué separar en capas?

► ¿Vue usa MVC?

✓ Resumen en 3 frases

1. **MVC** separa la app en 3 capas: **Model** (datos), **View** (interfaz), **Controller** (lógica de negocio).
2. La ventaja es la **separación de responsabilidades**: cambiar una capa no rompe las otras.
3. En frameworks como Vue, React o Angular, MVC se aplica de forma implícita: datos en el state, interfaz en el template, lógica en los methods.

 Volver al [índice de la unidad](#) · Anterior: [04 — Patrones de comportamiento](#) · Siguiente: [06 — Repository](#)

06 — Repository

 Estás en:  **UD 2.5 · Patrones de Diseño** → 06 · Repository

 *Repository: una capa entre tu lógica y los datos*

El patrón **Repository** separa el **acceso a datos** de la **lógica de negocio**. En vez de tener llamadas a localStorage, fetch o IndexedDB por todas partes, las centralizas en un Repository que expone una interfaz limpia.

La idea en una frase

Repository encapsula el acceso a datos; tu lógica de negocio no sabe si los

datos vienen de `localStorage`, una API o `IndexedDB`. Solo llama al **Repository**.

No mezcles lógica con almacenamiento.

¿Qué problema resuelve?

Sin Repository	Con Repository
<code>localStorage.getItem('tasks')</code> en 10 archivos	<code>TaskRepository.getAll()</code> en un solo lugar
Si cambias a <code>IndexedDB</code> , tocas 10 archivos	Solo cambias el Repository
La lógica depende del almacenamiento	La lógica es independiente del almacenamiento

Ejemplo: Repository para tareas

Repository base

```
// repositories/TaskRepository.js
class TaskRepository {
  constructor(storageKey = 'tasks') {
    this.storageKey = storageKey;
  }

  getAll() {
    return JSON.parse(localStorage.getItem(this.storageKey) || '[]');
  }

  getById(id) {
    const tasks = this.getAll();
    return tasks.find(t => t.id === id);
  }

  save(task) {
    const tasks = this.getAll();
    const index = tasks.findIndex(t => t.id === task.id);
    if (index > -1) {
      tasks[index] = task;
    } else {
      tasks.push(task);
    }
    localStorage.setItem(this.storageKey, JSON.stringify(tasks));
    return task;
  }

  delete(id) {
    const tasks = this.getAll().filter(t => t.id !== id);
    localStorage.setItem(this.storageKey, JSON.stringify(tasks));
  }

  count() {
    return this.getAll().length;
  }
}
```

Uso en la lógica de negocio

```
// En tu lógica (Model o Controller)
const taskRepo = new TaskRepository();

// Crear tarea
const newTask = { id: Date.now(), name: 'Nueva tarea', done: false };
taskRepo.save(newTask);

// Obtener todas
const allTasks = taskRepo.getAll();

// Obtener una
const task = taskRepo.getById(123);

// Eliminar
taskRepo.delete(123);
```

Si mañana cambias a IndexedDB

```
// Solo cambias el Repository, NO la lógica de negocio
class TaskRepositoryIndexedDB {
  constructor(dbName = 'TaskDB') {
    this.dbName = dbName;
  }

  async getAll() {
    // Implementación con IndexedDB
    // ...
  }

  async save(task) {
    // Implementación con IndexedDB
    // ...
  }
}

// La lógica sigue igual
const taskRepo = new TaskRepositoryIndexedDB();
const allTasks = await taskRepo.getAll(); // Misma interfaz
```

Comparativa: con Repository vs sin Repository

Sin Repository

En 5 archivos tengo 'localStorage.getItem'. El profesor dice 'usad IndexedDB'. Tengo que buscar y cambiar 5 archivos. Me equivoco en 2.

Con Repository

Tengo un TaskRepository. El profesor dice 'usad IndexedDB'. Cambio una clase. La lógica sigue igual. Un archivo, un cambio, cero errores.

Repository genérico reutilizable

```
// repositories/BaseRepository.js
class BaseRepository {
  constructor(storageKey) {
    this.storageKey = storageKey;
  }

  getAll() {
    return JSON.parse(localStorage.getItem(this.storageKey) || '[]');
  }

  getById(id) {
    return this.getAll().find(item => item.id === id);
  }

  save(item) {
    const items = this.getAll();
    const index = items.findIndex(i => i.id === item.id);
    if (index > -1) {
      items[index] = item;
    } else {
      items.push(item);
    }
    localStorage.setItem(this.storageKey, JSON.stringify(items));
    return item;
  }

  delete(id) {
    const items = this.getAll().filter(i => i.id !== id);
    localStorage.setItem(this.storageKey, JSON.stringify(items));
  }
}

// Repositories específicos que heredan
class TaskRepository extends BaseRepository {
  constructor() {
    super('tasks');
  }

  getPending() {
    return this.getAll().filter(t => !t.done);
  }

  getCompleted() {
    return this.getAll().filter(t => t.done);
  }
}

class UserRepository extends BaseRepository {
  constructor() {
    super('users');
  }

  getCurrentUser() {
    return this.getAll().find(u => u.isActive);
  }
}
```

► 🤔 ¿Merece la pena un Repository si solo uso localStorage?

Mini-chequeo

► ¿Qué hace Repository?

► ¿Por qué usarlo?

► ¿Necesito Repository en frontend-only?

✅ Resumen en 3 cambios

1. **Repository** encapsula el acceso a datos: tu lógica no sabe si los datos vienen de localStorage, IndexedDB o una API.
2. La ventaja principal es el **cambio futuro**: si cambias de almacenamiento, solo modificas el Repository, no toda la lógica.
3. Crea un **Repository por tipo de datos** (TaskRepository, UserRepository): mantiene el código organizado y testeable.

 Volver al [índice de la unidad](#) · Anterior: [05 — MVC](#) · Siguiente: [07 — Cuándo usar cada patrón](#)

07 — Cuándo usar cada patrón

📖 *No uses un patrón solo porque existe. Úsalo cuando lo necesites*

El peor error con los patrones es **usarlos donde no hace falta**. Un patrón se usa cuando hay un **problema concreto** que resuelve. Si no tienes el problema, no necesitas el patrón.

📖 La idea en una frase

Los patrones son herramientas: úsalos cuando el problema lo justifica. No añadas complejidad donde no la necesitas.

El mejor código es el que no tiene patrones innecesarios.

La tabla guía

Problema que tienes	Patrón	Ejemplo
“Solo quiero UNA instancia de configuración”	Singleton	Config, Logger
“Necesito crear diferentes tipos de objetos”	Factory	Notificaciones, Exportaciones
“Quiero construir objetos con muchos parámetros”	Builder	Queries, Configuraciones
“Tengo una interfaz incompatible”	Adapter	API vieja vs nueva
“Necesito simplificar una interfaz compleja”	Facade	Coordinar auth + DB + cache
“Quiero tratar individuos y grupos igual”	Composite	Menús, Tareas con subtarear
“Necesito notificar cambios a múltiples objetos”	Observer	Eventos, Notificaciones
“Quiero intercambiar algoritmos”	Strategy	Filtros, Ordenación
“Necesito deshacer/rehacer acciones”	Command	Editor, Historial
“Quiero separar datos de interfaz”	MVC	Cualquier app web
“Quiero separar lógica de almacenamiento”	Repository	Acceso a datos

Flujo de decisión

¿Necesitas UNA instancia global?
→ Sí: Singleton
→ No ↓

¿Creas diferentes tipos de objetos?
→ Sí: Factory
→ No ↓

¿El objeto tiene muchos parámetros opcionales?
→ Sí: Builder
→ No ↓

¿Necesitas notificar cambios?
→ Sí: Observer
→ No ↓

¿Necesitas intercambiar algoritmos?
→ Sí: Strategy
→ No ↓

¿Necesitas deshacer acciones?
→ Sí: Command
→ No ↓

¿Tienes múltiples subsistemas que coordinar?
→ Sí: Facade
→ No ↓

¿Tu interfaz no es compatible con algo?
→ Sí: Adapter
→ No ↓

No necesitas un patrón. Código simple.

Ejemplo de decisión real

Problema: “Necesito filtrar tareas por nombre, fecha y categoría”

Opción	Patrón	¿Por qué?
<pre>if (filtro === 'nombre') ... else if ...</pre>	Ninguno	Si son solo 3 opciones, un if/else basta
<pre>list.setFilter(new NameFilter())</pre>	Strategy	Si quieres intercambiar filtros sin modificar el código

Decisión: Si son 3 filtros fijos, no necesitas Strategy. Si van a crecer o quieres flexibilidad, Strategy.

Comparativa: patrón innecesario vs patrón necesario

Patrón innecesario

Usé Factory para crear un objeto que solo tiene 2 tipos. El código tiene 200 líneas más de lo necesario. Nadie lo entiende. Nadie lo necesita.

Patrón necesario

Usé Observer para notificar a 3 componentes cuando cambia el carrito. Sin Observer, tendría 3 llamadas directas acopladas. Con Observer, es limpio y extensible.

Aclaración: “¿Cuándo NO usar un patrón?”

► 🤔 ¿Hay situaciones donde NO debo usar ningún patrón?

Mini-chequeo

► ¿Cuándo usar Singleton?

► ¿Cuándo usar Strategy?

► ¿Cuándo NO usar patrones?

✅ Resumen en 3 frases

1. Usa **Singleton** para instancias globales, **Factory** para crear diferentes tipos, **Builder** para objetos complejos paso a paso.

2. Usa **Observer** para notificar cambios, **Strategy** para intercambiar algoritmos, **Command** para deshacer acciones.
3. **No uses patrones** donde no los necesitas: si tu código es simple y funciona, no le añadas complejidad innecesaria.

 Volver al [índice de la unidad](#) · Anterior: [o6 — Repository](#) · Siguiendo: [o8 — Template](#)
[patrones](#)

o8 — Template de selección de patrones

 **Estás en:**  **UD 2.5 · Patrones de Diseño** → [o8 · Template de selección de patrones](#)

 *Documenta tus decisiones de diseño para que el equipo las entienda*

Este template te ayuda a documentar **qué patrones usas** en tu proyecto, **por qué** los elegiste y **dónde** los aplicas. Es la documentación de arquitectura que el profesor espera ver y que tu equipo necesita.

La idea en una frase

Copia este template como `PATRONES.md` en tu repositorio y documenta qué patrones usas, por qué y dónde.

Sin documentación de diseño, nadie entiende tus decisiones.

Template completo

```
# Patrones de Diseño — [Nombre del proyecto]

## Decisiones de arquitectura

### Patrón principal: MVC

| Capa | Responsabilidad | Archivos |
|-----|-----|-----|
| **Model** | Datos y estado | `src/models/`, `src/stores/` |
| **View** | Interfaz de usuario | `src/components/`, `src/views/` |
| **Controller** | Lógica de negocio | `src/controllers/`, `src/services/` |

**Por qué MVC**: Separa datos de interfaz. Permite cambiar el diseño sin romper la lógica.

### Acceso a datos: Repository

| Repository | Tipo de datos | Almacenamiento |
|-----|-----|-----|
| `TaskRepository` | Tareas | localStorage |
| `UserRepository` | Usuarios | localStorage |

**Por qué Repository**: Si mañana cambiamos a IndexedDB o una API, solo cambiamos el Repository.

## Patrones específicos

| Patrón | Problema que resuelve | Dónde se aplica | Archivos |
|-----|-----|-----|-----|
| **Singleton** | Configuración global | `src/config/Config.js` | 1 archivo |
| **Observer** | Notificaciones de eventos | `src/events/EventBus.js` | 1 archivo |
| **Strategy** | Filtros de búsqueda | `src/strategies/FilterStrategy.js` | 3-4 estrategias |

### Singleton — Configuración

```javascript
// src/config/Config.js
class Config {
 // ... implementación Singleton
}
```

**Por qué:** Solo necesitamos una instancia de configuración global.

## Observer — EventBus

```
// src/events/EventBus.js
class EventBus {
 // ... implementación Observer
}
```

**Por qué:** Necesitamos notificar a múltiples componentes cuando cambian los datos.

## Strategy — Filtros

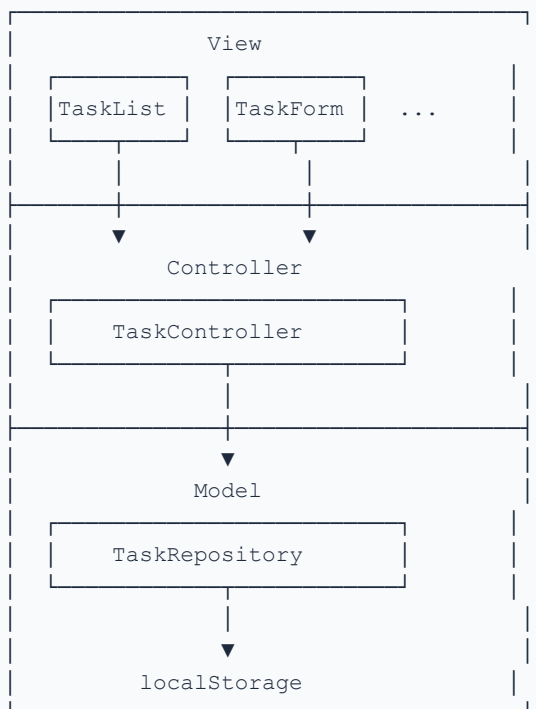
```
// src/strategies/FilterStrategy.js
const filterStrategies = {
 byName: (items, query) => items.filter(i => i.name.includes(query)),
 byDate: (items, query) => items.filter(i => i.date === query),
 byCategory: (items, query) => items.filter(i => i.category === query),
};
```

**Por qué:** Tenemos múltiples filtros que el usuario puede cambiar en tiempo de ejecución.

## Patrones que NO usamos (y por qué)

Patrón	Por qué no	Alternativa
<b>Factory</b>	Solo tenemos 1 tipo de cada cosa	<code>new Clase()</code> directo
<b>Builder</b>	Los constructores son simples	Constructores normales
<b>Adapter</b>	No hay APIs incompatibles	No hace falta
<b>Facade</b>	La app es pequeña	Llamadas directas
<b>Command</b>	No necesitamos deshacer	Funciones normales

## Diagrama de componentes



---

## Aclaración: "¿Necesito documentar todos los patrones?"

<details>

<summary>🤔 ¿No es excesivo documentar cada patrón que uso?</summary>

No. Documenta **las decisiones importantes**:

Documenta	No documentes
MVC (arquitectura principal)	`if/else` statements
Repository (acceso a datos)	Funciones auxiliares simples
Patrones que el equipo no conoce	Cosas obvias
Por qué elegiste un patrón sobre otro	Código que se explica solo

**Mínimo**: la tabla de patrones que usas + por qué los elegiste. Son 20 minutos de docu

</details>

---

## Mini-chequeo

<details>

<summary>¿Qué documentar en PATRONES.md?</summary>

Las **decisiones de arquitectura**: MVC, Repository, patrones específicos que usas, por  
</details>

<details>

<summary>¿Por qué documentar patrones que no uso?</summary>

Para que el equipo sepa **por qué no los elegiste**. Si alguien pregunta "¿por qué no F  
</details>

<details>

<summary>¿Cuánto tiempo toma documentar?</summary>

**20-30 minutos** para tener el template completo. Es una inversión que ahorra confusión  
</details>

---

## ✅ Resumen en 3 frases

1. Documenta **qué patrones usas**, **por qué** los elegiste y **dónde** los aplicas en
2. Incluye también **qué patrones NO usas** y por qué: si alguien pregunta, tienes la re
3. Son **20-30 minutos** de documentación que ahorran horas de confusión y demuestran co

---

> 🗺️ Volver al [índice de la unidad](../u2-5-patrones-diseno) · Anterior: [07 – Cuándo

---

## 09 – Cierre

> 🗺️ **Estás en:** 📖 **UD 2.5 · Patrones de Diseño** → 09 · Cierre

> 📖 Los patrones son herramientas, no religión\_

>

> ~~Has recorrido toda la unidad: desde qué son los patrones hasta cuándo~~

> usar cada uno, pasando por creacionales, <sup>540/743</sup>estructurales, comportamiento,

> MVC y Repository. Ahora toca **verificar que lo entiendes** y



```

> **aplicarlo a tu proyecto**.

🚩 La idea en una frase

> **Los patrones resuelven problemas. Si no tienes el problema, no necesitas el patrón.

Usa patrones cuando los necesites, no para impresionar.

🎯 Mini-chequeo final

Preguntas de comprensión

<details>
<summary>¿Cuáles son los 3 grupos de patrones?</summary>

Creacionales (cómo se crean: Singleton, Factory, Builder), **Estructurales** (cómo se
</details>

<details>
<summary>¿Qué hace Singleton?</summary>

Asegura que solo **exista una instancia** de una clase. Útil para configuración global,
</details>

<details>
<summary>¿Qué diferencia hay entre Factory y Builder?</summary>

Factory crea diferentes tipos de objetos (elige entre tipos). **Builder** construye
</details>

<details>
<summary>¿Qué hace MVC?</summary>

Separa la app en 3 capas: **Model** (datos), **View** (interfaz), **Controller** (lógica)
</details>

<details>
<summary>¿Qué es Repository?</summary>

Encapsula el acceso a datos: tu lógica no sabe si los datos vienen de localStorage,
</details>

<details>
<summary>¿Cuándo NO usar patrones?</summary>

Cuando el código es **simple**, no hay problema que resolver, o el equipo no lo entiende
</details>

Ejercicio práctico

En 2 horas, aplica patrones a tu proyecto:

1. **Identifica** (20 min): ¿Qué patrones necesitas? Usa la tabla guía del punto 07
2. **MVC** (30 min): Separa tu código en Model, View y Controller (o verifica que tu fr
3. **Repository** (30 min): Crea un Repository para tus datos principales
4. **Un patrón específico** (20 min): Implementa Observer, Strategy o el que necesites
5. **Documenta** (20 min): Rellena `PATRONES.md` con tus decisiones

**Si consigues tener MVC + Repository + al menos 1 patrón específico documentado → has c

📊 Tabla resumen de la unidad

```

```

|-----|-----|
| **Patrones** | Soluciones reutilizables a problemas comunes de diseño |
| **Singleton** | Una sola instancia global (Config, Logger) |
| **Factory** | Crear diferentes tipos de objetos |
| **Builder** | Construir objetos complejos paso a paso |
| **Adapter** | Convertir una interfaz incompatible en otra |
| **Facade** | Simplificar una interfaz compleja |
| **Composite** | Tratar individuos y compuestos igual |
| **Observer** | Notificar cambios a múltiples objetos |
| **Strategy** | Intercambiar algoritmos en tiempo de ejecución |
| **Command** | Encapsular acciones con execute() y undo() |
| **MVC** | Model-View-Controller: datos, interfaz, lógica |
| **Repository** | Separar acceso a datos de la lógica de negocio |

📖 Vocabulario rápido

| Término | Definición |
|-----|-----|
| **Patrón de diseño** | Solución reutilizable a un problema de diseño frecuente |
| **Creacional** | Patrones que controlan la creación de objetos |
| **Estructural** | Patrones que combinan clases y objetos |
| **Comportamiento** | Patrones que distribuyen responsabilidades |
| **Singleton** | Asegura una sola instancia de una clase |
| **Factory** | Crea objetos de diferentes tipos según la entrada |
| **Builder** | Construye objetos complejos paso a paso |
| **Adapter** | Convierte una interfaz incompatible en otra |
| **Facade** | Simplifica una interfaz compleja con un punto de entrada |
| **Composite** | Trata objetos individuales y compuestos igual |
| **Observer** | Notifica cambios a múltiples suscriptores |
| **Strategy** | Intercambia algoritmos en tiempo de ejecución |
| **Command** | Encapsula acciones como objetos con undo |
| **MVC** | Model-View-Controller: arquitectura de 3 capas |
| **Repository** | Capa de abstracción para acceso a datos |

🔗 Conexión con las siguientes unidades

| Unidad | Qué aprenderás | Cómo se conecta |
|-----|-----|-----|
| **UD 2.5 Diagramas** | C4, UML, secuencia | Modelarás la arquitectura que diseñaste co |
| **UD 2.6 IA como Copiloto** | LLMs, prompt engineering | Usarás IA para implementar pa |
| **UD 2.7 Despliegue** | Docker, CI/CD | Desplegarás la arquitectura que diseñaste |

✅ Resumen en 3 frases

1. Los patrones son **soluciones reutilizables**: Singleton (una instancia), Factory (creación)
2. **MVC** separa datos, interfaz y lógica; **Repository** separa lógica de almacenamiento
3. **No uses patrones donde no los necesitas**: si tu código es simple y funciona, no lo compliques

> 🗺️ Volver al [índice de la unidad](../u2-5-patrones-diseno) · Anterior: [08 – Templating]

01 – ¿Qué son los diagramas?

> 📚 **Estás en:** 📄 **UD 2.6 · Diagramas** → 01 · ¿Qué son los diagramas?

> _📖 Un diagrama comunica en 30 segundos lo que un texto tarda 10 minutos_
>
> Los diagramas son **representaciones visuales** de tu sistema: muestran
> cómo se estructura, cómo fluyen los datos y cómo interactúan los
> componentes. No son dibujos bonitos: son herramientas de comunicación

```

```

> que todo el equipo entiende.

🗨️ La idea en una frase

> **Un diagrama es una imagen que explica cómo funciona tu sistema: quién habla con quién. Si no puedes explicar tu sistema en un diagrama, no lo entiendes bien.

¿Qué tipo de diagramas existen?

| Tipo | Qué muestra | Cuándo usarlo |
|-----|-----|-----|
| **C4** | Arquitectura en 4 niveles | Para documentar la estructura general |
| **Entidad-Relación (ER)** | Datos y sus relaciones | Para modelar la base de datos |
| **Secuencia** | Interacción entre objetos | Para explicar un flujo concreto |
| **Componentes** | Partes del sistema | Para ver qué módulos existen |
| **Flujo (Flowchart)** | Proceso paso a paso | Para explicar un algoritmo |
| **Estado** | Ciclo de vida de un objeto | Para ver transiciones de estado |

¿Por qué son importantes?

Sin diagramas

- Cada persona imagina la arquitectura diferente
- Explicar un flujo toma 10 minutos de texto
- El profesor no entiende cómo se estructura tu app
- Nuevo miembro del equipo no sabe por dónde empezar

Con diagramas

- Todos ven la misma imagen
- Un flujo se entiende en 30 segundos
- El profesor ve arquitectura profesional
- Nuevo miembro entiende el sistema en minutos

Comparativa: sin diagramas vs con diagramas

> **Sin diagramas**
>
> El profesor: '¿Cómo funciona el login?' Yo: 'Pues mira, el usuario pone el email, luego...'

> **Con diagramas**
>
> El profesor: '¿Cómo funciona el login?' Yo: 'Mira este diagrama de secuencia'. Ve la imagen...

Aclaración: "¿No son los diagramas para empresas grandes?"

<details>
<summary>🤔 ¿Necesito diagramas para un proyecto de módulo?</summary>

Sí, pero ligeros. No necesitas diagramas de 50 componentes. Necesitas:

1. 1 diagrama C4 nivel 1 (contexto): qué hace tu app y quién la usa
2. 1 diagrama ER: tus tablas principales y sus relaciones
3. 2-3 diagramas de secuencia: los flujos más importantes (login, crear tarea, etc.)

Esto te toma 1-2 horas y le da al profesor una visión clara de tu arquitectura.

</details>

```

---

## ## Mini-chequeo

<details>

<summary>¿Qué tipo de diagramas debo conocer?</summary>

**\*\*C4\*\*** (arquitectura), **\*\*ER\*\*** (datos), **\*\*Secuencia\*\*** (flujos), **\*\*Componentes\*\*** (estructura)  
</details>

<details>

<summary>¿Por qué son importantes los diagramas?</summary>

**\*\*Comunican\*\*** en 30 segundos lo que un texto tarda 10 minutos. Todos ven la misma imagen  
</details>

<details>

<summary>¿Cuántos diagramas necesito para un proyecto de módulo?</summary>

**\*\*3-4 diagramas\*\***: 1 C4 nivel 1, 1 ER, 2-3 secuencia. Son 1-2 horas de trabajo que ahorran  
</details>

---

## ## Resumen en 3 frases

1. Los diagramas son **\*\*representaciones visuales\*\*** de tu sistema: muestran estructura, componentes y flujos.
2. Los más importantes son **\*\*C4\*\*** (arquitectura), **\*\*ER\*\*** (datos) y **\*\*Secuencia\*\*** (flujos).
3. No son burocracia: son la **\*\*documentación visual\*\*** que entiende todo el equipo y que sirve para comunicar.

---

>  Volver al [índice de la unidad] (../u2-6-diagramas) · Siguiente: [02 – Modelo C4] (./02-modelo-c4)

---

## ## 02 – Modelo C4

>  **\*\*Estás en:\*\***  **\*\*UD 2.6 · Diagramas\*\*** → 02 · Modelo C4

>  C4 es como un zoom: de lo general a lo específico\_

>

> El **\*\*modelo C4\*\*** documenta la arquitectura en **\*\*4 niveles\*\*** que van de lo más general (contexto) a lo más específico (código). Es como hacer zoom en Google Maps: primero ves el país, luego la ciudad, luego la calle y finalmente la casa.

## ## La idea en una frase

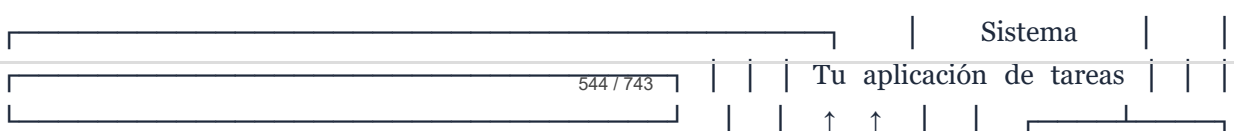
> **\*\*C4 documenta tu arquitectura en 4 niveles: Contexto (qué hace), Container (qué tecnologías), Componentes (qué partes) y Código (qué hace cada parte).\*\***  
Cada nivel responde una pregunta diferente.

---

## ## Los 4 niveles de C4

### ### Nivel 1: Contexto (System Context)

**\*\*Pregunta\*\***: ¿Qué hace mi sistema y quién lo usa?





Elemento	Descripción
<b>Tu sistema</b>	App de tareas para gestión de alumnos
<b>Alumno</b>	Crea y gestiona sus tareas
<b>Profesor</b>	Ve el progreso de sus alumnos

### Nivel 2: Container

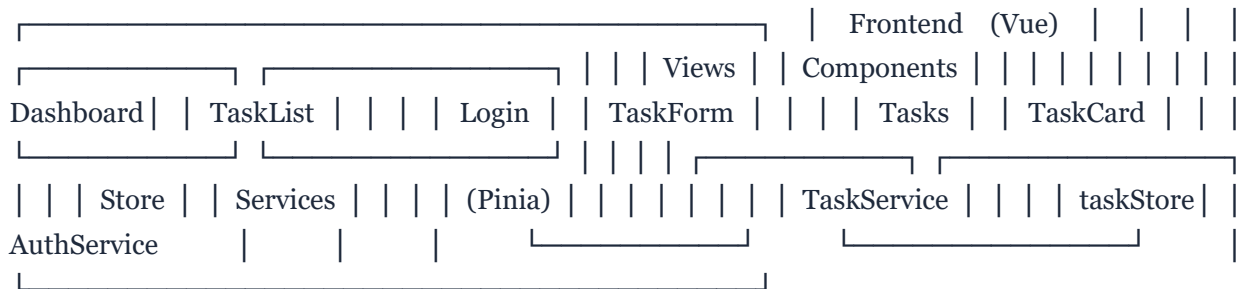
**Pregunta:** ¿Qué tecnologías componen mi sistema?



Container	Tecnología	Responsabilidad
Frontend	Vue + Tailwind	Interfaz de usuario
Backend API	Node.js + Express	Lógica de negocio
Base de datos	SQLite	Persistencia de datos

### Nivel 3: Componente

**Pregunta:** ¿Qué partes tiene mi frontend?



### Nivel 4: Código

**Pregunta:** ¿Cómo se implementa un componente concreto?

```
TaskService + getAll(): Task[]
TaskService + getById(id): Task
TaskService + save(task): Task
TaskService + delete(id): void
TaskRepository - repository: TaskRepository
```

---

## Comparativa: sin C4 vs con C4

> **Sin C4**

>

> El profesor: '¿Cómo se estructura tu app?' Yo: 'Pues tiene componentes, y un store, y

> **Con C4**

>

> El profesor: '¿Cómo se estructura tu app?' Yo: 'Nivel 1: contexto. Nivel 2: containers

---

## Cuántos niveles documentar para Proyecto 2

Nivel	¿Necesario?	Tiempo
**1 - Contexto**	✅ Siempre	15 min
**2 - Container**	✅ Siempre	20 min
**3 - Componente**	⚠️ Recomendado	30 min
**4 - Código**	❌ Solo si lo piden	30+ min

**Mínimo**: Niveles 1 y 2. Son 35 minutos y dan una visión completa de tu arquitectura.

---

## Aclaración: "¿Necesito los 4 niveles?"

<details>

<summary>🤖 ¿Tengo que documentar los 4 niveles de C4?</summary>

Para un proyecto de módulo, **no** necesitas los 4. Documenta:

- **Nivel 1 (Contexto)**: obligatorio. Qué hace tu app y quién la usa
- **Nivel 2 (Container)**: obligatorio. Qué tecnologías usas
- **Nivel 3 (Componente)**: recomendado. Qué partes tiene tu frontend
- **Nivel 4 (Código)**: opcional. Solo si el profesor lo pide

**Los niveles 1 y 2 son suficientes** para demostrar que entiendes la arquitectura de tu

</details>

---

## Mini-chequeo

<details>

<summary>¿Qué son los 4 niveles de C4?</summary>

**Contexto** (qué hace y quién lo usa), **Container** (qué tecnologías), **Componente**

<details>

<summary>¿Cuántos niveles documentar?</summary>

**Mínimo 2**: Contexto y Container. Recomendado 3: añadir Componente. Código solo si lo

<details>

<summary>¿Cuánto tiempo toma documentar C4?</summary>

**35-65 minutos** para los 2-3 primeros niveles. Es una inversión que da una visión clara

---  
## ✅ Resumen en 3 frases

1. El **modelo C4** documenta la arquitectura en 4 niveles: **Contexto** (qué hace), **Container** (cómo se hace), **Componentes** (qué partes tiene) y **Interfaces** (cómo se comunican).
2. Para Proyecto 2, documenta **mínimo 2 niveles** (Contexto y Container): son 35 minutos.
3. C4 es como un **zoom**: desde lo general hasta lo específico. El profesor puede hacer un zoom en cualquier nivel.

---

> 🏠 Volver al [índice de la unidad](../u2-6-diagramas) · Anterior: [01 - ¿Qué son los

---

## 03 - Diagrama de entidad-relación

> 📖 **Estás en:** 📁 **UD 2.6 · Diagramas** → 03 · Diagrama de entidad-relación

> 📺 Antes de programar, modela tus datos\_

>

> Un **diagrama de entidad-relación (ER)** muestra qué **datos** tiene tu sistema y cómo se **relacionan** entre sí. Es el plano de tu base de datos: sin él, estás creando tablas a ciegas y descubriendo problemas cuando ya tienes datos dentro.

## 🗨 La idea en una frase

> **Un diagrama ER muestra las entidades (tablas), sus atributos (columnas) y las relaciones.**

Modela antes de programar. Ahorra horas de refactorización.

---

## Los 3 elementos del ER

### 1. Entidad (tabla)

Una entidad es un "objeto" del sistema: algo de lo que guardas datos.



### ### 2. Atributo (columna)

Cada entidad tiene atributos: datos concretos que guardas.

```
| Entidad | Atributos |
|-----|-----|
| **Usuario** | id, nombre, email, rol |
| **Tarea** | id, titulo, descripcion, fecha_limite, prioridad |
| **Categoría** | id, nombre, color |
```

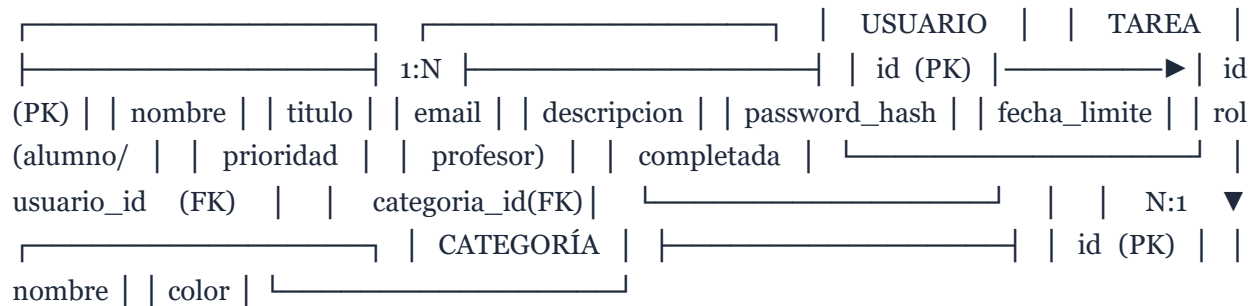
### ### 3. Relación (conexión)

Las relaciones muestran cómo se conectan las entidades:

```
| Relación | Significado | Símbolo |
|-----|-----|-----|
| **1:N** (uno a muchos) | Un usuario tiene muchas tareas | Línea con tridente |
| **N:M** (muchos a muchos) | Un alumno está en muchas clases | Línea con rombos |
| **1:1** (uno a uno) | Un usuario tiene un perfil | Línea simple |
```

---

## Ejemplo completo: App de tareas



### ### Leyendo el diagrama

- **Usuario 1:N Tarea**: Un usuario tiene muchas tareas. Una tarea pertenece a un usuario.
- **Tarea N:1 Categoría**: Una categoría tiene muchas tareas. Una tarea tiene una categoría.
- **PK** = Primary Key (clave primaria)
- **FK** = Foreign Key (clave foránea)

---

## Las 3 relaciones en detalle

### ### 1:N (Uno a muchos)

USUARIO — 1:N — TAREA

Un usuario tiene muchas tareas. Cada tarea tiene UN usuario.

**En código**: `usuario.tareas` es un array. `tarea.usuario` es un objeto.

### ### N:M (Muchos a muchos)

ALUMNO — N:M — CLASE



Un alumno está en muchas clases. Una clase tiene muchos alumnos.

**\*\*En código\*\*:** necesitas una **\*\*tabla intermedia\*\*** `ALUMNO\_CLASE`.

### 1:1 (Uno a uno)

USUARIO — 1:1 — PERFIL

Un usuario tiene un perfil. Un perfil pertenece a un usuario.

**\*\*En código\*\*:** `usuario.perfil` es un objeto.

---

## Comparativa: modelar antes vs modelar después

> **\*\*Modelar después\*\***

>

> Creo las tablas sobre la marcha. A mitad del proyecto descubro que la relación tarea-c

> **\*\*Modelar antes\*\***

>

> Dibujo el ER al principio. Detecto que necesito una tabla intermedia. La creo antes de

---

## Aclaración: "¿Necesito un diagrama ER si uso localStorage?"

<details>

<summary>🤔 ¿No es un diagrama ER para bases de datos SQL?</summary>

No necesariamente. Un ER también sirve para **\*\*modelar datos en localStorage/IndexedDB\*\***

- Te ayuda a ver **\*\*qué objetos\*\*** guardas
- Te ayuda a ver **\*\*cómo se relacionan\*\*** entre sí
- Si mañana migras a una BD real, ya tienes el modelo

**\*\*Mínimo\*\*:** dibuja un ER simple con tus entidades principales y sus relaciones. Te toma

</details>

---

## Mini-chequeo

<details>

<summary>¿Qué son las 3 relaciones?</summary>

**\*\*1:N\*\*** (uno a muchos): un usuario tiene muchas tareas. **\*\*N:M\*\*** (muchos a muchos): un al

</details>

<details>

<summary>¿Qué es PK y FK?</summary>

**\*\*PK\*\*** (Primary Key): identificador único de cada fila. **\*\*FK\*\*** (Foreign Key): referencia

</details>

<details>

<summary>¿Necesito ER si uso localStorage?</summary>

Sí, para **\*\*modelar\*\*** qué objetos guardas y cómo se relacionan. Te da claridad y facilita

</details>

---

## ✅ Resumen en 3 frases

1. Un **\*\*diagrama ER\*\*** muestra entidades (tablas), atributos (columnas) y relaciones (1:1
2. Las relaciones clave son **\*\*1:N\*\*** (un usuario tiene muchas tareas), **\*\*N:M\*\*** (necesita
3. Modela **\*\*antes de programar\*\***: 30 minutos de ER ahorran horas de refactorización cuar

---

```

> 🏠 Volver al [índice de la unidad](../u2-6-diagramas) · Anterior: [02 - Modelo C4](..)

04 - Diagrama de secuencia

> 📖 **Estás en:** 📁 **UD 2.6 · Diagramas** → 04 · Diagrama de secuencia

> 📖 Un diagrama de secuencia cuenta la historia de una interacción_
>
> Un **diagrama de secuencia** muestra **quién habla con quién** y **en qué
> orden** durante un proceso concreto. Es como un guion de teatro: cada
> actor (componente) tiene su turno y las líneas muestran qué mensaje
> recibe cada uno.

🗨️ La idea en una frase

> **Un diagrama de secuencia muestra la interacción entre componentes en el tiempo: quién
> habla con quién y en qué orden. Cuenta la historia de un flujo concreto, paso a paso.

Los elementos del diagrama de secuencia

```

```

Usuario Frontend Store localStorage | | | | |—click—▶| | | | |—getTasks()—▶| | | |
|—getItem()—▶| | | | ◀—data——| | | ◀—tasks——| | | ◀—render——| | | | |

```

```

| Elemento | Símbolo | Ejemplo |
|-----|-----|-----|
| **Actor** | Recuadro en la parte superior | Usuario, Frontend, Store |
| **Línea de vida** | Línea vertical punteada | Existe durante todo el flujo |
| **Mensaje** | Flecha horizontal | `getTasks()`, `save()` |
| **Respuesta** | Flecha punteada de vuelta | `tasks[]`, `undefined` |
| **Actividad** | Rectángulo en la línea de vida | Procesando datos |

Ejemplo: flujo de login

```

```

Usuario LoginView AuthStore localStorage Router | | | | |—click—▶| | | | | “Login” | |
| | | |—login()—▶| | | | |—getItem()—▶| | | | | ◀—users——| | | | | | | |
|—validar()—| | | | | | | | | ◀—success——| | | | | | | | |
|—redirect()———| | | | | | | | | | | | | | |
| ◀—navegar———| | | | | | | | | | | | | | |

```

### Leyendo el diagrama

1. El **usuario** hace click en "Login"
2. **LoginView** llama a ``AuthStore.login()``
3. **AuthStore** busca usuarios en ``localStorage``
4. **AuthStore** valida las credenciales
5. Si es correcto, devuelve ``success``
6. **LoginView** redirige con el Router
7. El **usuario** navega al dashboard

---

## Los 4 tipos de mensajes

Tipo	Símbolo	Ejemplo
-----	-----	-----
<b>Síncrono</b>	Flecha sólida →	<code>`getTasks()`</code> (espera respuesta)
<b>Asíncrono</b>	Flecha abierta ⇨	<code>`notificar()`</code> (no espera)
<b>Respuesta</b>	Flecha punteada ···→	<code>`tasks[]`</code> (devuelve datos)
<b>Creación</b>	Flecha con <code>`create`</code>	<code>`new Task()`</code> (crea objeto)

---

## Ejemplo: flujo de crear tarea



---

## Comparativa: secuencia vs texto

> **Explicación en texto**

>

> El usuario hace submit. El formulario llama al store. El store llama al repository. El

> **Diagrama de secuencia**

>

> [Dibujo con 6 flechas que muestra el mismo flujo en una imagen]

---

## Cuándo crear diagramas de secuencia

Flujo	¿Merece un diagrama?	
----- -----		
Login	✓ Sí (flujo central)	
Crear tarea	✓ Sí (funcionalidad principal)	
Exportar datos	✓ Sí (flujo complejo)	
Cargar una página	⚠ Opcional (simple)	
Hacer click en un botón	✗ No (demasiado trivial)	

**Regla:** si el flujo tiene **más de 3 pasos** o **más de 2 componentes**, un diagrama

---

## Aclaración: "¿Cómo dibujo un diagrama de secuencia?"

<details>

<summary>🤖 ¿Qué herramienta uso para crear diagramas de secuencia?</summary>

Tres opciones principales:

1. **Mermaid** (recomendado): escribes el diagrama en código de texto. Se renderiza automáticamente.
2. **draw.io**: herramienta visual gratuita para dibujar
3. **Excalidraw**: estilo manuscrito, rápido y bonito

**Para Mermaid**, el ejemplo del login sería:

```
sequenceDiagram
 participant U as Usuario
 participant V as LoginView
 participant S as AuthStore
 participant L as localStorage

 U->>V: click "Login"
 V->>S: login(email, password)
 S->>L: getItem('users')
 L-->>S: users[]
 S->>S: validar credenciales
 S-->>V: success
 V->>V: redirect('/dashboard')
 V-->>U: navegar al dashboard
```

► ¿Qué muestra un diagrama de secuencia?

► ¿Cuándo crear un diagrama de secuencia?

► ¿Qué herramienta usar?

## ✓ Resumen en 3 frases

1. Un **diagrama de secuencia** muestra la interacción entre componentes en el tiempo: quién envía qué mensaje, en qué orden y qué devuelve.
2. Los elementos son: **actores** (componentes), **líneas de vida**, **mensajes** (flechas) y **respuestas** (flechas punteadas).
3. Crea un diagrama cuando el flujo tenga **más de 3 pasos** o **más de 2 componentes**: los flujos simples no lo necesitan.

 Volver al [índice de la unidad](#) · Anterior: [03 — Diagrama ER](#) · Siguiente: [05 — Símbolos UML](#)

## 05 — Símbolos UML

 Estás en:  **UD 2.6 · Diagramas** → 05 · Símbolos UML

 *UML es el alfabeto de los diagramas: conoce los símbolos básicos*

**UML** (Unified Modeling Language) es el estándar para diagramas de software. No necesitas conocer los 14 tipos de diagramas UML: necesitas conocer los **símbolos esenciales** que aparecen en los diagramas que vas a crear en Proyecto 2.

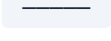
## La idea en una frase

**UML es el lenguaje común de los diagramas: conoce los símbolos básicos (clases, relaciones, actores) y podrás leer y crear cualquier diagrama.**

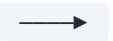
No memorices todo. Memoriza lo esencial.

## Los 10 símbolos esenciales

### Para diagramas de clases

Símbolo	Nombre	Ejemplo
	<b>Clase</b>	Una clase con nombre, atributos y métodos
	<b>Público</b>	<code>+ nombre: string</code>
	<b>Privado</b>	<code>- password: string</code>
	<b>Protegido</b>	<code># id: number</code>
	<b>Herencia</b>	Clase hija → Clase padre

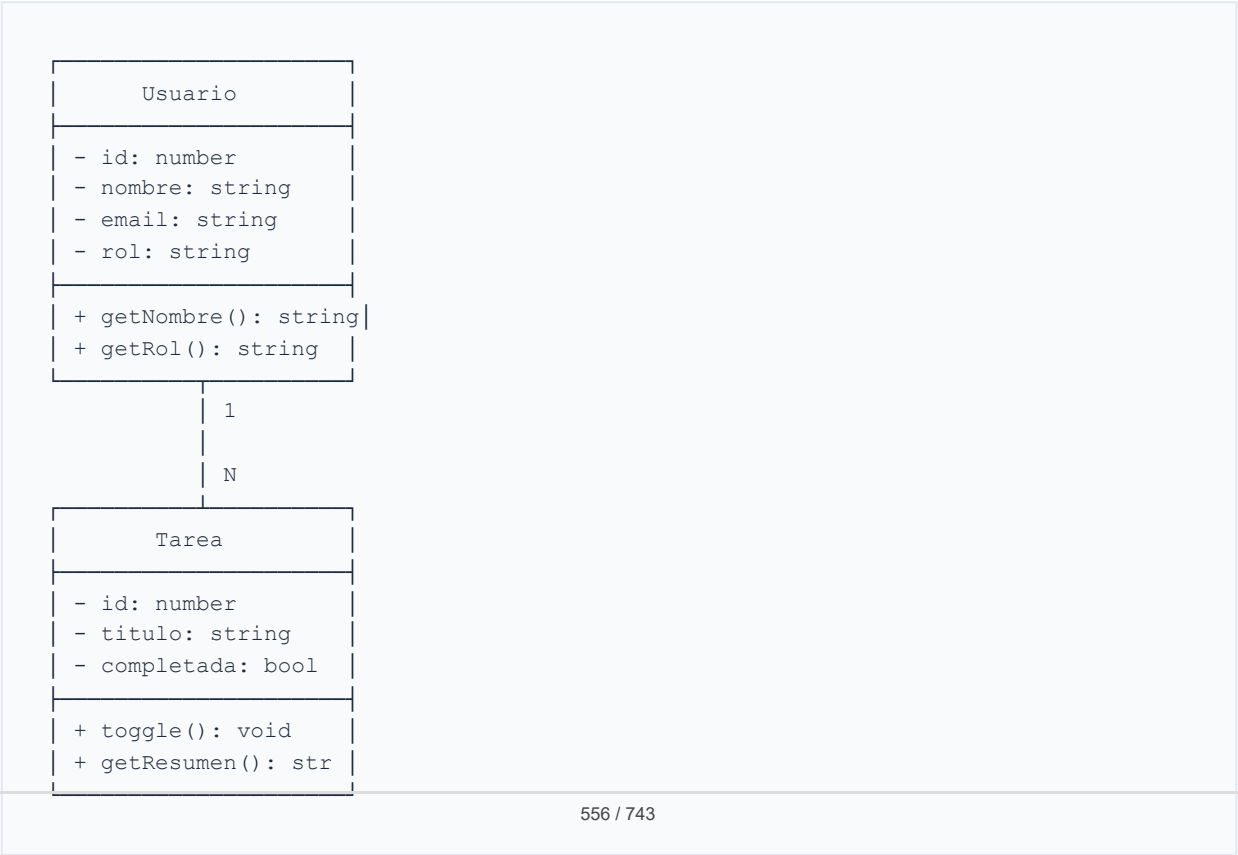
### Para diagramas de relación

Símbolo	Nombre	Significado
	Composición	“Tiene un” (fuerte, sin el todo no existe la parte)
	Agregación	“Tiene un” (débil, la parte existe sin el todo)
	Asociación	“Conoce a” (hay relación)
	Dependencia	“Usa a” (temporal)
	Herencia	“Es un tipo de”

Para diagramas de secuencia

Símbolo	Nombre	Ejemplo
	Línea de vida	Componente que existe durante el flujo
	Mensaje síncrono	<code>getTasks()</code> (espera respuesta)
	Respuesta	<code>tasks[]</code> (devuelve datos)

Ejemplo: diagrama de clases UML





## Leyendo el diagrama

- **Usuario** tiene 3 atributos privados ( - ) y 2 métodos públicos ( + )
- **Tarea** tiene 3 atributos y 2 métodos
- La relación **1:N** indica que un usuario tiene muchas tareas

## Las relaciones más importantes

### Composición vs Agregación

#### Composición (◆)

Una TAREA no existe sin su USUARIO. Si borras el usuario, borras sus tareas. Es dependencia fuerte.

#### Agregación (◇)

Una CATEGORÍA existe sin las TAREAS. Si borras la categoría, las tareas siguen ahí. Es dependencia débil.

Relación	Significado	Ejemplo
<b>Composición ◆</b>	La parte NO existe sin el todo	Tarea sin usuario → no existe
<b>Agregación ◇</b>	La parte SÍ existe sin el todo	Tarea sin categoría → sigue existiendo

## Aclaración: “¿Necesito memorizar todo UML?”

► 🤔 ¿Tengo que dominar los 14 tipos de diagramas UML?

# Mini-chequeo

► ¿Qué significan +, -, # en UML?

► ¿Cuál es la diferencia entre composición y agregación?

► ¿Cuántos símbolos UML debo memorizar?

## ✓ Resumen en 3 frases

1. Los símbolos UML esenciales son **clase** (recuadro con nombre, atributos, métodos) y **relaciones** (composición, agregación, asociación, herencia).
2. **Composición** ◆ = dependencia fuerte (la parte no existe sin el todo). **Agregación** ◇ = dependencia débil (la parte sobrevive).
3. No memorices los 14 tipos de diagramas UML: **conoce los 10 símbolos esenciales** y úsalos como referencia.

 Volver al [índice de la unidad](#) · Anterior: [04 — Diagrama de secuencia](#) · Siguiendo: [06 — Mermaid](#)

## 06 — Mermaid

 Estás en:  **UD 2.6 · Diagramas** → 06 · Mermaid

 *Mermaid: diagramas como código, no como dibujos*

**Mermaid** es un lenguaje de diagramas en **código de texto**: escribes el diagrama en una sintaxis simple y se renderiza automáticamente. GitHub, GitLab y muchas herramientas lo soportan. Es la forma más rápida de crear diagramas que se mantienen actualizados.

## La idea en una frase

**Mermaid convierte código de texto en diagramas: escribes la sintaxis, se renderiza automáticamente. Es como Markdown pero para diagramas.**

Si sabes escribir Markdown, sabes crear diagramas Mermaid.

## Flujo (Flowchart)

```
graph TD
 A[Inicio] --> B{¿Está logueado?}
 B -->|Sí| C[Mostrar dashboard]
 B -->|No| D[Mostrar login]
 D --> E[Introducir credenciales]
 E --> F{¿Son correctas?}
 F -->|Sí| C
 F -->|No| G[Mostrar error]
 G --> D
```

## Sintaxis básica

```
graph TD
 A[Texto del nodo] --> B[Otro nodo]
 A --> C[Tercer nodo]
 C -->|Sí| D[Resultado positivo]
 C -->|No| E[Resultado negativo]
```

Símbolo	Significado
-->	Flecha
--> texto	Flecha con etiqueta
[texto]	Nodo rectangular
{texto}	Nodo diamante (decisión)
((texto))	Nodo circular
TD	Top-Down (arriba abajo)
LR	Left-Right (izquierda a derecha)

## Secuencia (Sequence)

```
sequenceDiagram
 participant U as Usuario
 participant V as Frontend
 participant S as Store
 participant L as localStorage

 U->>V: Hace click en "Guardar"
 V->>S: saveTask(task)
 S->>L:.setItem('tasks', tasks)
 L-->>S: OK
 S-->>V: success
 V-->>U: Tarea guardada
```

## Sintaxis básica

```
sequenceDiagram
 participant A as NombreCorto
 participant B as OtroNombre
 A->>B: Mensaje síncrono
 B-->>A: Respuesta
 A-xB: Mensaje async (no espera)
```

## Clases (Class Diagram)

```

classDiagram
 class Usuario {
 -id: number
 -nombre: string
 -email: string
 +getNombre() string
 +getRol() string
 }

 class Tarea {
 -id: number
 -titulo: string
 -completada: bool
 +toggle() void
 }

 Usuario "1" --> "*" Tarea : tiene

```

## Sintaxis básica

```

classDiagram
 class NombreClase {
 -atributoPrivado: tipo
 +metodoPublico() tipo
 }

 ClaseA "1" --> "*" ClaseB : relación
 ClaseA <|-- ClaseB : herencia

```

## Entidad-Relación (ER)

```

erDiagram
 USUARIO ||--o{ TAREA : tiene
 TAREA }o--|| CATEGORIA : pertenece

 USUARIO {
 int id PK
 string nombre
 string email
 }

 TAREA {
 int id PK
 string titulo
 date fecha_limite
 int usuario_id FK
 int categoria_id FK
 }

 CATEGORIA {
 int id PK
 string nombre
 string color
 }

```

## Sintaxis básica

```

erDiagram
 ENTIDAD_A ||--o{ ENTIDAD_B : relación
 ENTIDAD_A {
 tipo atributo PK
 tipo atributo FK
 }

```

Símbolo	Significado
<code>\\   --o\\ </code>	Uno a uno
<code>\\   --o{</code>	Uno a muchos
<code>}o--o{</code>	Muchos a muchos

## Flujo de datos (Pie Chart)

```

pie title Distribución de tareas
 "Completadas" : 45
 "En progreso" : 30
 "Pendientes" : 25

```

# Comparativa: Mermaid vs draw.io

## draw.io

Abro la herramienta, dibujo recuadros, arrastro flechas, le pongo texto. Bonito pero toma 20 minutos. Si cambio algo, redibujo todo.

## Mermaid

Escribo 10 líneas de código. Se renderiza automáticamente. Si cambio algo, 编辑 o una línea. Se actualiza solo.

Aspecto	Mermaid	draw.io
Velocidad	Rápido (código)	Lento (dibujar)
Mantenimiento	Fácil (editar texto)	Difícil (redibujar)
Colaboración	Git (versionado)	Archivo binario
Calidad visual	Buena	Excelente
Complejidad	Limitada	Ilimitada

## Aclaración: “¿Dónde puedo probar Mermaid?”

► 🤖 ¿Dónde veo mis diagramas Mermaid renderizados?

## Mini-chequeo

► ¿Qué es Mermaid?

► ¿Qué tipos de diagramas soporta?

► ¿Dónde ver los diagramas renderizados?

## ✓ Resumen en 3 frases

1. **Mermaid** es un lenguaje de diagramas en código de texto: escribes la sintaxis y se renderiza automáticamente.
2. Soporta **flowcharts** (flujo), **sequence** (secuencia), **class** (clases), **ER** (datos) y más: es el equivalente a Markdown pero para diagramas.
3. **GitHub** renderiza Mermaid automáticamente: crea un `.md` con código Mermaid y lo ves en el repositorio sin instalar nada.

 Volver al [índice de la unidad](#) · Anterior: [05 — Símbolos UML](#) · Siguiente: [07 — Herramientas de diagramas](#)

## 07 — Herramientas de diagramas

 **Estás en:**  **UD 2.6 · Diagramas** → 07 · Herramientas de diagramas

 *La herramienta correcta para cada tipo de diagrama*

Hay muchas herramientas para crear diagramas. No todas son iguales: algunas son visuales, otras son código; algunas son gratuitas, otras de pago. Elige la que mejor se adapte a lo que necesitas.



**draw.io para diagramas visuales completos, Excalidraw para bocetos rápidos, Mermaid para diagramas en código. Cada uno tiene su momento.**

No hay una herramienta perfecta. Hay la herramienta correcta para cada caso.

## Las 3 herramientas principales

### 1. draw.io (diagrams.net)

Característica	Detalle
Tipo	Visual, gratuita
URL	draw.io o diagrams.net
Ideal para	Diagramas completos, arquitectura, ER, secuencia
Exporta	PNG, SVG, PDF, XML
Integra con	GitHub, Google Drive, Notion

**Ventajas:** Gratuita, sin registro, muchas plantillas, exporta a cualquier formato. **Desventajas:** Puede ser lenta con diagramas grandes.

### 2. Excalidraw

Característica	Detalle
Tipo	Manuscrito, gratuito
URL	excalidraw.com
Ideal para	Bocetos, brainstorming, diagramas informales
Exporta	PNG, SVG
Integra con	VS Code (extensión)

**Ventajas:** Rápido, bonito, estilo manuscrito, colaborativo. **Desventajas:** Menos preciso que draw.io, sin plantillas formales.

### 3. Mermaid

Característica	Detalle
Tipo	Código de texto, gratuito
URL	mermaid.live
Ideal para	Diagramas que cambian frecuentemente, versionados en Git
Exporta	PNG, SVG (desde el editor)
Integra con	GitHub, GitLab, VS Code

**Ventajas:** Versionable en Git, fácil de mantener, rápido de editar. **Desventajas:** Menos flexible visualmente, curva de aprendizaje.

## Comparativa: las 3 herramientas

### draw.io

Necesito un diagrama de arquitectura completo para el profesor. Lo dibujo con colores, formas y etiquetas. Exporto PNG y lo subo al README.

### Excalidraw

Necesito un boceto rápido para discutir con el equipo. Lo dibujo en 5 minutos estilo manuscrito. Comparto el enlace.

### Mermaid

Necesito un diagrama de secuencia que cambia cada sprint. Lo escribo en código, lo commiteo a Git. Cuando cambia,编辑o una línea.

Criterio	draw.io	Excalidraw	Mermaid
Velocidad	Media	Rápida	Rápida
Calidad visual	Excelente	Buena	Buena
Mantenimiento	Difícil	Medio	Fácil
Git	No (XML binario)	No (PNG)	Sí (texto)
Gratis	✓	✓	✓
Sin registro	✓	✓	✓

## ¿Qué herramienta usar para cada diagrama?

Diagrama	Herramienta recomendada	Por qué
C4 arquitectura	draw.io	Necesitas precisión y formas
ER (datos)	draw.io o Mermaid	Ambas funcionan bien
Secuencia	Mermaid	Fácil de editar y versionar
Flujo	Mermaid o Excalidraw	Depende de si es formal o boceto
Clases	draw.io	Necesitas UML formal
Boceto	Excalidraw	Rápido y visual

## Aclaración: “¿Cuál es la mejor para empezar?”

► 🤔 ¿Qué herramienta uso si solo puedo elegir una?

► ¿Cuáles son las 3 herramientas principales?

► ¿Qué herramienta para diagramas formales?

► ¿Qué herramienta para diagramas que cambian?

## ✓ Resumen en 3 frases

1. **draw.io** para diagramas visuales completos (arquitectura, ER formal): gratuita, exporta PNG/PDF.
2. **Excalidraw** para bocetos rápidos y brainstorming: estilo manuscrito, colaborativo.
3. **Mermaid** para diagramas que cambian frecuentemente: código de texto, versionable en Git, renderiza en GitHub automáticamente.

 Volver al [índice de la unidad](#) · Anterior: [o6 — Mermaid](#) · Siguiente: [o8 — Template diagrams](#)

## o8 — Template de diagramas

 Estás en:  **UD 2.6 · Diagramas** → [o8 · Template de diagramas](#)

 *Las plantillas que te ahorran empezar desde cero*

Este template incluye las plantillas de Mermaid para los diagramas esenciales de Proyecto 2: C4 contexto, ER, secuencia y flujo. Cópialos, adapta los nombres a tu proyecto y tendrás tu documentación visual en 30 minutos.

**Copia estas plantillas en un archivo `DIAGRAMAS.md` en tu repositorio, adapta los nombres y tendrás la documentación visual de tu proyecto.**

Los diagramas son documentación. Trátalos como tal.

## Template 1: C4 Nivel 1 — Contexto

```
graph TD
 subgraph "Sistema"
 APP["Tu aplicación
(Vue + Node.js)"]
 end

 ALUMNO["👤 Alumno"] -->|"Usa"| APP
 PROFESOR["👨‍🏫 Profesor"] -->|"Usa"| APP
 APP -->|"Guarda datos"| DB["localStorage
o SQLite"]

 style APP fill:#4CAF50,color:#fff
 style ALUMNO fill:#2196F3,color:#fff
 style PROFESOR fill:#FF9800,color:#fff
 style DB fill:#9C27B0,color:#fff
```

## Template 2: C4 Nivel 2 — Containers

```

graph LR
 subgraph Frontend
 VUE["Vue.js
Componentes"]
 PINIA["Pinia
State Management"]
 end

 subgraph Backend
 NODE["Node.js
API REST"]
 end

 subgraph Data
 DB["SQLite
Base de datos"]
 end

 VUE <-->|"HTTP"| NODE
 PINIA <-->|"Estado"| VUE
 NODE <-->|"Query"| DB

 style VUE fill:#4CAF50,color:#fff
 style PINIA fill:#8BC34A,color:#fff
 style NODE fill:#2196F3,color:#fff
 style DB fill:#9C27B0,color:#fff

```

## Template 3: Diagrama ER

```

erDiagram
 USUARIO ||--o{ TAREA : "tiene"
 TAREA }o--|| CATEGORIA : "pertenece a"

 USUARIO {
 int id PK
 string nombre
 string email
 string rol
 }

 TAREA {
 int id PK
 string titulo
 string descripcion
 date fecha_limite
 int prioridad
 boolean completada
 int usuario_id FK
 int categoria_id FK
 }

 CATEGORIA {
 int id PK
 string nombre
 string color
 }

```

## Template 4: Diagrama de secuencia — Login

```
sequenceDiagram
 participant U as  Usuario
 participant V as  LoginView
 participant S as  AuthStore
 participant L as  localStorage

 U->>V: Introduce credenciales
 V->>S: login(email, password)
 S->>L: getItem('users')
 L-->>S: users[]
 S->>S: Buscar usuario por email
 S->>S: Comparar contraseñas

 alt Credenciales correctas
 S-->>V: { success: true, user }
 V->>V: Guardar sesión
 V-->>U: Redirigir al dashboard
 else Credenciales incorrectas
 S-->>V: { success: false, error }
 V-->>U: Mostrar "Email o contraseña incorrectos"
 end
end
```

## Template 5: Diagrama de secuencia — Crear tarea

```
sequenceDiagram
 participant U as  Usuario
 participant F as  TaskForm
 participant T as  TaskStore
 participant R as  TaskRepository
 participant L as  localStorage

 U->>F: Rellena formulario y envía
 F->>F: Validar campos
 F->>T: createTask({ titulo, fecha })
 T->>T: Generar ID único
 T->>R: save(task)
 R->>L: setItem('tasks', tasks)
 L-->>R: OK
 R-->>T: task guardada
 T-->>F: { success: true }
 F->>F: Limpiar formulario
 F-->>U: Mostrar "Tarea creada"
```

## Template 6: Flujo de decisión

```
graph TD
 A[Inicio: Recibir tarea] --> B{¿Está completa?}
 B -->|Sí| C[Marcar como completada]
 B -->|No| D{¿Tiene fecha límite?}
 D -->|Sí| E{¿Ha pasado la fecha?}
 D -->|No| F[Mantener pendiente]
 E -->|Sí| G[Marcar como atrasada]
 E -->|No| H[Verificar progreso]
 C --> I[Actualizar estadísticas]
 G --> I
 F --> I
 H --> I
 I --> J[Fin]

 style A fill:#4CAF50,color:#fff
 style I fill:#2196F3,color:#fff
 style J fill:#9C27B0,color:#fff
```

## Aclaración: “¿Cómo uso estas plantillas?”

► 🤖 ¿Cómo adapto estas plantillas a mi proyecto?

## Mini-chequeo

► ¿Qué templates incluye?

► ¿Cómo adapto los templates?

► ¿Dónde guardo los diagramas?

## ✅ Resumen en 3 frases

1. El template incluye **6 diagramas**: C4 contexto, C4 containers, ER, secuencia login, secuencia crear tarea y flujo de decisión.



2. **Copia, cambia nombres y adapta** a tu proyecto: no necesitas entender toda la sintaxis de Mermaid.

3. Guarda todo en `DIAGRAMAS.md`: GitHub lo renderiza automáticamente y el profesor puede ver tus diagramas.

 Volver al [índice de la unidad](#) · Anterior: [07 — Herramientas](#) · Siguiendo: [09 — Cierre](#)

## 09 — Cierre

 Estás en:  **UD 2.6 · Diagramas** → 09 · Cierre

 *Ya puedes comunicar tu arquitectura con imágenes, no con palabras*

Has recorrido toda la unidad: desde qué son los diagramas hasta Mermaid y las herramientas, pasando por C4, ER, secuencia y UML. Ahora toca **verificar que lo entiendes** y **aplicarlo a tu proyecto**.

### La idea en una frase

**Un diagrama bien hecho comunica en 30 segundos lo que un texto tarda 10 minutos. Si puedes explicar tu sistema en un diagrama, lo entiendes de verdad.**

Los diagramas no son burocracia. Son comprensión.

## Preguntas de comprensión

► ¿Qué son los 4 niveles de C4?

► ¿Qué son las 3 relaciones del ER?

► ¿Qué muestra un diagrama de secuencia?

► ¿Qué significan +, -, # en UML?

► ¿Qué es Mermaid?

► ¿Qué herramienta usar para diagramas formales?

---

## Ejercicio práctico

**En 2 horas, crea los diagramas de tu proyecto:**

1. **C4 contexto** (20 min): Dibuja qué hace tu app y quién la usa
2. **C4 containers** (20 min): Dibuja qué tecnologías componen tu sistema
3. **ER** (30 min): Modela tus entidades principales y sus relaciones
4. **Secuencia** (30 min): Dibuja el flujo de login y crear tarea
5. **Documenta** (20 min): Guarda todo en `DIAGRAMAS.md`

**Si consigues tener C4 + ER + 2 secuencias documentados → has entendido la unidad.**

---



## Tabla resumen de la unidad

Concepto	Idea clave
<b>C4</b>	4 niveles: Contexto, Container, Componente, Código
<b>ER</b>	Entidades, atributos y relaciones (1:N, N:M, 1:1)
<b>Secuencia</b>	Interacción entre componentes en el tiempo
<b>UML</b>	Símbolos estándar: clase, relación, herencia
<b>Mermaid</b>	Diagramas en código de texto, renderiza en GitHub
<b>draw.io</b>	Herramienta visual gratuita para diagramas formales
<b>Excalidraw</b>	Bocetos rápidos estilo manuscrito
<b>PK/FK</b>	Primary Key / Foreign Key: identificador y referencia

---

## Vocabulario rápido

Término	Definición
<b>C4</b>	Modelo de arquitectura en 4 niveles: Contexto, Container, Componente, Código
<b>ER</b>	Entidad-Relación: diagrama que modela datos y sus relaciones
<b>Secuencia</b>	Diagrama que muestra interacciones entre componentes en el tiempo
<b>UML</b>	Unified Modeling Language: estándar de diagramas de software
<b>Mermaid</b>	Lenguaje de diagramas en código de texto
<b>draw.io</b>	Herramienta visual gratuita para diagramas
<b>Excalidraw</b>	Herramienta de bocetos estilo manuscrito
<b>PK</b>	Primary Key: identificador único de cada fila
<b>FK</b>	Foreign Key: referencia a la PK de otra tabla
<b>1:N</b>	Relación uno a muchos
<b>N:M</b>	Relación muchos a muchos (requiere tabla intermedia)
<b>1:1</b>	Relación uno a uno
<b>Flowchart</b>	Diagrama de flujo con nodos y flechas
<b>Clase</b>	Entidad UML con nombre, atributos y métodos

## Conexión con las siguientes unidades

Unidad	Qué aprenderás	Cómo se conecta
<b>UD 2.6 IA como Copiloto</b>	LLMs, prompt engineering	Usarás IA para generar diagramas Mermaid
<b>UD 2.7 Despliegue</b>	Docker, CI/CD	Diagramarás la infraestructura de despliegue
<b>UD 2.8 Monitorización</b>	Prometheus, Grafana	Diagramarás el flujo de datos y métricas

## ✓ Resumen en 3 frases

1. **C4** documenta arquitectura en 4 niveles (Contexto → Container → Componente → Código): para Proyecto 2, documenta mínimo los 2 primeros.
2. **ER** modela datos con entidades, atributos y relaciones (1:N, N:M, 1:1): dibuja tu modelo de datos antes de programar.
3. **Mermaid** crea diagramas en código de texto que GitHub renderiza automáticamente: es la herramienta más rápida para documentar tu arquitectura.

 Volver al [índice de la unidad](#) · Anterior: [o8 — Template diagramas](#) · Siguiente: [UD 2.7 — IA como Copiloto](#)

## o1 — Qué hace bien y mal la IA

 Estás en:  [UD 2.7 · IA como Copiloto](#) → [o1 · Qué hace bien y mal la IA](#)

 *La IA no es mágica. Es una herramienta con límites.*

Antes de usar la IA en tu proyecto, necesitas entender **dónde brilla y dónde falla**. Usarla sin conocer sus limitaciones es como conducir un coche sin saber frenar: al principio vas rápido, pero al final chocas.

## La idea en una frase

**La IA es excelente para tareas repetitivas y patrones conocidos, pero no entiende tu contexto ni tu negocio.**

# Qué hace BIEN la IA

## 1. Generar código boilerplate

La IA brilla cuando necesitas código repetitivo que siga un patrón:

- Componentes CRUD (crear, leer, actualizar, borrar)
- Formularios con validación
- Endpoints de API con estructura estándar
- Tests unitarios básicos

## 2. Explicar código existente

Puedes pegar código y pedir que te lo explique línea por línea. Es como tener un compañero que siempre tiene paciencia.

## 3. Sugerir mejoras

La IA detecta patrones subóptimos: - Código duplicado que podría ser una función - Variables mal nombradas - Posibles bugs por null checks faltantes

## 4. Aprender新技术

¿No sabes cómo usar un framework? La IA puede generar ejemplos básicos que te dan un punto de partida.

## 5. Convertir entre lenguajes

Tienes un snippet en Python y necesitas la versión en JavaScript. La IA lo hace en segundos.

---

# Qué hace MAL la IA

## 1. Entender tu negocio

La IA no sabe que en tu app de citas los usuarios son profesores jubilados, no jóvenes de 20 años. Generará código genérico, no adaptado a tu contexto.

## 2. Arquitectura compleja

La IA puede crear un componente, pero no puede diseñar la arquitectura completa de tu aplicación. Eso requiere comprensión del negocio y decisiones de diseño.

### 3. Dependencias

La IA inventa paquetes que no existen o usa versiones obsoletas. Siempre verifica que las dependencias que sugiere sean reales y actualizadas.

### 4. Seguridad

La IA genera código con vulnerabilidades comunes: SQL injection, XSS, secrets hardcodeados. **Nunca** confíes ciegamente en el código de seguridad que genere.

### 5. Contexto largo

Si tu archivo tiene 500 líneas y le pides que modifique la línea 47, puede “olvidar” el contexto de las primeras 100 líneas y generar algo que rompe lo existente.

---

## Comparativa: IA vs Desarrollador

---

Puedo generar un componente de login en 30 segundos con email, contraseña, validación y manejo de errores.

---

Puedo decidir si el login necesita OAuth, si la validación debe ser client-side o server-side, y si el error debe ser genérico o específico.

Tarea	IA	Desarrollador
Generar boilerplate	✅ Rápido y correcto	🐢 Lento pero personalizado
Explicar código	✅ Clara y paciente	✅ Entiende el contexto
Arquitectura	❌ No entiende el negocio	✅ Toma decisiones informadas
Seguridad	⚠️ Genera código vulnerable	✅ Conoce los riesgos
Testing	✅ Genera tests básicos	✅ Diseña estrategia de testing
Debugging	⚠️ A veces acierta	✅ Entiende la causa raíz

## La regla del 80/20

La IA puede hacer el **80% del trabajo** en tiempo de un desarrollador junior. Pero ese último 20% — la comprensión del negocio, la seguridad, la arquitectura — es lo que separa un proyecto funcional de uno profesional.

**Consejo:** Usa la IA para el 80% repetitivo y dedica tu tiempo al 20% que requiere pensamiento humano.

## Aclaración: “¿La IA me va a quitar el trabajo?”

► 🤖 ¿Debería preocuparme de que la IA reemplace a los desarrolladores?



# Mini-chequeo

► ¿Qué tipo de código genera mejor la IA?

► ¿Qué NO deberías delegar a la IA?

► ¿Qué es la regla del 80/20?

## ✓ Resumen en 3 frases

1. La IA es excelente para **código repetitivo, explicaciones y aprendizaje**, pero no entiende tu negocio ni tu contexto.
2. **Nunca confíes ciegamente** en código de seguridad generado por IA: siempre revisa y valida.
3. Usa la IA como acelerador, no como sustituto: **tú tomas las decisiones, ella ejecuta lo mecánico.**

 Volver al [índice de la unidad](#) · Siguiente: [02 — Prompts efectivos](#)

## 02 — Prompts efectivos

 Estás en:  UD 2.7 · IA como Copiloto → 02 · Prompts efectivos

 *El arte de pedir bien para recibir bien*

Un prompt mal escrito te da código inútil. Un prompt bien escrito te ahorra horas de trabajo. La diferencia entre un buen uso de la IA y uno malo no es la herramienta: es

cómo le pides las cosas.

## La idea en una frase

**Un buen prompt es específico, da contexto y describe el resultado esperado.**

## Estructura de un buen prompt

### Fórmula básica

```
CONTEXTO: [Qué estás haciendo]
TAREA: [Qué necesitas que haga la IA]
RESTRICCIONES: [Qué NO debe incluir]
EJEMPLO: [Cómo debería verse el resultado]
```

### Ejemplo malo vs bueno

#### Malo:

```
Hazme un login
```

#### Bueno:

```
CONTEXTO: Estoy creando una app de tareas con Vue 3 y Tailwind.
TAREA: Crea un componente de login con email y contraseña.
RESTRICCIONES: Sin frameworks externos, solo Vue 3 Composition API.
EJEMPLO: El componente debe tener un formulario centrado, dos campos y un botón.
```

## Los 5 principios de un buen prompt

### 1. Sé específico

Genérico	Específico
“Hazme un API”	“Crea un endpoint GET /api/users que devuelva una lista de usuarios desde un array en memoria”
“Un componente bonito”	“Un componente de tarjeta con imagen arriba, título, descripción y botón. Estilo Tailwind.”
“Tests”	“Tests unitarios para la función <code>calcularTotal</code> que suma precios con IVA”

## 2. Da contexto del proyecto

Proyecto: App de tareas con Vue 3 + Vite + Tailwind CSS  
Ya tengo: Componente TaskList.vue que muestra una lista de tareas  
Necesito: Un componente TaskForm.vue para crear nuevas tareas

## 3. Define el formato de salida

Genera el código en un solo archivo .vue  
Incluye <script setup> con Composition API  
Usa Tailwind para los estilos  
Incluye validación básica del formulario

## 4. Pide explicaciones

Añade al prompt:

Explica brevemente cada parte del código

## 5. Itera

Si el primer resultado no es correcto, refina el prompt:

El código anterior tiene un error: no valida el email.  
Corrige la validación y añade un mensaje de error debajo del campo.

# Prompts según la tarea

## Para generar código

```
Crea un componente React que:
- Muestra una lista de items
- Tiene un botón para añadir nuevo item
- Permite borrar items con un botón
- Usa useState para gestionar el estado
- Estilos con Tailwind CSS
```

## Para explicar código

```
Explica este código línea por línea. Indica:
1. Qué hace cada función
2. Por qué se usa ese patrón
3. Posibles mejoras
[Código aquí]
```

## Para refactorizar

```
Refactoriza este código para:
- Extraer la lógica de validación a una función separada
- Usar async/await en vez de .then()
- Añadir manejo de errores
[Código aquí]
```

## Para debugging

```
Este código tiene un bug: [descripción del problema].
El error ocurre cuando [pasos para reproducir].
Revisa el código y sugiere la corrección.
[Código aquí]
```

## Para tests

```
Genera tests unitarios para esta función:
- Test happy path (cuando todo va bien)
- Test edge case (valores límite)
- Test error (cuando algo falla)
Usa [framework de tests] como framework.
[Código aquí]
```

## Vue 3

```
Crea un componente Vue 3 con <script setup>:
- Composition API
- Props: [lista de props]
- Emits: [lista de eventos]
- Template con Tailwind CSS
```

## Node.js / Express

```
Crea un endpoint Express:
- Método: GET/POST/PUT/DELETE
- Ruta: /api/[recurso]
- Valida los campos de entrada
- Devuelve JSON con status code adecuado
- Maneja errores con try/catch
```

## Base de datos

```
Genera la consulta SQL para:
- Crear tabla [nombre] con campos [lista]
- Insertar datos de ejemplo
- Consulta con JOIN entre tablas [tablas]
```

---

## Ejemplo completo: Prompt y resultado

### Prompt

```
CONTEXTO: App de tareas con Vue 3 + Tailwind
TAREA: Componente TaskForm para crear tareas
RESTRICCIONES:
- Solo Composition API (<script setup>)
- Sin dependencias externas
- Validación: título obligatorio, fecha no puede ser pasada
SALIDA: Componente .vue completo
```

### Resultado esperado

```

<script setup>
import { ref } from 'vue'

const emit = defineEmits(['add-task'])
const titulo = ref('')
const fecha = ref('')
const error = ref('')

function handleSubmit() {
 error.value = ''

 if (!titulo.value.trim()) {
 error.value = 'El título es obligatorio'
 return
 }

 const hoy = new Date().toISOString().split('T')[0]
 if (fecha.value && fecha.value < hoy) {
 error.value = 'La fecha no puede ser pasada'
 return
 }

 emit('add-task', {
 titulo: titulo.value,
 fecha: fecha.value || null
 })

 titulo.value = ''
 fecha.value = ''
}
</script>

<template>
 <form @submit.prevent="handleSubmit" class="max-w-md mx-auto p-4">
 <input
 v-model="titulo"
 type="text"
 placeholder="Título de la tarea"
 class="w-full p-2 border rounded mb-2"
 />
 <input
 v-model="fecha"
 type="date"
 class="w-full p-2 border rounded mb-2"
 />
 <p v-if="error" class="text-red-500 text-sm mb-2">{{ error }}</p>
 <button
 type="submit"
 class="w-full bg-blue-500 text-white p-2 rounded"
 >
 Añadir tarea
 </button>
 </form>
</template>

```

## Errores comunes en prompts

Error	Ejemplo	Cómo corregirlo
Demasiado vago	“Haz algo bonito”	“Crea un card con imagen, título y botón”
Sin contexto	“Hazme un login”	“Login con email en Vue 3, Composition API”
Sin restricciones	“Un componente”	“Sin dependencias externas, solo Vue 3”
Un solo intento	“Hazlo bien”	Itera: “Corrige el error de validación”
Sin explicación	Solo código	“Explica qué hace cada parte”

## Aclaración: “¿Y si no sé qué contexto dar?”

► 🤖 ¿Cómo escribo un buen prompt si no domino el tema?

## Mini-chequeo

► ¿Cuáles son los 5 principios de un buen prompt?

► ¿Qué fórmula estructura un buen prompt?

► ¿Qué hago si el primer resultado no es correcto?

## ✅ Resumen en 3 frases

1. Un buen prompt es **específico, con contexto y restricciones claras** que la IA pueda seguir.
2. Usa la fórmula **CONTEXTO + TAREA + RESTRICCIONES + EJEMPLO** para estructurar tus peticiones.
3. **Itera:** el primer raro rara vez es perfecto; refina el prompt con cada intento.

## 03 — GitHub Copilot

 Estás en:  UD 2.7 · IA como Copiloto → 03 · GitHub Copilot

 *Tu compañero de código siempre disponible*

GitHub Copilot es el asistente de IA de GitHub que sugiere código mientras escribes. No reemplaza tu criterio, pero acelera enormemente el desarrollo rutinario. Aprender a usarlo bien es como tener un copiloto que nunca se cansa.

### La idea en una frase

**Copilot sugiere, tú decides. Nunca aceptes una sugerencia sin entender qué hace.**

## Qué es GitHub Copilot

GitHub Copilot es un asistente de IA que se integra directamente en tu editor (VS Code, JetBrains, etc.) y sugiere código en tiempo real mientras escribes.

### Cómo funciona

1. Escribes un comentario o código
2. Copilot analiza el contexto



3. Sugiere una o varias opciones de código
4. Tú aceptas, modifies o rechazas

## Dónde funciona

Editor	Soporte
VS Code	✓ Excelente
JetBrains (WebStorm, etc.)	✓ Bueno
Neovim	✓ Básico
GitHub.com	✓ En el editor web

## Cómo obtenerlo

1. Ve a [github.com/features/copilot](https://github.com/features/copilot)
2. Activa la prueba gratuita (30 días para estudiantes)
3. Instala la extensión en VS Code
4. Inicia sesión con tu cuenta de GitHub
5. Empieza a escribir código

## Comandos básicos de Copilot

### En VS Code

Acción	Atajo
Aceptar sugerencia	Tab
Rechazar sugerencia	Esc
Ver siguientes sugerencias	Alt + ]
Ver sugerencias anteriores	Alt + [
Abrir panel de sugerencias	Ctrl + Enter

## Estrategias de uso

### 1. Comentarios como instrucciones

Escribe un comentario claro y Copilot generará el código:

```
// Función que valida un email con expresión regular
function validarEmail(email) {
 // Copilot sugiere el código completo
}
```

### 2. Firmas de función

Escribe la firma y deja que Copilot complete el cuerpo:

```
def calcular_precio_total(precios, iva=0.21):
 # Copilot implementa la función
```

### 3. Nombres descriptivos

El nombre de la función o variable es un prompt:

```
// Copilot entiende qué quieres por el nombre
const filtrarUsuariosActivos = (usuarios) => {
 // ...
}
```

### 4. Código existente como contexto

Copilot mira el resto de tu archivo para sugerir código coherente:

```
Si ya tienes definida una clase Usuario...
class Usuario:
 def __init__(self, nombre, email):
 self.nombre = nombre
 self.email = email

Copilot sugerirá funciones relacionadas con Usuario
def buscar_usuario_por_email(usuarios, email):
 # ...
```

## Ejemplo práctico: CRUD con Copilot

### Paso 1: Modelo

```
// models/Task.ts
interface Task {
 id: number;
 titulo: string;
 completada: boolean;
 fechaCreacion: Date;
}
```

### Paso 2: Copilot sugiere la función de crear

```
// function to create a new task
function crearTarea(titulo: string): Task {
 // Copilot sugiere:
 return {
 id: Date.now(),
 titulo,
 completada: false,
 fechaCreacion: new Date()
 };
}
```

### Paso 3: Copilot sugiere el filtrado

```
// filter tasks by status
function filtrarPorEstado(tareas: Task[], completada: boolean): Task[] {
 // Copilot sugiere:
 return tareas.filter(t => t.completada === completada);
}
```

# Qué NO hacer con Copilot

## 1. No aceptes sin leer

Copilot puede sugerir código que compila pero que tiene bugs lógicos o problemas de seguridad. **Siempre lee lo que aceptas.**

## 2. No confíes en dependencias

Copilot a veces sugiere paquetes que no existen o están abandonados. Verifica siempre en npm o PyPI.

## 3. No uses para código sensible

Nunca pegues en Copilot: - Contraseñas o API keys - Datos de clientes reales - Código propietario de tu empresa

## 4. No esperes arquitectura

Copilot genera líneas y funciones, no diseña la arquitectura de tu aplicación. Eso es tu trabajo.

---

## Aclaración: “¿Es copiar si uso Copilot?”

► 🤖 ¿Usar Copilot es hacer trampas o copiar?

---

## Mini-chequeo

► ¿Qué atajo acepta una sugerencia en VS Code?

► ¿Qué información le da contexto a Copilot?

► ¿Qué NUNCA deberías pegar en Copilot?

## ✓ Resumen en 3 frases

1. Copilot sugiere código en tiempo real basándose en **comentarios, nombres y contexto** del archivo.
2. Usa **comentarios descriptivos** y **nombres claros** para obtener mejores sugerencias.
3. **Siempre lee y entiende** lo que Copilot genera antes de aceptarlo: tú eres el responsable del código.

 Volver al [índice de la unidad](#) · Anterior: [02 — Prompts efectivos](#) · Siguiente: [04 — Testing con IA](#)

## 04 — Testing con IA

 Estás en:  UD 2.7 · IA como Copiloto → 04 · Testing con IA

 Si la IA genera el código, ¿quién lo testea?

Cuando la IA genera código, surge una pregunta: ¿cómo sabemos que funciona bien? La respuesta es doble: testea el código generado **y** usa la IA para ayudarte a escribir tests. Es un círculo virtuoso.

## La idea en una frase

**El código generado por IA necesita tests tanto o más que el escrito por humanos: nunca asumas que es correcto.**

# Por qué testear código de IA

La IA genera código que: - **Compila** → pero puede tener bugs lógicos - **Se ve correcto** → pero falla en edge cases - **Usa patrones conocidos** → pero no validados contra tu negocio

## Ejemplo real

La IA genera esto para validar un email:

```
function validarEmail(email) {
 return email.includes('@');
}
```

¿Funciona? Sí, para emails simples. Pero: - `@.@` → pasa la validación (es inválido) - `user@.com` → pasa (inválido) - `user@domain` → no pasa (válido si hay DNS local)

**Lección:** el código generado por IA necesita tests para validar comportamiento real, no solo casos felices.

## Estrategia: usar IA para testear con IA

### Paso 1: Genera el código con IA

```
Crea una función calcularDescuento(precio, porcentaje) que:
- Aplique un descuento al precio
- No permita descuentos negativos ni mayores al 100%
- Redondee a 2 decimales
```

### Paso 2: Pide tests con IA

```
Genera tests unitarios para la función calcularDescuento:
- Happy path: descuento válido
- Edge cases: 0%, 100%, descuento negativo
- Errores: precio negativo, porcentaje > 100
Usa Jest como framework.
```

### Paso 3: Ejecuta y verifica

```
// Tests generados
describe('calcularDescuento', () => {
 test('aplica descuento correctamente', () => {
 expect(calcularDescuento(100, 20)).toBe(80);
 });

 test('no permite descuento negativo', () => {
 expect(() => calcularDescuento(100, -10)).toThrow();
 });

 test('no permite descuento mayor al 100%', () => {
 expect(() => calcularDescuento(100, 150)).toThrow();
 });

 test('redondea a 2 decimales', () => {
 expect(calcularDescuento(100, 33.333)).toBe(66.67);
 });
});
```

## Tipos de tests que la IA genera bien

Tipo	Utilidad	Limitaciones
Tests unitarios de funciones puras	✅ Excelente	No cubre integración
Tests de validación	✅ Muy bueno	Puede olvidar edge cases
Tests de componentes básicos	⚠️ Aceptable	No prueba interacción real
Tests de integración	⚠️ Limitado	No conoce tu arquitectura
Tests E2E	❌ Malo	No puede simular usuario real

## Qué NO delegar a la IA en testing

### 1. Estrategia de testing

La IA genera tests individuales, pero no diseña la **estrategia**: - ¿Qué módulos son críticos? - ¿Cuánta cobertura necesitas? - ¿Qué escenarios de usuario son prioritarios?

## 2. Tests de negocio

La IA no entiende que “un profesor no puede crear un examen para mañana si el plazo mínimo es de 3 días”. Ese test requiere conocimiento del dominio.

## 3. Tests de rendimiento

La IA no puede simular 1000 usuarios concurrentes. Para eso necesitas herramientas específicas (k6, Artillery).

## Patrón: Test-Driven con IA

1. Escribe el test primero (tú o con IA)
2. Pide a la IA que implemente la función
3. Ejecuta el test
4. Si falla, refina el prompt
5. Repite hasta pasar

## Ejemplo

```
// 1. Test primero
test('formato fecha DD/MM/YYYY', () => {
 expect(formatoFecha(new Date(2025, 0, 15))).toBe('15/01/2025');
});

// 2. Pides a la IA que implemente
// "Implementa formatoFecha que convierta Date a DD/MM/YYYY"

// 3. Ejecutas y verificas
```

## Mini-chequeo

► ¿Por qué testear código generado por IA?

► ¿Qué tipo de tests genera mejor la IA?

► ¿Qué es el patrón Test-Driven con IA?



## ✓ Resumen en 3 frases

1. El código generado por IA **necesita tests** porque puede tener bugs lógicos que no son evidentes.
2. Usa la IA para **generar tests unitarios y de validación**, pero diseña la estrategia de testing tú mismo.
3. Aplica el patrón **Test-Driven con IA**: test primero, implementación con IA, verificación y refinamiento.

 Volver al [índice de la unidad](#) · Anterior: [03 — GitHub Copilot](#) · Siguiente: [05 — Seguridad con IA](#)

## 05 — Seguridad con IA

 Estás en:  UD 2.7 · IA como Copiloto → 05 · Seguridad con IA

 *La IA genera código rápido, pero no genera código seguro por defecto*

La IA aprendió de millones de repositorios públicos, incluyendo muchos con malas prácticas de seguridad. Cuando genera código, puede repetir vulnerabilidades que vio en su entrenamiento. Tu trabajo es detectarlas antes de que lleguen a producción.

## La idea en una frase

**La IA no distingue entre código seguro e inseguro: asume que todo lo que genera es correcto. Tú eres el único filtro de seguridad.**

# Vulnerabilidades comunes en código de IA

## 1. SQL Injection

La IA a veces genera concatenación de strings en queries SQL:

```
// ❌ VULNERABLE - La IA puede generar esto
const query = `SELECT * FROM users WHERE id = ${userId}`;
db.query(query);

// ✅ SEGURO - Siempre usa parámetros
const query = 'SELECT * FROM users WHERE id = ?';
db.query(query, [userId]);
```

## 2. XSS (Cross-Site Scripting)

```
// ❌ VULNERABLE
element.innerHTML = userInput;

// ✅ SEGURO
element.textContent = userInput;
// O usa frameworks que escapan automáticamente (Vue, React)
```

## 3. Secrets hardcodeados

```
// ❌ VULNERABLE - La IA puede generar esto
const API_KEY = 'sk-1234567890abcdef';

// ✅ SEGURO
const API_KEY = process.env.API_KEY;
```

## 4. Autenticación débil

```
// ❌ VULNERABLE
if (user.password === inputPassword) { ... }

// ✅ SEGURO
const isValid = await bcrypt.compare(inputPassword, user.passwordHash);
```

## 5. CORS demasiado permisivo

```
// ❌ VULNERABLE
app.use(cors({ origin: '*' }));

// ✅ SEGURO
app.use(cors({ origin: 'https://tu-dominio.com' }));
```

## Reglas de seguridad al usar IA

### 1. Nunca pegues secrets en prompts

- ❌ "API key: sk-1234... genera un endpoint que la use"
- ✅ "Genera un endpoint que lea la API\_KEY de variables de entorno"

### 2. Siempre pregunta por seguridad

Añade a tus prompts:

```
"Incluye manejo seguro de errores sin exponer información sensible"
```

### 3. Valida contra OWASP Top 10

Revisa que el código generado no tenga: - Inyección SQL - XSS - Configuración insegura - Autenticación rota - Exposición de datos

### 4. Usa dependencias de seguridad

```
{
 "dependencies": {
 "helmet": "^7.0.0",
 "express-rate-limit": "^7.0.0",
 "bcrypt": "^5.1.0"
 }
}
```

## Checklist de seguridad post-IA

- ☐ No hay secrets hardcodeados (usa <sup>599/743</sup> variables de entorno)

- ☐ Las queries SQL usan parámetros, no concatenación
- ☐ El HTML se escapa (usa `textContent` o frameworks)
- ☐ Las contraseñas se hashean con `bcrypt`
- ☐ CORS está configurado con orígenes específicos
- ☐ Los errores no exponen stack traces en producción
- ☐ Las dependencias no tienen CVEs conocidos ( `npm audit` )

---

## Aclaración: “¿La IA nunca genera código seguro?”

- ▶ 🤖 ¿Es tan peligroso usar IA para código de seguridad?

---

## Mini-chequeo

- ▶ ¿Cuáles son las 3 vulnerabilidades más comunes en código de IA?

- ▶ ¿Qué hago siempre antes de aceptar código de IA?

- ▶ ¿Qué comando verifica dependencias vulnerables?

---

## ✅ Resumen en 3 frases

1. La IA genera código con **vulnerabilidades comunes** como SQL injection, XSS y secrets hardcodeados.
2. Siempre verifica que no haya **credenciales hardcodeadas**, que se usen **parámetros en queries** y que el **input se escape**.
3. Usa `npm audit` y preguntas de seguridad en tus prompts para **reducir riesgos**.

## 06 — Ética de la IA

 Estás en:  UD 2.7 · IA como Copiloto → 06 · Ética de la IA

 *Poder usar la IA no significa que debas usarla para todo*

La IA es una herramienta poderosa, pero con poder viene responsabilidad. Usarla de forma ética significa ser honesto sobre su uso, respetar el trabajo de otros y entender el impacto de lo que generas.

### La idea en una frase

**Usa la IA como herramienta, no como sustituto de tu criterio y tu responsabilidad.**

## Principios éticos al usar IA

### 1. Transparencia

**Sé honesto sobre cuánto usaste la IA.**

En un proyecto académico: - Indica en la documentación qué partes usaron IA - El profesor valora que sepas usar herramientas, pero también que seas honesto - Si toda la app la generó la IA, ¿qué has aprendido tú?

En un entorno profesional: - Muchas empresas requieren declarar el uso de IA - El código generado por IA puede tener implicaciones legales (licencias)<sup>43</sup> Tu equipo necesita saber qué partes no fueron revisadas a fondo

## 2. Comprensión

**No uses código que no entiendas.**

---

La IA generó un componente de autenticación. Lo leí, entendí cómo funciona, modifiqué lo que no me gustaba y lo integré.

---

La IA generó un componente de autenticación. Lo copié, lo pegué, funciona. No sé cómo funciona por dentro.

## 3. Calidad

**No entregues código que no has verificado.**

- El código de IA puede tener bugs, vulnerabilidades y malas prácticas
- Tú eres el responsable de la calidad, no la IA
- Si lo entregas, debes poder explicarlo y mantenerlo

## 4. Licencias y derechos de autor

**Respetar las licencias del código generado.**

- La IA entrenó con código público, pero eso no significa que sea libre
- Algunos modelos generan código que viola licencias existentes
- Verifica que el código generado no esté copiado de un proyecto con licencia restrictiva

---

## Casos éticos reales

### Caso 1: El alumno que copió todo de IA

Un alumno generó toda su aplicación con IA, la entregó sin entenderla. En la defensa, el profesor preguntó: “¿Cómo funciona la autenticación?”. El alumno no pudo explicarlo. **Resultado:** o en la asignatura.

**Lección:** Saber usar IA es una habilidad. Pero si no entiendes lo que generas, no has aprendido.

## Caso 2: La empresa que usó código con licencia problemática

Una startup usó IA para generar código. Parte del código generado era idéntico a código de un proyecto con licencia GPL. La empresa tuvo que abrir su código fuente o enfrentar una demanda.

**Lección:** Verifica siempre las licencias del código generado.

## Caso 3: El desarrollador que automatizó la revisión de código

Un desarrollador usó IA para revisar pull requests de su equipo. La IA aprobó código con vulnerabilidades porque no tenía contexto del proyecto. **Resultado:** una brecha de seguridad en producción.

**Lección:** La IA ayuda, pero no sustituye la revisión humana en decisiones críticas.

---

## Lo que la IA NO debe hacer

### 1. Generar código discriminatorio

La IA puede generar sesgos de género, raza o edad en: - Sistemas de recomendación - Filtros de búsqueda - Algoritmos de puntuación

**Acción:** Revisa el código buscando sesgos y patrones discriminatorios.

### 2. Crear deepfakes o contenido engañoso

Usar IA para: - Generar imágenes falsas de personas - Crear textos que parezcan de alguien real - Suplantar identidades

**Acción:** Nunca uses IA para engañar o suplantar.

### 3. Automatizar despidos sin supervisión humana

Usar IA para decidir quién es despedido sin que un humano revise cada caso.

**Acción:** Las decisiones sobre personas siempre requieren supervisión humana.

# Guía de uso responsable

Situación	Uso ético	Uso no ético
Aprendizaje	Usar IA para aprender conceptos nuevos	Copiar sin entender
Proyecto académico	Declarar qué partes usaron IA	Entregar todo como propio
Código en producción	Revisar, testear y validar	Copiar sin probar
Documentación	Usar IA como borrador, luego revisar	Publicar sin leer
Decisiones de negocio	Decidir tú, usar IA como input	Dejar que la IA decida por ti

## Aclaración: “¿Es deshonesto usar IA en un proyecto?”

▶ 🤔 ¿Debería declarar que usé IA en mi proyecto?

## Mini-chequeo

▶ ¿Cuáles son los 4 principios éticos al usar IA?

▶ ¿Por qué no debes entregar código que no entiendes?

▶ ¿Qué dice la ética sobre usar IA para decidir sobre personas?

 **Resumen en 3 frases**



2. **Nunca entregues código que no entiendas:** la responsabilidad es tuya, no de la IA.
3. Verifica siempre las **licencias** y revisa el código buscando **sesgos y vulnerabilidades**.

 Volver al [índice de la unidad](#) · Anterior: [05 — Seguridad con IA](#) · Siguiente: [07 — Errores comunes](#)

## 07 — Errores comunes

 Estás en:  UD 2.7 · IA como Copiloto → 07 · Errores comunes

 *Los errores que comete todo el mundo (y cómo evitarlos)*

Todos cometen los mismos errores cuando empiezan a usar IA. Conocerlos de antemano te ahorrará horas de frustración. Son errores predecibles y evitables si sabes buscarlos.

### La idea en una frase

**Los errores más peligrosos con IA son los que no detectas: código que funciona pero que no entiendes.**

## Los 8 errores más comunes

### Error 1: Copiar sin leer

**El problema:** Copias el código de IA y lo pegas directamente.

**La consecuencia:** Tienes código que compila pero que puede tener bugs, vulnerabilidades o lógica incorrecta.

**Cómo evitarlo:** Lee cada línea antes de aceptar. Si no entiendes algo, pregunta a la IA que te lo explique.

---

## Error 2: Dependencias inventadas

**El problema:** La IA sugiere un paquete que no existe o está abandonado.

**Ejemplo:**

```
npm install super-ultimate-validator
Error: 404 Not Found
```

**Cómo evitarlo:** Verifica siempre en npm, PyPI o la documentación oficial que la dependencia existe y está activa.

---

## Error 3: Contexto perdido

**El problema:** Le pides a la IA que modifique una función, pero olvida el resto del archivo.

**La consecuencia:** La modificación rompe código existente que la IA no vio.

**Cómo evitarlo:** Incluye en el prompt el contexto relevante: “Ten en cuenta que esta función se usa en el componente X y espera un parámetro de tipo Y”.

---

## Error 4: Variables inventadas

**El problema:** La IA usa variables o funciones que no existen en tu código.

**Ejemplo:**

```
// La IA asume que existe una función validarPermisos
if (validarPermisos(usuario)) {
 mostrarDashboard();
}
// Pero validarPermisos no está definida en ningún lado
```

**Cómo evitarlo:** Verifica que todo lo que la IA referencia exista en tu código.

## Error 5: Lógica aparentemente correcta

**El problema:** El código parece correcto pero falla en edge cases.

**Ejemplo:**

```
function promedio(numeros) {
 return numeros.reduce((a, b) => a + b) / numeros.length;
}
// ¿Qué pasa con un array vacío? → NaN
// ¿Qué pasa con [1, 2, "3"]? → "123/3" (concatenación, no suma)
```

**Cómo evitarlo:** Piensa en edge cases: arrays vacíos, tipos mixtos, valores nulos, límites numéricos.

---

## Error 6: Soluciones genéricas

**El problema:** La IA da una solución genérica que no se adapta a tu caso específico.

**Ejemplo:** Le pides un sistema de autenticación y te da JWT básico, pero tu proyecto necesita OAuth con Google.

**Cómo evitarlo:** Sé específico en tus prompts: “Autenticación con OAuth 2.0 de Google, no JWT básico”.

---

## Error 7: No testear cambios

**El problema:** Aceptas cambios de IA sin ejecutar tests.

**La consecuencia:** Introduces bugs que no detectas hasta producción.

**Cómo evitarlo:** Ejecuta los tests después de cada cambio generado por IA. Si no hay tests, escríbelos primero.

---

## Error 8: Demasiada confianza

**El problema:** Confías en que la IA siempre tiene razón.

**La consecuencia:** No cuestionas decisiones de arquitectura, patrones o prácticas que pueden ser incorrectas.

**Cómo evitarlo:** Recuerda que la IA es una herramienta, no un experto. Siempre verifica con documentación oficial y buenas prácticas reconocidas.

## Tabla resumen de errores

#	Error	Consecuencia	Prevención
1	Copiar sin leer	Bugs y vulnerabilidades	Lee cada línea
2	Dependencias inventadas	npm install falla	Verifica en npm
3	Contexto perdido	Código roto	Incluye contexto en prompt
4	Variables inventadas	Runtime errors	Verifica que existan
5	Lógica incorrecta	Bugs en edge cases	Testea edge cases
6	Soluciones genéricas	No se adapta a tu caso	Sé específico
7	No testear	Bugs en producción	Ejecuta tests siempre
8	Demasiada confianza	Malas prácticas	Verifica con docs

## Mini-chequeo

- ¿Cuál es el error más peligroso al usar IA?
- ¿Cómo evitas el error de “contexto perdido”?
- ¿Qué hago si la IA usa una variable que no existe en mi código?

1. Los errores más comunes son **copiar sin leer**, **dependencias inventadas** y **contexto perdido**: todos evitables con atención.
2. Siempre **verifica** que lo que la IA genera existe, funciona y se adapta a tu caso específico.
3. **Ejecuta tests** después de cada cambio de IA y no confíes ciegamente: **tú eres el responsable del código**.

 Volver al [índice de la unidad](#) · Anterior: [o6 — Ética de la IA](#) · Siguiente: [o8 — Template de uso](#)

## o8 — Template de uso de IA

 Estás en:  UD 2.7 · IA como Copiloto → [o8](#) · Template de uso de IA

 *Documenta tu uso de IA para ser transparente y aprender*

Si usaste IA en tu proyecto, es bueno documentar qué, cómo y por qué. Esto te ayuda a ser transparente, a aprender de la experiencia y a mejorar tu uso de la herramienta en el futuro.

### La idea en una frase

**Documentar tu uso de IA demuestra madurez profesional y te ayuda a aprender de la experiencia.**

## 1. Resumen de uso

Aspecto	Detalle
Herramientas usadas	[GitHub Copilot, ChatGPT, Claude, etc.]
Porcentaje estimado de código con IA	[Ej: 30%]
Qué partes usaron IA	[Ej: Componentes CRUD, tests, formularios]
Qué partes fueron manuales	[Ej: Arquitectura, UX, seguridad]

## 2. Uso detallado por módulo

Módulo: [Nombre del módulo]

Qué se hizo con IA	Prompt utilizado	Resultado	Modificaciones manuales
[Ej: Componente de login]	[Prompt que usaste]	[Qué generó]	[Qué cambiaste]
[Ej: Tests unitarios]	[Prompt que usaste]	[Qué generó]	[Qué cambiaste]

## 3. Lecciones aprendidas

**Qué funcionó bien con IA:** - [Ej: Generar CRUDs rápidamente] - [Ej: Tests unitarios de funciones puras]

**Qué no funcionó:** - [Ej: Arquitectura del proyecto - la IA no entendía el contexto] - [Ej: Seguridad - generaba código vulnerable]

**Qué haría diferente la próxima vez:** - [Ej: Dar más contexto en los prompts] - [Ej: Testear antes de aceptar cada cambio]

## 4. Prompt bank (banco de prompts)

Guarda los prompts que mejor funcionaron para reusarlos:

```
Prompts efectivos

CRUD para Vue 3
"Crea un componente [Nombre] con <script setup>, Composition API, Tailwind CSS. Funciones: listar, crear, editar, eliminar. Sin dependencias externas."
```

```
Tests con Jest
"Genera tests unitarios para la función [nombre]:
- Happy path
- Edge cases (vacío, nulo, límites)
- Errores
Framework: Jest"
```

```
Refactorizar
"Refactoriza este código para:
- Extraer lógica a funciones separadas
- Usar async/await
- Añadir manejo de errores
- Explica cada cambio"
```

## 5. Métricas de productividad

Métrica	Sin IA	Con IA	Ahorro
Tiempo por componente	[Ej: 2h]	[Ej: 30min]	[75%]
Tests escritos	[Ej: 5/h]	[Ej: 20/h]	[75%]
Bugs encontrados post-deploy	[Ej: 3]	[Ej: 1]	[67%]

## Ejemplo relleno

### 1. Resumen de uso

Aspecto	Detalle
Herramientas usadas	GitHub Copilot, ChatGPT
Porcentaje estimado	40%
Partes con IA	CRUDs, tests, formularios, utilidades
Partes manuales	Arquitectura, autenticación, UX

## 2. Uso detallado

Qué	Prompt	Resultado	Cambios
TaskForm.vue	“Componente Vue 3 form con validación”	Formulario completo	Cambié validación de fecha
Tests de utils	“Tests para funciones de utilidad”	12 tests	Añadí 3 edge cases

## 3. Lecciones

**Bien:** CRUDs y tests se generan rápido. **Mal:** Arquitectura y seguridad necesitan supervisión humana. **Próxima vez:** Más contexto en prompts de arquitectura.

## Cuándo usar esta plantilla

Contexto	¿Es necesario?	Beneficio
Proyecto académico	Recomendado	Transparencia con el profesor
Proyecto personal	Opcional	Aprendizaje y mejora
Proyecto en empresa	Generalmente sí	Cumplimiento de políticas
Código abierto	Recomendado	Los contribuidores saben qué esperar

**Aclaración: “¿Es mucho trabajo documentar esto?”**



► 🤔 ¿Merece la pena documentar el uso de IA?

## Mini-chequeo

► ¿Qué secciones tiene la plantilla de uso de IA?

► ¿Por qué guardar un “prompt bank”?

► ¿Qué métricas puedes medir?

## ✅ Resumen en 3 frases

1. Documenta tu uso de IA con la plantilla: **resumen, uso detallado, lecciones, prompts y métricas**.
2. Guarda los **prompts efectivos** en un prompt bank para reusarlos en futuros proyectos.
3. La documentación demuestra **transparencia y madurez profesional**, y te ayuda a aprender de la experiencia.

 Volver al [índice de la unidad](#) · Anterior: [07 — Errores comunes](#) · Siguiente: [09 — Cierre](#)

## 09 — Cierre

 Estás en:  UD 2.7 · IA como Copiloto → 09 · Cierre

 *Ya sabes usar la IA como un profesional*

Has recorrido toda la unidad: desde qué hace bien y mal la IA hasta cómo documentar tu uso de forma ética. Ahora toca **verificar que lo entiendes** y **prepararte para el despliegue**.

## La idea en una frase

**La IA es una herramienta poderosa que usada bien te hace más rápido, y usada mal te crea problemas invisibles.**

Si recuerdas esto, has entendido la unidad.

## Mini-chequeo final

Responde estas preguntas antes de seguir:

### Preguntas de comprensión

► ¿Qué hace bien la IA en desarrollo?

► ¿Qué hace MAL la IA?

► ¿Cuál es la fórmula de un buen prompt?

► ¿Qué vulnerabilidades de seguridad debes buscar en código de IA?

► ¿Por qué documentar el uso de IA?

## Ejercicio práctico

En 10 minutos, completa la plantilla del punto 08 con tu uso de IA:

- 1. Lista las herramientas de IA que usaste
- 2. Estima el porcentaje de código generado con IA
- 3. Documenta 2-3 usos específicos con prompts
- 4. Anota qué funcionó y qué no
- 5. Guarda 2-3 prompts efectivos en tu prompt bank

Si consigues documentar tu uso de IA de forma clara → has entendido la unidad.

## Tabla resumen de la unidad

Concepto	Idea clave
Qué hace bien	Boilerplate, explicaciones, aprendizaje
Qué hace mal	Negocio, arquitectura, seguridad
Prompts	CONTEXTO + TAREA + RESTRICCIONES + EJEMPLO
Copilot	Sugerencias en tiempo real, úsalo con comentarios claros
Testing	Testea código de IA, usa IA para generar tests
Seguridad	Nunca hardcodees secrets, usa parámetros en queries
Ética	Sé transparente, entiende lo que generas, respeta licencias
Errores	Copiar sin leer es el error más peligroso
Template	Documenta tu uso para aprender y ser transparente

## Vocabulario rápido

Término	Definición
LLM	Large Language Model — modelo de IA que genera texto y código
Prompt	Instrucción que le das a la IA para obtener un resultado
Prompt engineering	Arte de escribir prompts efectivos
Copilot	Asistente de IA que sugiere código en tiempo real
Hallucination	Cuando la IA inventa datos, paquetes o funcionalidades que no existen
Boilerplate	Código repetitivo y estructurado que sigue un patrón
OWASP	Organización que lista las vulnerabilidades web más comunes
Deepfake	Contenido falso generado por IA que parece real

## Conexión con las siguientes unidades

Unidad	Qué aprenderás	Cómo se conecta
UD 2.7: Despliegue	Docker, CI/CD, producción	Usarás IA para generar Dockerfiles y workflows
UD 2.8: Monitorización	Logs, métricas, dashboards	Usarás IA para configurar logging
UD 2.9: Propuesta Final	Proyecto Full-Stack completo	Documentarás el uso de IA en tu proyecto final

## Consejo para el siguiente punto

Si vas a desplegar tu proyecto:

1. Usa la IA para generar el **Dockerfile** (punto 02 de UD 2.7)
2. Pide ayuda para configurar **GitHub Actions** (punto 03 de UD 2.7)
3. Recuerda: **revisa siempre** lo que la IA genere de seguridad

## ✓ Resumen en 3 frases

1. La IA es excelente para **código repetitivo y aprendizaje**, pero no entiende tu negocio ni tu contexto.
2. Usa la fórmula **CONTEXTO + TAREA + RESTRICCIONES + EJEMPLO** y siempre **revisa, prueba y valida** el código generado.
3. Documenta tu uso de IA de forma **transparente y ética**: demuestra madurez profesional y te ayuda a aprender.

 Volver al [índice de la unidad](#) · Anterior: [08 — Template de uso](#) · Siguiente: [UD 2.8 — Despliegue](#)

## 01 — Qué es el despliegue

 **Estás en:**  **UD 2.8 · Despliegue** → 01 · Qué es el despliegue

 *Desplegar es más que subir archivos a un servidor*

Cuando tu aplicación funciona en localhost, está en tu burbuja. El despliegue es sacarla de ahí y ponerla donde cualquiera pueda usarla. Pero no es solo “copiar archivos”: implica configuración, seguridad, dominio y mantenimiento.

## La idea en una frase

Desplegar es hacer tu aplicación accesible desde internet con dominio, HTTPS y disponibilidad garantizada.

## Qué cambia de localhost a producción

Aspecto	localhost	Producción
URL	localhost:3000	tu-app.com
Acceso	Solo tu ordenador	Cualquiera con internet
HTTPS	No necesario	Obligatorio
Puerto	3000 (libre)	80/443 (estándar)
Base de datos	Local	Remota
Variables de entorno	En tu máquina	En el servidor
Logs	Terminal	Archivos o servicio externo
Errores	Los ves tú	Los ve el usuario

## Componentes de un despliegue

### 1. Servidor (hosting)

El ordenador que ejecuta tu aplicación 24/7:

- **Gratuito:** GitHub Pages, Netlify, Vercel (solo frontend)
- **Freemium:** Render, Railway, Fly.io (frontend + backend)
- **De pago:** AWS, Google Cloud, Azure (escala completa)

### 2. Dominio

La dirección que los usuarios escriben en el navegador:

- **Gratis:** tu-app.netlify.app, tu-usuario.github.io

- **Barato:** .com (~10€/año), .dev (~12€/año)
- **Propio:** Estableces la marca de tu proyecto

### 3. Certificado HTTPS

Cifra la comunicación entre el navegador y tu servidor:

- **Let's Encrypt:** gratuito y automático
- **Cloudflare:** gratuito con CDN
- **Incluido:** Vercel y Netlify lo dan automáticamente

### 4. Base de datos remota

Si tu app usa base de datos, necesitas una versión remota:

- **Supabase:** PostgreSQL gratuito (500MB)
- **PlanetScale:** MySQL gratuito (5GB)
- **MongoDB Atlas:** MongoDB gratuito (512MB)
- **Firebase:** Firestore gratuito (1GB)

## El flujo completo de despliegue

```
Desarrollo → Tests → Build → Deploy → Verificación → Monitoreo
 | | | | | |
 | Code | Test | Build | Upload | Smoke | Watch
 |_____|_____|_____|_____|_____|_____|
```

1. **Desarrollo:** escribes código en tu máquina
2. **Tests:** verificas que todo funciona
3. **Build:** compilas/optimizas el código
4. **Deploy:** subes el código al servidor
5. **Verificación:** compruebas que funciona en producción
6. **Monitoreo:** vigilas que siga funcionando

## Aclaración: “¿Necesito desplegar mi proyecto?”

## Mini-chequeo

► ¿Cuáles son los 4 componentes de un despliegue?

► ¿Qué cambia más de localhost a producción?

► ¿Qué plataforma de hosting es gratuita para frontend + backend?

### ✓ Resumen en 3 frases

1. Desplegar es hacer tu aplicación **accesible desde internet** con dominio, HTTPS y disponibilidad garantizada.
2. Los 4 componentes son **servidor, dominio, HTTPS y base de datos remota** si la necesitas.
3. El flujo es **desarrollo → tests → build → deploy → verificación → monitoreo**.

 Volver al [índice de la unidad](#) · Siguiente: [02 — Docker](#)

## 02 — Docker

 Estás en:  **UD 2.8 · Despliegue** → 02 · Docker

 *Docker resuelve el problema de 'en mi máquina funciona'*



“En mi máquina funciona” es la frase más peligrosa del desarrollo. Docker la elimina: empaquetas tu aplicación con todo lo que necesita para funcionar, y funciona igual en cualquier ordenador, servidor o plataforma en el mundo.

## La idea en una frase

**Docker empaqueta tu app en un contenedor que funciona igual en todas partes: en tu máquina, en el servidor del profesor o en la nube.**

## Qué es Docker

Docker es una herramienta que crea **contenedores**: cajas aisladas que contienen tu aplicación y todo lo que necesita para funcionar (sistema operativo, dependencias, configuración).

### Imagen vs Contenedor

Concepto	Qué es	Ejemplo
<b>Imagen</b>	La receta (plantilla)	<code>node:20-alpine</code>
<b>Contenedor</b>	La ejecución de la receta	Un contenedor Node.js corriendo

## Dockerfile: el corazón de Docker

Un Dockerfile es un archivo de texto con instrucciones para construir la imagen.

### Ejemplo: Dockerfile para una app Node.js

```
1. Imagen base
FROM node:20-alpine

2. Directorio de trabajo
WORKDIR /app

3. Copiar dependencias
COPY package*.json ./

4. Instalar dependencias
RUN npm ci --only=production

5. Copiar código fuente
COPY . .

6. Exponer puerto
EXPOSE 3000

7. Comando de inicio
CMD ["node", "src/server.js"]
```

## Línea por línea

```
FROM node:20-alpine # Usa Node.js 20 con Alpine Linux (ligero)
WORKDIR /app # Crea la carpeta /app dentro del contenedor
COPY package*.json ./ # Copia package.json y package-lock.json
RUN npm ci --only=production # Instala solo dependencias de producción
COPY . . # Copia todo el código fuente
EXPOSE 3000 # Documenta que el contenedor usa el puerto 3000
CMD ["node", "src/server.js"] # Comando que se ejecuta al iniciar el contenedor
```

---

## Docker Compose: multi-servicio

Si tu app tiene frontend + backend + base de datos, usas docker-compose.yml:

```
version: '3.8'

services:
 frontend:
 build: ./frontend
 ports:
 - "3000:3000"
 depends_on:
 - backend

 backend:
 build: ./backend
 ports:
 - "4000:4000"
 environment:
 - DATABASE_URL=postgres://user:pass@db:5432/mydb
 depends_on:
 - db

 db:
 image: postgres:16-alpine
 environment:
 - POSTGRES_USER=user
 - POSTGRES_PASSWORD=pass
 - POSTGRES_DB=mydb
 volumes:
 - pgdata:/var/lib/postgresql/data

volumes:
 pgdata:
```

## Comandos esenciales de Docker

Comando	Qué hace
<code>docker build -t mi-app .</code>	Construye la imagen desde el Dockerfile
<code>docker run -p 3000:3000 mi-app</code>	Ejecuta un contenedor
<code>docker ps</code>	Lista contenedores en ejecución
<code>docker stop &lt;id&gt;</code>	Detiene un contenedor
<code>docker-compose up -d</code>	Levanta todos los servicios
<code>docker-compose down</code>	Detiene todos los servicios
<code>docker logs &lt;id&gt;</code>	Muestra los logs del contenedor

# Variables de entorno: configuración que cambia entre entornos

Todo lo que cambia entre desarrollo, pruebas y producción (credenciales, URLs, puertos) debe ser una **variable de entorno**, nunca un valor fijo en el código.

```
// ❌ MAL: hardcodeado
const DB_URL = "mongodb://localhost:27017/mi_app";
const API_KEY = "sk-1234567890abcdef";

// ✅ BIEN: variables de entorno
const DB_URL = process.env.DB_URL || "mongodb://localhost:27017/mi_app";
const API_KEY = process.env.API_KEY;
```

## El archivo `.env` y `.env.example`

En desarrollo se usa el archivo `.env`; en producción, las variables se configuran en la plataforma.

```
.env (desarrollo local - NUNCA subir al repo)
DB_HOST=localhost
DB_PASSWORD=mipassword
NODE_ENV=development
PORT=3000
```

```
.env.example (si se sube al repo - valores vacíos)
DB_HOST= # Host de la base de datos
DB_PASSWORD= # Contraseña (nunca valores reales)
NODE_ENV= # development o production
PORT= # Puerto del servidor
```

**Reglas de oro:** 1. **Nunca subas** `.env` al repositorio. Añádalo a `.gitignore`. 2. Crea `.env.example` con las variables sin valores reales, para que quien clone el repo sepa qué configurar. 3. Usa **valores diferentes** para desarrollo y producción. 4. No pongas **valores por defecto peligrosos** en los secrets: mejor fallar rápido si falta una variable que arrancar con configuración incompleta.

```
.gitignore
.env
.env.*
!.env.example
```

## En Docker Compose

En Compose, las credenciales no van directamente en el YAML. Se referencian desde `.env`:

```
backend:
 environment:
 - DB_HOST=${DB_HOST}
 - DB_PASSWORD=${DB_PASSWORD}
```

 **Vocabulario:** un **secret** es una variable de entorno sensible (credenciales, claves API). Un **.env.example** es la plantilla documentada de variables sin valores reales.

## Dockerfile en producción: endurecer la imagen

Un Dockerfile no es solo “que funcione”, es “que funcione bien”. Cuatro reglas para producción:

### 1. Usa una base ligera (alpine/slim)

```
❌ MAL: imagen completa (~900 MB)
FROM node:20

✅ BIEN: variante alpine (~180 MB) o slim (~250 MB)
FROM node:20-alpine
```

### 2. Usuario no-root

Por defecto los contenedores ejecutan como root. Si alguien explota tu app dentro del contenedor, tiene acceso root.

```
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser
```

### 3. CMD en exec-form (array)

```
✅ BIEN: exec-form. Node es PID 1 y recibe señales de terminación
CMD ["node", "src/server.js"]

❌ MAL: shell-form. El shell es PID 1; Docker no puede parar limpiamente
CMD node src/server.js
```

### 4. Orden del COPY para aprovechar la caché

Copia `package*.json` primero e instala dependencias antes de copiar el resto. Si solo cambia el código pero no las dependencias, Docker reutiliza la capa de instalación.

```
✅ Copia dependencias primero → caché aprovechada
COPY package*.json ./
RUN npm ci --only=production
COPY . .
```

### Errores comunes

Error	Problema
<code>COPY . .</code> + <code>npm install</code>	Copia dependencias de tu máquina. Instala dentro del contenedor
Olvidar <code>.dockerignore</code>	<code>.git</code> , <code>node_modules</code> , <code>.env</code> se cuelan en la imagen y la inflan
<code>FROM node:20</code> completo	Imagen pesada (~900 MB) e insegura
Usuario root	Superficie de ataque si explotan tu app
Sin multi-stage	La imagen final incluye código fuente, tests y tools de desarrollo

**Regla de oro:** una imagen ligera sube más rápido al registro, arranca más rápido y tiene menos superficie de ataque.

### Multi-stage build: optimizar imágenes

```
Etapa 1: construir
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

Etapa 2: producción (solo lo necesario)
FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

**Ventaja:** La imagen final es mucho más ligera porque no incluye herramientas de desarrollo.

## Comparativa: sin Docker vs con Docker

‘En mi máquina funciona’ - el servidor tiene Node 18, yo tengo Node 20. El profesor tiene Windows, yo tengo Mac. El despliegue falla.

El contenedor incluye Node 20, las dependencias exactas y la configuración. Funciona igual en cualquier máquina.

## .dockerignore

Crea un archivo `.dockerignore` para excluir archivos innecesarios:

```
node_modules
.git
.env
*.md
.vscode
coverage
dist
```

**Por qué:** Estos archivos no deben estar en la imagen porque: - `node_modules` se instala con `npm ci` - `.git` ocupa mucho espacio - `.env` tiene secretos

## Aclaración: “¿Necesito Docker para mi proyecto?”

► 🤔 ¿Siempre necesito Docker?

## Más información sobre Docker

Si quieres profundizar en Docker, tienes un curso completo gratuito en:

👉 **Curso de Introducción a Docker** — Conceptos, comandos, Dockerfile, Docker Compose y buenas prácticas explicadas paso a paso.

## Mini-chequeo

► ¿Qué es un Dockerfile?

► ¿Qué hace `docker-compose.yml`?

► ¿Qué es un multi-stage build?

► ¿Por qué se copia `package*.json` antes que el resto del código?



► ¿Por qué usar un usuario no-root en el Dockerfile?

► ¿Qué es `.env.example` y por qué se sube al repo?

## ✓ Resumen en 3 frases

1. Docker **empaqueta tu app en un contenedor** que funciona igual en cualquier entorno, eliminando “en mi máquina funciona”.
2. El **Dockerfile** define la imagen y **docker-compose.yml** orquesta múltiples servicios (frontend + backend + BD).
3. Usa **multi-stage builds** para reducir el tamaño, **variables de entorno** para la configuración y **usuario no-root + alpine** para endurecer la imagen en producción.

 Volver al [índice de la unidad](#) · Anterior: [01 — Qué es el despliegue](#) · Siguiente: [03 — CI/CD](#)

## 03 — CI/CD y despliegue

 Estás en:  **UD 2.8 · Despliegue** → [03 · CI/CD y despliegue](#)

 *Automatizar el despliegue es como tener un asistente que nunca se equivoca*

Cada vez que haces un push a main, ¿quieres que se ejecuten los tests, se construya la app y se despliegue automáticamente? Eso es CI/CD: automatizar todo el proceso de code → test → build → deploy.

**CI/CD automatiza el camino de tu código desde tu máquina hasta producción: tests, build y deploy sin intervención manual.**

## CI vs CD

Siglas	Significado	Qué hace
CI	Continuous Integration	Ejecuta tests automáticamente en cada push
CD	Continuous Delivery	Prepara el código para despliegue manual
CD	Continuous Deployment	Despliega automáticamente en cada push

## GitHub Actions: la herramienta gratuita

GitHub Actions es el sistema de CI/CD de GitHub. Es gratuito para repos públicos y tiene 2000 minutos/mes para privados.

### Estructura de un workflow

Los workflows se definen en `.github/workflows/` :

```
.github/workflows/deploy.yml
name: CI/CD Pipeline

on:
 push:
 branches: [main]
 pull_request:
 branches: [main]

jobs:
 test:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-node@v4
 with:
 node-version: 20
 - run: npm ci
 - run: npm test

 deploy:
 needs: test
 if: github.ref == 'refs/heads/main'
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - run: npm ci
 - run: npm run build
 - run: npm run deploy
```

---

## Workflow completo para Node.js

```
name: CI/CD Pipeline

on:
 push:
 branches: [main, develop]
 pull_request:
 branches: [main]

jobs:
 lint:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-node@v4
 with:
 node-version: 20
 - run: npm ci
 - run: npm run lint

 test:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-node@v4
 with:
 node-version: 20
 - run: npm ci
 - run: npm test

 build:
 needs: [lint, test]
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-node@v4
 with:
 node-version: 20
 - run: npm ci
 - run: npm run build
 - uses: actions/upload-artifact@v4
 with:
 name: dist
 path: dist/

 deploy:
 needs: build
 if: github.ref == 'refs/heads/main'
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: actions/download-artifact@v4
 with:
 name: dist
 path: dist/
 - run: npm run deploy
 env:
 DEPLOY_TOKEN: ${ secrets.DEPLOY_TOKEN }
```

Nunca guardes tokens o contraseñas en el código. Usa GitHub Secrets:

1. Ve a tu repo → Settings → Secrets and variables → Actions
2. Crea un secret (ej: `DEPLOY_TOKEN` )
3. En el workflow, accede con `${{ secrets.DEPLOY_TOKEN }}`

---

## Workflow para desplegar en Vercel

```
name: Deploy to Vercel

on:
 push:
 branches: [main]

jobs:
 deploy:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: amondnet/vercel-action@v25
 with:
 vercel-token: ${ secrets.VERCEL_TOKEN }
 vercel-org-id: ${ secrets.VERCEL_ORG_ID }
 vercel-project-id: ${ secrets.VERCEL_PROJECT_ID }
 vercel-args: '--prod'
```

---

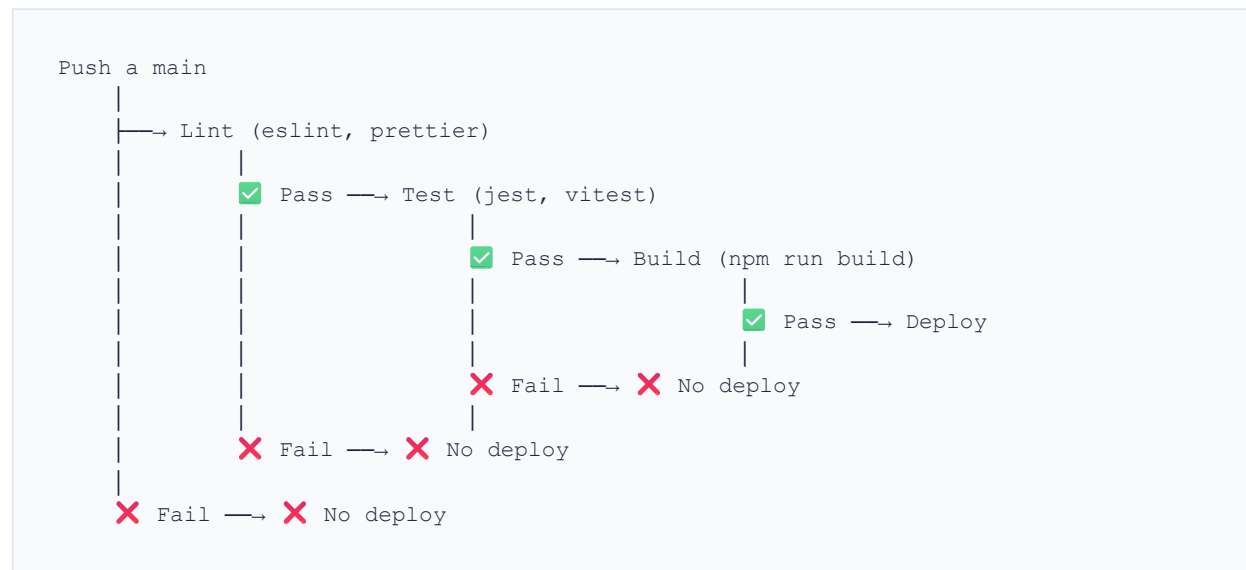
## Workflow para desplegar en Netlify

```
name: Deploy to Netlify

on:
 push:
 branches: [main]

jobs:
 deploy:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - run: npm ci && npm run build
 - uses: nwtgck/actions-netlify@v3
 with:
 publish-dir: './dist'
 production-deploy: true
 netlify-auth-token: ${ secrets.NETLIFY_AUTH_TOKEN }
 netlify-site-id: ${ secrets.NETLIFY_SITE_ID }
```

## Flujo visual del CI/CD



## Aclaración: “¿CI/CD es complicado?”

- 🤔 ¿Merece la pena configurar CI/CD para un proyecto pequeño?

## Mini-chequeo

- ¿Qué significan CI y CD?

- ¿Dónde se definen los workflows de GitHub Actions?

- ¿Por qué usar GitHub Secrets?

1. CI/CD **automatiza** el proceso de test → build → deploy cada vez que haces push a main.
2. GitHub Actions es **gratuito** y se configura con archivos YAML en `.github/workflows/`.
3. Usa **GitHub Secrets** para tokens y credenciales, nunca los guardes en el código.

 Volver al [índice de la unidad](#) · Anterior: [02 — Docker](#) · Siguiente: [04 — Plataformas](#)

## 04 — Plataformas de despliegue

 **Estás en:**  **UD 2.8 · Despliegue** → [04 · Plataformas de despliegue](#)

 *No todas las plataformas sirven para todo: elige la adecuada*

Hay muchas opciones para desplegar, pero no todas sirven para lo mismo. GitHub Pages es genial para sitios estáticos pero no para backends. Vercel es excelente para frontend pero limitado para Node.js pesado. Elegir bien te ahorra problemas.

### La idea en una frase

**Elige la plataforma según lo que necesitas: solo frontend, frontend + backend, o escalabilidad completa.**

## Comparativa de plataformas

Plataforma	Frontend	Backend	Base de datos	Gratis	Ideal para
GitHub Pages	✓	✗	✗	✓	Sockets estáticos, docs
Netlify	✓	⚠ Functions	✗	✓ (100GB)	Frontend + serverless
Vercel	✓	⚠ Serverless	✗	✓ (100GB)	Next.js, frontend moderno
Render	✓	✓	✓	✓ (750h)	Full-Stack completo
Railway	✓	✓	✓	✓ (5€)	Full-Stack, rápido
Fly.io	✓	✓	✓	✓ (3VMs)	Docker, escalabilidad

## GitHub Pages

**Ideal para:** Sitios estáticos, documentación, portfolios.

### Cómo funciona

1. Tu repo tiene una rama `main` o una carpeta `docs/`
2. GitHub Actions construye el sitio
3. Se sirve en `tu-usuario.github.io/tu-repo`

### Limitaciones

- Solo archivos estáticos (HTML, CSS, JS)
- No hay backend
- No hay base de datos
- Max 1GB por repo

### Ejemplo: deploy con Astro



```
.github/workflows/deploy.yml
name: Deploy to GitHub Pages

on:
 push:
 branches: [main]

jobs:
 build:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-node@v4
 with:
 node-version: 20
 - run: npm ci
 - run: npm run build
 - uses: actions/upload-pages-artifact@v3
 with:
 path: dist/
 deploy:
 needs: build
 runs-on: ubuntu-latest
 permissions:
 pages: write
 id-token: write
 environment:
 name: github-pages
 steps:
 - uses: actions/deploy-pages@v4
```

---

## Netlify

**Ideal para:** Frontend + functions serverless.

### Ventajas

- Deploy automático desde Git
- Formularios integrados
- Serverless functions (JavaScript)
- Headers de seguridad automáticos

### Limitaciones

- Functions tienen 10s de timeout (gratis)
- No hay base de datos incluida
- Pago para uso intensivo

```
[build]
 command = "npm run build"
 publish = "dist"

[[redirects]]
 from = "/api/*"
 to = "/.netlify/functions/:splat"
 status = 200
```

---

## Vercel

**Ideal para:** Frontend moderno, Next.js, frameworks de React/Vue.

### Ventajas

- Optimizado para frameworks modernos
- Preview deployments en cada PR
- Analytics integrado
- Edge functions

### Limitaciones

- Backend limitado (serverless functions)
- No hay contenedores Docker
- Pago para uso intensivo

### vercel.json

```
{
 "buildCommand": "npm run build",
 "outputDirectory": "dist",
 "framework": "vite",
 "rewrites": [
 { "source": "/(.*)", "destination": "/index.html" }
]
}
```

---

## Render

**Ideal para:** Full-Stack completo con base de datos.

## Ventajas

- Backend + frontend + base de datos
- Docker support
- Deploy automático
- SSL gratuito

## Limitaciones

- 750 horas/mes gratis (se agota)
  - Pago para uso 24/7
  - Build veces lentos en el plan gratuito
- 

## Railway

**Ideal para:** Full-Stack rápido, prototipos.

## Ventajas

- Deploy en minutos
- Base de datos incluida
- Variables de entorno fáciles
- Pricing predecible (5€/mes)

## Limitaciones

- Solo 5€ de crédito gratis
  - No hay plan gratuito permanente
- 

## VPS + Nginx: la alternativa con control total

Las plataformas managed son lo más cómodo, pero existe una alternativa clásica: un **VPS** (máquina virtual en la nube con acceso root). Tienes control total, pero también toda la responsabilidad.

## Cuándo elegir VPS

- Quieres **aprender administración de servidores** (muy valorado en entrevistas)

- Necesitas **control total**: WebSocket, cron jobs, puertos concretos
- Tu proyecto tiene requisitos que las plataformas managed no cubren

## Flujo básico

```
1. Conectar al servidor
ssh root@tu-servidor.com

2. Actualizar el sistema
apt update && apt upgrade -y

3. Instalar Docker
curl -fsSL https://get.docker.com | sh

4. Clonar y configurar
git clone https://github.com/tu-usuario/tu-proyecto.git
cd tu-proyecto
cp .env.example .env
nano .env

5. Arrancar la app
docker-compose up -d
```

## Nginx como reverse proxy

Tu app corre en el puerto 3000 dentro del contenedor, pero el usuario debe entrar por el puerto 80/443 con un dominio. **Nginx** redirige las peticiones del puerto 80 a tu contenedor:

```
/etc/nginx/sites-available/tu-app
server {
 listen 80;
 server_name tu-dominio.com;

 location / {
 proxy_pass http://localhost:3000;
 proxy_http_version 1.1;
 proxy_set_header Upgrade $http_upgrade;
 proxy_set_header Host $host;
 proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
 }
}
```

```
Activar la configuración
ln -s /etc/nginx/sites-available/tu-app /etc/nginx/sites-enabled/
nginx -t
systemctl restart nginx
```

►  ¿Cuál es la diferencia entre plataforma managed y VPS?

# Auto-deploy: despliega con cada push

Casi todas las plataformas ofrecen **auto-deploy**: cada push a `main` despliega la nueva versión automáticamente.

- **Vercel / Netlify**: conectas el repo; cada push a `main` despliega y cada PR crea una preview deployment
- **GitHub Pages**: usas GitHub Actions; cada push a `main` ejecuta el workflow
- **Render / Railway**: activas “Auto Deploy” en los settings del servicio

Con un **VPS**, el auto-deploy también es posible: en el servidor puedes añadir un **webhook** que haga `git pull && docker-compose up -d --build` cuando llegue el push. Así, aunque uses un VPS, no tienes que entrar a mano en cada despliegue.

## Comparativa: ¿Cuál elijo?

GitHub Pages (gratis, simple) o Vercel (mejor DX, preview deployments)

Netlify (serverless functions) o Vercel (edge functions)

Render (gratis con limitaciones) o Railway (5€/mes, más rápido)

# Aclaración: “¿Qué pasa si me quedo sin espacio gratis?”

- ▶ 🤔 ¿Las plataformas gratuitas son suficientes para mi proyecto?

## Mini-chequeo

- ▶ ¿Qué plataforma es mejor para un sitio estático?

- ▶ ¿Qué plataforma sirve para Full-Stack con base de datos?

- ▶ ¿Qué hace Vercel que no hace GitHub Pages?

- ▶ ¿Qué es un VPS y cuándo conviene?

- ▶ ¿Para qué sirve Nginx en un VPS?

## ✅ Resumen en 3 frases

1. **GitHub Pages** es ideal para sitios estáticos (gratis, simple). **Vercel** para frontend moderno con functions.
2. **Render** y **Railway** sirven para Full-Stack con backend y base de datos. **Un VPS + Nginx** da control total cuando necesitas administrar el servidor tú mismo.
3. Elige según tus necesidades: **solo frontend** → **Pages/Vercel**; **Full-Stack** → **Render/Railway**; **control total** → **VPS**.

## 05 — Zero-downtime deployments

 **Estás en:**  **UD 2.8 · Despliegue** → 05 · Zero-downtime deployments

 *El despliegue perfecto es el que el usuario no nota*

Imagina que estás usando una web y, de repente, aparece un error 502 o la página no carga. Eso es downtime: el tiempo en que tu aplicación no está disponible. Zero-downtime significa que el usuario nunca ve ese error, incluso durante un despliegue.

### La idea en una frase

**Zero-downtime es desplegar la nueva versión sin que el usuario note ningún corte de servicio.**

## Por qué hay downtime

Durante un despliegue tradicional:

- |                               |                                                                                                          |
|-------------------------------|----------------------------------------------------------------------------------------------------------|
| 1. Paras el servidor antiguo  | →  La app no responde |
| 2. Subes el código nuevo      | →  La app no responde |
| 3. Instalas dependencias      | →  La app no responde |
| 4. Arrancas el servidor nuevo | →  La app vuelve      |

El usuario ve un error durante los pasos 1-3. Con zero-downtime, **nunca** deja de responder.

## Estrategias de zero-downtime

### 1. Rolling update (actualización gradual)

El servidor ejecuta la versión antigua y la nueva simultáneamente:

```
Estado 1: [v1] [v1] [v1] [v1] → Todo tráfico a v1
Estado 2: [v1] [v1] [v1] [v2] → 75% v1, 25% v2
Estado 3: [v1] [v1] [v2] [v2] → 50% v1, 50% v2
Estado 4: [v1] [v2] [v2] [v2] → 25% v1, 75% v2
Estado 5: [v2] [v2] [v2] [v2] → Todo tráfico a v2
```

**Ventaja:** Si la v2 tiene un bug, solo afecta a una parte del tráfico.

## 2. Blue-green deployment (despliegue azul-verde)

Tienes dos entornos idénticos: uno activo (azul) y uno en espera (verde):

```
1. Verde tiene la v2 lista
2. Cambias el tráfico de azul a verde
3. Si hay problemas, vuelves a azul
4. Si todo va bien, eliminas azul
```

**Ventaja:** Rollback instantáneo si algo falla.

## 3. Canary deployment (despliegue canario)

Envías un porcentaje pequeño de tráfico a la nueva versión:

```
1. 5% del tráfico va a v2 (canario)
2. Si no hay errores, subes a 25%
3. Si no hay errores, subes a 100%
4. Si hay errores, vuelves a v1
```

**Ventaja:** Detectas problemas antes de afectar a todos los usuarios.

---

## Cómo funciona en la práctica

### Con Docker + nginx



```
upstream backend {
 server app-v1:3000;
 server app-v2:3000;
}

server {
 listen 80;

 location / {
 proxy_pass http://backend;
 }
}
```

nginx distribuye el tráfico entre v1 y v2 automáticamente.

## Con Vercel/Netlify

Estas plataformas hacen zero-downtime automáticamente: 1. Despliegan la nueva versión en un entorno aislado 2. Verifican que funciona 3. Cambian el tráfico a la nueva versión 4. Eliminan la versión antigua

## Con Kubernetes

```
apiVersion: apps/v1
kind: Deployment
spec:
 replicas: 3
 strategy:
 type: RollingUpdate
 rollingUpdate:
 maxSurge: 1 # Máximo 1 pod adicional
 maxUnavailable: 0 # Nunca menos pods de los deseados
```

## ¿Cuándo necesitas zero-downtime?

Situación	¿Necesario?	Prioridad
Proyecto de clase	No	Baja
Portfolio personal	No	Baja
App con pocos usuarios	Recomendado	Media
App con muchos usuarios	Obligatorio	Alta
E-commerce / pagos	Obligatorio	Crítica

## Aclaración: “¿Mi proyecto necesita zero-downtime?”

- ▶ 🤔 ¿Para un proyecto de clase necesito zero-downtime?

## Mini-chequeo

- ▶ ¿Qué es zero-downtime?

- ▶ ¿Cuáles son las 3 estrategias principales?

- ▶ ¿Qué plataformas hacen zero-downtime automáticamente?

### ✅ Resumen en 3 frases

1. Zero-downtime significa que **el usuario nunca ve errores** durante un despliegue.
2. Las 3 estrategias son **rolling update** (gradual), **blue-green** (dos entornos) y **canary** (porcentaje de tráfico).
3. Plataformas como **Vercel** y **Netlify** lo hacen automáticamente; para más control usa Docker + nginx.

 [Volver al índice de la unidad](#) · Anterior: [04 — Plataformas](#) · Siguiente: [06 — Checklist](#)

📖 *Un despliegue sin checklist es una apuesta*

Antes de desplegar, necesitas verificar que todo está listo. Un checklist te evita olvidar algo importante: variables de entorno, dependencias, configuración de HTTPS o puertos. Imprime esta lista y marca cada paso antes de hacer push.

## 📖 La idea en una frase

**Un checklist de despliegue te garantiza que no olvidas nada crítico antes de poner tu app en producción.**

## Checklist pre-despliegue

### 1. Configuración del proyecto

- ☐ `package.json` tiene scripts de build y start correctos
- ☐ Las dependencias están en `dependencies`, no en `devDependencies` (las que necesitas en producción)
- ☐ `package-lock.json` o `yarn.lock` está commiteado
- ☐ No hay dependencias con versiones `latest` o `*`

### 2. Variables de entorno

- ☐ Todas las variables de entorno están documentadas en `.env.example`
- ☐ No hay `.env` en el repositorio (está en `.gitignore`)
- ☐ Las variables de producción están configuradas en la plataforma
- ☐ No hay secrets hardcodeados en el código

### 3. Seguridad

- ☐ HTTPS está habilitado
- ☐ No hay CORS con `origin: '*'` en producción

- ☐ Los errores no exponen stack traces
- ☐ Las dependencias no tienen CVEs ( `npm audit` )
- ☐ No hay archivos sensibles en el repo ( `.env` , `*.key` , `*.pem` )

## 4. Build y optimización

- ☐ El build funciona sin errores: `npm run build`
- ☐ El build es reproducible (mismo resultado en otra máquina)
- ☐ Los assets están optimizados (imágenes comprimidas, CSS minificado)
- ☐ No hay archivos innecesarios en el build ( `node_modules` , `.git` , `src` )

## 5. Tests

- ☐ Todos los tests pasan: `npm test`
- ☐ La cobertura es aceptable (>60%)
- ☐ No hay tests pendientes o skipped sin justificación

## 6. Documentación

- ☐ README tiene instrucciones de instalación claras
- ☐ README explica cómo ejecutar en desarrollo y producción
- ☐ La documentación de la API está actualizada (si aplica)

## 7. Docker (si aplica)

- ☐ Dockerfile funciona: `docker build -t mi-app .`
- ☐ docker-compose.yml funciona: `docker-compose up`
- ☐ `.dockerignore` excluye archivos innecesarios
- ☐ Las imágenes son ligeras (multi-stage build)

## 8. CI/CD

- ☐ El workflow de GitHub Actions funciona
- ☐ Los tests se ejecutan en cada push
- ☐ El deploy se ejecuta solo en `main`
- ☐ Los secrets están configurados en GitHub

## 9. Dominio y DNS

- ☐ El dominio apunta al servidor correcto
- ☐ El certificado SSL funciona
- ☐ La redirección HTTP → HTTPS funciona <sup>648/743</sup>

- ☐ Los subdominios están configurados (si aplica)

## 10. Verificación post-despliegue

- ☐ La URL carga correctamente
  - ☐ El login funciona
  - ☐ Las operaciones CRUD principales funcionan
  - ☐ Los formularios envían datos
  - ☐ Los errores se muestran correctamente (no 500)
  - ☐ Los logs se generan correctamente
- 

## Checklist para el día de la presentación

- ☐ La URL está preparada y accesible
  - ☐ Los datos de demo están cargados
  - ☐ El HTTPS funciona (candado verde en el navegador)
  - ☐ Los tiempos de carga son aceptables (<3s)
  - ☐ La app funciona en el navegador del tribunal
  - ☐ Tienes un plan B (código local) si falla internet
- 

## Template de checklist

Copia este checklist en tu documentación:

```
Checklist de despliegue

Pre-despliegue
- [] Build funciona: `npm run build`
- [] Tests pasan: `npm test`
- [] Variables de entorno configuradas
- [] No hay secrets hardcodeados
- [] HTTPS habilitado
- [] `npm audit` sin vulnerabilities críticas

Deploy
- [] Push a main ejecuta CI/CD
- [] Deploy completado sin errores
- [] URL carga correctamente

Post-despliegue
- [] Funcionalidades principales verificadas
- [] Logs funcionando
- [] Rendimiento aceptable
```

---

## Mini-chequeo

► ¿Cuántas secciones tiene el checklist?

► ¿Qué debes verificar después de desplegar?

► ¿Qué hacer el día de la presentación?

---

## ✓ Resumen en 3 frases

1. El checklist verifica **configuración, seguridad, build, tests y documentación** antes de desplegar.
2. Después del despliegue, **verifica que todo funciona**: URL, login, CRUDs, formularios y logs.
3. El día de la presentación, **prepárate para el peor escenario**: URL lista, datos de demo y plan B.

## 07 — Errores comunes de despliegue

 **Estás en:**  **UD 2.8 • Despliegue** → 07 • Errores comunes de despliegue

 *Los errores de despliegue son predecibles y evitables*

Casi todo el mundo comete los mismos errores la primera vez que despliega. Conocerlos de antemano te ahorra horas de frustración. Son errores que suenan complicados pero tienen soluciones simples.

### La idea en una frase

**El 90% de los errores de despliegue se resuelven con: variables de entorno, dependencias y configuración de puertos.**

## Los 10 errores más comunes

### Error 1: Variables de entorno no configuradas

**Síntoma:** La app falla con “Cannot read property of undefined” o “DATABASE\_URL not set”.

**Causa:** Las variables de entorno que tienes en `.env` local no están en el servidor.

**Solución:** Configura las variables en la plataforma (Settings → Environment Variables).

## Error 2: Dependencias de desarrollo en producción

**Síntoma:** “Module not found” o “Command not found” durante el build.

**Causa:** Dependencias como `typescript`, `vite` o `jest` están en `devDependencies` y no se instalan en producción.

**Solución:** Asegúrate de que las dependencias necesarias para el build estén en `dependencies`, o ejecuta `npm ci` en vez de `npm ci --only=production`.

---

## Error 3: Puerto incorrecto

**Síntoma:** “EADDRINUSE” o la app no responde.

**Causa:** Tu app escucha en el puerto 3000, pero la plataforma espera otro puerto.

**Solución:** Usa `process.env.PORT || 3000` en tu servidor.

```
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
 console.log(`Server running on port ${PORT}`);
});
```

---

## Error 4: Rutas absolutas rotas

**Síntoma:** Los assets (CSS, JS, imágenes) no cargan, 404.

**Causa:** Usas rutas absolutas como `/assets/style.css` pero la app está en un subdirectorio.

**Solución:** Configura la base URL en tu framework: - Vite: `base: '/tu-repo/'` - Next.js: `basePath: '/tu-repo'` - React Router: ``

---

## Error 5: Archivos case-sensitive

**Síntoma:** “Module not found” pero el archivo existe.

**Causa:** En tu Mac funciona `import User from './User'` pero en Linux el archivo se llama `user.js`.

**Solución:** Usa siempre minúsculas en nombres de archivos y sé consistente con las importaciones.



## Error 6: Build que no es reproducible

**Síntoma:** El build funciona en tu máquina pero falla en el servidor.

**Causa:** Versión de Node.js diferente, dependencias sin lock file, o sistema operativo distinto.

**Solución:** Especifica la versión de Node en `.nvmrc` o `package.json` :

```
{
 "engines": {
 "node": ">=20.0.0"
 }
}
```

## Error 7: Gitignore incompleto

**Síntoma:** El build falla porque intenta compilar archivos que no deberían estar.

**Causa:** `.env` , `node_modules` o archivos del sistema están commiteados.

**Solución:** Verifica tu `.gitignore` :

```
node_modules/
.env
.env.local
dist/
coverage/
.DS_Store
```

## Error 8: CORS en producción

**Síntoma:** “Access-Control-Allow-Origin” error en el navegador.

**Causa:** Tu backend tiene CORS configurado para `localhost:3000` pero en producción la URL es diferente.

**Solución:** Configura CORS con la URL de producción:

```
app.use(cors({
 origin: process.env.FRONTEND_URL || 'http://localhost:3000'
}));
```

## Error 9: Base de datos no accesible

**Síntoma:** “Connection refused” o “ECONNREFUSED”.

**Causa:** La base de datos es local y el servidor no puede acceder a ella.

**Solución:** Usa un servicio de BD en la nube (Supabase, PlanetScale, MongoDB Atlas) y configura la URL en variables de entorno.

---

## Error 10: Archivos grandes en el repo

**Síntoma:** El deploy es lento o falla por timeout.

**Causa:** Imágenes, videos o `node_modules` están en el repo.

**Solución:** Usa Git LFS para archivos grandes, comprime imágenes y asegúrate de que `node_modules` está en `.gitignore` .

---

## Tabla resumen

#	Error	Síntoma	Solución rápida
1	Variables de entorno	“Cannot read property”	Configurar en la plataforma
2	Dependencias dev	“Module not found”	Mover a dependencies
3	Puerto	“EADDRINUSE”	Usar process.env.PORT
4	Rutas	Assets 404	Configurar base URL
5	Case-sensitive	“Module not found”	Usar minúsculas
6	Build no reproducible	Falla en servidor	Especificar versión Node
7	Gitignore	Build falla	Actualizar .gitignore
8	CORS	Access-Control error	Configurar origin URL
9	BD inaccesible	“Connection refused”	BD en la nube
10	Archivos grandes	Deploy lento	Git LFS, comprimir

## Aclaración: “¿Cómo depuro errores de despliegue?”

► 🤔 ¿Qué hago cuando el despliegue falla?

## Mini-chequeo

► ¿Cuál es el error más común en despliegue?

► ¿Cómo evitas el error de “puerto incorrecto”?

► ¿Qué hago si el build funciona en mi máquina pero falla en el servidor?

## ✓ Resumen en 3 frases

1. El 90% de los errores de despliegue son **variables de entorno, dependencias y configuración de puertos**.
2. Usa `process.env.PORT` para puertos, `.env.example` para documentar variables y `.gitignore` completo.
3. Cuando algo falle, **lee el log completo** y busca el primer error: la cadena de errores empieza por uno solo.

 Volver al [índice de la unidad](#) · Anterior: [o6 — Checklist](#) · Siguiente: [o8 — Template](#)

## o8 — Template de despliegue

 Estás en:  **UD 2.8 · Despliegue** → [o8 · Template de despliegue](#)

 *Documenta tu despliegue para que cualquier persona lo reproduzca*

Un buen despliegue se documenta. Si mañana otro desarrollador (o tú mismo) necesita replicar el proceso, la documentación es la clave. Esta plantilla te guía para documentar cada aspecto de tu despliegue.

## La idea en una frase

**Si tu despliegue no se puede replicar con la documentación, no está bien documentado.**

# Plantilla: Documentación de despliegue

## 1. Resumen del despliegue

Aspecto	Detalle
Plataforma	[Ej: Vercel, Netlify, Render]
URL de producción	[Ej: https://mi-app.vercel.app]
URL de staging	[Ej: https://mi-app-staging.vercel.app]
Último despliegue	[Fecha y commit]
Estado	✅ Operativo / ⚠️ Problemas / ❌ Caído

## 2. Stack de despliegue

Capa	Tecnología	Versión
Frontend	[Ej: Vue 3 + Vite]	[v3.4]
Backend	[Ej: Node.js + Express]	[v20 LTS]
Base de datos	[Ej: Supabase PostgreSQL]	[v15]
Hosting	[Ej: Vercel]	[N/A]
CI/CD	[Ej: GitHub Actions]	[N/A]
Dominio	[Ej: mi-app.com]	[Cloudflare]

## 3. Variables de entorno

<pre># .env.example (commiteado) DATABASE_URL=          # URL de conexión a la BD API_KEY=               # Clave de la API externa FRONTEND_URL=          # URL del frontend (para CORS) NODE_ENV=              # production   development PORT=                  # Puerto del servidor (asignado por plataforma)</pre>
657 / 743

## Variables en la plataforma (no commiteadas):

Variable	Valor	Dónde está
DATABASE_URL	postgres://...	Plataforma → Settings → Env Vars
API_KEY	sk-...	Plataforma → Settings → Env Vars

## 4. Proceso de despliegue

```
1. Asegurar que todo está commiteado
git status
git add .
git commit -m "feat: descripción del cambio"
git push origin main

2. GitHub Actions ejecuta automáticamente:
- Lint (eslint)
- Tests (jest/vitest)
- Build (npm run build)
- Deploy (a la plataforma)

3. Verificar el despliegue
- Abrir la URL de producción
- Probar funcionalidades principales
- Revisar logs si hay errores
```

## 5. Comandos útiles

```
Build local
npm run build

Ejecutar en modo producción local
npm run start

Verificar variables de entorno
node -e "console.log(process.env.DATABASE_URL)"

Build de Docker
docker build -t mi-app .

Ejecutar Docker
docker run -p 3000:3000 -e DATABASE_URL=... mi-app

Verificar que el build es correcto
npm run build && npm run start
```

## 6. Troubleshooting (resolución de problemas)

Problema	Causa probable	Solución
“Cannot find module”	Dependencia faltante	<code>npm install</code> y <code>commitear package-lock.json</code>
“PORT in use”	Puerto ya en uso	Usar <code>process.env.PORT</code>
“CORS error”	Origen no permitido	Configurar <code>origin</code> en CORS
“Build failed”	Error en el código	Revisar logs del build
“Database connection”	URL incorrecta	Verificar <code>DATABASE_URL</code> en env vars

## 7. Monitoreo post-despliegue

- ☐ URL funciona y carga en <3 segundos
- ☐ Login/autenticación funciona
- ☐ Operaciones CRUD principales funcionan
- ☐ Formularios envían datos correctamente
- ☐ Logs se generan (verificar en la plataforma)
- ☐ No hay errores en la consola del navegador

## 8. Rollback (volver atrás)

Si algo falla después del despliegue:

**En Vercel:** 1. Ve a tu proyecto → Deployments 2. Busca el último deployment que funcionaba 3. Haz clic en “Promote to Production”

**En GitHub Pages:**

```
git revert HEAD
git push origin main
```

**En Render:** 1. Ve a tu servicio → Manual Deploy 2. Selecciona un commit anterior 3. Haz clic en “Deploy”

## Ejemplo rellenado

### Resumen

Aspecto	Detalle
Plataforma	Vercel
URL	https://taskmanager-sergarb1.vercel.app
Último despliegue	2025-01-15, commit abc123
Estado	<input checked="" type="checkbox"/> Operativo

### Stack

Capa	Tecnología
Frontend	Vue 3 + Vite + Tailwind
Backend	Vercel Serverless Functions
BD	Supabase PostgreSQL
CI/CD	GitHub Actions

## Mini-chequeo

► ¿Qué secciones tiene la plantilla de despliegue?



► ¿Por qué documentar las variables de entorno?

► ¿Qué es un rollback?

## ✓ Resumen en 3 frases

1. Documenta tu despliegue con **resumen, stack, variables de entorno, proceso, comandos y troubleshooting**.
2. Incluye un apartado de **rollback** por si algo falla: saber volver atrás es tan importante como avanzar.
3. La documentación debe permitir que **cualquier persona reproduzca tu despliegue** sin preguntarte.

 Volver al [índice de la unidad](#) · Anterior: [07 — Errores comunes](#) · Siguiente: [09 — Cierre](#)

## 09 — Cierre

 Estás en:  UD 2.8 · Despliegue → 09 · Cierre

 *Ya sabes llevar tu proyecto de localhost a producción*

Has recorrido toda la unidad: desde qué es el despliegue hasta cómo resolver errores comunes. Ahora toca **verificar que lo entiendes** y **prepararte para monitorizar tu aplicación**.

**Desplegar es hacer tu aplicación accesible para el mundo: con dominio, HTTPS, automatización y un plan por si algo falla.**

Si recuerdas esto, has entendido la unidad.

## Mini-chequeo final

Responde estas preguntas antes de seguir:

### Preguntas de comprensión

► ¿Qué es Docker y para qué sirve?

► ¿Qué es CI/CD y por qué importa?

► ¿Cuáles son las 3 estrategias de zero-downtime?

► ¿Cuál es el error más común de despliegue?

► ¿Qué debes hacer después de desplegar?

### Ejercicio práctico

**En 15 minutos, configura el despliegue de tu proyecto:**

1. Crea un `Dockerfile` básico (si tu proyecto lo necesita)
2. Configura un workflow de GitHub Actions para test + build
3. Elige una plataforma (Vercel para frontend, Render para Full-Stack)
4. Despliega y verifica que funciona
5. Documenta el proceso con la plantilla del punto 08

Si consigues desplegar tu app y acceder a ella desde internet → has entendido la unidad.

## Laboratorio de Tortura: depura el docker-compose.yml

**Duración estimada: 25-35 minutos.** Un equipo te pasa su configuración de despliegue. *Parece correcta...* y no lo es. Encuentra los 4 problemas.

```
docker-compose.yml
version: '3.8'

services:
 backend:
 build: ./backend
 ports:
 - "3000:3000"
 environment:
 - DB_HOST=localhost
 - DB_PASSWORD=root123
 depends_on:
 - database

 database:
 image: mysql:8.0
 ports:
 - "3306:3306"
 environment:
 - MYSQL_ROOT_PASSWORD=root123
 - MYSQL_DATABASE=mi_app
```

```
Dockerfile del backend
FROM node:20
WORKDIR /app
COPY . .
RUN npm ci --only=production
EXPOSE 3000
CMD ["node", "src/server.js"]
```

```
.gitignore
node_modules
```

Intenta encontrarlos tú primero. Si te atas, abre las pistas una a una:

► **Pista 1 — La red interna**

► **Pista 2 — Secrets**

► Pista 3 — Persistencia y puertos

► ...Solución completa...

## Tabla resumen de la unidad

Concepto	Idea clave
<b>Despliegue</b>	De localhost a producción con dominio y HTTPS
<b>Docker</b>	Empaquetar en contenedores para portabilidad
<b>CI/CD</b>	Automatizar test → build → deploy con GitHub Actions
<b>Plataformas</b>	Pages (estático), Vercel (frontend), Render (Full-Stack)
<b>Zero-downtime</b>	Desplegar sin que el usuario note cortes
<b>Checklist</b>	Verificar todo antes y después de desplegar
<b>Errores</b>	Variables de entorno, dependencias, puertos
<b>Template</b>	Documentar el proceso para replicabilidad
<b>Laboratorio</b>	<code>DB_HOST=database</code> (no localhost), secrets en <code>.env</code> , volúmenes y puertos mínimos

## Vocabulario rápido

Término	Definición
<b>Dockerfile</b>	Archivo de instrucciones para construir una imagen Docker
<b>Contenedor</b>	Caja aislada que ejecuta tu aplicación con todo lo que necesita
<b>CI/CD</b>	Continuous Integration / Continuous Deployment: automatización de build y deploy
<b>Workflow</b>	Pipeline de CI/CD definido en YAML
<b>Zero-downtime</b>	Despliegue sin interrupciones de servicio
<b>Rolling update</b>	Cambio gradual de versiones
<b>Blue-green</b>	Dos entornos idénticos, uno activo y uno en espera
<b>Canary</b>	Despliegue a un porcentaje pequeño de tráfico primero
<b>Rollback</b>	Volver a la versión anterior del despliegue

## Conexión con las siguientes unidades

Unidad	Qué aprenderás	Cómo se conecta
<b>UD 2.8: Monitorización</b>	Logs, métricas, dashboards	Monitorizarás la app que desplegaste
<b>UD 2.9: Propuesta Final</b>	Proyecto completo	Tu app desplegada será la base del proyecto final
<b>Proyecto real</b>	Deploy en producción	Aplicarás todo lo aprendido aquí

## Consejo para el siguiente punto

Si vas a monitorizar tu app desplegada:

1. Empieza por **logs básicos** (punto 01 de UD 2.8)
2. Añade **métricas simples** como uptime y tiempo de respuesta

3. No necesitas Prometheus y Grafana para empezar: **console.log** y un servicio de **monitoreo básico** bastan

**Lo importante es empezar a observar, no tener el sistema perfecto.**

## ✓ Resumen en 3 frases

1. Docker **empaqueta tu app en contenedores** que funcionan igual en cualquier entorno.
2. CI/CD **automatiza** test → build → deploy con GitHub Actions, y las plataformas como Vercel y Render lo facilitan.
3. Después de desplegar, **verifica todo**: URL, funcionalidades, logs y rendimiento. Y ten un **plan de rollback** por si algo falla.

 Volver al [índice de la unidad](#) · Anterior: [o8 — Template](#) · Siguiente: [UD 2.9 — Monitorización](#)

## 01 — Qué son los logs

 **Estás en:**  **UD 2.9 · Monitorización** → 01 · Qué son los logs

 *Los logs son el diario de tu aplicación*

Un log es un registro de lo que pasa en tu aplicación. Cuando algo falla, el primer sitio donde miras es los logs. Si no tienes logs, estás a ciegas: no sabes qué pasó, cuándo pasó ni por qué pasó.

## La idea en una frase

Los logs son el diario de tu aplicación: registran qué pasa, cuándo y por qué, para que puedas diagnosticar problemas.

## Los 5 niveles de log

Nivel	Cuándo usarlo	Ejemplo
ERROR	Algo ha fallado	“No se pudo conectar a la base de datos”
WARN	Algo sospechoso pero funciona	“La consulta tardó 3 segundos (umbral: 1s)”
INFO	Información normal del sistema	“Servidor arrancado en puerto 3000”
DEBUG	Detalles para desarrollo	“Recibida petición GET /api/users”
TRACE	Muy detallado (solo desarrollo)	“Variable userId = 123 antes de la consulta”

## Regla de uso

Producción:	ERROR + WARN + INFO
Desarrollo:	ERROR + WARN + INFO + DEBUG + TRACE

## Estructura de un log

Un buen log no es solo un mensaje. Es una línea estructurada:

```
{
 "timestamp": "2025-01-15T10:30:45.123Z",
 "level": "error",
 "message": "Error al conectar con la base de datos",
 "service": "api-users",
 "requestId": "req-abc-123",
 "error": {
 "name": "ConnectionError",
 "message": "ECONNREFUSED 127.0.0.1:5432",
 "stack": "..."
 }
}
```

## Por qué estructurar

Sin estructura	Con estructura
Error al conectar	<code>{"level":"error","message":"...","service":"api-users"}</code>
No sabes cuándo	Tienes timestamp preciso
No sabes de dónde	Tienes service y requestId
No puedes buscar	Puedes filtrar por nivel, servicio, etc.

## Logs en Node.js con Winston



```

const winston = require('winston');

const logger = winston.createLogger({
 level: process.env.LOG_LEVEL || 'info',
 format: winston.format.combine(
 winston.format.timestamp(),
 winston.format.json()
),
 transports: [
 new winston.transports.File({ filename: 'logs/error.log', level: 'error' }),
 new winston.transports.File({ filename: 'logs/combined.log' }),
],
});

// En desarrollo, también imprime en consola
if (process.env.NODE_ENV !== 'production') {
 logger.add(new winston.transports.Console({
 format: winston.format.simple()
 }));
}

// Uso
logger.info('Servidor arrancado', { port: 3000 });
logger.error('Error al conectar', { error: err.message });
logger.warn('Consulta lenta', { duration: 3200, query: 'SELECT *' });

```

---

## Logs en Python con logging

```

import logging
import json

Configuración básica
logging.basicConfig(
 level=logging.INFO,
 format='{"timestamp":"%(asctime)s","level":"%(levelname)s","message":"%(message)s"}'
)

logger = logging.getLogger(__name__)

Uso
logger.info("Servidor arrancado en puerto 3000")
logger.error("Error al conectar con la base de datos")

```

---

## Dónde guardar logs

Opción	Ventaja	Desventaja
Archivo local	Simple	Se llena el disco, difícil de buscar
Consola	Visible en desarrollo	Se pierde al reiniciar
Servicio externo (Sentry, LogRocket)	Búsqueda, alertas	Depende de un tercero
ELK Stack	Potente, escalable	Complejo de configurar

## Aclaración: “¿Necesito logs para un proyecto pequeño?”

▶ 🤔 ¿Siempre necesito configurar logs estructurados?

## Mini-chequeo

▶ ¿Cuáles son los 5 niveles de log?

▶ ¿Por qué estructurar los logs en JSON?

▶ ¿Qué librería de logs se usa en Node.js?

## ✅ Resumen en 3 frases

1. Los logs son el **diario de tu aplicación**: registran qué pasa, cuándo y por qué.
2. Usa **5 niveles** (ERROR, WARN, INFO, DEBUG, TRACE) y **estructura JSON** para poder buscar y filtrar.
3. Configura **Winston** (Node.js) o **logging** (Python) desde el principio para tener logs útiles en producción.

## 02 — Métricas clave

 Estás en:  **UD 2.9 · Monitorización** → 02 · Métricas clave

 *Lo que no mides, no puedes mejorar*

Los logs te dicen qué pasó. Las métricas te dicen **cómo está** tu aplicación ahora mismo: ¿es rápida? ¿tiene errores? ¿cuántos usuarios tiene? Las métricas son el pulso de tu sistema.

### La idea en una frase

**Las métricas son los números que te dicen si tu aplicación funciona bien o mal: latencia, errores y throughput.**

## Las 3 métricas principales (RED)

El método RED (Rate, Errors, Duration) es el estándar:

### 1. Rate (Throughput) — Cuántas peticiones por segundo

`Throughput = Peticiones / Tiempo`

Ejemplo: 150 requests/segundo

**Por qué importa:** Si el throughput cae de golpe, algo está fallando.

## 2. Errors — Tasa de errores

Error Rate = (Errores / Total peticiones) × 100

Ejemplo: 2.3% de errores (4xx + 5xx)

**Por qué importa:** Si la tasa de errores sube, hay un bug o el servidor está sobrecargado.

## 3. Duration (Latency) — Cuánto tarda en responder

Latency = Tiempo desde request hasta response

Percentiles:

- P50 (mediana): 120ms → La mitad de las peticiones son más rápidas
- P95: 450ms → El 95% son más rápidas
- P99: 1200ms → El 99% son más rápidas

**Por qué importa:** Si P95 sube de 500ms, los usuarios notan la app lenta.

# Métricas de infraestructura

Además de RED, monitoriza el servidor:

Métrica	Qué mide	Umbral típico
CPU	Uso del procesador	>80% es preocupante
Memoria	RAM utilizada	>85% es preocupante
Disco	Espacio en disco	>90% es crítico
Red	Ancho de banda	Depende del plan

# Métricas de aplicación

Métrica	Qué mide	Ejemplo
Tiempo de respuesta de BD	Cuánto tardan las queries	<100ms es bueno
Tasa de cache hit	Cuántas peticiones usan cache	>80% es bueno
Conexiones activas	Peticiones simultáneas	Depende del servidor
Uptime	Tiempo sin caídas	>99.9% es bueno

## Ejemplo: métricas de una API REST

```
// Métricas básicas que deberías trackear

// 1. Tiempo de respuesta por endpoint
app.use((req, res, next) => {
 const start = Date.now();
 res.on('finish', () => {
 const duration = Date.now() - start;
 logger.info('Request completado', {
 method: req.method,
 path: req.path,
 statusCode: res.statusCode,
 duration,
 });
 });
 next();
});

// 2. Errores por tipo
app.use((err, req, res, next) => {
 logger.error('Error no manejado', {
 error: err.message,
 stack: err.stack,
 path: req.path,
 });
 res.status(500).json({ error: 'Internal Server Error' });
});
```

## Comparativa: logs vs métricas

Te dicen QUÉ pasó: ‘Error al conectar con BD a las 10:30:45’. Útil para diagnosticar un error específico.

Te dicen CÓMO ESTÁ tu app ahora: ‘95% de peticiones responden en <200ms’. Útil para ver tendencias y alertas.

### Cuándo usar cada métrica

Situación	Métrica clave	Qué buscar
App lenta	Latency (P95, P99)	¿Qué endpoint tarda más?
Muchos errores	Error Rate	¿Qué tipo de error sube?
Pico de tráfico	Throughput	¿Cuántas peticiones manejo?
Servidor saturado	CPU, Memoria	¿Necesito escalar?
Caída del servicio	Uptime	¿Cuánto tiempo ha estado caído?

### Aclaración: “¿Necesito todas estas métricas?”

► 🤖 ¿Para un proyecto de clase necesito monitorizar todo esto?

### Mini-chequeo

► ¿Cuáles son las 3 métricas RED?

► ¿Qué es el percentil P95?

► ¿Cuándo es preocupante el uso de CPU?

## ✓ Resumen en 3 frases

1. Las 3 métricas RED son **throughput** (peticiones/segundo), **error rate** (tasa de errores) y **latency** (tiempo de respuesta).
2. Usa **percentiles** (P50, P95, P99) para medir latencia: el P95 es el más indicativo de la experiencia real del usuario.
3. Para un proyecto básico, mide **uptime**, **tiempo de respuesta** y **errores 5xx**: son suficientes para demostrar monitorización.

 Volver al [índice de la unidad](#) · Anterior: [01 — Qué son los logs](#) · Siguiente: [03 — Prometheus + Grafana](#)

## 03 — Prometheus y Grafana

 Estás en:  **UD 2.9 · Monitorización** → [03 · Prometheus y Grafana](#)

 *Prometheus recopila, Grafana visualiza*

Prometheus y Grafana son las herramientas de monitorización más usadas en la industria. Prometheus **recopila métricas** de tu aplicación y Grafana las **visualiza en dashboards**. Juntas, forman el stack estándar de observabilidad.

**Prometheus es la base de datos de métricas, Grafana es el panel de control que las muestra en tiempo real.**

## Qué es Prometheus

Prometheus es un sistema de **recopilación y almacenamiento de métricas**. Funciona con un modelo pull: Prometheus **pregunta** a tu aplicación cada X segundos “¿cuáles son tus métricas?”.

## Cómo funciona

```
Tu app expone métricas en /metrics
 |
 ▼
Prometheus las recoge cada 15s
 |
 ▼
Almacena en su base de datos
 |
 ▼
Grafana consulta y visualiza
```

## Ejemplo de endpoint /metrics

```
HELP http_requests_total Total de peticiones HTTP
TYPE http_requests_total counter
http_requests_total{method="GET",path="/api/users",status="200"} 1523
http_requests_total{method="POST",path="/api/users",status="201"} 89
http_requests_total{method="GET",path="/api/users",status="500"} 12

HELP http_request_duration_seconds Duración de peticiones HTTP
TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{method="GET",path="/api/users",le="0.1"} 1200
http_request_duration_seconds_bucket{method="GET",path="/api/users",le="0.5"} 1450
http_request_duration_seconds_bucket{method="GET",path="/api/users",le="1"} 1500
```

## Qué es Grafana

Grafana es una plataforma de **visualización de datos**. Conecta con Prometheus (y otras fuentes) y crea dashboards interactivos con gráficas, tablas y alertas.



## Características principales

- Dashboards en tiempo real
- Múltiples fuentes de datos (Prometheus, MySQL, etc.)
- Alertas visuales y por email
- Plugins para diferentes tipos de gráficas
- Acceso multi-usuario con permisos

---

## Configuración básica con Docker Compose

```
version: '3.8'

services:
 prometheus:
 image: prom/prometheus:latest
 ports:
 - "9090:9090"
 volumes:
 - ./prometheus.yml:/etc/prometheus/prometheus.yml
 - prometheus-data:/prometheus

 grafana:
 image: grafana/grafana:latest
 ports:
 - "3001:3000"
 environment:
 - GF_SECURITY_ADMIN_PASSWORD=admin
 volumes:
 - grafana-data:/var/lib/grafana
 depends_on:
 - prometheus

volumes:
 prometheus-data:
 grafana-data:
```

---

## Configuración de Prometheus

```
prometheus.yml
global:
 scrape_interval: 15s
 evaluation_interval: 15s

scrape_configs:
 - job_name: 'mi-app'
 static_configs:
 - targets: ['app:3000']
 metrics_path: '/metrics'
 scrape_interval: 10s
```

---

## Integración con Node.js (prom-client)

```
const client = require('prom-client');

// Recopilar métricas del sistema
client.collectDefaultMetrics();

// Métrica personalizada: peticiones HTTP
const httpRequestsTotal = new client.Counter({
 name: 'http_requests_total',
 help: 'Total de peticiones HTTP',
 labelNames: ['method', 'path', 'status'],
});

const httpRequestDuration = new client.Histogram({
 name: 'http_request_duration_seconds',
 help: 'Duración de peticiones HTTP en segundos',
 labelNames: ['method', 'path'],
 buckets: [0.01, 0.05, 0.1, 0.5, 1, 2, 5],
});

// Middleware para registrar métricas
app.use((req, res, next) => {
 const end = httpRequestDuration.startTimer({ method: req.method, path: req.path });
 res.on('finish', () => {
 httpRequestsTotal.inc({ method: req.method, path: req.path, status: res.statusCode });
 end();
 });
 next();
});

// Endpoint /metrics para Prometheus
app.get('/metrics', async (req, res) => {
 res.set('Content-Type', client.register.contentType);
 res.end(await client.register.metrics());
});
```

---

## Integración con Python (prometheus\_client)

```

from prometheus_client import Counter, Histogram, generate_latest, CONTENT_TYPE_LATEST
from flask import Flask, request, Response

app = Flask(__name__)

Métricas
http_requests_total = Counter(
 'http_requests_total',
 'Total de peticiones HTTP',
 ['method', 'path', 'status']
)

http_request_duration = Histogram(
 'http_request_duration_seconds',
 'Duración de peticiones HTTP',
 ['method', 'path'],
 buckets=[0.01, 0.05, 0.1, 0.5, 1, 2, 5]
)

@app.before_request
def before_request():
 request.start_time = time.time()

@app.after_request
def after_request(request):
 duration = time.time() - request.start_time
 http_requests_total.labels(
 method=request.method,
 path=request.path,
 status=response.status_code
).inc()
 http_request_duration.labels(
 method=request.method,
 path=request.path
).observe(duration)
 return response

@app.route('/metrics')
def metrics():
 return Response(generate_latest(), mimetype=CONTENT_TYPE_LATEST)

```

## Queries útiles en Prometheus

```

Tasa de errores (errores por segundo)
rate(http_requests_total{status=~"5.."}[5m])

Latencia P95
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))

Peticiones por segundo
sum(rate(http_requests_total[5m]))

Endpoints más lentos
topk(5, histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m])))

```

# Aclaración: “¿Prometheus y Grafana son complicados?”

▶ 🤔 ¿Merece la pena configurar Prometheus y Grafana para un proyecto de clase?

## Mini-chequeo

▶ ¿Qué hace Prometheus?

▶ ¿Qué hace Grafana?

▶ ¿Qué endpoint expone las métricas de Node.js?

## ✅ Resumen en 3 frases

1. **Prometheus** recopila métricas de tu app cada X segundos con el modelo pull y las almacena para consultas.
2. **Grafana** conecta con Prometheus y crea **dashboards interactivos** con gráficas y alertas en tiempo real.
3. Para Node.js usa `prom-client` para exponer métricas en `/metrics`, y Docker Compose para levantar el stack completo.

 Volver al [índice de la unidad](#) · Anterior: [02 — Métricas clave](#) · Siguiente: [04 — Alertas](#)

📺 Las alertas te avisan antes de que el usuario se dé cuenta

No puedes mirar un dashboard 24/7. Las alertas hacen el trabajo por ti: cuando algo va mal (errores, latencia alta, servidor caído), te envían un email, un mensaje de Slack o una notificación. Detectas problemas antes de que los usuarios se quejen.

## 📺 La idea en una frase

**Las alertas son ojos automáticos que vigilan tu aplicación y te avisan cuando algo no va bien.**

## Qué monitorizar con alertas

Métrica	Condición de alerta	Urgencia
Uptime	App no responde	🔴 Crítica
Error rate	>5% de errores 5xx	🔴 Alta
Latencia	P95 >2 segundos	🟡 Media
CPU	>80% durante 5 min	🟡 Media
Memoria	>85% durante 5 min	🟡 Media
Disco	>90%	🔴 Alta
Certificado SSL	Expira en <30 días	🟡 Media

## 1. Alerta de caída (down)

```
Prometheus: alerta si la app no responde
groups:
 - name: app-alerts
 rules:
 - alert: AppDown
 expr: up{job="mi-app"} == 0
 for: 1m
 labels:
 severity: critical
 annotations:
 summary: "La aplicación está caída"
 description: "La app no responde desde hace más de 1 minuto"
```

## 2. Alerta de errores altos

```
- alert: HighErrorRate
 expr: |
 sum(rate(http_requests_total{status=~"5.."}[5m]))
 / sum(rate(http_requests_total[5m])) > 0.05
 for: 2m
 labels:
 severity: warning
 annotations:
 summary: "Tasa de errores alta"
 description: "Más del 5% de peticiones fallan"
```

## 3. Alerta de latencia alta

```
- alert: HighLatency
 expr: |
 histogram_quantile(0.95,
 rate(http_request_duration_seconds_bucket[5m])
) > 2
 for: 3m
 labels:
 severity: warning
 annotations:
 summary: "Latencia P95 alta"
 description: "El 95% de peticiones tardan más de 2 segundos"
```

## 4. Alerta de recursos

```
- alert: HighCPU
 expr: 100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 80
 for: 5m
 labels:
 severity: warning
 annotations:
 summary: "CPU alta"
 description: "El uso de CPU supera el 80% durante 5 minutos"
```

# Canales de notificación

## Email

```
alertmanager.yml
route:
 receiver: 'email-notifications'

receivers:
- name: 'email-notifications'
 email_configs:
 - to: 'tu-email@gmail.com'
 from: 'alertas@tudominio.com'
 smarthost: 'smtp.gmail.com:587'
 auth_username: 'tu-email@gmail.com'
 auth_password: 'tu-password-de-aplicacion'
```

## Slack

```
receivers:
- name: 'slack-notifications'
 slack_configs:
 - api_url: 'https://hooks.slack.com/services/T.../B...'
 channel: '#alerts'
 title: '{{ .GroupLabels.alertname }}'
 text: '{{ .Annotations.description }}'
```

## Discord

```
receivers:
- name: 'discord-notifications'
 webhook_configs:
 - url: 'https://discord.com/api/webhooks/...'
```

---

## Configuración completa de Alertmanager

```
alertmanager.yml
global:
 resolve_timeout: 5m

route:
 group_by: ['alertname']
 group_wait: 30s
 group_interval: 5m
 repeat_interval: 4h
 receiver: 'slack-notifications'
 routes:
 - match:
 severity: critical
 receiver: 'email-notifications'

receivers:
 - name: 'slack-notifications'
 slack_configs:
 - api_url: 'https://hooks.slack.com/services/...'
 channel: '#alerts'

 - name: 'email-notifications'
 email_configs:
 - to: 'tu-email@gmail.com'
 from: 'alertas@tudominio.com'
 smarthost: 'smtp.gmail.com:587'
 auth_username: 'tu-email@gmail.com'
 auth_password: 'contraseña-de-aplicacion'

inhibit_rules:
 - source_match:
 severity: 'critical'
 target_match:
 severity: 'warning'
 equal: ['alertname']
```

---

## Aclaración: “¿Necesito alertas para un proyecto de clase?”

► 🤔 ¿Debo configurar alertas para mi proyecto académico?

---

## Mini-chequeo

► ¿Qué métricas deberías monitorizar con alertas?

► ¿Qué hace Alertmanager?



## ✓ Resumen en 3 frases

1. Las alertas te avisan **automáticamente** cuando algo va mal: caída, errores altos, latencia o recursos saturados.
2. Configura **Alertmanager** con canales de notificación (email, Slack) y define umbrales claros para cada métrica.
3. Para empezar rápido, usa **UptimeRobot** o **Grafana Cloud** sin necesidad de Prometheus completo.

 Volver al [índice de la unidad](#) · Anterior: [03 — Prometheus + Grafana](#) · Siguiente: [05 — Stack mínimo](#)

## 05 — Stack mínimo de monitorización

 Estás en:  **UD 2.9 · Monitorización** → 05 · Stack mínimo de monitorización

 *No necesitas Prometheus y Grafana para empezar*

El stack completo de monitorización puede ser overkill para un proyecto pequeño. Hay opciones más simples que te dan el 80% de la información con el 20% del esfuerzo. Empieza pequeño y escala cuando lo necesites.

## La idea en una frase

Empieza con la solución más simple que te dé la información que necesitas, y complica solo cuando sea necesario.

## Niveles de monitorización

### Nivel 1: Básico (o configuración)

**Herramientas:** `console.log` , `console.error` , logs en archivo

**Qué mides:** Errores visibles en la terminal o en archivos

**Cuándo usarlo:** Desarrollo local, proyectos muy pequeños

```
// Lo más básico que puedes hacer
app.use((err, req, res, next) => {
 console.error(`[${new Date().toISOString()}] ERROR: ${err.message}`);
 res.status(500).json({ error: 'Internal Server Error' });
});
```

### Nivel 2: Servicio externo gratuito

**Herramientas:** UptimeRobot, Better Stack (gratuito)

**Qué mides:** Uptime, tiempo de respuesta, caídas

**Cuándo usarlo:** Cualquier proyecto desplegado

Servicio	Plan gratuito	Qué ofrece
UptimeRobot	50 monitors	Uptime, response time, alertas
Better Stack	5 monitors	Uptime, logs, métricas
Freshping	50 checks	Uptime, alertas

**Configuración** (2 minutos): 1. Crear cuenta en UptimeRobot 2. Añadir monitor con la URL de tu app 3. Configurar alertas por email

## Nivel 3: Docker Compose simple

**Herramientas:** Prometheus + Grafana en Docker

**Qué mides:** Métricas completas, dashboards, alertas

**Cuándo usarlo:** Cuando necesitas métricas de aplicación (no solo uptime)

```
docker-compose.monitoring.yml
version: '3.8'
services:
 prometheus:
 image: prom/prometheus
 ports: ["9090:9090"]
 volumes: [".:/prometheus.yml:/etc/prometheus/prometheus.yml"]

 grafana:
 image: grafana/grafana
 ports: ["3001:3000"]
 environment:
 GF_SECURITY_ADMIN_PASSWORD: admin
 depends_on: [prometheus]
```

## Nivel 4: Stack completo

**Herramientas:** Prometheus + Grafana + Loki + Alertmanager

**Qué mides:** Logs, métricas y trazas (observabilidad completa)

**Cuándo usarlo:** Producción con múltiples servicios

## Comparativa: qué necesitas según tu proyecto

Nivel 1-2: console.log + UptimeRobot. Suficiente para demostrar que monitorizas.

Nivel 2-3: UptimeRobot + Grafana básico. Demuestra conocimientos reales.

Nivel 3-4: Prometheus + Grafana + Loki. Observabilidad completa.

## Stack mínimo recomendado

Para un proyecto de clase o portfolio:

```
Tu app (Node.js/Python)
├── Logs estructurados (Winston/logging)
│ └── Archivo local o servicio externo
├── Métricas básicas (prom-client)
│ └── Endpoint /metrics
├── Uptime monitoring (UptimeRobot)
│ └── Alertas por email
└── Dashboard básico (Grafana)
 └── Conecta con Prometheus
```

## Aclaración: “¿Cuándo debo complicar mi stack de monitorización?”

► 🤔 ¿Cuándo paso del nivel 1 al nivel 2 o 3?

## Mini-chequeo

► ¿Cuáles son los 4 niveles de monitorización?

► ¿Qué servicio gratuito monitorea uptime?

► ¿Cuándo necesito Prometheus y Grafana?

## ✓ Resumen en 3 frases

1. **Empieza simple:** console.log + UptimeRobot te dan el 80% de la información con el 20% del esfuerzo.
2. **Escalas cuando lo necesites:** Prometheus + Grafana cuando necesites métricas de aplicación.
3. **No compliques por complicar:** elige el nivel según las necesidades reales de tu proyecto.

 Volver al [índice de la unidad](#) · Anterior: [04 — Alertas](#) · Siguiente: [06 — Configuración](#)

## 06 — Configuración

 Estás en:  **UD 2.9 · Monitorización** → 06 · Configuración

 *Código listo para copiar y adaptar*

Ahora toca ponerse técnicos. Aquí tienes configuraciones reales de Winston para logs y prom-client para métricas. Copia, adapta a tu proyecto y tienes monitorización funcional en minutos.

**Winston gestiona logs estructurados y prom-client expone métricas para Prometheus: juntos cubren las dos caras de la monitorización.**

## Winston: logs estructurados

### Instalación

```
npm install winston
```

### Configuración completa

```

// src/config/logger.js
const winston = require('winston');

const logger = winston.createLogger({
 level: process.env.LOG_LEVEL || 'info',
 format: winston.format.combine(
 winston.format.timestamp({ format: 'YYYY-MM-DD HH:mm:ss.SSS' }),
 winston.format.errors({ stack: true }),
 winston.format.json()
),
 defaultMeta: {
 service: process.env.SERVICE_NAME || 'api',
 environment: process.env.NODE_ENV || 'development'
 },
 transports: [
 // Archivo para errores
 new winston.transports.File({
 filename: 'logs/error.log',
 level: 'error',
 maxsize: 5242880, // 5MB
 maxFiles: 5,
 }),
 // Archivo para todos los niveles
 new winston.transports.File({
 filename: 'logs/combined.log',
 maxsize: 10485760, // 10MB
 maxFiles: 10,
 }),
],
});

// En desarrollo, también imprime en consola con color
if (process.env.NODE_ENV !== 'production') {
 logger.add(new winston.transports.Console({
 format: winston.format.combine(
 winston.format.colorize(),
 winston.format.simple()
)
 }));
}

module.exports = logger;

```

## Uso en la aplicación

```

const logger = require('./config/logger');

// Log informativo
logger.info('Servidor arrancado', { port: 3000 });

// Log de petición
logger.info('Petición recibida', {
 method: 'GET',
 path: '/api/users',
 userId: req.user?.id,
});

// Log de error
try {
 await db.query('SELECT * FROM users');
} catch (error) {
 logger.error('Error al consultar usuarios', {
 error: error.message,
 stack: error.stack,
 query: 'SELECT * FROM users',
 });
}

// Log de warning
logger.warn('Consulta lenta detectada', {
 duration: 3200,
 query: 'SELECT * FROM orders',
 threshold: 1000,
});

```

## Middleware de logging para Express

```

// src/middleware/requestLogger.js
const logger = require('../config/logger');

const requestLogger = (req, res, next) => {
 const start = Date.now();

 res.on('finish', () => {
 const duration = Date.now() - start;
 const logData = {
 method: req.method,
 path: req.path,
 statusCode: res.statusCode,
 duration,
 ip: req.ip,
 userAgent: req.get('user-agent'),
 };

 if (res.statusCode >= 400) {
 logger.warn('Petición con error', logData);
 } else {
 logger.info('Petición completada', logData);
 }
 });

 next();
};

module.exports = requestLogger;

```



# prom-client: métricas para Prometheus

## Instalación

```
npm install prom-client
```

## Configuración completa

```
// src/config/metrics.js
const client = require('prom-client');

// Recopilar métricas del sistema (CPU, memoria, etc.)
client.collectDefaultMetrics({
 prefix: 'app_',
 gcDurationBuckets: [0.001, 0.01, 0.1, 1, 2, 5],
});

// Métrica: total de peticiones HTTP
const httpRequestsTotal = new client.Counter({
 name: 'app_http_requests_total',
 help: 'Total de peticiones HTTP recibidas',
 labelNames: ['method', 'path', 'status_code'],
});

// Métrica: duración de peticiones HTTP
const httpRequestDuration = new client.Histogram({
 name: 'app_http_request_duration_seconds',
 help: 'Duración de peticiones HTTP en segundos',
 labelNames: ['method', 'path', 'status_code'],
 buckets: [0.01, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5, 10],
});

// Métrica: peticiones activas
const httpRequestsActive = new client.Gauge({
 name: 'app_http_requests_active',
 help: 'Número de peticiones HTTP activas actualmente',
});

// Métrica: errores por tipo
const errorsByType = new client.Counter({
 name: 'app_errors_total',
 help: 'Total de errores por tipo',
 labelNames: ['type', 'severity'],
});

module.exports = {
 client,
 httpRequestsTotal,
 httpRequestDuration,
 httpRequestsActive,
 errorsByType,
};
```

```
// src/middleware/metricsCollector.js
const {
 httpRequestsTotal,
 httpRequestDuration,
 httpRequestsActive
} = require('../config/metrics');

const metricsCollector = (req, res, next) => {
 httpRequestsActive.inc();

 const end = httpRequestDuration.startTimer({
 method: req.method,
 path: req.path,
 });

 res.on('finish', () => {
 end({ status_code: res.statusCode });
 httpRequestsTotal.inc({
 method: req.method,
 path: req.path,
 status_code: res.statusCode,
 });
 httpRequestsActive.dec();
 });

 next();
};

module.exports = metricsCollector;
```

## Endpoint /metrics

```
// src/routes/metrics.js
const express = require('express');
const { client } = require('../config/metrics');

const router = express.Router();

router.get('/metrics', async (req, res) => {
 try {
 res.set('Content-Type', client.register.contentType);
 res.end(await client.register.metrics());
 } catch (error) {
 res.status(500).end();
 }
});

module.exports = router;
```

## Integración en el servidor principal

```
// src/server.js
const express = require('express');
const logger = require('./config/logger');
const requestLogger = require('./middleware/requestLogger');
const metricsCollector = require('./middleware/metricsCollector');
const metricsRoutes = require('./routes/metrics');

const app = express();

// Middlewares
app.use(express.json());
app.use(requestLogger);
app.use(metricsCollector);

// Rutas
app.use('/api', metricsRoutes);
app.use('/api/users', usersRouter);
// ... más rutas

// Error handler
app.use((err, req, res, next) => {
 logger.error('Error no manejado', {
 error: err.message,
 stack: err.stack,
 path: req.path,
 });
 res.status(500).json({ error: 'Internal Server Error' });
});

// Arrancar servidor
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
 logger.info(`Servidor arrancado en puerto ${PORT}`);
});
```

---

## Aclaración: “¿Cuánto espacio ocupan los logs?”

► 🤔 ¿Los logs no llenan el disco del servidor?

---

## Mini-chequeo

► ¿Qué librería de logs se usa en Node.js?

► ¿Qué librería de métricas se usa con Prometheus?

## ✓ Resumen en 3 frases

1. **Winston** configura logs estructurados con JSON, rotación de archivos y múltiples transports.
2. **prom-client** expone métricas HTTP (peticiones, duración, errores) y del sistema (CPU, memoria) en `/metrics`.
3. Ambos se integran con **middlewares de Express** que registran logs y métricas automáticamente en cada petición.

 Volver al [índice de la unidad](#) · Anterior: [05 — Stack mínimo](#) · Siguiente: [07 — Dashboard](#)

## 07 — Construir dashboards

 Estás en:  **UD 2.9 · Monitorización** → [07 · Construir dashboards](#)

 *Un buen dashboard te dice todo de un vistazo*

Un dashboard es un panel visual que muestra el estado de tu aplicación en tiempo real. Con un vistazo deberías saber si todo va bien o si algo falla. Grafana te permite crear dashboards potentes con pocas líneas de configuración.

## La idea en una frase

Un buen dashboard muestra en una pantalla lo esencial: uptime, errores, latencia y tráfico.

## Qué mostrar en tu dashboard

### Panel 1: Estado general (stat)

```
{
 "title": "Estado de la aplicación",
 "type": "stat",
 "targets": [{
 "expr": "up{job=\"mi-app\"}",
 "legendFormat": "{{instance}}"
 }],
 "fieldConfig": {
 "defaults": {
 "mappings": [
 { "type": "value", "options": { "0": { "text": "CAÍDA", "color": "red" }, "1": { } } }
]
 }
 }
}
```

### Panel 2: Peticiones por segundo (graph)

```
{
 "title": "Peticiones por segundo",
 "type": "timeseries",
 "targets": [{
 "expr": "sum(rate(app_http_requests_total[1m]))",
 "legendFormat": "req/s"
 }]
}
```

### Panel 3: Tasa de errores (gauge)

```
{
 "title": "Tasa de errores",
 "type": "gauge",
 "targets": [{
 "expr": "sum(rate(app_http_requests_total{status_code=~\"5..\"}[5m])) / sum(rate(ap",
 "legendFormat": "Errores %"
 }],
 "fieldConfig": {
 "defaults": {
 "min": 0,
 "max": 100,
 "thresholds": {
 "steps": [
 { "value": 0, "color": "green" },
 { "value": 1, "color": "yellow" },
 { "value": 5, "color": "red" }
]
 }
 }
 }
}
```

## Panel 4: Latencia P95 (graph)

```
{
 "title": "Latencia P95 (ms)",
 "type": "timeseries",
 "targets": [{
 "expr": "histogram_quantile(0.95, sum(rate(app_http_request_duration_seconds_bucket",
 "legendFormat": "P95"
 }])
}
```

## Panel 5: Errores por endpoint (table)

```
{
 "title": "Errores por endpoint",
 "type": "table",
 "targets": [{
 "expr": "topk(10, sum(rate(app_http_requests_total{status_code=~\"5..\"}[5m])) by (e",
 "instant": true
 }])
}
```

## Panel 6: Uso de CPU y memoria (graph)

```
{
 "title": "CPU y Memoria",
 "type": "timeseries",
 "targets": [
 {
 "expr": "100 - (avg(rate(node_cpu_seconds_total{mode=\"idle\"}[5m])) * 100)",
 "legendFormat": "CPU %"
 },
 {
 "expr": "(1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100",
 "legendFormat": "Memoria %"
 }
]
}
```

## Layout recomendado

[Estado]	[Req/s]	[Errores %]	[P95 Latency]	← Fila 1: KPIs principales
Peticiones por segundo		Latencia P95		← Fila 2: Tendencias
Errores por endpoint		CPU y Memoria		← Fila 3: Detalles

## Variables del dashboard

Las variables permiten filtrar por servicio, endpoint o timeframe:

```
{
 "templating": {
 "list": [
 {
 "name": "service",
 "type": "query",
 "query": "label_values(app_http_requests_total, service)",
 "refresh": 2
 },
 {
 "name": "interval",
 "type": "interval",
 "options": ["1m", "5m", "15m", "1h"],
 "current": { "text": "5m", "value": "5m" }
 }
]
 }
}
```

## Aclaracion: “¿Cuántos paneles necesito?”

► 🤔 ¿Debería llenar el dashboard de gráficas?

## Mini-chequeo

► ¿Cuántos paneles debería tener un dashboard básico?

► ¿Qué es una variable en Grafana?

► ¿Qué tipo de panel usar para el estado de la app?

## ✅ Resumen en 3 frases

1. Un dashboard básico tiene **6-8 paneles**: Estado, Throughput, Errores, Latencia, Errores por endpoint y Recursos.
2. Usa **variables** para filtrar por servicio y timeframe, y **thresholds** de color para detectar problemas de un vistazo.
3. **No llenes el dashboard de gráficas**: menos paneles, más claridad. Si no ves el problema en 5 segundos, reorganiza.

 Volver al [índice de la unidad](#) · Anterior: [06 — Configuración](#) · Siguiente: [08 — Template](#)



📖 Estás en: 🇪🇸 UD 2.9 · Monitorización → o8 · Template de monitorización

📖 Documenta tu monitorización para que cualquier persona la configure

Si configuraste logs, métricas y dashboards, documenta cómo lo hiciste. Esto ayuda a tu equipo (y a tu yo futuro) a entender qué monitorea tu app y cómo resolver problemas.

📌 La idea en una frase

Si no puedes explicar qué monitorizas y cómo, tu monitorización no está completa.

Plantilla: Documentación de monitorización

1. Resumen

Aspecto	Detalle
Herramientas de logs	[Ej: Winston, console.log]
Herramientas de métricas	[Ej: prom-client, UptimeRobot]
Dashboard	[Ej: Grafana en localhost:3001]
Alertas	[Ej: Email + Slack]
Nivel de monitorización	[1: Básico / 2: Externo / 3: Docker / 4: Completo]

Aspecto	Detalle
Librería	[Ej: Winston]
Niveles habilitados	[Ej: error, warn, info]
Formato	[Ej: JSON estructurado]
Archivos	[Ej: logs/error.log, logs/combined.log]
Rotación	[Ej: 5MB, 5 archivos máximo]
Transportes	[Ej: Archivo + Consola (dev)]

Ejemplo de log:

```
{
 "timestamp": "2025-01-15T10:30:45.123Z",
 "level": "error",
 "message": "Error al conectar con la base de datos",
 "service": "api-users",
 "error": { "name": "ConnectionError", "message": "ECONNREFUSED" }
}
```

3. Métricas

Métrica	Tipo	Descripción
app_http_requests_total	Counter	Total de peticiones HTTP
app_http_request_duration_seconds	Histogram	Duración de peticiones
app_http_requests_active	Gauge	Peticiones activas
app_errors_total	Counter	Errores por tipo

Endpoint de métricas: GET /metrics

4. Dashboard de Grafana

Panel	Tipo	Métrica principal
Estado de la app	Stat	<code>up{job="mi-app"}</code>
Peticiones/segundo	Timeseries	<code>sum(rate(app_http_requests_total[1m]))</code>
Tasa de errores	Gauge	<code>rate(erroses) / rate(total) * 100</code>
Latencia P95	Timeseries	<code>histogram_quantile(0.95, ...)</code>
Erros por endpoint	Table	<code>topk(10, sum by (path))</code>
CPU y Memoria	Timeseries	<code>node_cpu</code> , <code>node_memory</code>

## 5. Alertas

Alerta	Condición	Canal	Urgencia
AppDown	<code>up == 0</code> durante 1m	Email + Slack	Crítica
HighErrorRate	>5% errores durante 2m	Slack	Alta
HighLatency	P95 >2s durante 3m	Slack	Media
HighCPU	>80% durante 5m	Email	Media

## 6. Comandos útiles

```
Ver logs en tiempo real
tail -f logs/combined.log

Buscar errores en logs
grep '"level":"error"' logs/combined.log

Ver métricas actuales
curl http://localhost:3000/metrics

Levantar stack de monitorización
docker-compose -f docker-compose.monitoring.yml up -d

Verificar que Prometheus está recogiendo métricas
curl http://localhost:9090/api/v1/targets
```

## 7. Troubleshooting

Problema	Causa	Solución
No hay logs	Winston no está configurado	Verificar <code>logger.js</code> y ruta de archivos
No hay métricas	Endpoint /metrics no existe	Verificar ruta y middleware
Grafana no conecta	Prometheus apunta mal	Verificar <code>prometheus.yml</code> <code>target</code>
Alertas no llegan	SMTP no configurado	Verificar <code>alertmanager.yml</code>

## Mini-chequeo

► ¿Qué secciones tiene la plantilla de monitorización?

► ¿Qué información documentar sobre los logs?

► ¿Por qué documentar las métricas?

## ✓ Resumen en 3 frases

1. Documenta **logs** (librería, formato, rotación), **métricas** (nombres, tipos, descripción) y **dashboard** (paneles, consultas).
2. Incluye las **alertas** configuradas con sus condiciones y canales, y un apartado de **troubleshooting** para problemas comunes.
3. La documentación debe permitir que **cualquier persona reproduzca tu monitorización** sin preguntarte.

## 09 — Cierre

 Estás en:  UD 2.9 · Monitorización → 09 · Cierre

 *Ya sabes qué pasa con tu aplicación en producción*

Has recorrido toda la unidad: desde qué son los logs hasta cómo construir dashboards en Grafana. Ahora toca **verificar que lo entiendes** y **prepararte para la propuesta final**.

### La idea en una frase

**La monitorización te da datos reales sobre tu aplicación: logs para diagnosticar, métricas para tendencias, dashboards para visualizar.**

Si recuerdas esto, has entendido la unidad.

### Mini-chequeo final

Responde estas preguntas antes de seguir:

#### Preguntas de comprensión

► ¿Cuáles son los 5 niveles de log?

► ¿Qué son las métricas RED?

► ¿Qué hacen Prometheus y Grafana?

► ¿Qué es el stack mínimo de monitorización?

► ¿Cuántos paneles debería tener un dashboard?

---

## Ejercicio práctico

**En 15 minutos, configura la monitorización básica de tu proyecto:**

1. Añade **Winston** con logs estructurados a tu app
2. Configura un **middleware** que registre cada petición
3. Crea un endpoint `/metrics` con prom-client
4. Configura un **monitor en UptimeRobot** con la URL de tu app
5. Documenta la configuración con la plantilla del punto 08

**Si consigues ver logs estructurados y tu app aparece en UptimeRobot → has entendido la unidad.**

---



## Tabla resumen de la unidad

Concepto	Idea clave
<b>Logs</b>	Diario de tu app: qué pasa, cuándo y por qué
<b>Niveles</b>	ERROR, WARN, INFO, DEBUG, TRACE
<b>Métricas RED</b>	Rate, Errors, Duration
<b>Prometheus</b>	Recopila métricas (pull model)
<b>Grafana</b>	Visualiza en dashboards
<b>Alertas</b>	Te avisan cuando algo va mal
<b>Stack mínimo</b>	Winston + UptimeRobot para empezar
<b>Dashboard</b>	6-8 paneles, no más

## Vocabulario rápido

Término	Definición
<b>Log</b>	Registro de eventos de la aplicación
<b>Winston</b>	Librería de logs para Node.js
<b>prom-client</b>	Librería de métricas para Prometheus en Node.js
<b>Prometheus</b>	Sistema de recopilación de métricas (pull model)
<b>Grafana</b>	Plataforma de visualización y dashboards
<b>Alertmanager</b>	Gestor de alertas de Prometheus
<b>Counter</b>	Métrica que solo sube (total acumulado)
<b>Histogram</b>	Métrica que distribuye valores en rangos (buckets)
<b>Gauge</b>	Métrica que sube y baja (valor actual)
<b>Percentil</b>	P95 = 95% de los valores son menores a ese

Unidad	Qué aprenderás	Cómo se conecta
<b>UD 2.9: Propuesta Final</b>	Proyecto completo	Incluirás monitorización en tu proyecto final
<b>Proyecto real</b>	Producción	Aplicarás logs, métricas y dashboards
<b>Portfolio</b>	Demostrar habilidades	La monitorización te diferencia de otros

### Consejo para el siguiente punto

Si vas a preparar la propuesta final:

1. Revisa todo lo aprendido en las 9 UDs
2. Define qué proyecto quieres hacer
3. Documenta tu stack tecnológico incluyendo monitorización
4. Prepara una demo funcional

**La propuesta final es donde todo se une: requisitos, SCRUM, Git, patrones, IA, despliegue y monitorización.**

### Resumen en 3 frases

1. Los **logs** (Winston) registran qué pasa, las **métricas** (prom-client) dicen cómo está, y los **dashboards** (Grafana) lo visualizan.
2. Para empezar, necesitas **logs estructurados + UptimeRobot**. Para más, añade **Prometheus + Grafana**.
3. Un buen dashboard tiene **6-8 paneles** que muestran en un vistazo: Estado, Throughput, Errores, Latencia y Recursos.



## 01 — Briefing final

 Estás en:  UD 2.10 · Propuesta Final → 01 · Briefing final

 *Antes de construir, hay que entender qué se espera*

Este es el momento de entender **exactamente** qué se espera de tu proyecto final. No es solo “hacer una app”: es demostrar que puedes planificar, desarrollar, testear, desplegar y monitorizar una aplicación Full-Stack completa con metodología profesional.

### La idea en una frase

**El proyecto final demuestra que puedes llevar una idea de la conceptualización a producción con todas las habilidades aprendidas.**

## Qué se evalúa

Criterio	Peso	Qué demuestra
Funcionalidad	25%	Que la app funciona como debe
Calidad técnica	20%	Arquitectura, patrones, código limpio
Testing	15%	Tests unitarios, integración, cobertura
Despliegue	15%	Docker, CI/CD, HTTPS, producción
Documentación	10%	README, memoria, diagramas
Monitorización	5%	Logs, métricas, dashboard
Presentación	10%	Demo, defensa, explicación

## Stack mínimo esperado

Capa	Mínimo	Ideal
Frontend	Framework moderno (Vue, React, Angular)	+ Tailwind CSS
Backend	API REST con Node.js o Python	+ Autenticación JWT
BD	PostgreSQL o MongoDB	+ Migraciones
Testing	Tests unitarios	+ Integración + E2E
Despliegue	HTTPS funcionando	+ Docker + CI/CD
Monitorización	Logs básicos	+ Métricas + Dashboard

## Hitos del proyecto

Hito	Cuándo	Qué entregar
Propuesta	Inicio	Idea, stack, cronograma
Sprint 1-2	Semana 2-3	Backend funcional
Sprint 3-4	Semana 4-5	Frontend funcional
Sprint 5-6	Semana 6-7	Integración + tests
Sprint 7-8	Semana 8-9	Despliegue + monitorización
Entrega	Semana 10	Código + documentación + demo

## Qué NO hacer

### 1. No empieces a codificar sin planificar

El 70% de los proyectos que fracasan lo hacen porque el equipo empezó a programar sin definir requisitos, stack ni cronograma.

### 2. No elijas un proyecto demasiado grande

Un proyecto ambicioso que no terminas es peor que uno sencillo bien hecho. Prioriza **calidad sobre cantidad**.

### 3. No dejes el despliegue para última hora

El despliegue y la monitorización deben empezar en la semana 6-7, no en la última semana.

### 4. No ignores el testing

Los tests automatizados son **obligatorios**. Sin tests, el proyecto está incompleto.

## Aclaración: “¿Puedo hacer un proyecto solo?”

► 🤖 ¿Necesito equipo o puedo hacer el proyecto solo?

## Mini-chequeo

► ¿Cuáles son las 7 áreas de evaluación?

► ¿Cuándo debe empezar el despliegue?

► ¿Qué es lo más importante del proyecto final?

### ✓ Resumen en 3 frases

1. El proyecto final demuestra que puedes llevar una idea **de la conceptualización a producción** con metodología profesional.
2. Las 7 áreas de evaluación son **funcionalidad, calidad, testing, despliegue, documentación, monitorización y presentación**.
3. **Planifica antes de codificar**, elige un alcance realista y empieza el despliegue en la semana 6-7.

 Volver al [índice de la unidad](#) · Siguiendo: [02 — Ideas finales](#)

## 02 — Generar ideas de proyecto

 Estás en:  **UD 2.10 · Propuesta Final** → 02 · Generar ideas de proyecto

No necesitas una idea revolucionaria. Necesitas una idea **que puedas terminar en 8 semanas**, que demuestre las habilidades que has aprendido y que te motive lo suficiente para no abandonar a mitad.



## La idea en una frase

**La mejor idea de proyecto es la que se ajusta a tu tiempo, tus habilidades y tu motivación.**

## Métodos para generar ideas

### 1. Resuelve un problema real

Pregúntate: - ¿Qué me molesta de mi vida diaria? - ¿Qué hago de forma manual que podría automatizarse? - ¿Qué herramienta necesito que no existe?

### 2. Mejora algo existente

Toma una app que uses y piensa: - ¿Qué le falta? - ¿Qué haría diferente? - ¿Para quién no funciona bien?

### 3. Combina dos ideas

Idea A	Idea B	Combinación
Gestión de tareas	Gamificación	App de tareas con puntos y logros
Red social	Educación	Plataforma de estudio colaborativo
E-commerce	Local	Marketplace de productos de tu barrio

### 4. Usa un framework de ideas

Responde estas preguntas:

1. **¿Para quién?** → [Usuario objetivo]

2. **¿Qué problema resuelve?** → [Problema concreto]

3. **¿Cómo lo resuelve?** → [Solución técnica]
4. **¿Por qué es diferente?** → [Ventaja competitiva]
5. **¿Lo puedo terminar en 8 semanas?** → [Viabilidad temporal]

---

## Ejemplos de proyectos Realistas

### Bien valorados

- **Gestor de biblioteca** con préstamo, devolución y multi-usuario
- **Plataforma de eventos** con inscripción, pagos y notificaciones
- **App de recetas** con búsqueda, filtros y planificación de menú
- **Dashboard de métricas** con visualización en tiempo real
- **Sistema de reservas** con calendario y confirmación por email

### Evitar

- Red social completa (demasiado grande para 8 semanas)
- Clone de YouTube/Netflix (necesitas infraestructura de video)
- Juego 3D (necesitas habilidades que no son DAW)
- App móvil nativa (necesitas Swift/Kotlin, no es el enfoque)

---

## Criterios de selección de idea

Criterio	Pregunta	Peso
Viabilidad	¿Lo puedo terminar en 8 semanas?	30%
Aprendizaje	¿Demuestra las habilidades que necesito?	25%
Motivación	¿Me importa este proyecto?	20%
Complejidad	¿Tiene retos técnicos interesantes?	15%
Originalidad	¿Es diferente a lo que hacen otros?	10%

## Aclaración: “¿Qué pasa si no se me ocurre nada?”

▶ 🤔 ¿Qué hago si no tengo ninguna idea de proyecto?

### Mini-chequeo

▶ ¿Cuáles son los 4 métodos para generar ideas?

▶ ¿Cuál es el criterio más importante al elegir idea?

▶ ¿Qué proyectos debo evitar?

### ✓ Resumen en 3 frases

1. Genera ideas con **problemas reales**, **mejoras existentes**, **combinaciones** o un **framework de preguntas**.
2. Selecciona por **viabilidad** (¿lo termino?), **aprendizaje** (¿demuestra lo que sé?) y **motivación** (¿me importa?).
3. Si no se te ocurre nada, elige un proyecto clásico y **añade un giro único** que lo diferencie.

 Volver al [índice de la unidad](#) · Anterior: [01 — Briefing final](#) · Siguiente: [03 — Selección](#)

## 03 — Selección de la idea final

📌 La decisión final: elegir una y comprometerte con ella

Tienes varias ideas. Ahora toca elegir **una sola** y comprometerte con ella. No hay vuelta atrás: una vez empieces a desarrollar, cambiar de idea es perder tiempo. Elige bien y no mires atrás.

📌 La idea en una frase

**Elige la idea que puedas terminar, que te motive y que demuestre lo que sabes hacer.**

## Matriz de decisión

Evalúa cada idea en una escala de 1-5:

Criterio	Peso	Idea A	Idea B	Idea C
<b>Viabilidad temporal</b> (¿lo termino?)	30%	?/5	?/5	?/5
<b>Aprendizaje</b> (¿demuestra habilidades?)	25%	?/5	?/5	?/5
<b>Motivación</b> (¿me importa?)	20%	?/5	?/5	?/5
<b>Complejidad técnica</b> (¿retos interesantes?)	15%	?/5	?/5	?/5
<b>Originalidad</b> (¿es diferente?)	10%	?/5	?/5	?/5
<b>Puntuación total</b>		?	?	?



## Ejemplo: 3 ideas comparadas

Viabilidad: 5 (muy realista). Aprendizaje: 3 (CRUD básico). Motivación: 3 (aburrido).  
Total: 3.7

Viabilidad: 3 (ambicioso). Aprendizaje: 5 (múltiples entidades). Motivación: 4 (interesante). Total: 3.85

Viabilidad: 4 (realista). Aprendizaje: 4 (WebSockets, gráficas). Motivación: 5 (me encanta).  
Total: 4.25

## Preguntas de validación

Antes de decidir, responde honestamente:

### Viabilidad

1. ¿Tengo las habilidades necesarias o puedo aprenderlas a tiempo?
2. ¿Cuánto tiempo tengo realmente? (no el que te gustaría)
3. ¿Qué pasa si me equivoco en una estimación? ¿Tengo buffer?

### Aprendizaje

4. ¿El proyecto demuestra las competencias que se evalúan?

5. ¿Hay retos técnicos (no solo CRUD)?
6. ¿Puedo explicar las decisiones técnicas que tomaré?

## Motivación

7. ¿Me importa este proyecto o solo lo hago por la nota?
8. ¿Podré hablar de ello con entusiasmo en la presentación?
9. ¿Lo incluiría en mi portfolio?

---

## El test del “5 minutos”

Imagina que estás en la presentación final. El profesor te pregunta: “¿Por qué este proyecto?”

Si puedes responder con **entusiasmo y claridad** en 5 minutos → es la idea correcta.

Si necesitas justificar o explicar por qué no elegiste otra → quizás no es la mejor opción.

---

## Cómo comprometerte

Una vez elegida la idea:

1. **Documenta la decisión:** escribe por qué elegiste esta idea
2. **Comparte con el equipo:** todos deben estar de acuerdo
3. **No mires atrás:** no compares con otras ideas una vez empezado
4. **Adapta, no cambies:** si algo no funciona, adapta la idea, no la reemplaces

---

## Aclaración: “¿Qué pasa si me arrepiento?”

► 🤔 ¿Puedo cambiar de idea una vez empezado a desarrollar?

► ¿Qué es la matriz de decisión?

► ¿Cuál es el test más importante antes de decidir?

► ¿Cuándo Sí puedes cambiar de idea?

## ✓ Resumen en 3 frases

1. Evalúa cada idea con la **matriz de decisión**: viabilidad (30%), aprendizaje (25%), motivación (20%), complejidad (15%), originalidad (10%).
2. Usa el **test de los 5 minutos**: si puedes explicar con entusiasmo por qué elegiste esta idea, es la correcta.
3. Una vez elegida, **comprométete**: documenta la decisión, no mires atrás y adapta en vez de cambiar.

 Volver al [índice de la unidad](#) · Anterior: [02 — Ideas finales](#) · Siguiente: [04 — Stack final](#)

## 04 — Stack tecnológico final

 Estás en:  **UD 2.10 · Propuesta Final** → [04 · Stack tecnológico final](#)

 *El stack correcto es el que se ajusta a tu proyecto, no al más popular*

Elegir tecnologías es una de las decisiones más importantes del proyecto. Pero no se trata de usar la tecnología más nueva o popular: se trata de elegir la que mejor se adapte a tus necesidades, tu experiencia y tu tiempo.

El stack correcto es el que conoces (o puedes aprender rápido) y que se adapta a lo que tu proyecto necesita.

## Stack mínimo recomendado

Capa	Opción 1 (segura)	Opción 2 (moderna)
Frontend	Vue 3 + Vite + Tailwind	React + Next.js + Tailwind
Backend	Node.js + Express	Python + FastAPI
BD	PostgreSQL (Supabase)	MongoDB (Atlas)
Auth	JWT manual	Supabase Auth / Clerk
Testing	Jest/Vitest	Jest/Vitest + Cypress
Despliegue	Vercel + Render	Netlify + Railway
CI/CD	GitHub Actions	GitHub Actions
Monitorización	UptimeRobot + Winston	Grafana + prom-client

## Criterios de elección

### 1. Conocimiento vs aprendizaje

Situación	Recomendación
Conoces bien la tecnología	Úsala
La has usado poco	Úsala si es fácil de aprender
No la conoces pero es similar a algo que sabes	Úsala con tiempo de aprendizaje
Es completamente nueva	Evítala en el proyecto final

## 2. Complejidad del proyecto

Proyecto	Stack recomendado
CRUD simple	Vue + Express + PostgreSQL
Tiempo real (chat, notificaciones)	React + Socket.io + MongoDB
Dashboard con gráficas	Vue + D3.js/Chart.js + PostgreSQL
E-commerce	Next.js + Stripe + PostgreSQL

## 3. Despliegue

Stack	Despliegue más fácil
Frontend puro	Vercel / Netlify / GitHub Pages
Full-Stack Node.js	Render / Railway
Full-Stack Python	Render / Railway
Con Docker	Cualquier VPS (DigitalOcean, Hetzner)

## Stack según el tipo de proyecto

### CRUD básico (gestor de tareas, biblioteca, recetas)

```
Frontend: Vue 3 + Vite + Tailwind
Backend: Node.js + Express
BD: PostgreSQL (Supabase)
Auth: JWT
Deploy: Vercel (FE) + Render (BE)
```

## Tiempo real (chat, collaborative editing)

```
Frontend: React + Socket.io-client
Backend: Node.js + Socket.io
BD: MongoDB (Atlas)
Auth: JWT
Deploy: Render + MongoDB Atlas
```

## Dashboard con métricas

```
Frontend: Vue 3 + Chart.js/D3.js
Backend: Node.js + Express + prom-client
BD: PostgreSQL
Auth: JWT
Deploy: Vercel (FE) + Render (BE)
Monitor: Prometheus + Grafana
```

## Justificación del stack

En tu documentación, justifica cada elección:

```
Stack tecnológico

Frontend: Vue 3 + Vite + Tailwind
- **Por qué Vue**?: Conocimiento previo, documentación clara, Composition API
- **Por qué Vite**?: Rápido, configuración mínima, hot reload instantáneo
- **Por qué Tailwind**?: CSS utility-first, responsive rápido, sin CSS custom

Backend: Node.js + Express
- **Por qué Node.js**?: JavaScript en todo el stack, no necesito cambiar de lenguaje
- **Por qué Express**?: Simple, flexibles, mucha documentación

BD: PostgreSQL (Supabase)
- **Por qué PostgreSQL**?: Relacional, escalable, gratis en Supabase
- **Por qué Supabase**?: Dashboard incluido, auth, real-time, sin configurar servidor
```

## Aclaración: “¿Debo usar lo último de lo último?”

- ▶ 🤔 ¿Debo usar la tecnología más nueva o la más popular?

## Mini-chequeo

- ▶ ¿Cuáles son las capas del stack mínimo?

- ▶ ¿Cuál es el criterio más importante al elegir stack?

- ▶ ¿Qué hacer si no conozco ninguna tecnología del stack?

## ✅ Resumen en 3 frases

1. El stack correcto es el que **conoces o puedes aprender rápido** y se adapta a lo que tu proyecto necesita.
2. Justifica cada elección: **por qué ese framework, por qué esa BD, por qué ese despliegue.**
3. No uses tecnología nueva solo por novedad: en el proyecto final, **lo importante es que funcione y que lo entiendas.**

 Volver al [índice de la unidad](#) · Anterior: [03 — Selección](#) · Siguiente: [05 — Features](#)

### 📖 Menos features, más calidad: la clave del proyecto exitoso

La tentación es hacer “todo lo que se me ocurra”. Pero en 8 semanas no puedes hacer todo. La clave es definir **pocas features bien hechas** que demuestren tus habilidades, no muchas features a medias.

## 📖 La idea en una frase

Define 5-8 features principales que puedas terminar con calidad, no 20 que hagas a medias.

## Método MoSCoW para priorizar

Prioridad	Significado	Cantidad típica
<b>Must</b>	Imprescindible para que el proyecto tenga sentido	3-4 features
<b>Should</b>	Importante pero no vital	2-3 features
<b>Could</b>	Deseable si sobra tiempo	1-2 features
<b>Won't</b>	Futuro, no va en el proyecto final	Las que quieras

### Regla práctica

- **Must:** Lo que presentas al tribunal como “esto es mi proyecto”
- **Should:** Lo que añades si el profesor pregunta “¿qué más?”
- **Could:** Lo que documentas como “futuras mejoras”
- **Won't:** Lo que no mencionas



# Ejemplo: Gestor de tareas

## Must (MVP)

1. CRUD de tareas (crear, listar, editar, eliminar)
2. Filtrado por estado (pendiente/hecha) y prioridad
3. Autenticación de usuarios (registro/login)
4. Dashboard con estadísticas básicas

## Should

5. Recordatorios por fecha
6. Exportar tareas a CSV
7. Modo oscuro

## Could

8. Colaboración (compartir tareas)
9. Etiquetas personalizadas
10. Estadísticas avanzadas con gráficas

## Won't

- App móvil
- Chat integrado
- Pagos
- Notificaciones push

---

## Criterios de calidad para cada feature

Cada feature Must debe cumplir:

- ☐ **Funciona:** no tiene bugs visibles
- ☐ **Valida datos:** campos obligatorios, formatos correctos
- ☐ **Maneja errores:** muestra mensajes claros al usuario
- ☐ **Es responsive:** funciona en móvil y escritorio
- ☐ **Tiene tests:** al menos tests unitarios
- ☐ **Está documentada:** el README la menciona

# Cuántas features por area de evaluación

Área	Features necesarias
Funcionalidad	3-4 Must + 1-2 Should
Testing	Tests para cada feature Must
Despliegue	La app desplegada con HTTPS
Documentación	README + memoria
Monitorización	Logs + uptime

## Aclaracion: “¿Y si termino antes?”

► 🤔 ¿Qué hago si termino las features antes de tiempo?

## Mini-chequeo

► ¿Cuántas features Must debería tener mi proyecto?

► ¿Qué hace una feature “de calidad”?

► ¿Qué hago si termino antes de tiempo?

✅ Resumen en 3 frases

1. Define **3-4 features Must** que sean el corazón de tu proyecto, **2-3 Should** importantes y **1-2 Could** deseadas.
2. Cada feature Must debe **funcionar, validar, manejar errores, ser responsive, tener tests** y estar documentada.
3. **Menos features bien hechas** que muchas a medias: la calidad se valora más que la cantidad.

 Volver al [índice de la unidad](#) · Anterior: [04 — Stack final](#) · Siguiente: [06 — Cronograma](#)

## 06 — Cronograma final

 Estás en:  **UD 2.10 · Propuesta Final** → [06 · Cronograma final](#)

 *Un cronograma realista te salva de la angustia de última hora*

El 70% de los proyectos se terminan en la última semana. Eso genera estrés, errores y código chapucero. Un cronograma con hitos claros te permite distribuir el trabajo y entregar con calma.

### La idea en una frase

**Distribuir el trabajo en sprints de 1-2 semanas con hitos claros, y nunca llegarás desesperado a la fecha de entrega.**

## Cronograma estándar (10 semanas)

Semana	Fase	Actividad	Entregable
1	Planificación	Definir idea, stack, features, cronograma	Propuesta escrita
2	Backend基础	Setup proyecto, estructura, BD	Backend arranca
3	Backend API	CRUDs, autenticación	API funcional
4	Backend avanzado	Lógica de negocio, validaciones	Backend completo
5	Frontend基础	Setup, componentes, routing	Frontend arranca
6	Frontend integración	Conectar con API, formularios	Frontend funcional
7	Testing	Tests unitarios, integración	Cobertura >60%
8	Despliegue	Docker, CI/CD, HTTPS	App en producción
9	Monitorización	Logs, métricas, dashboard	Monitorización activa
10	Preparación	Documentación, demo, presentación	Todo listo

## Hitos obligatorios

Hito	Semana	Qué debe funcionar
Backend	3-4	API REST completa con auth
Frontend	5-6	UI conectada con API
Testing	7	Tests ejecutándose en CI
Producción	8-9	App accesible con HTTPS
Entrega	10	Código + docs + demo

# Consejos de planificación

## 1. Estima en días, no en horas

Feature: CRUD de tareas

- Día 1: Modelo de datos + migración

- Día 2: Endpoints GET/POST

- Día 3: Endpoints PUT/DELETE

- Día 4: Validaciones + tests

- Día 5: Buffer (imprevistos)

## 2. Añade buffer

Nunca planifiques al 100% del tiempo. Reserva **20% para imprevistos**:

Tiempo total: 10 semanas

Desarrollo: 8 semanas (80%)

Buffer: 2 semanas (20%)

## 3. Define “done” para cada sprint

Sprint 1 (Backend): DONE cuando

- [ ] La API responde en /api/users

- [ ] CRUD de usuarios funciona

- [ ] Auth JWT funciona

- [ ] Tests pasan

## 4. Usa un tablero visual

Pendiente	En progreso	Hecho
Feature 3	Feature 2	Feature 1
Feature 4		Feature 0

# Errores de planificación

Error	Consecuencia	Cómo evitarlo
No poner fechas	“Lo hago mañana”	Asigna fecha a cada tarea
No poner buffer	Imposible terminar	Reserva 20% del tiempo
Estimar en horas optimistas	Siempre te pasas	Duplica la estimación
No priorizar	Todo es urgente	Usa MoSCoW
Cambiar de features	Pierdes tiempo	Comprométete con la lista

## Aclaracion: “¿Y si me atraso?”

► 🤔 ¿Qué hago si voy retrasado?

## Mini-chequeo

► ¿Cuántas semanas de planificación necesitas?

► ¿Cuánto buffer debo reservar?

► ¿Qué es un “hito” en el cronograma?

## ✅ Resumen en 3 frases

1. Planifica en **10 semanas** con hitos claros: planificación → backend → frontend → testing → despliegue → monitorización → entrega.
2. Reserva **20% del tiempo para imprevistos** y define “done” para cada sprint con criterios concretos.

3. Si te atrasas, **corta features**, no calidad: un proyecto desplegado con 3 features se valora más que uno local con 6.

 Volver al [índice de la unidad](#) · Anterior: [05 — Features](#) · Siguiente: [07 — Entregables](#)

## 07 — Entregables

 **Estás en:**  **UD 2.10 · Propuesta Final** → 07 · Entregables

 *Entregar bien es tan importante como desarrollar bien*

Puedes tener un proyecto increíble pero si no lo entregas correctamente, pierdes puntos. Los entregables son la “caja” en la que presentas tu trabajo. Si la caja está desordenada, el tribunal no verá la calidad de lo que hay dentro.

### La idea en una frase

**Los entregables son la primera impresión del tribunal: organízalos bien para que tu trabajo brille.**

## Lista de entregables

### 1. Repositorio GitHub

Aspecto	Qué esperar
<b>README.md</b>	Descripción, stack, cómo ejecutar, URL de producción
<b>Estructura</b>	Carpetas organizadas (src/, tests/, docs/)
<b>Historial</b>	Commits claros con mensajes descriptivos
<b>Ramas</b>	Estrategia de branching documentada (GitFlow o similar)
<b>.gitignore</b>	Completo (node_modules, .env, dist)
<b>LICENSE</b>	Licencia del proyecto

## 2. Código fuente

Aspecto	Qué esperar
<b>Backend</b>	API REST funcional con auth
<b>Frontend</b>	UI completa y funcional
<b>Tests</b>	Tests unitarios y/o de integración
<b>Configuración</b>	Variables de entorno documentadas
<b>Docker</b>	Dockerfile y docker-compose.yml (si aplica)

## 3. Despliegue

Aspecto	Qué esperar
<b>URL de producción</b>	App accesible con HTTPS
<b>CI/CD</b>	GitHub Actions configurado
<b>Monitorización</b>	Logs y/o uptime monitoring

## 4. Documentación



Documento	Contenido
<b>README.md</b>	Instrucciones de instalación y uso
<b>Memoria</b>	Comparativa con Proyecto 1, decisiones técnicas
<b>Diagramas</b>	Diagrama C4 mínimo, diagrama ER
<b>Guía de usuario</b>	Cómo usar la aplicación

## 5. Demo

Aspecto	Qué esperar
<b>Preparada</b>	Funciona sin errores
<b>Con datos</b>	Tiene datos de ejemplo cargados
<b>Probada</b>	La has probado en el navegador del tribunal
<b>Backup</b>	Tienes una versión local por si falla internet

## Checklist de entrega

### Pre-entrega (1 semana antes)

- ☐ Todas las features Must funcionan
- ☐ Tests ejecutándose y pasando
- ☐ App desplegada con HTTPS
- ☐ README actualizado con instrucciones
- ☐ Memoria escrita
- ☐ Diagramas incluidos
- ☐ Demo probada en otro navegador
- ☐ Datos de demo cargados

### Día de la entrega

- ☐ Último commit con cambios finales
- ☐ Repositorio limpio (sin archivos innecesarios)
- ☐ URL de producción funciona

- ☐ Demo lista para presentar
- ☐ Backup local preparado

# Template de README

```
Nombre del Proyecto

Descripción breve del proyecto (1-2 frases).

Stack tecnológico
- Frontend: [Framework + CSS]
- Backend: [Framework]
- BD: [Base de datos]
- Despliegue: [Plataforma]

Cómo ejecutar

Requisitos
- Node.js v20+
- npm

Instalación
```bash
git clone https://github.com/usuario/proyecto.git
cd proyecto
npm install
cp .env.example .env
npm run dev
```

Variables de entorno

Variable	Descripción	Ejemplo
DATABASE_URL	URL de la BD	postgres://...
JWT_SECRET	Secreto JWT	mi-secreto

Despliegue

URL de producción: https://mi-app.vercel.app

Autor

[Tu nombre] - [Tu email] ```

Aclaración: “¿Qué es la memoria?”

▶ 🤔 ¿Qué debo incluir en la memoria del proyecto?

Mini-chequeo

▶ ¿Cuáles son los 5 entregables principales?

▶ ¿Qué debe tener el README?

▶ ¿Qué es la memoria del proyecto?

✅ Resumen en 3 frases

1. Los 5 entregables son **repositorio limpio, código funcional, despliegue con HTTPS, documentación completa y demo preparada**.
2. El README debe tener **descripción, stack, instrucciones de instalación y URL de producción**.
3. Prepara la demo **una semana antes**: prueba en otros navegadores, carga datos de ejemplo y ten un backup local.

 [Volver al índice de la unidad](#) · Anterior: [o6 — Cronograma](#) · Siguiente: [o8 — Evaluación](#)

📖 Saber cómo se evalúa te permite enfocar tu esfuerzo

No es lo mismo “hacer un proyecto” que “hacer un proyecto que cumple los criterios de evaluación”. Conocer qué se valora te permite distribuir tu tiempo en lo que realmente puntúa.

📖 La idea en una frase

Conoce los criterios de evaluación y enfoca tu esfuerzo en lo que realmente puntúa.

Desglose de evaluación

Criterio	Peso	Qué se valora
Funcionalidad	25%	Que las features principales funcionen correctamente
Calidad técnica	20%	Arquitectura, patrones, código limpio, buenas prácticas
Testing	15%	Tests unitarios, integración, cobertura
Despliegue	15%	Docker, CI/CD, HTTPS, producción funcionando
Documentación	10%	README, memoria, diagramas
Monitorización	5%	Logs, métricas, uptime
Presentación	10%	Demo fluida, explicación clara, respuestas

Cómo maximizar cada criterio

Funcionalidad (25%)

- Las features **Must** funcionan sin errores visibles
- Los formularios **validan** datos
- Los errores se **muestran** al usuario (no 500 genérico)
- La app es **responsive** (móvil y escritorio)

Calidad técnica (20%)

- **Patrones de diseño** aplicados (Factory, Strategy, etc.)
- **Separación de capas** (controllers, services, routes)
- **Código limpio** (nombres descriptivos, funciones pequeñas)
- **Sin código duplicado** (DRY)
- **Manejo de errores** consistente

Testing (15%)

- Tests para cada feature Must
- Tests de integración para la API
- Cobertura >60%
- Tests ejecutándose en CI/CD

Despliegue (15%)

- App accesible con **HTTPS**
- **CI/CD** configurado (GitHub Actions)
- **Docker** funcional (si aplica)
- Variables de entorno **no hardcodeadas**

Documentación (10%)

- README con instrucciones claras
- Memoria con decisiones justificadas
- Diagrama C4 mínimo
- Diagrama ER completo

Monitorización (5%)

- Logs estructurados (Winston o similar)
- Uptime monitoring (UptimeRobot o similar)

- Dashboard básico (Grafana o similar)

Presentación (10%)

- Demo **preparada y probada**
- Explicación **clara y concisa** (5-10 minutos)
- Respuestas a preguntas del tribunal
- **Comparativa con Proyecto 1**

Errores que penalizan

Error	Consecuencia
App no desplegada	-15% (despliegue)
Sin tests	-15% (testing)
Código sin documentar	-10% (documentación)
Variables hardcodeadas	-5% (calidad)
Demo que no funciona	-10% (presentación)
No explicar decisiones	-10% (calidad)

Aclaracion: “¿Qué se valora más, la cantidad o la calidad?”

► 🤔 ¿Es mejor tener muchas features o pocas bien hechas?

Mini-chequeo

► ¿Cuáles son los 7 criterios de evaluación?

► ¿Qué error penaliza más?

► ¿Qué es lo más importante en la presentación?

✓ Resumen en 3 frases

1. Los criterios principales son **funcionalidad (25%)**, **calidad técnica (20%)**, **testing (15%)** y **despliegue (15%)**.
2. **Pocas features bien hechas** se valora más que muchas a medias: el tribunal prefiere calidad sobre cantidad.
3. Los errores que más penalizan son **no desplegar la app** y **no tener tests**: evítalos a toda costa.

 Volver al [índice de la unidad](#) · Anterior: [07 — Entregables](#) · Siguiente: [09 — Cierre](#)

09 — Cierre

 Estás en:  **UD 2.10 · Propuesta Final** → 09 · Cierre

 *Estás listo para empezar tu proyecto final*

Has recorrido toda la guía: desde qué se espera hasta cómo se evalúa. Ahora toca **verificar que lo entiendes** y **empezar a trabajar en tu proyecto. Esto es solo el comienzo.

Un proyecto final exitoso se resume en: **planificar bien, ejecutar con calidad, entregar a tiempo y explicar lo que hiciste.**

Si recuerdas esto, has entendido la unidad.

Mini-chequeo final

Responde estas preguntas antes de empezar tu proyecto:

Preguntas de comprensión

► ¿Cuáles son los 7 criterios de evaluación y sus pesos?

► ¿Cuántas features Must debería definir?

► ¿Cuándo debe empezar el despliegue?

► ¿Qué documentos debes entregar?

► ¿Qué es lo más importante en la presentación?

Ejercicio final

En 30 minutos, completa tu propuesta de proyecto:

1. Escribe el **problema** que resuelve tu proyecto (1-2 frases)
2. Define **3-4 features Must** con criterios de calidad
3. Elige tu **stack tecnológico** con justificación
4. Crea un **cronograma** con hitos por semana
5. Prepara la **matriz de decisión** si tienes varias ideas

Si consigues completar estos 5 puntos → **estás listo para empezar.**

Tabla resumen de la unidad

Concepto	Idea clave
Briefing	Qué se espera del proyecto final
Ideas	Genera varias y evalúa con criterios
Selección	Elige una y comprométete
Stack	Elige lo que conozcas y se adapte
Features	3-4 Must, calidad sobre cantidad
Cronograma	10 semanas con hitos y buffer
Entregables	Repo, código, deploy, docs, demo
Evaluación	7 criterios, enfoca en los de mayor peso

Vocabulario rápido

Término	Definición
MoSCoW	Método de priorización: Must, Should, Could, Won't
MVP	Minimum Viable Product: la versión más pequeña que funciona
Hito	Punto concreto en el tiempo con un entregable definido
Buffer	Tiempo de reserva para imprevistos
Stack	Conjunto de tecnologías que usa un proyecto
CRUD	Create, Read, Update, Delete: operaciones básicas
CI/CD	Continuous Integration / Continuous Deployment
Memoria	Documento que justifica las decisiones técnicas

UD	Qué aprendiste	Cómo lo usas
2.1 Requisitos Avanzado	Criterios de aceptación, INVEST	Definir qué hace tu app
2.2 SCRUM Avanzado	Métricas, burndown, retrospectivas	Organizar tu trabajo
2.3 Git Avanzado	GitFlow, hooks, CI/CD básico	Controlar versiones
2.4 Patrones de Diseño	Factory, Strategy, Observer	Diseñar la arquitectura
2.5 Diagramas	C4, ER, secuencia	Modelar tu sistema
2.6 IA como Copiloto	Prompts, testing, éticas	Usar IA responsablemente
2.7 Despliegue	Docker, CI/CD, producción	Llevar tu app a producción
2.8 Monitorización	Logs, métricas, dashboards	Saber qué pasa en producción
2.9 Propuesta Final	Todo junto	Definir y entregar tu proyecto

Consejo final

El proyecto no es solo una nota: es tu **portfolio**. Un proyecto bien hecho, desplegado y documentado te diferencia de otros desarrolladores. No lo veas como un examen, véalo como una oportunidad de **crear algo que puedas enseñar**.

Empieza hoy. Planifica esta semana. Codifica las próximas. Despliega en la penúltima. Presenta con orgullo.

Resumen en 3 frases

1. Un proyecto exitoso se resume en: **planificar bien, ejecutar con calidad, entregar a tiempo y explicar lo que hiciste.**

2. Los criterios principales son **funcionalidad, calidad técnica, testing y despliegue**: enfoca tu esfuerzo ahí.
3. **Empieza hoy, planifica esta semana, codifica las próximas, despliega en la penúltima y presenta con orgullo.**



Volver al [índice de la unidad](#) · Anterior: [o8 — Evaluación](#) · Volver a [Guía Didáctica Proyecto 2](#)